DEEP LEARNING

AI · ML · DS / REVISION NOTES

UDEMY

# Convolutional Neural Networks with PyTorch

Deep-learning fundamentals and convolutional networks in PyTorch

22 SECTIONS

96 FIGURES

129 PAGES

Compiled October 2026

github.com/ankitgupta1729/AI-ML-DS

# Contents

LECTURE 1 Introduction 3
LECTURE 2 What is a Neural Network 8
LECTURE 3 Use cases of Neural Networks 12
LECTURE 4 Mathematical view of General Neural Networks 18
LECTURE 5 Type of Neural Networks models 25
LECTURE 6 Overview Convolutional Neural Network 29
LECTURE 7 Overview of Convolutional Layer 31
LECTURE 8 Overview of Pooling Layer 57
LECTURE 9 Overview of Fully Connected Layer 61
LECTURE 10 Typical Convolutional Neural Network architecture 65
LECTURE 11 Mathematical view of training a Neural Network 71
LECTURE 12 Practical view of training a Neural Network 81
LECTURE 13 Train with batches 87
LECTURE 14 Practical example presentation 93
LECTURE 15 Dataset 97
Load dataset 97
LECTURE 16 Image and Iter Function 103
Visualize data 103
LECTURE 17 Create Convolutional Neural Network model 106
LECTURE 18 Define cost function and optimizer 113
LECTURE 19 Train model train dataset 114
LECTURE 20 Train model validation dataset 118
LECTURE 21 Analyze model on test dataset and metrics 122
LECTURE 22 Practical advices about training 127

LECTURE 1

# Introduction

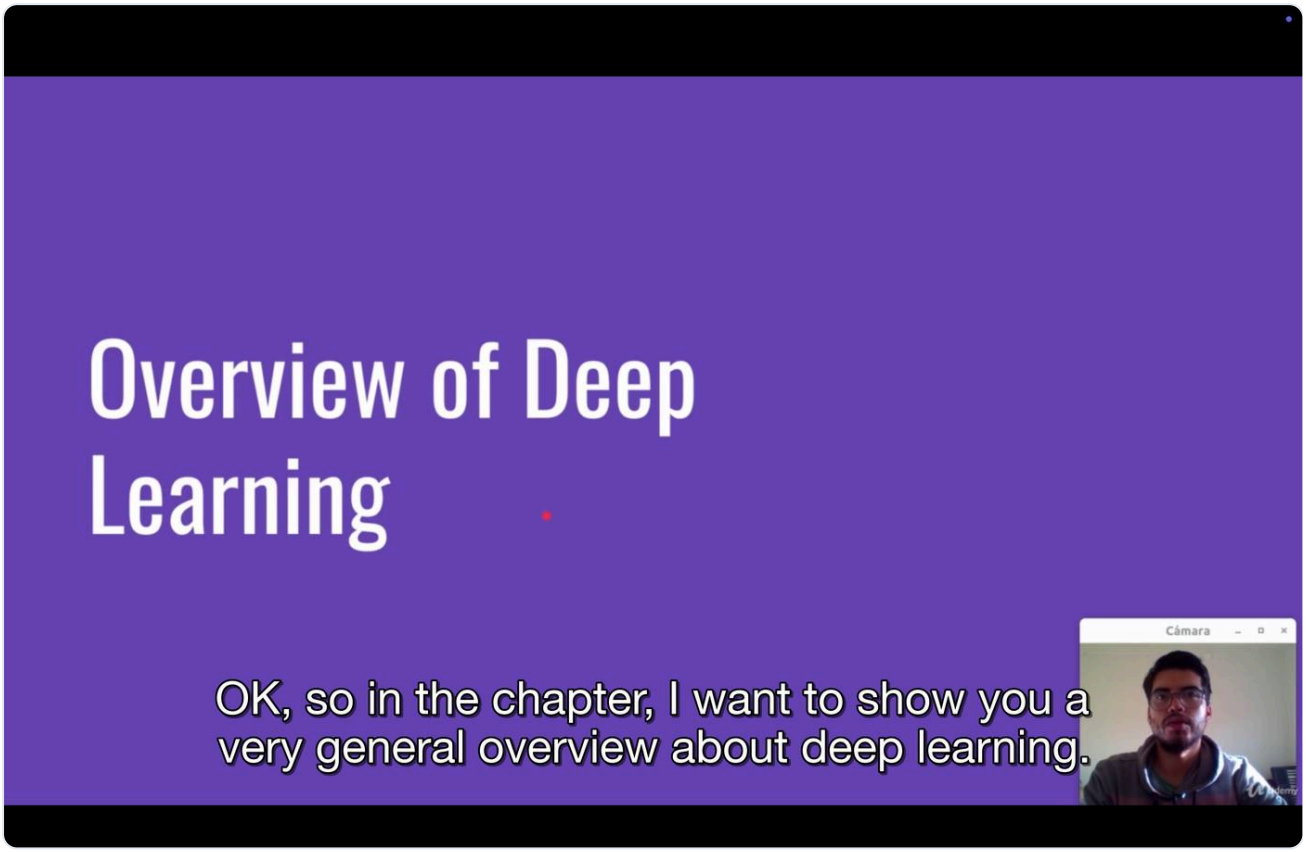



FIGURE 1.1

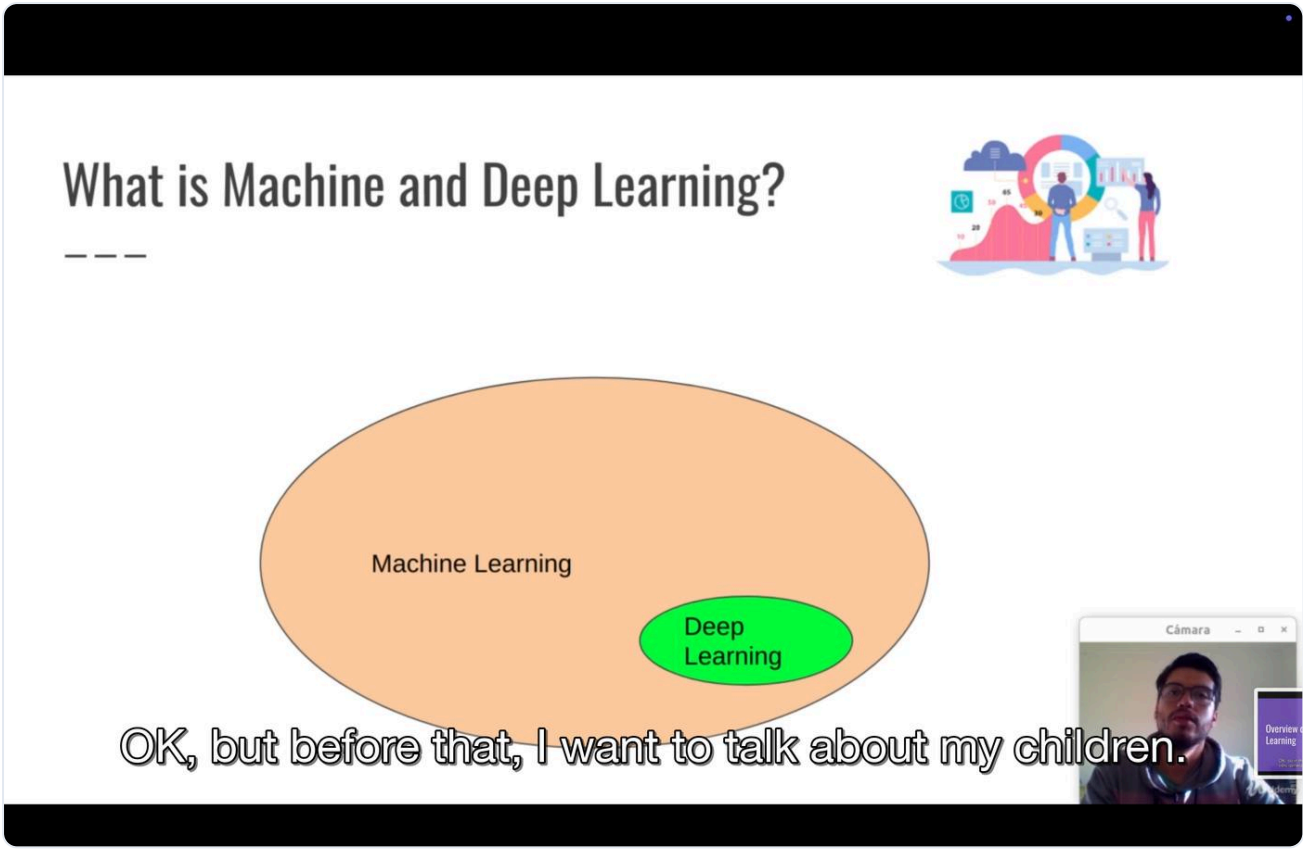



FIGURE 1.2 | Machine Learning

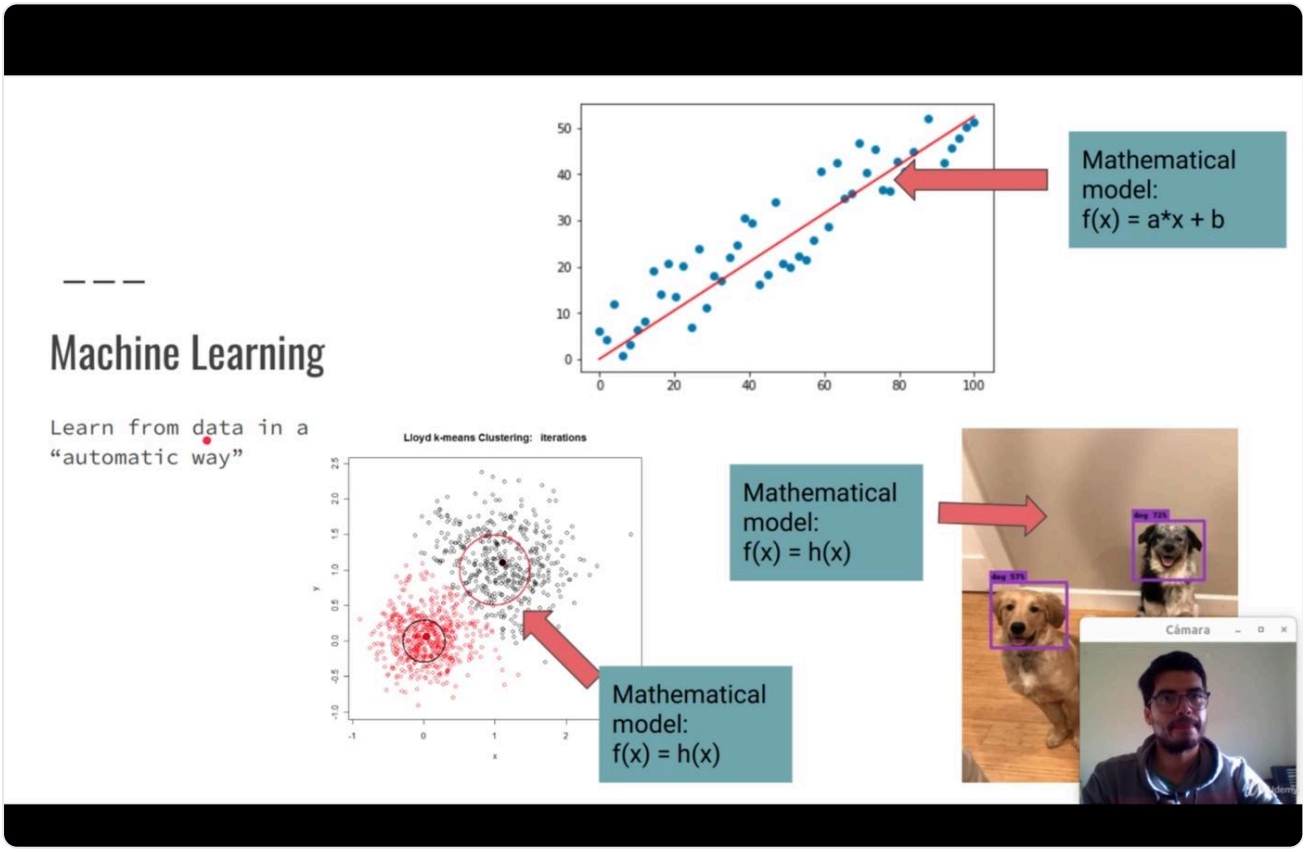



FIGURE 1.3

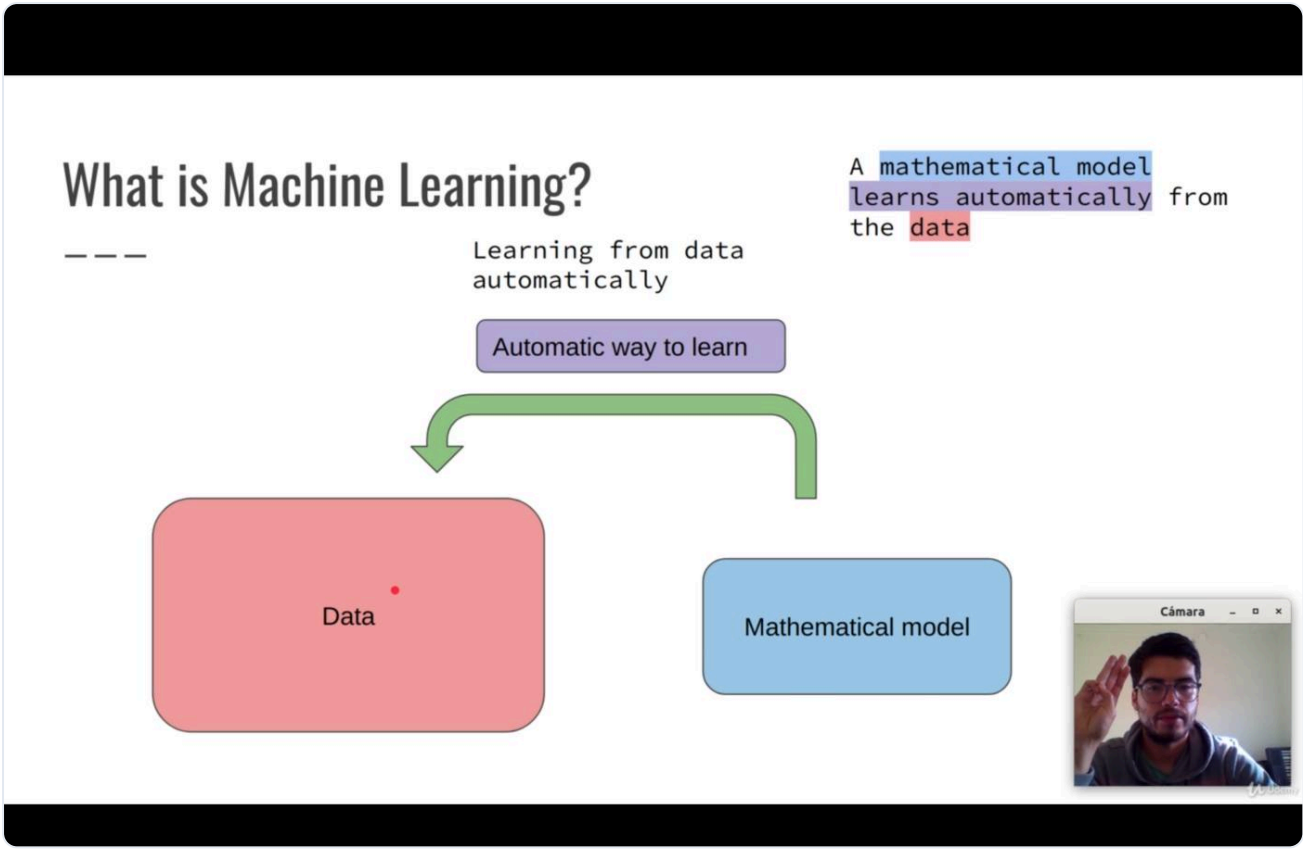



FIGURE 1.4 | A Mathematical Model

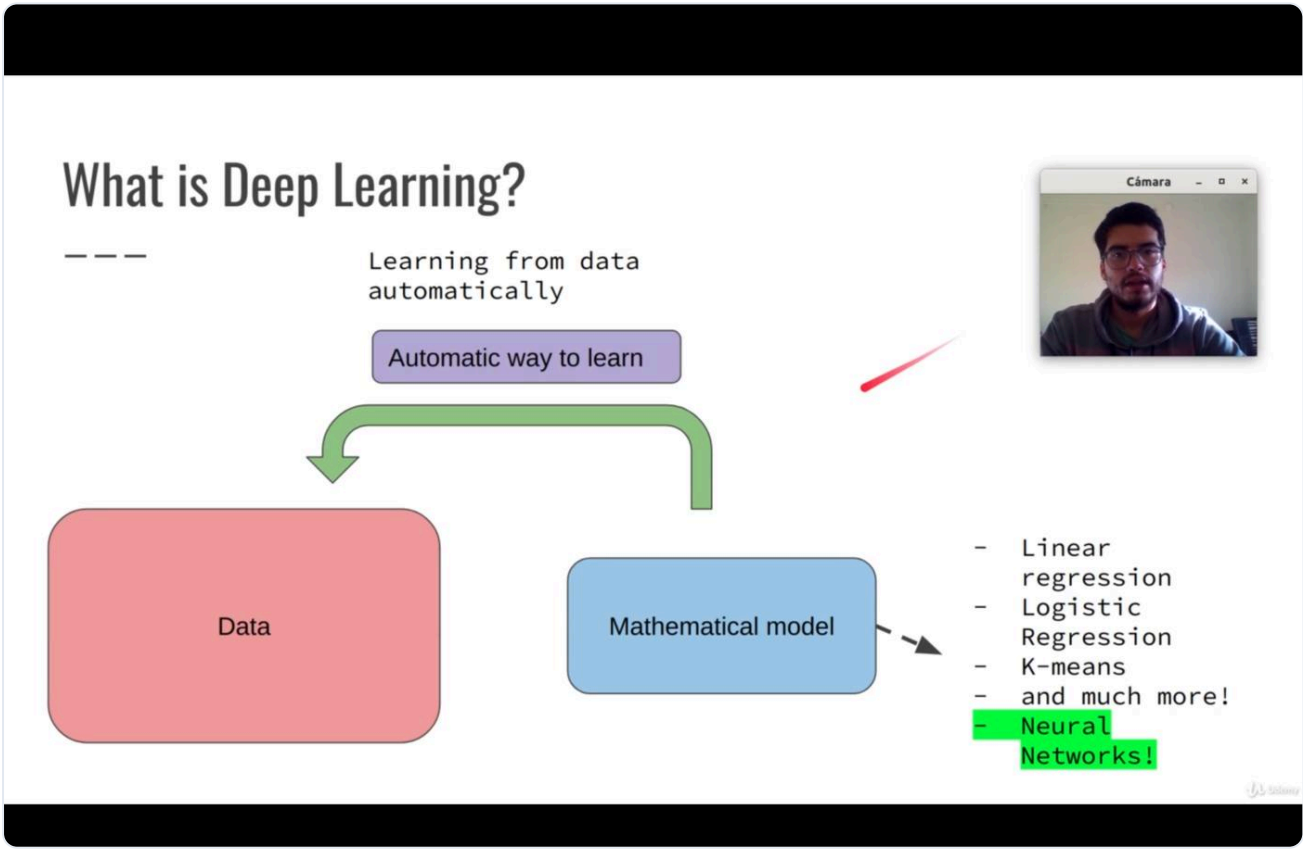



FIGURE 1.5 | What Is Deep Learning? Cn

OK, so in this chapter, I want to show you a very general overview of deep learning. OK, but before that, I want to talk about machine learning. OK, so what you're learning is this area, this kind of field here, and a specific area of machine learning is deep learning. OK, this area here. OK, so first we have to answer the question: what is machine learning? OK, so in a very general way, machine learning is a set of different methods that allow a mathematical model to learn from data in an automatic way.

OK, so let's see this in the next example. So we are always going to need three different ingredients in machine learning. OK, so the first one is data. OK. So in this example here, the different blue points are the data, OK, so that's the data, and so on. OK. The second ingredient is that we are going to need a mathematical model. OK, so in this example we have this red line, and this red line is a mathematical model. If you remember from school, this is a mathematical model.

OK, this is a function, the equation of this mathematical model. OK, so first we have the data, the blue points, and then we have the mathematical model, this red line here, OK? And then there is a third ingredient in any machine learning problem: the automatic way to learn. That means that we are going to have an algorithm that allows this mathematical model to best fit this data. And basically this algorithm is going to find the best line, in this example, for this data.

OK, so again, we're going to have the data, the blue points here. Then we're going to have the mathematical model, the red line here. And then we're going to have an algorithm that allows us to find the best red line for these blue points. OK, in this other example, we're going to have the same three ingredients. The first one is going to be the data, the black points and the red points. Then we're going to have the mathematical model. And in this example, the mathematical model is these kinds of circles, or the centers of the circles.

OK, and these circles are represented by a mathematical model. And the third thing is going to be the algorithm that is going to allow us to find the best circles for these different groups. OK, and in this final example, we're going to have the same thing. OK, the first thing is that we are going to need the data, OK? And the data in this example is the image. OK, so here we have the image of two dogs. The second ingredient is going to be the model. OK, so in this example, this is going to be a very complex model.

But finally, this is going to be a function of the image. Right. And this model is going to identify the faces of these dogs. OK, and then there is the algorithm that is going to allow this mathematical model to learn where the faces of the dogs are, OK. So here we have this kind of very general machine learning project, and here we have the three different ingredients. OK, so the first one is going to be the data, the second one is going to be the mathematical model, and the third one is going to be the automatic way to learn.

And that means that we are going to have a mathematical model that is going to learn in an automated way to do something with the data. OK, so give me a second. In machine learning, we can use different mathematical models. OK, one of the most famous is linear regression. Linear regression, in this example, can be this red line. OK, so this red line is actually a linear regression. OK, another very famous mathematical model is logistic regression.

OK, this is another model that allows us to do some kind of classification. Then we can use, for example, another mathematical model, and this is K-means. K-means is a very, very famous algorithm that allows us to do clustering. So, for example, these circles are done using the K-means algorithm, and many more we can use. There are actually tons of different mathematical models that allow us to learn from the data. OK, so this is machine learning. OK, deep learning specifically is a sub-area of machine learning.

But instead of using many different mathematical models, we're going to use only the neural network model. In the next section we're going to learn what a neural network is. But basically a neural network is a specific mathematical model. OK, so if we come back to our schema here, we're going to have the machine learning area, and in machine learning we have basically different algorithms and different mathematical models, but specifically in deep learning, we are only going to use neural network mathematical models.

OK.

LECTURE 2

# What is a Neural Network

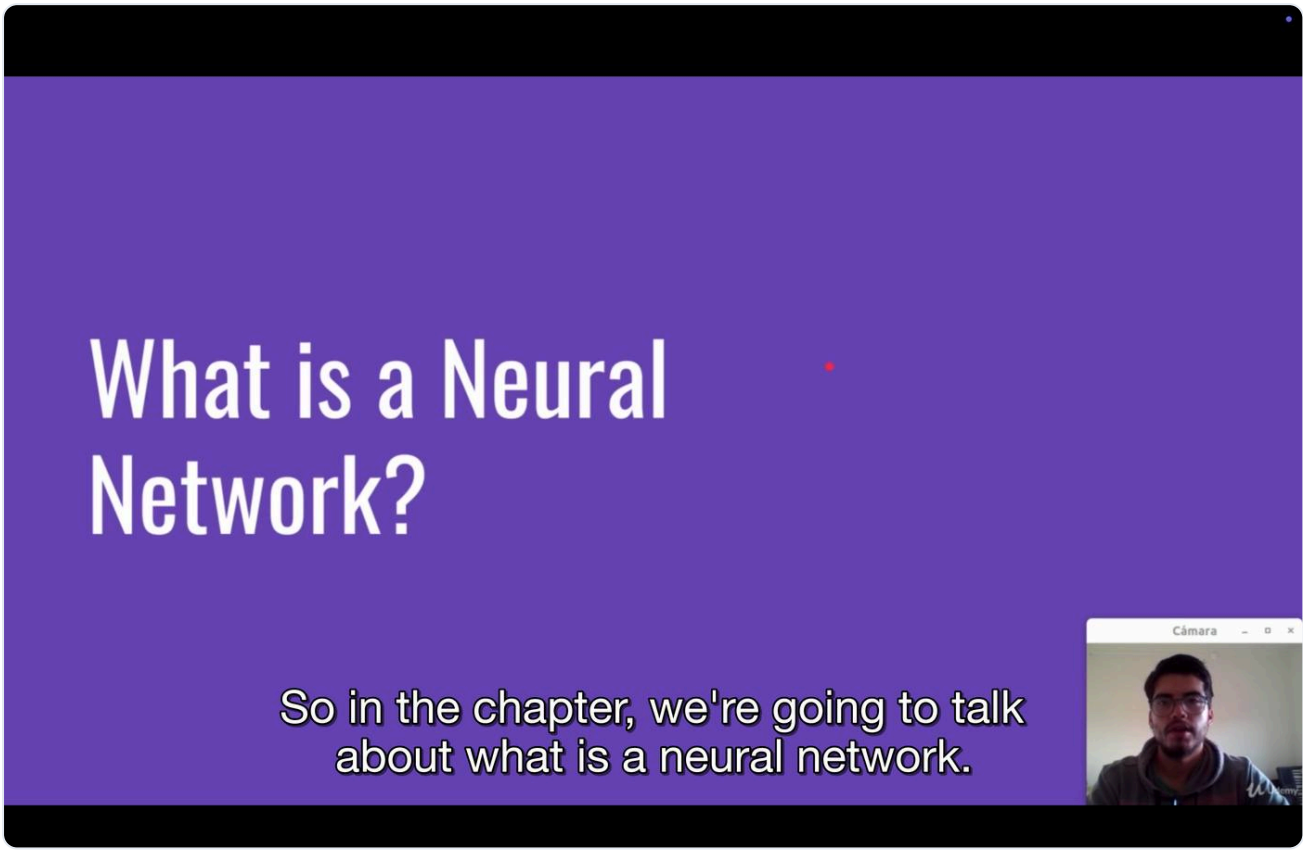



FIGURE 2.1

So in this chapter, we're going to talk about what a neural network is. OK, so it turns out that the neural network that we see in deep learning is inspired by the real biological system of the neurons that we have in our brain. OK, so here we have a representation, a very basic representation, of a real biological neural network. OK, so here we have different neurons. OK, here we have neuron one, two and three.

OK, so the way in which this biological system works is this: we're going to have the information, and it can be any signal that we can have in our brain, like electrical, I don't know, right. And this information is going to arrive at the first neuron. OK, so this first neuron is going to receive that information, and this information is going to flow through this neuron until it arrives at the end of this neuron. OK, this neuron is going to pass this information to the next neuron, right?

This next neuron is going to take the information and is going to flow this information until the end. At the end of this neuron, it is going to pass this information to the next neuron, OK? And the information is going to flow through it. And then it's going to arrive at the final piece of this neuron, and then this neuron can send information to our system. OK, so this part of this neuron can be thought of as the output. OK, so first we're going to have an input and then we're going to have an output.

And information in this biological system is going to flow through the different neurons until it arrives at the output. OK, that is the biological system. It turns out that the neural network in deep learning is based on this. We're going to have an input, and then we're going to have the output, OK? And it turns out that the information from the input is going to flow through the entire system. This system, the neural network system, or neural network, is going...

This information from the input is going to flow through this entire system, and then we're going to have an output. OK, so that is the first thing that we have in common. It turns out that this neural network, this mathematical neural network, is composed of different layers. OK, so we have here the input layer. Then we have here the output layer, on the right side. And between the input and output, we are going to have a set of different layers. OK, so here we have the input layer.

Here we have the output layer. And between these we're going to have these layers. And the name of these layers usually is going to be the hidden layers. OK, so in this example, we have hidden layer one and hidden layer two. OK, so each of these layers has these circles, OK? And each of these circles is a neuron. OK, so a neuron, in this mathematical view, you can think of each of these neurons as something like this biological one here. OK, so how this works: basically we are going to have the input layer.

This can be, for example, three numbers. This can be three pixels of an image. This can be three, I don't know, three points of a time series, and so on. It can be anything. OK, so this input layer, this input data, is going to flow through the input layer to the next layer, to the hidden layer one.

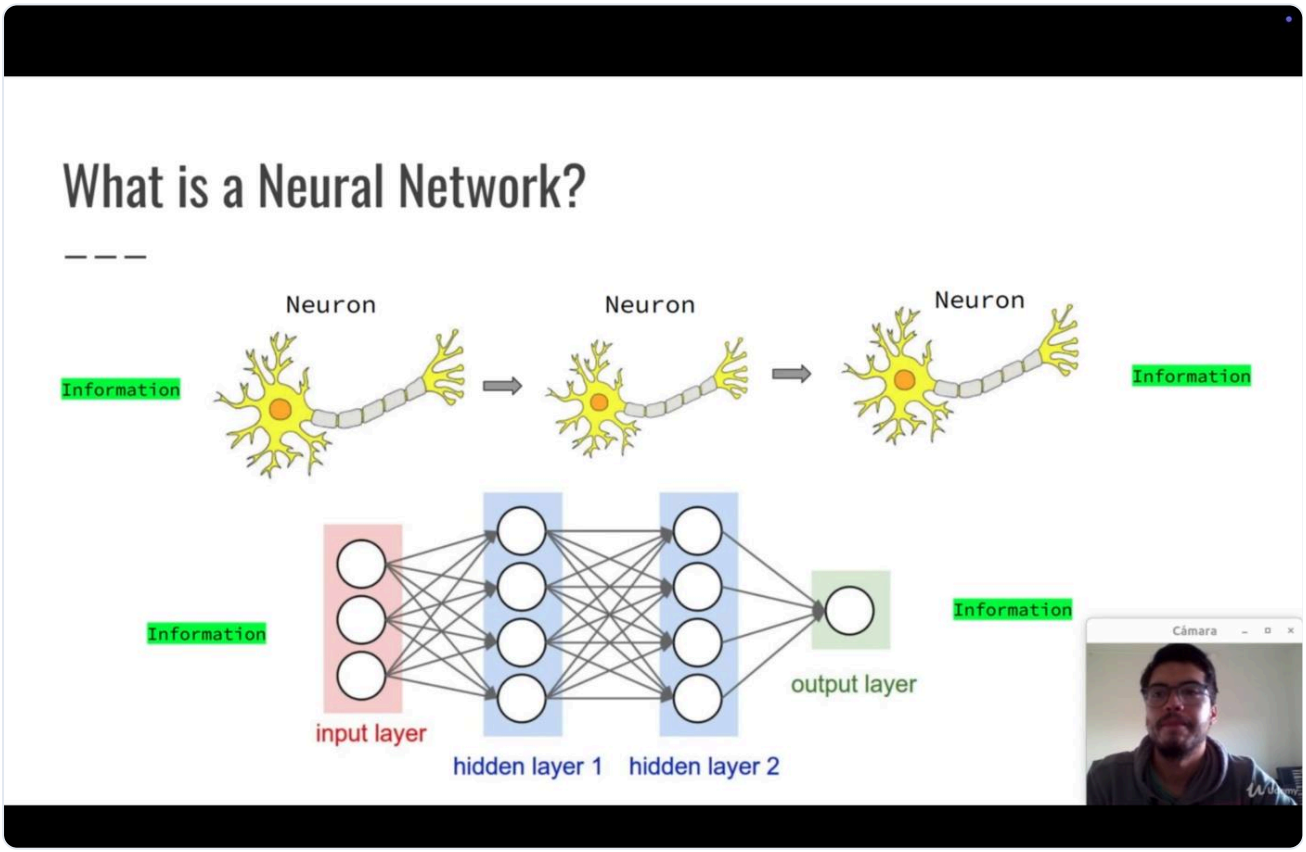



FIGURE 2.2 | What Is a Neural Network?

OK, the information from hidden layer one is going to flow to the next layer, hidden layer two, and the information from hidden layer two is going to flow to the output layer, and that's going to be the output of this system.

OK, so this kind of flow of information that we're going to have in this system is the same that we have in the biological system. OK, so each of these units that we have in each of these layers is a neuron, right. And basically a neuron is a unit, and these units are going to receive information from previous layers. For example, in this neuron here, we're going to receive these three different arrows. Right. The arrow from...

The first, from the first element of the input layer; then we're going to receive this line here, the information from the second neuron of the input layer; and then we're going to receive this one here, and this one here is the information from the third element of the input layer. OK, so each of these neurons is going to receive information from the previous layer, is going to do something here, and then we're going to see what's going on here.

But this information, this information that arrives at this neuron, is going to be processed, and then in this same neuron, we're going to have an output. So the output, in this example of this model, is going to flow to each of the units of the next layer. So, for example, here we have the connection with the first unit of hidden layer two, then we have the connection with the second element of hidden layer two, then we have the connection with the third element, and then we have the connection with the fourth element.

OK, so basically each of these neurons in each of these layers has a connection with the previous layer, processes the information, and then outputs the information to the next layer. OK.

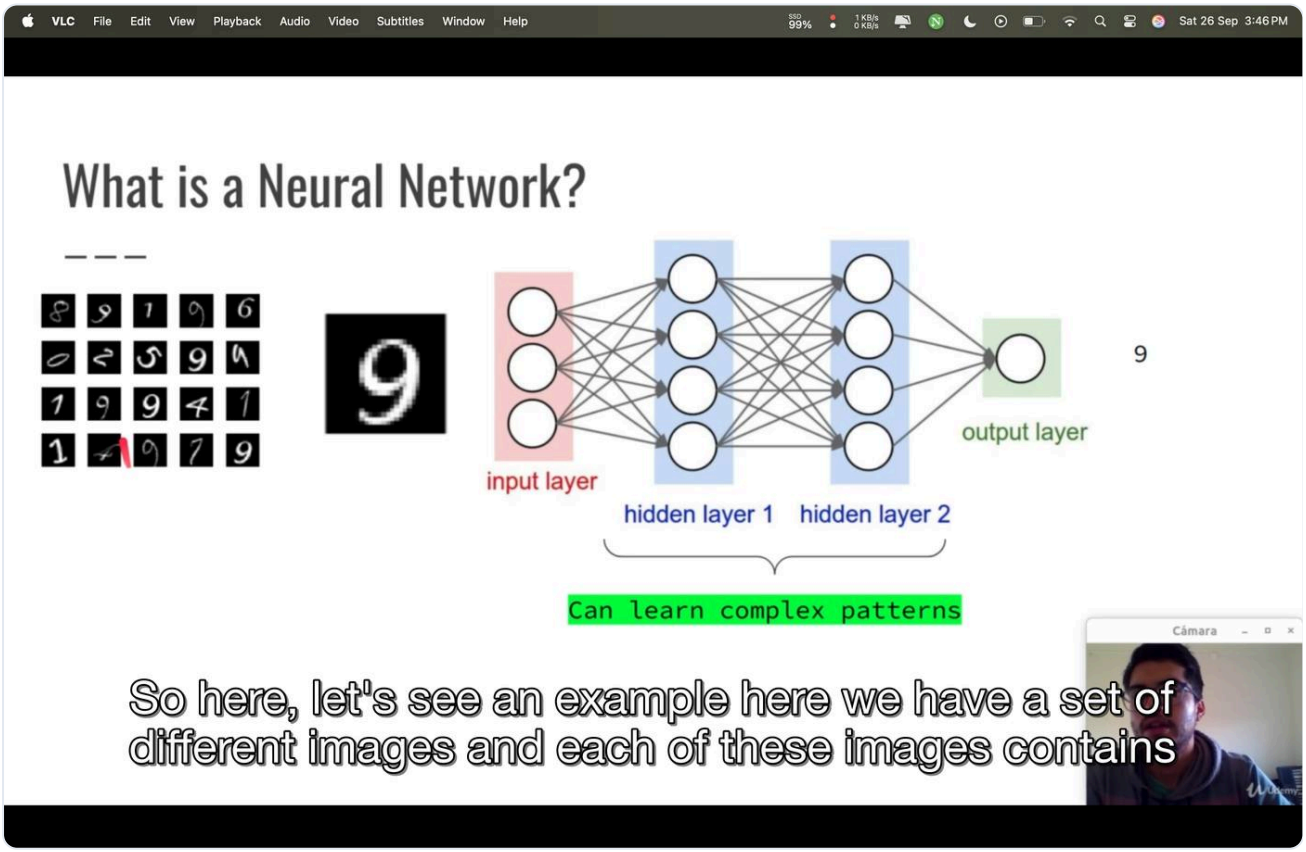



FIGURE 2.3

So here, let's see an example. Here we have a set of different images, and each of these images contains a handwritten number, an image of a handwritten number. So, for example, here we have the eight. Then we have the nine, and we have the one, nine, six, zero, two, five and so on.

OK, so each of these images is a different digit from zero to nine. OK, so here we have an example, and this is an image of a nine. Right. So how does this neural network work? So here we have the same system that we had here. Right. So here we have the input layer, two hidden layers and the output layer. OK, so in the input layer, we can, for example, have each of the pixels of this image. So for example, this first element can be the pixel at that point.

Then the second element can be the pixel here. Then the third element can be the pixel here, and then we can have dozens of different pixels. Excuse me, we are going to have a neuron for each of the pixels of these images. OK, so instead of feeding this image as a square matrix, we can feed this image completely flattened into a layer. Right. So let's say that each of these neurons is a pixel of this image.

OK, so here we're going to have the input layer, and it is going to be the pixels. Right. So what we can do with this model: for example, we can be interested in classifying what digit is in this image. So for example, if we feed this nine, we want our model to predict that this image is a nine. So in the input layer, we're going to feed the image, and in the output layer we should have this nine, the nine prediction. OK, then, for example, if we feed this image of a one to this model, in the output we want to have the one.

OK, so how this works: basically, as we said previously, we're going to have the flattened image here in the input layer, and each of these pixels, each of these neurons with the pixel as the input information, is going to flow through the different layers. OK, and finally, in the output layer, we're going to have the prediction. OK, so in this layer we're going to have all the pixels. Then all these pixels are going to be processed and sent to the next layer.

In this example, hidden layer one, and then from hidden layer one the information is going to flow to hidden layer two, and then after hidden layer two we're going to have the output layer. OK, so how is our model capable of doing this? Well, it turns out that all the different layers that we're going to have between the input and the output can learn complex patterns. OK, so each of these neurons is a mathematical model. And this mathematical model, with all these connections and with all the different layers, can learn very, very complex patterns.

So the idea is that, for example, this layer here can learn a specific pattern. For example, let's say that this layer can learn if in this area we have a kind of curve, for example, and then in this other layer, we can learn if in this area we have this kind of curved line, but in this direction, and then, for example, we can have another layer, and this layer can learn if we have a circle in this area of the image. OK, so that kind of very complex pattern can be learned by this model.

So that is the way in which a neural network works. You have to input any kind of data, and these hidden layers are going to learn what kind of patterns they can extract from the data in order to do, for example, some classification.

LECTURE 3

# Use cases of Neural Networks

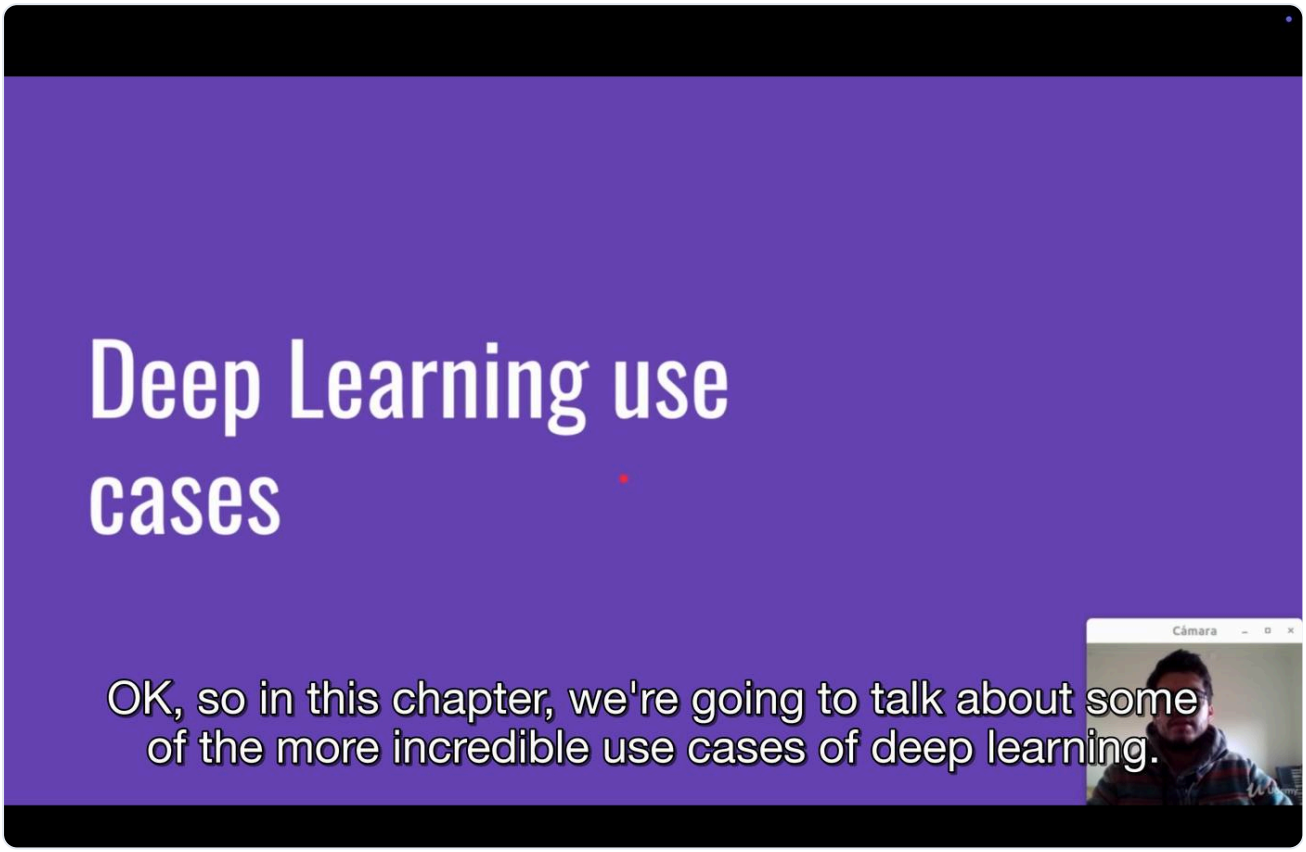



FIGURE 3.1

OK, so in this chapter, we're going to talk about some of the most incredible use cases of deep learning. OK, so the first one is that deep learning has had a very huge impact on self-driving cars. How does this work? Well, basically, a self-driving car is full of different cameras that are always recording the outside of the car. Right.

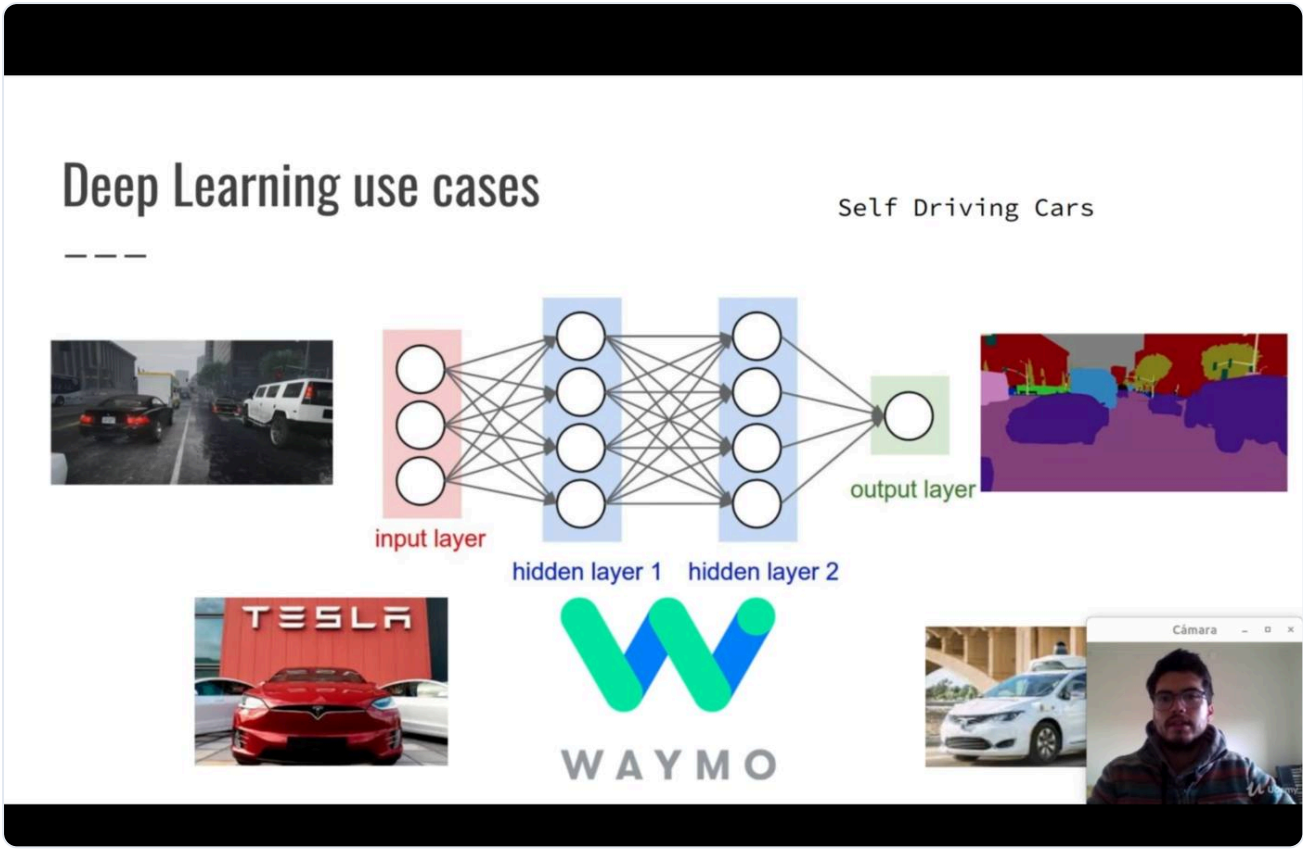



FIGURE 3.2 | Deep Learning Use Cases

And one of the cameras can be this one. OK, so you have a car, and this car is in the street, and in the street there are different cars, street signals and so on.

OK, so a neural network has been used to do something called segmentation, in which this image can be represented as this image here, in which, as you can see here, each of the elements is represented by a different color. OK, so for example, the cars are represented by this color; this is one car, this is another car, for example. Then we can have the street, and the street can be represented as another color.

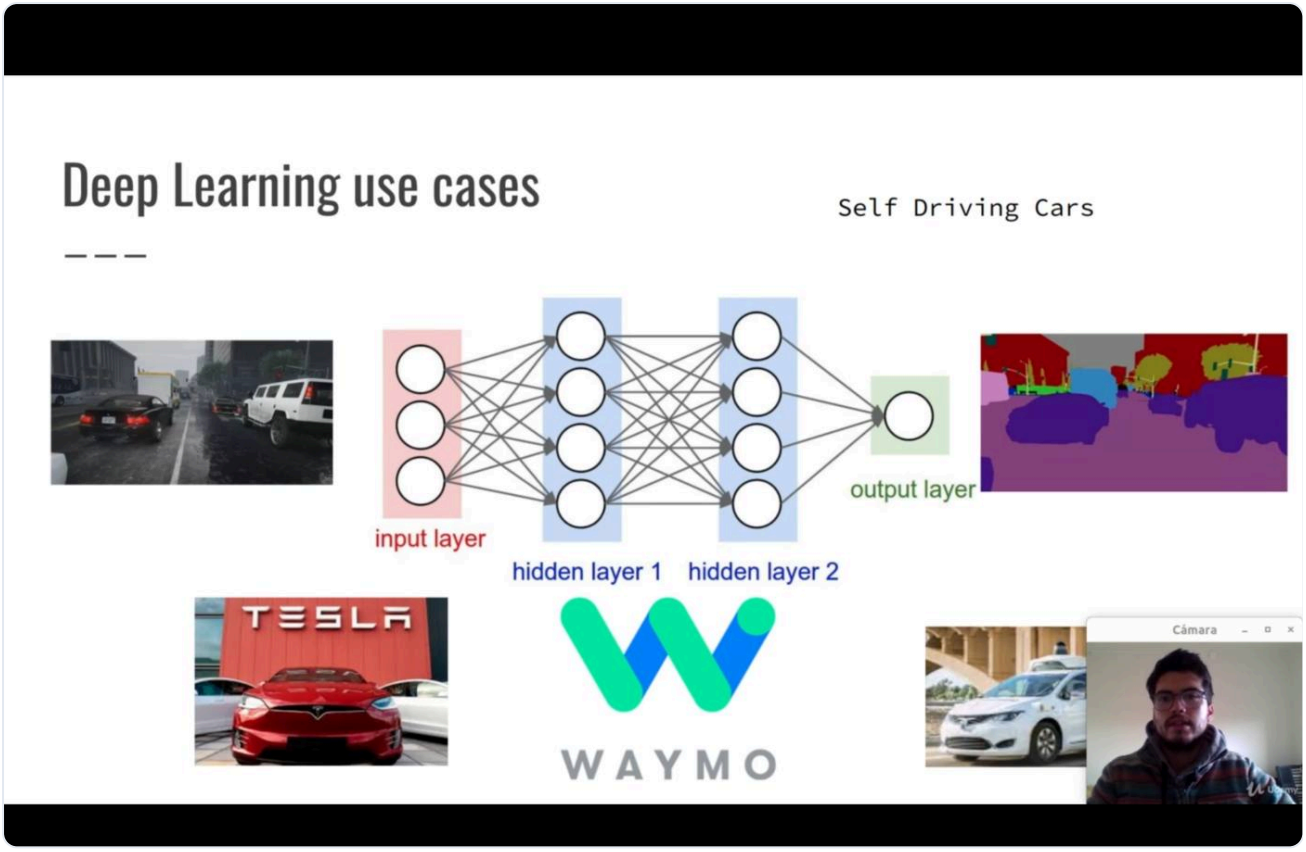



FIGURE 3.3 | Deep Learning Use Cases

Different cars can be represented by different colors.

And then we have the buildings and so on. OK, so to do this they are using neural networks. OK, so basically they feed it into this neural network here, this kind of neural network. Obviously the neural networks that they are using are much, much more complicated.

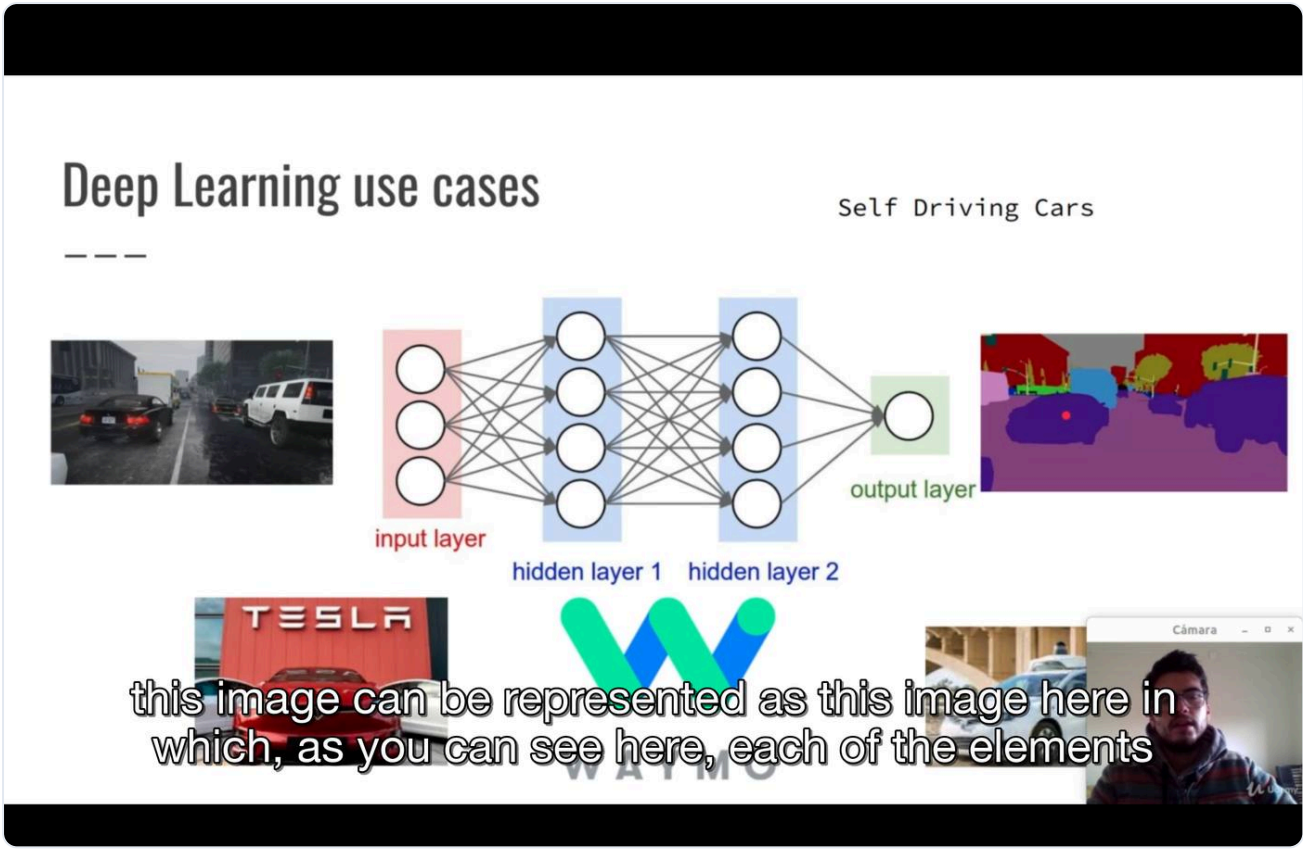



FIGURE 3.4 | Self Driving Cars

But at the end of the day, they are using these neural networks. OK, so the way in which this works is that they feed images into this neural network, and the output of this... excuse me, they are feeding the images into this model, into the input layer.

And this information is flowing through the entire system, the neural network, and the output of this layer can be another image. Instead of just one number, this can be an entire image, and this entire image can be this. OK, one of the companies that is building this kind of technology is Tesla.

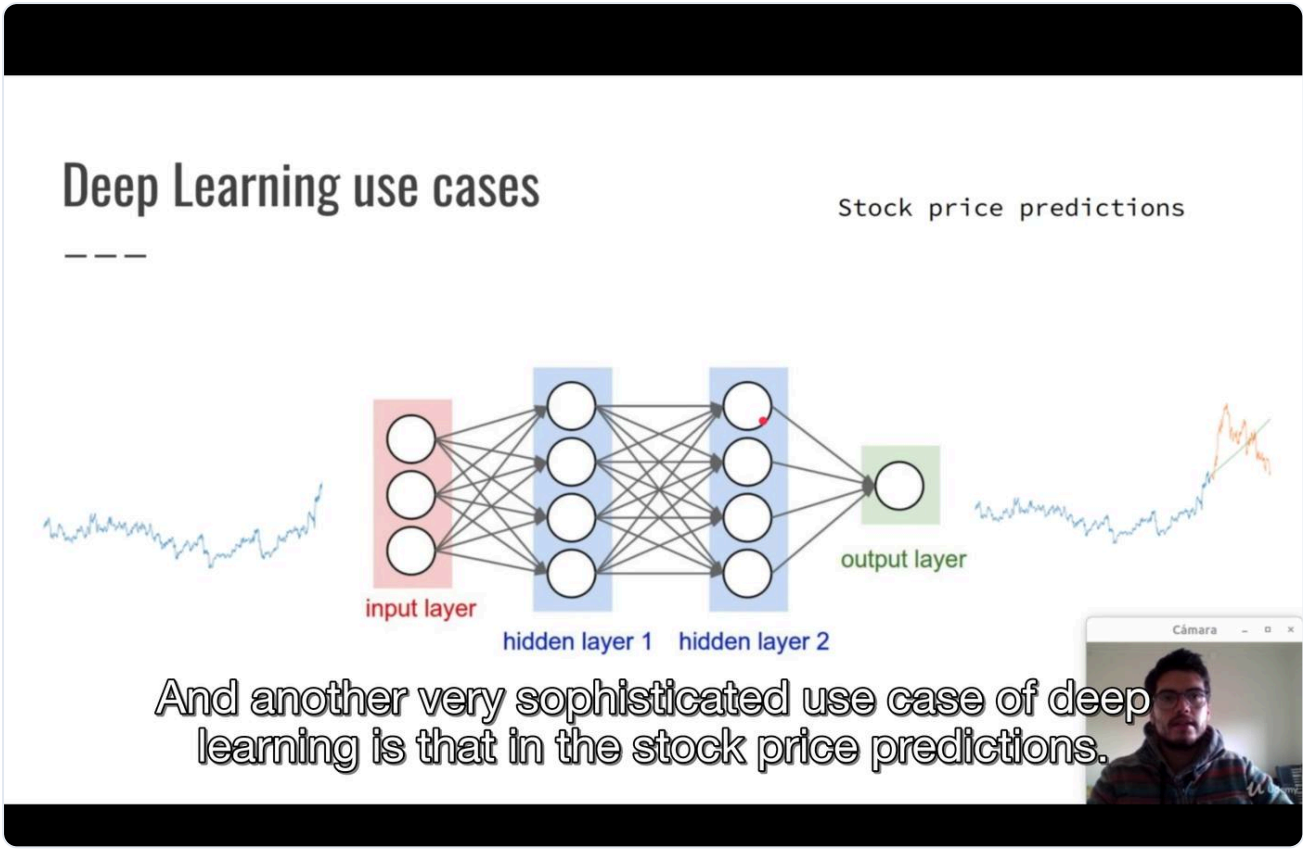



FIGURE 3.5 | Deep Learning Use Cases

And another very sophisticated use case of deep learning is stock price prediction. OK, so they can have this kind of neural network.

And the input of these neural networks is, in this example, the time series of a specific stock price. OK, so as you can see here, this is a very complicated time series because it has a lot of variability. But they're using this. So this is the input, and this is the time series. And the output is the same time series, but with predictions. OK.

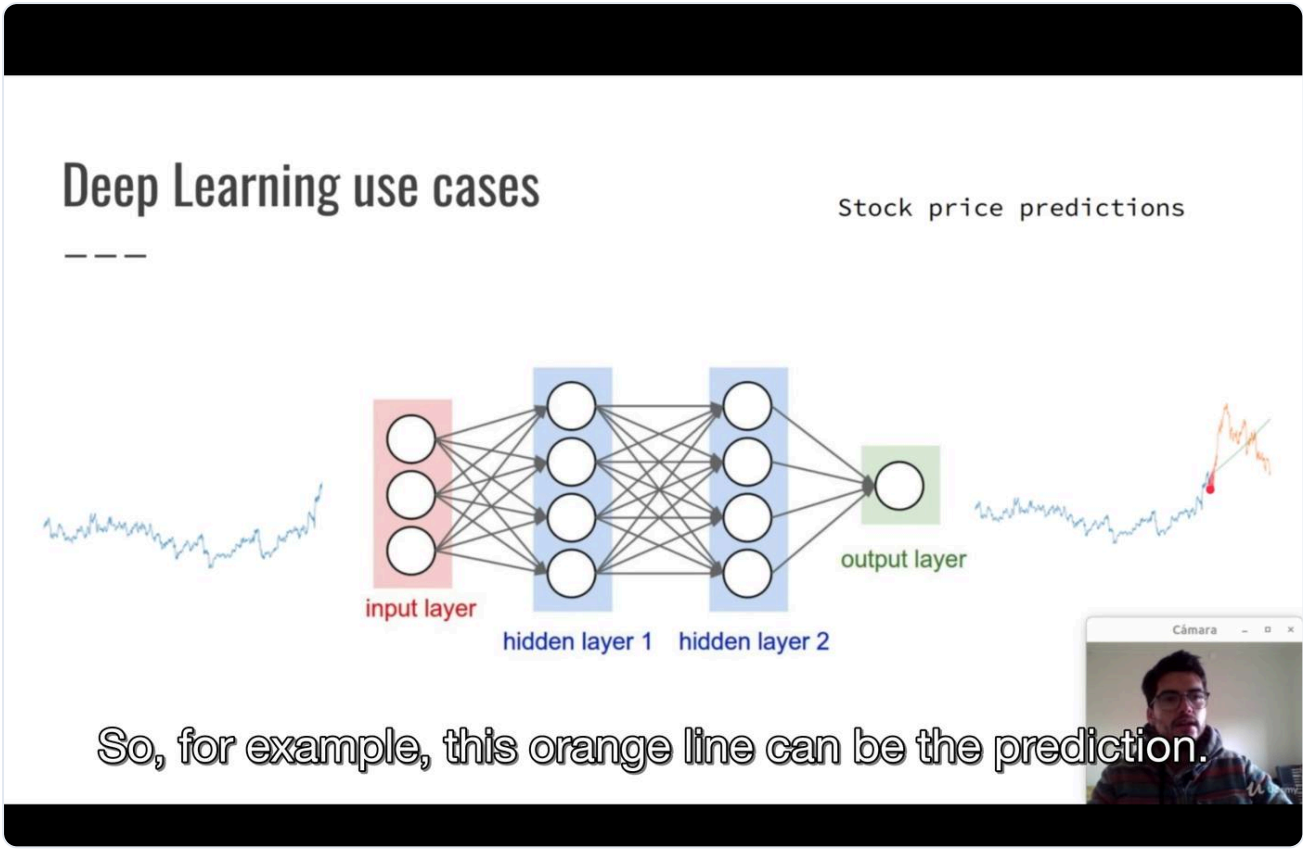



FIGURE 3.6 | Deep Learning Use Cases

So, for example, this orange line can be the prediction. OK, so in order to do this, they are using this neural network model.

OK, so the way in which this works, in a very general way obviously, is that they input this time series. This is done in the input layer, and then this time series is flowing through the entire system here, excuse me, layer by layer, and then finally arriving at the final layer, in which, in the final layer, we can have this time series here. OK.

LECTURE 4

# Mathematical view of General Neural Networks

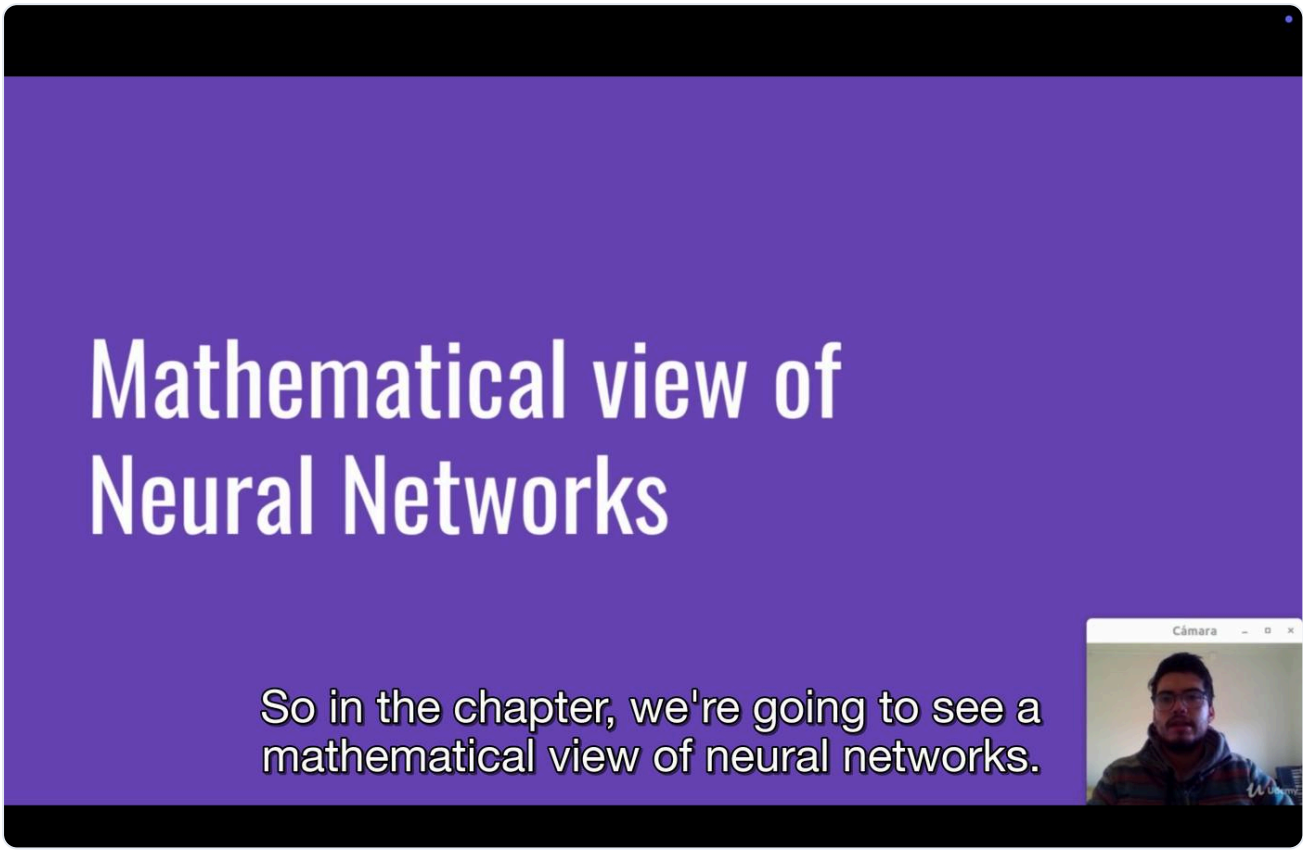



FIGURE 4.1

So in this chapter, we're going to see a mathematical view of neural networks.

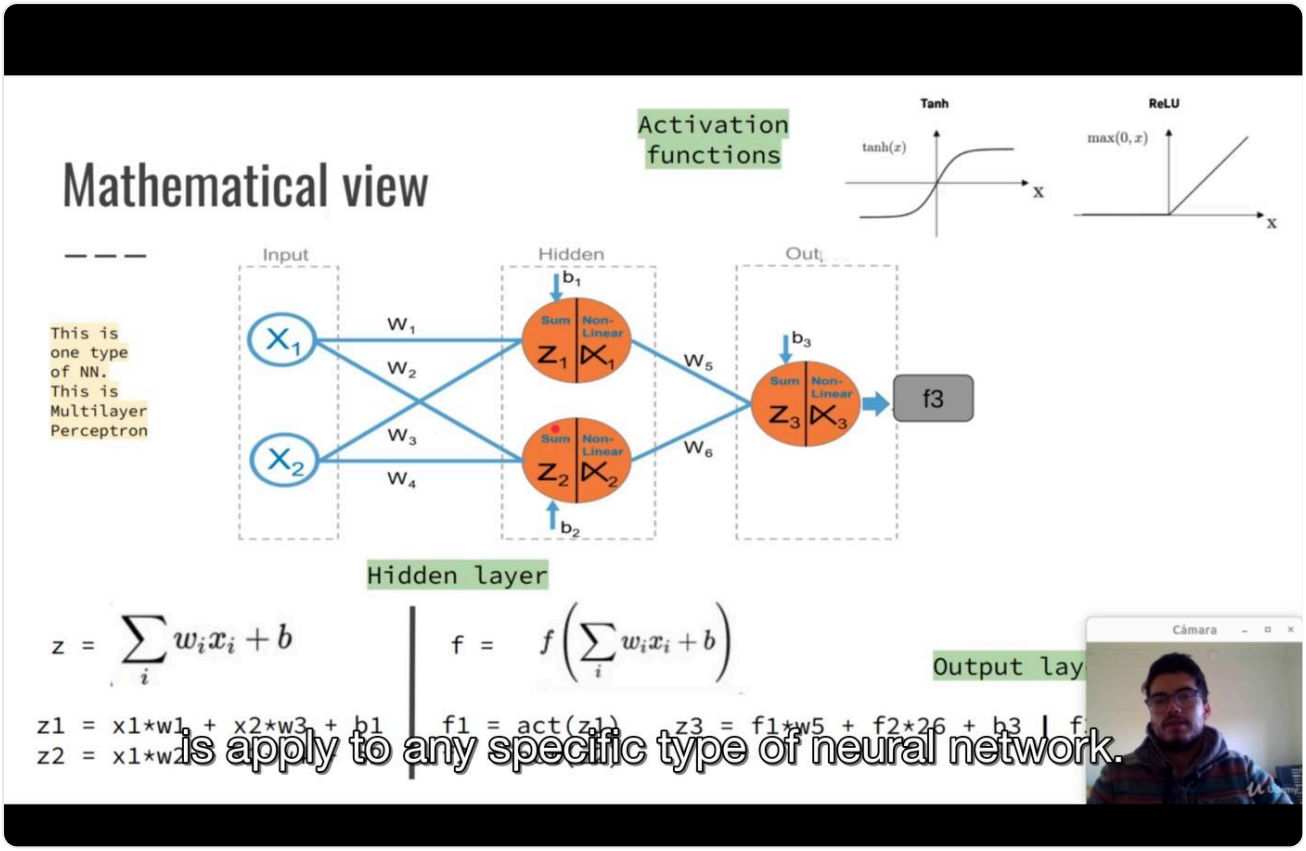



FIGURE 4.2

OK, but first I want to give a disclaimer, OK: the math we're going to see here applies to a specific type of neural network, and this is the multilayer perceptron.

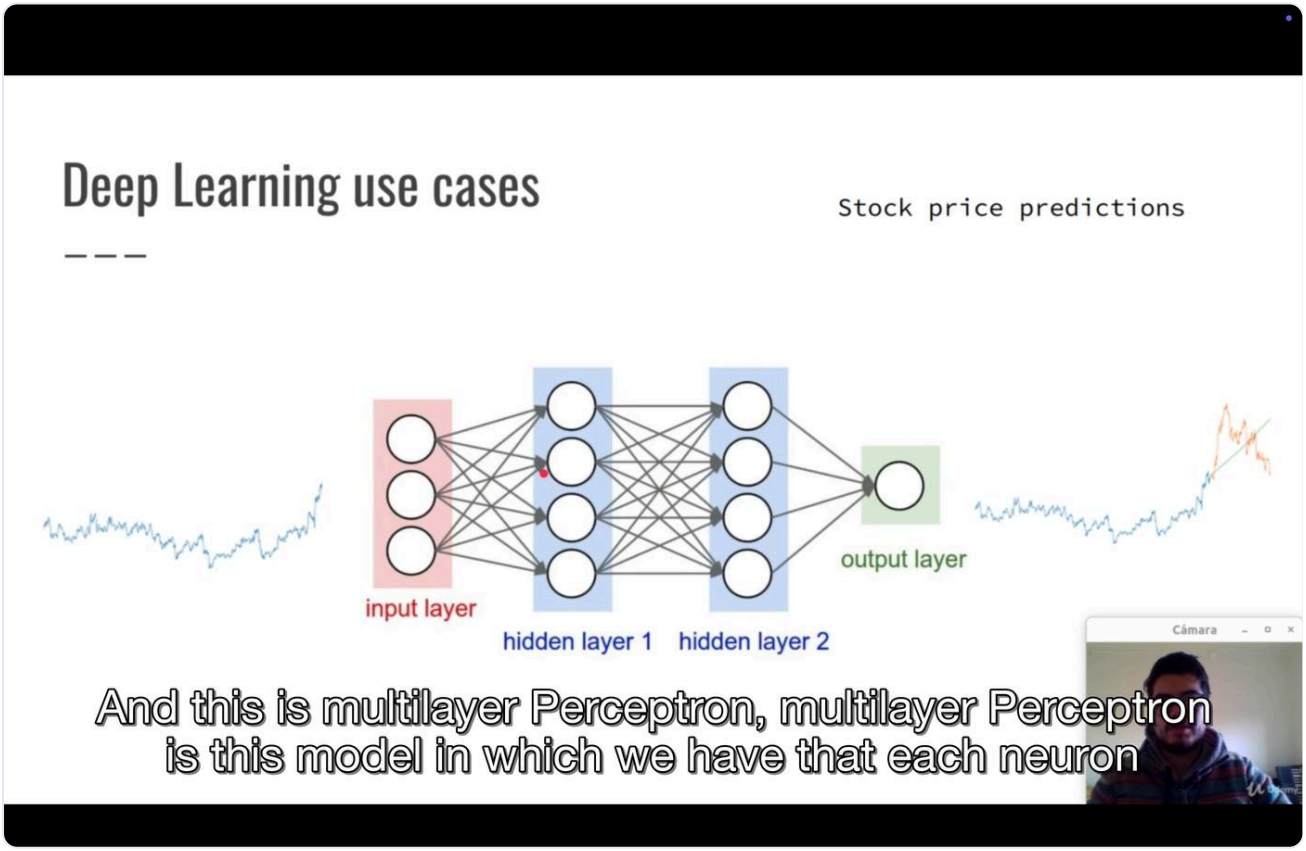



FIGURE 4.3

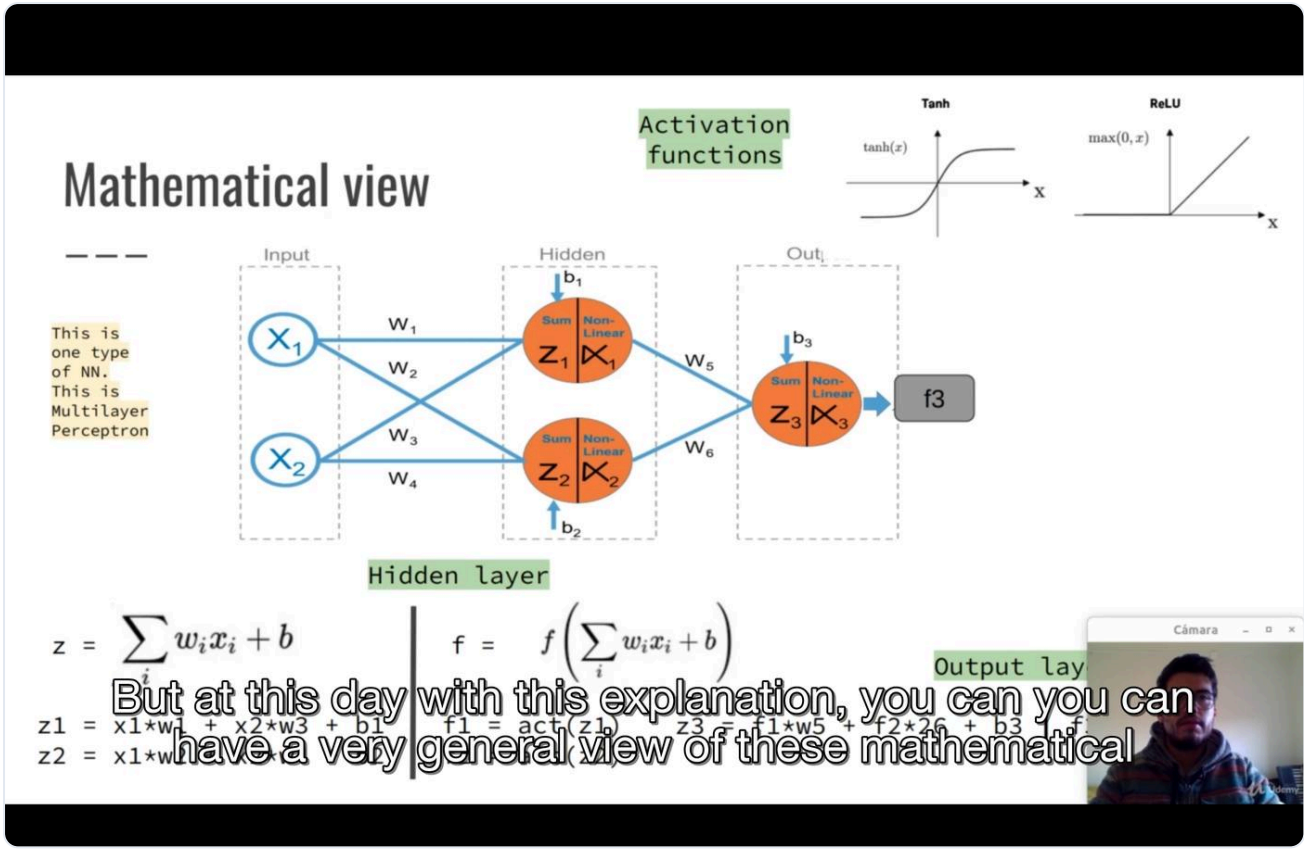



FIGURE 4.4

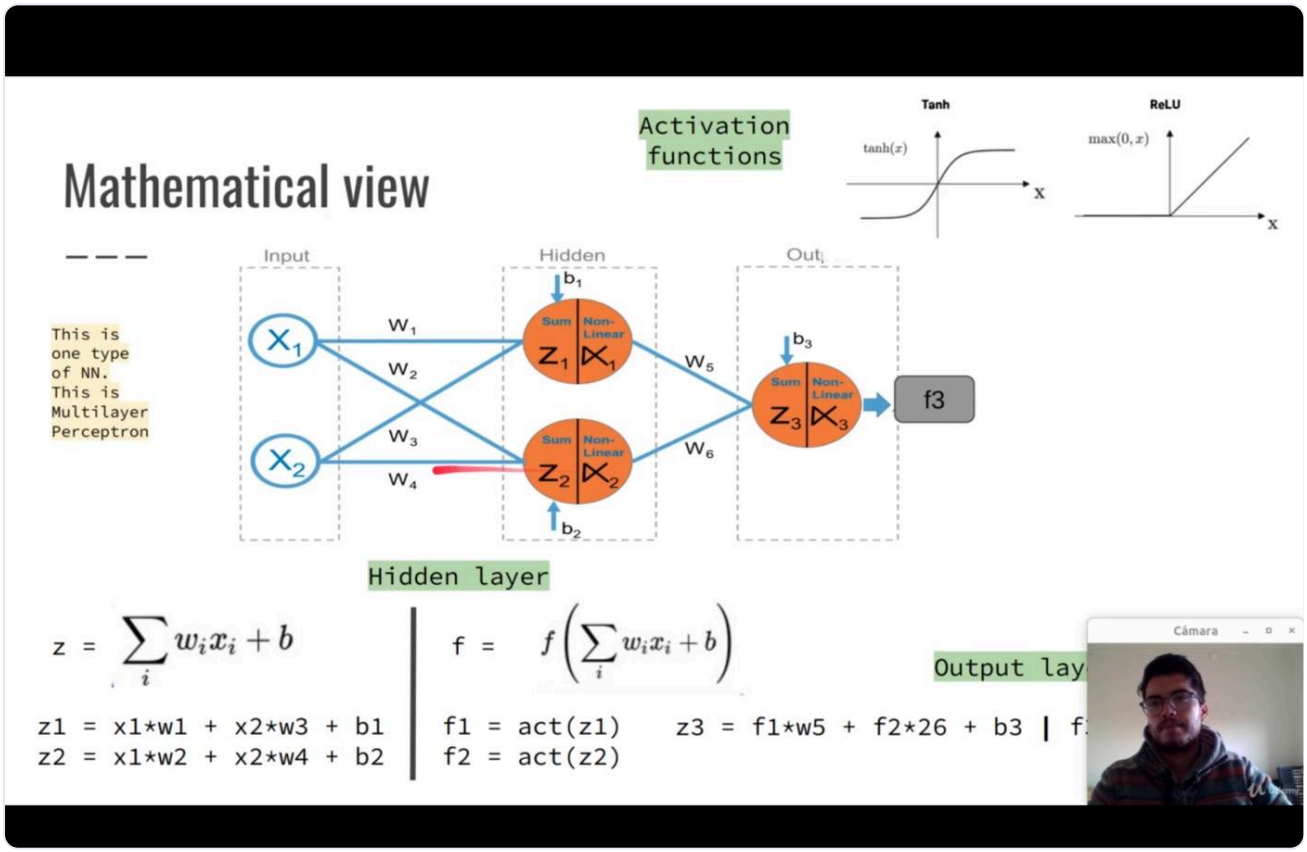



FIGURE 4.5 | Mathematical View

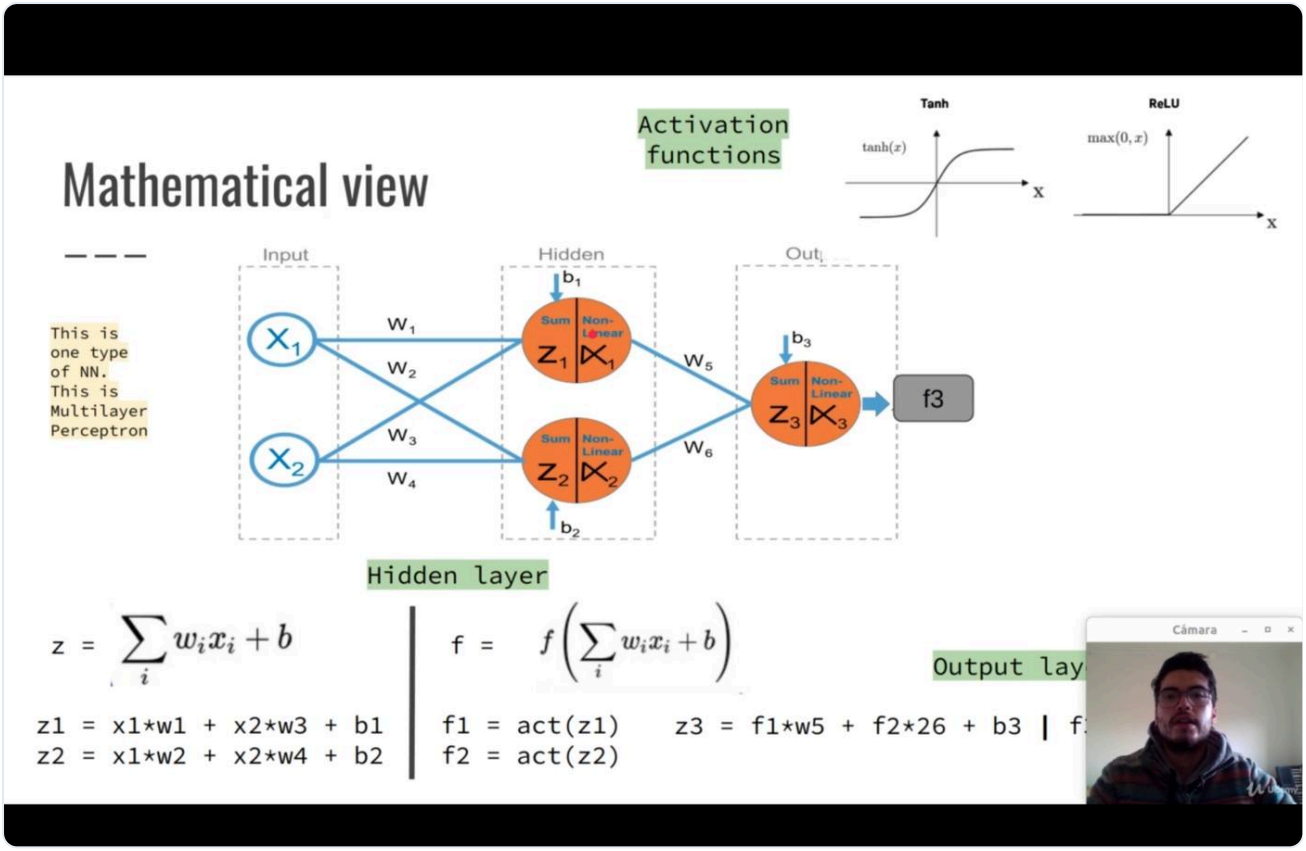



FIGURE 4.6 | Mathematical View

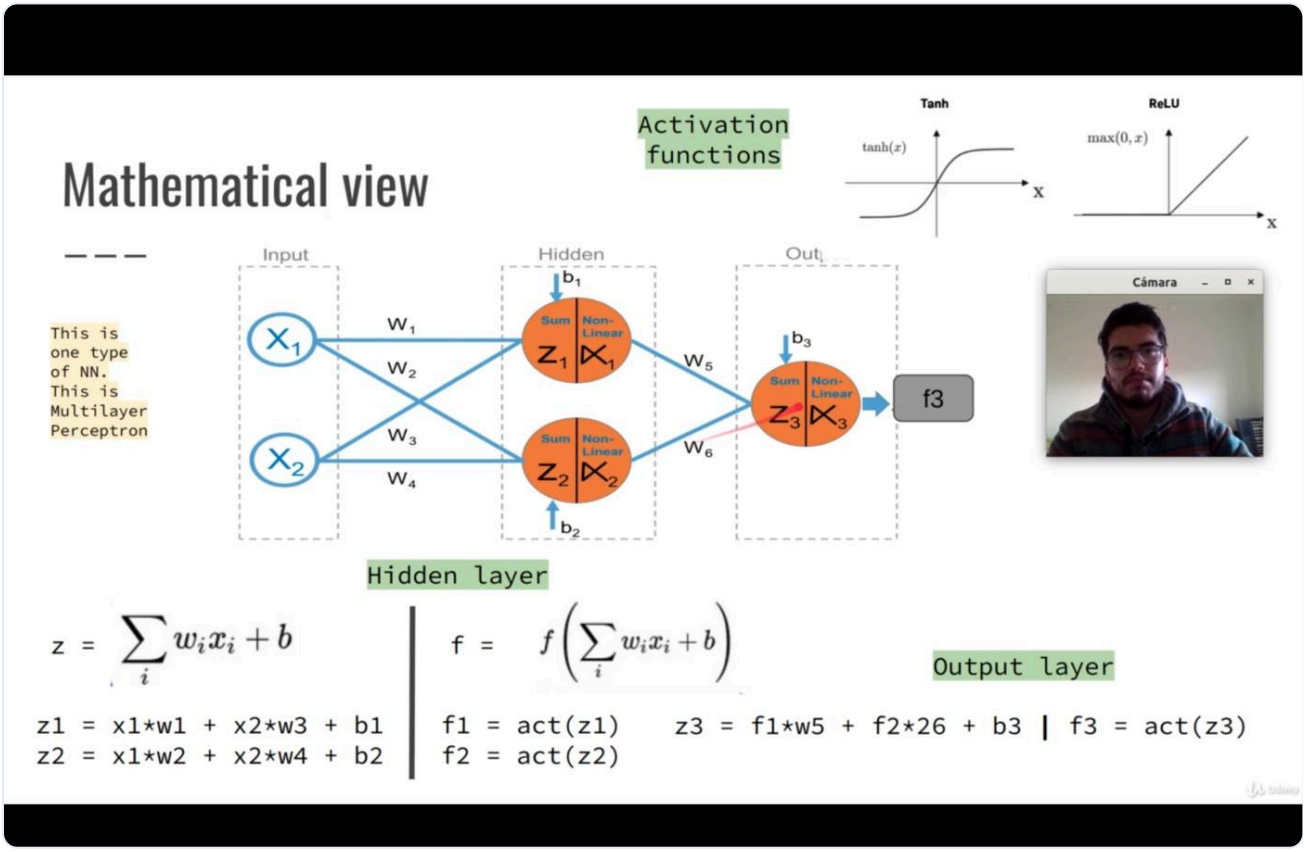



FIGURE 4.7

The multilayer perceptron is this model in which each neuron of each layer is connected to each of the neurons of the next layer. So this kind of model, or architecture, is the multilayer perceptron.

OK, but this math is a kind of general overview. OK, if you have another type of neural network model, the mathematics that you are going to have in that model is going to be different. Sometimes it is going to be slightly different, and sometimes it can be completely different. But at the end of the day, with this explanation, you can have a very general view of the mathematics. OK, so let's begin. Here we have a multilayer perceptron model in which we have the input layer with two neurons.

Then we have a hidden layer with two neurons, and then we have the output layer with one neuron. OK, so as you can see here, the input layer is connected to each of the neurons in the hidden layer. OK, this connection here and this connection here. And the second element is also connected to each of these neurons, and also each neuron in the hidden layer is connected to the final layer, to the one neuron of the final layer. OK, so the first thing that we have to note here is that we have a connection between this input here and the neuron here.

OK, so the connection is this one, OK? Each of the connections that we're going to have between units, in this example the input and the first neuron, is going to have a weight parameter. In this example, we say weight, OK, so this is going to be a number. For example, it can be 0.1, 0.3, one, ten and so on. Basically it's a parameter, OK. And each of the connections that we're going to have in the neural network is going to have a parameter.

OK, so for example, the connection between input one and neuron one is W1. This is the parameter. The connection between input one and neuron two is this connection here, and the connection here has another parameter. OK, it is the same for each of these connections. So we're going to have three, four, five, six. Right. And these are the different connections. OK, so, um, the way in which a neuron works is represented by a mathematical formula.

OK, so the mathematical formula for each of these neurons is this one. Actually we are going to have two steps. The first step is going to be a summation, a sum, and then the second step is going to be the application of a non-linear function. So let's see the first part. OK, so the first part of each neuron is going to be a sum, in which we are going to have a sum of the multiplication of each of the elements that arrive at the neuron, multiplied by the weight of that connection.

So let's see an example for neuron one. Excuse me. Before that, we're going to have this general formula, in which we have the summation over i, and i is each of the connections to the specific neuron. In this example, for neuron one, we're going to have the connection with one and the connection with two. Right. And this is going to be a summation of the multiplication between the weights, for example W1 and W3, multiplied by the X, and this can be X1 and X2.

OK, so this is going to be a summation, but also we're going to have a B term, and this is the bias, and each of the neurons is going to have a bias term. In this case, this is B1, and neuron two is going to have the bias B2. OK, so let's see how the connections come into the formula for neuron one, the sum part. OK, so neuron one is C1, and here we have C1. OK, so C1 has two connections, right: the connection with X1 and the connection with X2.

So the sum is going to be X1 multiplied by W1, and this is this one, X1 multiplied by W1, and this is the first connection. And then we have to add the other connection, and the other connection is this one, right. So this one is going to be X2 multiplied by W3. And here we have this term, X2 multiplied by W3. OK, so here, with this one, we're going to have the connection with X1 multiplied by W1. This is the first connection.

And then we're going to have the second connection, and the second connection is going to be X2 multiplied by W3. W3 is the parameter for this connection here. OK, and so that is the term for this sum. OK, basically you are summing the different connections that that specific neuron has, and then we're going to sum... then we're going to add the bias; for this specific example, this is B1. OK, so the formula, the first part, the sum for the first neuron, is this one here.

OK, and it is the same for neuron two, the sum part. So the sum part for neuron two is going to be the multiplication for each of the connections. So here we have two connections, and the first connection is going to be X1 multiplied by the parameter W2. Right. So this is the first connection. Then we're going to add the second connection. The second connection is going to be X2 multiplied by W4. These are the two terms of this sum.

Right. And finally we're going to sum the bias term, and this is B2. OK, so basically the sum part of the neuron is going to be a summation of the different connections, plus the bias. OK, so then the next step in the same neuron is going to be the application of a nonlinear function. OK, so basically this is the formula for the nonlinear function. OK, so we are going to apply a function F to C, and C is this expression here.

As you can see here, C is this expression here, and this is the same as this. OK, so basically in this step, the nonlinear function, we're applying a function F. OK. So, for example, for neuron one, we're going to have F1, and F1 is going to be the function F, and this function is usually called the activation function. OK, so that is why we have F here, for activation. OK, so F1 is going to be the output of the first neuron: it is going to be the activation function of C1, and C1 is going to be this formula here.

OK, so basically the output of this neuron is going to be the application of the nonlinear function to this one. OK, so the nonlinear function, the function that we apply to C, is called the activation function. And this activation function is going to be a function that maps the C output to some number. OK, so one of the most famous activation functions is the hyperbolic tangent, and this is the function. Another very famous activation function is ReLU.

And this is the maximum between zero and X, OK. And you can have many different activation functions. But the idea of the activation function is to apply a nonlinear behavior, a nonlinear function, to the neuron, because if you don't apply the nonlinear function, the neuron would be just a summation of different terms, and this is going to be linear. But with linear terms you cannot learn complex patterns, and so you use nonlinear functions in order to learn complex patterns.

OK, so that is the reason why we add a nonlinear function: because we want to learn very complex patterns, and the way in which we can do this is by applying different nonlinear functions. OK, so for neuron one we're going to have the activation function of C1; for neuron two, we're go-

ing to have the activation function of C2. OK, so the outputs of these two neurons are going to be F1 and F2, OK, and finally in the output layer we are going to have the same: in the neuron, we're going to have first the sum. It's going to be C3, and C3 is going to be the sum of the different connections plus the bias.

So C3 is going to use this connection here and this connection here, which is going to be the output of this neuron. In this example, we say that this is F1, so it is going to be the multiplication of F1 by the W5 term. Right. So this is the first connection. And then we're going to add the second term. The second connection is going to be F2 here, which we have to multiply by W6. OK, excuse me for that, but this is W6. OK, and then we're going to add the bias, and this is B3.

OK, so this is the sum part. And then we are going to apply the nonlinear function, and this is going to be the activation function of C3. So finally, the output of this neuron is going to be F3. Excuse me. So finally, this is the way in which each neuron works. The idea is that in each neuron we're going to have this summation part, and then we're going to have the application of some nonlinear function. OK, but basically, then, a neural network is finally a function.

And if you see the formula of F3, the final output F3 is this function here, in which we have the parameters W and B, and then we have the F values, and the F values are going to be an activation function of C1, and C1 is another sum plus bias. Right. So basically the output, or the model, of a neural network is going to be a really large function of nonlinear functions plus linear functions. OK.

LECTURE 5

# Type of Neural Networks models

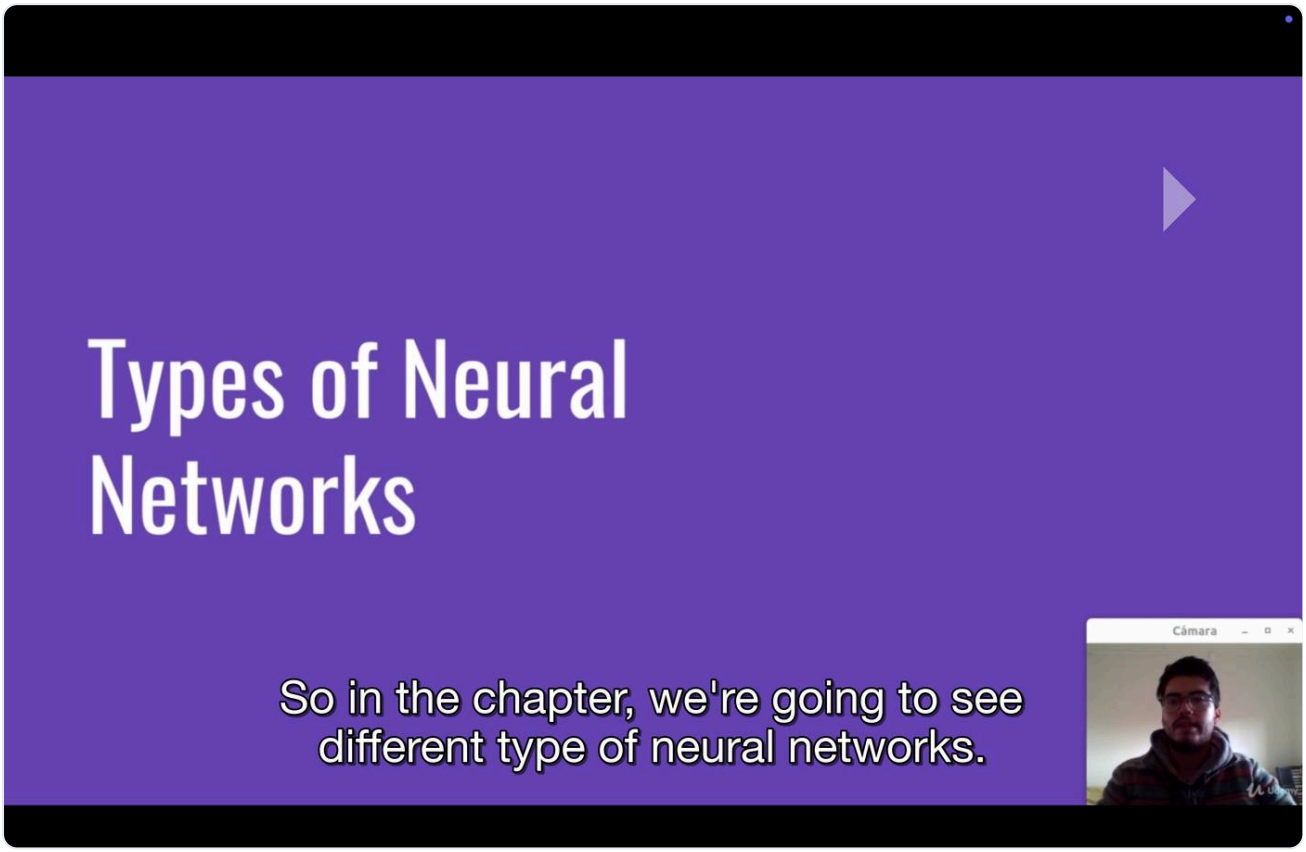



FIGURE 5.1

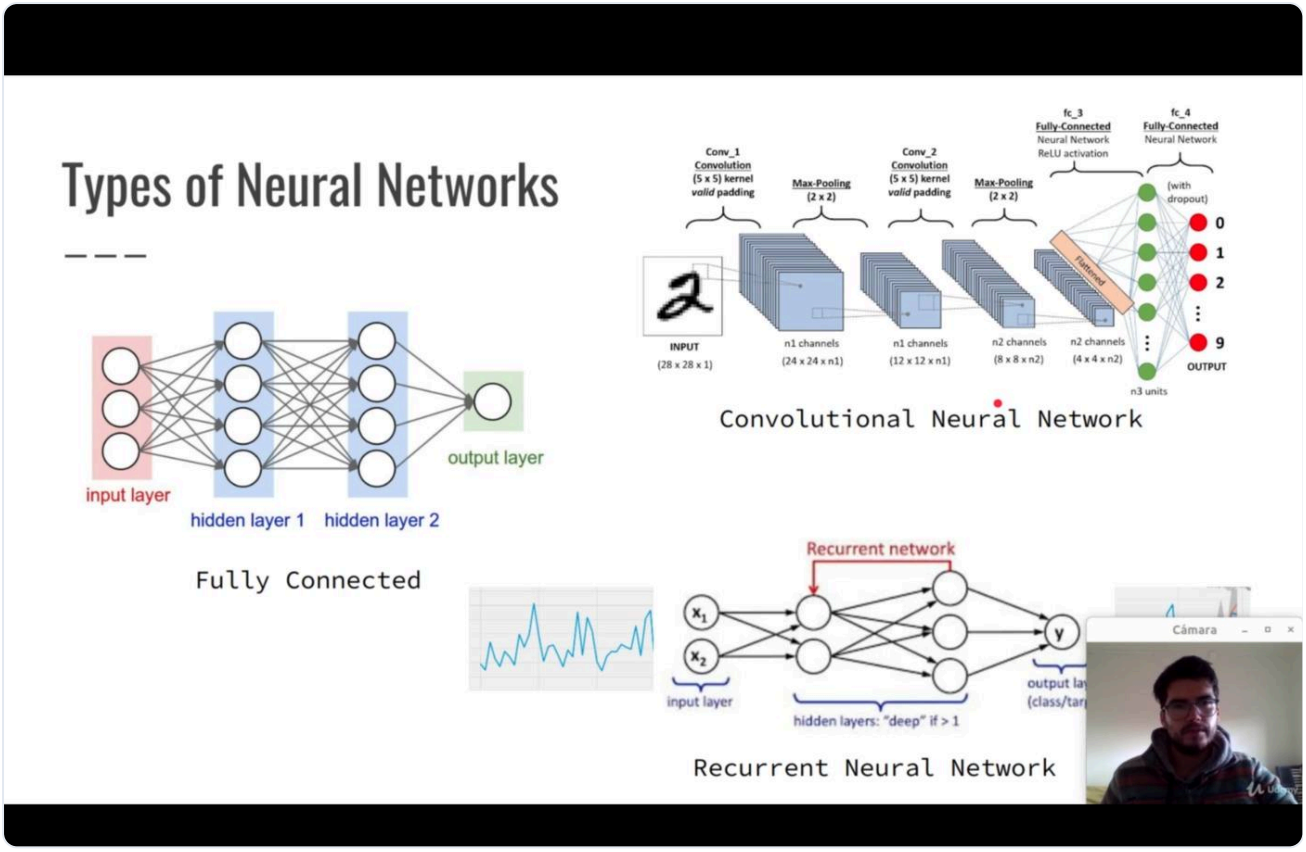



FIGURE 5.2 | Fully Connected

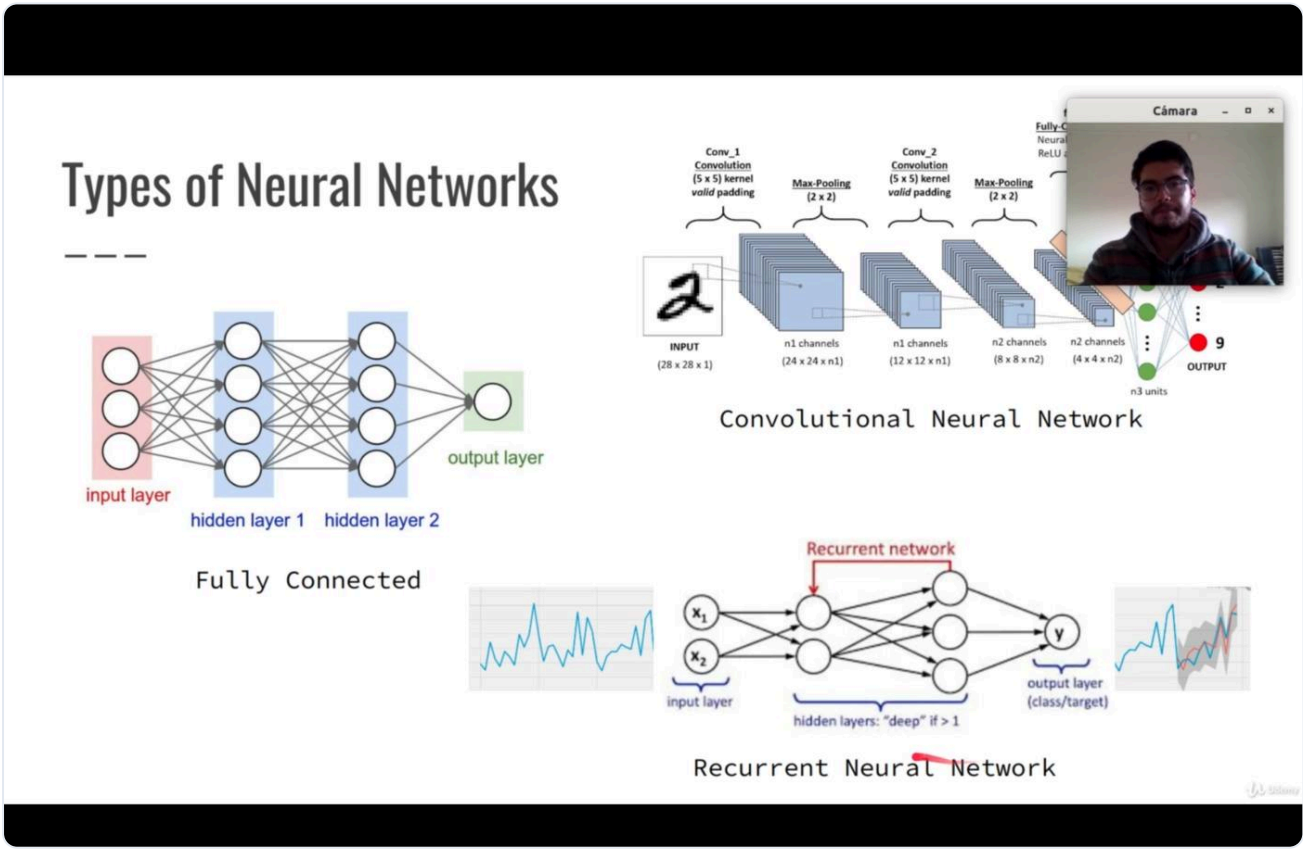



FIGURE 5.3

So in this chapter, we're going to see different types of neural networks. OK, so the first model, like we were talking about previously, was this one. And this is the multilayer perceptron. And this is also known as the fully connected model, because every neuron is connected with the previous and the next layer. OK, and this is the basic model we can use in neural networks. OK, but we have more models, or different architectures. The architecture is the way in which the layers and the neurons are organized.

OK, so another very typical neural network is the convolutional neural network. OK, the convolutional neural network is the neural network in which we are going to have convolutions inside the system, or inside the network, and the convolution is a special mathematical operation which we usually apply to a segment of the data. So for example, if we have this image here, we're going to apply a convolution to this area here, and then we can apply the convolution to an area here, another area, and so on.

OK, so instead of applying a mathematical operation to each pixel, we apply a mathematical operation to an area of the input data. OK, we usually use these convolutions with different filters. OK, so for example, we can have different filters. Each of these kinds of squares is a filter of different convolutions. Inside a convolutional neural network, we have convolutional layers, but also we have different other layers, like pooling or flatten, which we use for different goals inside the convolutional network, for example, to reduce the dimensionality or to flatten the output of the neural network.

OK, but basically the general idea of convolutional neural networks is that we are going to have convolutions inside the architecture. And the convolution is a mathematical operation that we apply to areas in the image, or in the data, in the input data. OK, convolutional networks are very good at finding patterns in images. Actually, they are very popular with images because with convolutions you can find patterns; for example, you can find the horizontal lines of this image, or you can find vertical lines in the images.

So with this mix of vertical and horizontal lines, you can try to find different patterns. Obviously everything automatically, but that is the idea. OK, and another very popular model nowadays is the recurrent neural network. OK, the recurrent neural network is a model that takes time into consideration. OK, so you can work with time data, like, for example, time series. If you use recurrent networks on time series, this kind of model is going to work really well, because the recurrence of the recurrent neural network model allows the model to kind of save, or store, information over time, or extract patterns from time data.

OK, usually when we have the fully connected network, we have connections, right, and all the connections are in one direction. So for example, all the information is flowing in this direction, right, in that direction. When we have recurrent models, we usually have recurrence, or a kind of feedback of information, in which one neuron, instead of sending the information exclusively to the next layer, is sending information to the previous layer. And this is a recurrence.

OK, and so, again, this kind of model is very useful when you are working with time, with data that has time as one of the dimensions, like, for example, time series. So, for example, we can use this time series, and we can use the recurrent neural network to do some predictions on this time

series. So we can use, for example, this... excuse me, we can use this data to get these predictions. OK. A very important point to have in mind here is that, depending on the type of data, the model is going to work well or not.

So different kinds of models work differently on different types of data. OK. Usually when you are working with images, convolutional neural networks work really well, or if you are working with time data, recurrence, or recurrent neural networks, work really well. OK, but another very important challenge is to adapt the model to the type of data. So, for example, if you are going to have an image and you want to find patterns in the image, yes, you can use convolutions, but then you have another challenge.

And this challenge is to adapt the architecture of the network in order to work well with that specific type of image. OK, so for example, instead of using one hundred convolutions, you can use two hundred convolutions; instead of having a convolution and then a pooling, you can have a convolution and pooling, then another convolution and pooling, and so on. OK, so yes, different kinds of models are going to work on different types of data, but also you have to adapt the model to the specific type of data that you are working on.

LECTURE 6

# Overview Convolutional Neural Network

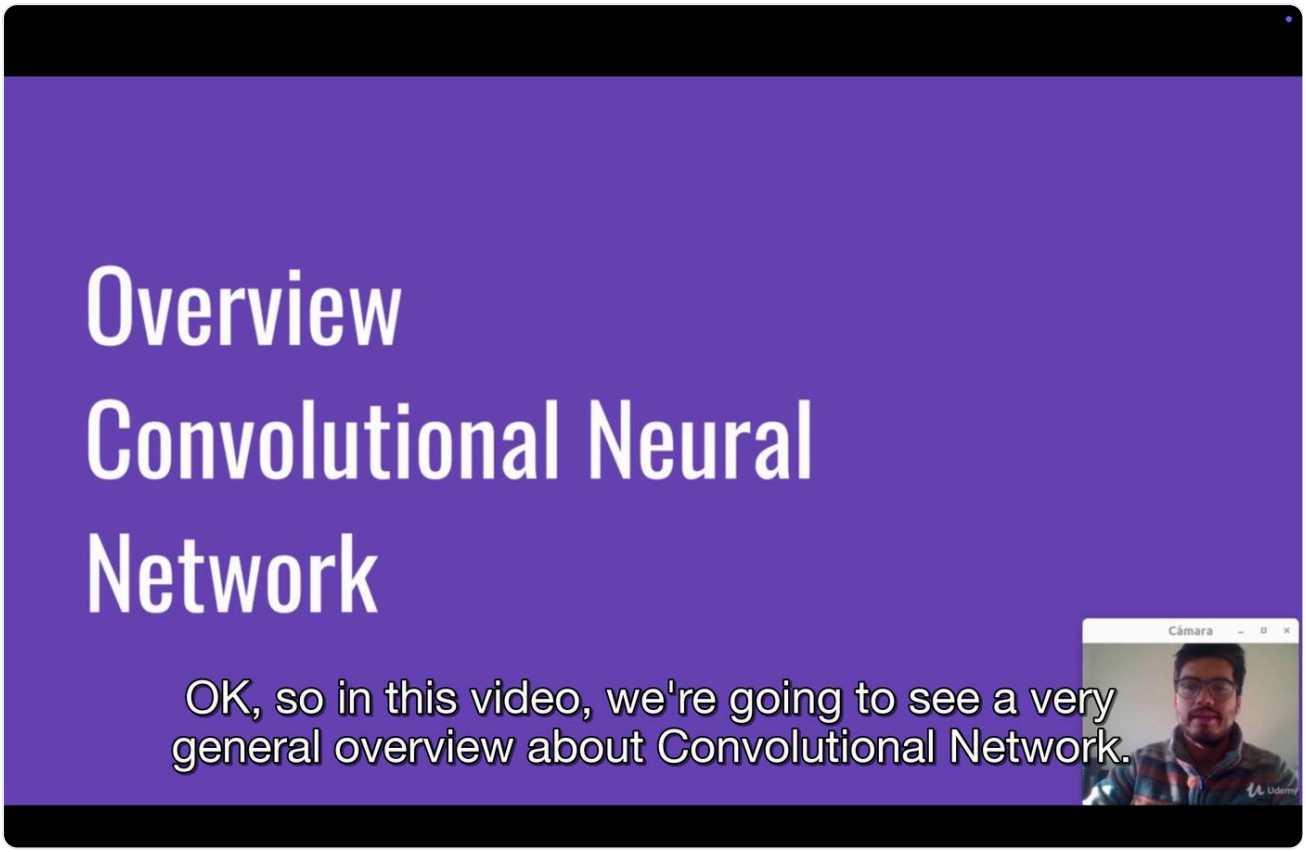



FIGURE 6.1 | Convolutional Neural

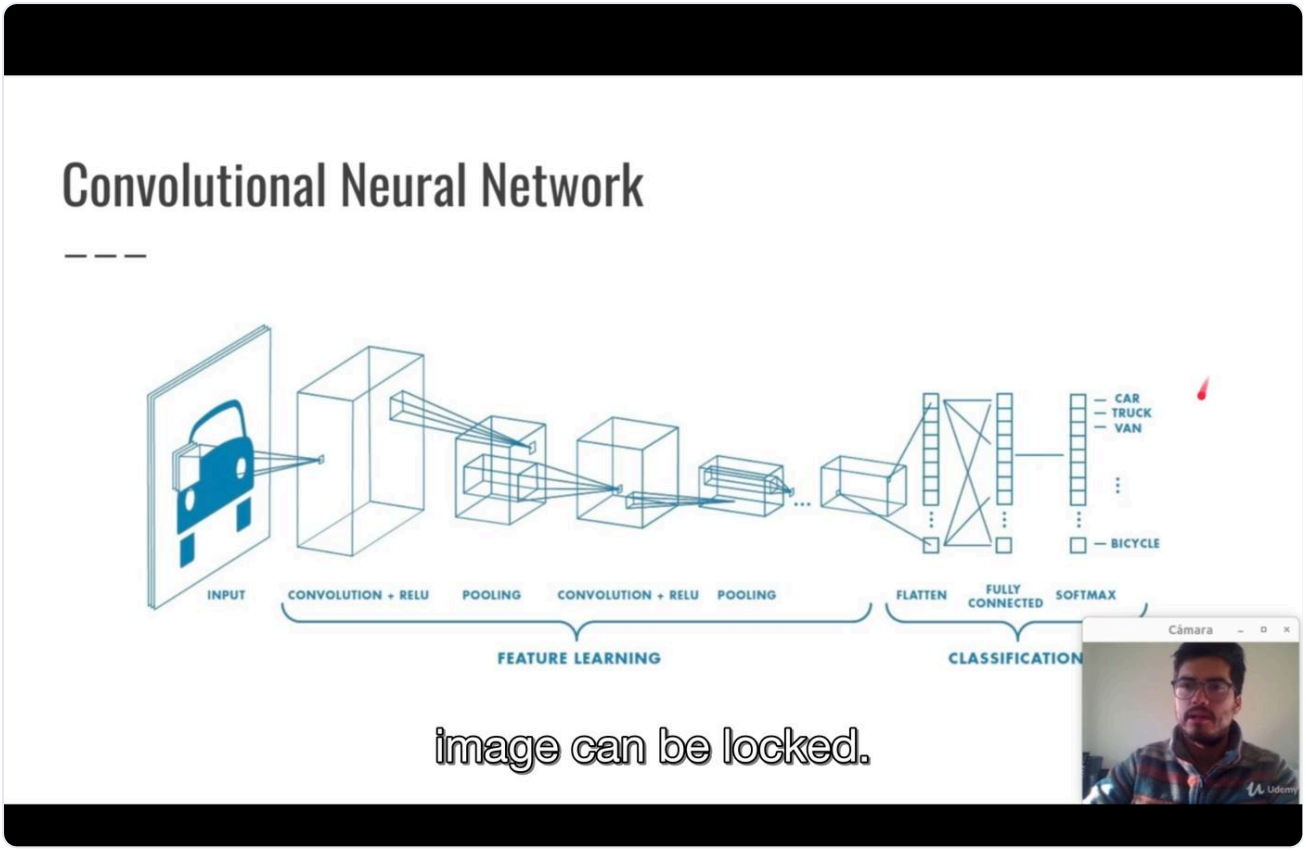



FIGURE 6.2 | Convolutional Neural Network

OK, so in this video, we're going to see a very general overview of convolutional networks. OK, so here is a very classical, very typical neural network model, and on this side we are going to have the input. So, for example, this can be a 2D image, a two-dimensional image here in the input, and then in the output, if we are talking about a classification problem, we're going to have the different classes to which this image can belong. OK, so between these two sides, we are going to have the convolutional layers.

OK, so a very typical architecture of a convolutional network has two parts. OK, so the first part is going to be a part in which we are going to learn the different features, or we're going to find different patterns automatically, in order to identify the different interesting parts that this image has. OK. So for example, in this feature learning, excuse me, in this part, we can have, for example, a feature that can learn if there are wheels in the image, OK?

And so basically in this part, we're going to learn different patterns that we are going to find in the input data. OK, and then we're going to have a classification stage, excuse me, and in this classification stage, we're going to take all the features that we extracted in the previous stage. And we're going to join these different features, and we're going to apply, generally, a fully connected layer in order to join these different features.

And finally, we're going to have the output of the different classes. OK, so basically, in the feature learning stage, we are going to find the different parts that we can find in the image, or in the input data. And then we're going to take these different parts and we're going to use them to classify the different classes that we are going to have in this problem.

OK.

LECTURE 7

# Overview of Convolutional Layer

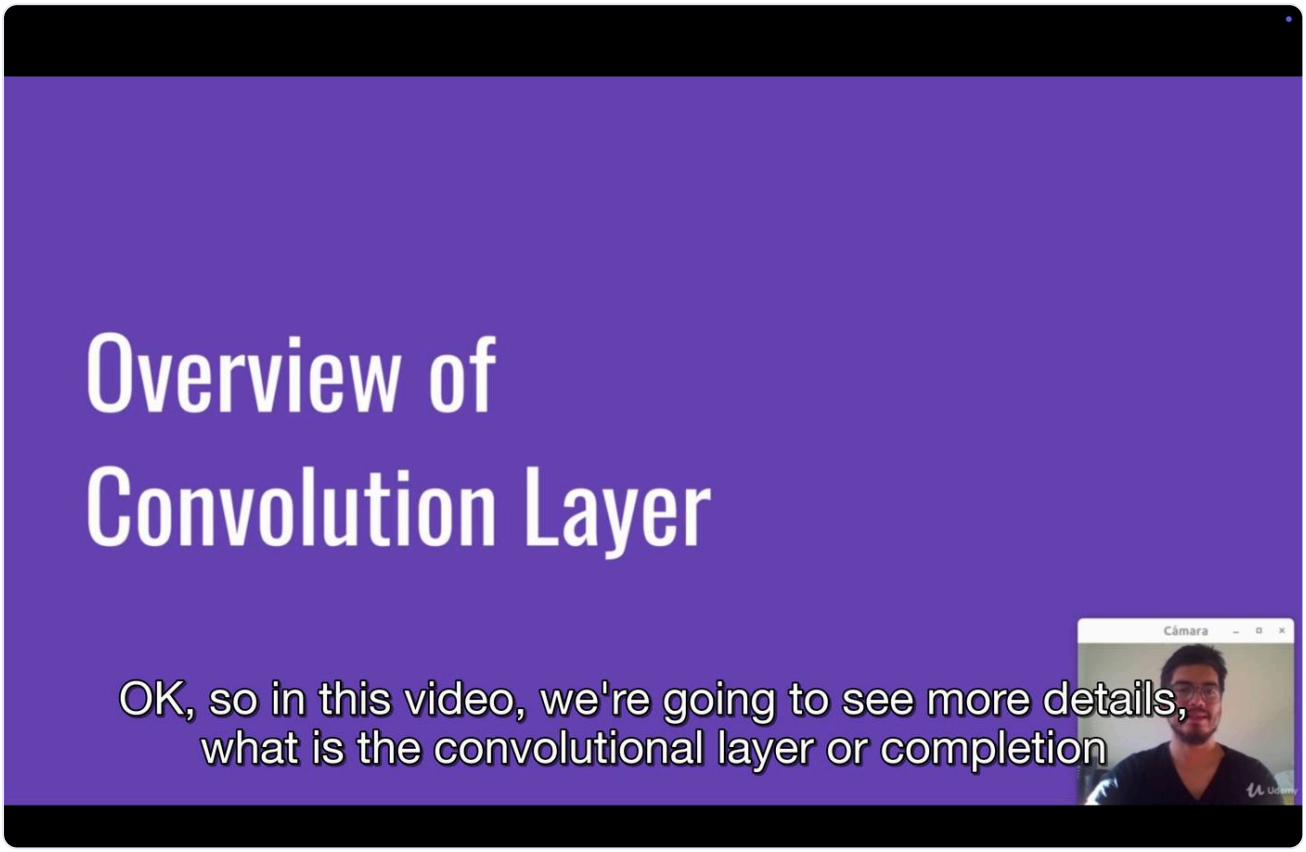



FIGURE 7.1 | Overview of

OK, so in this video, we're going to see in more detail what the convolutional layer, or convolution layer, is.

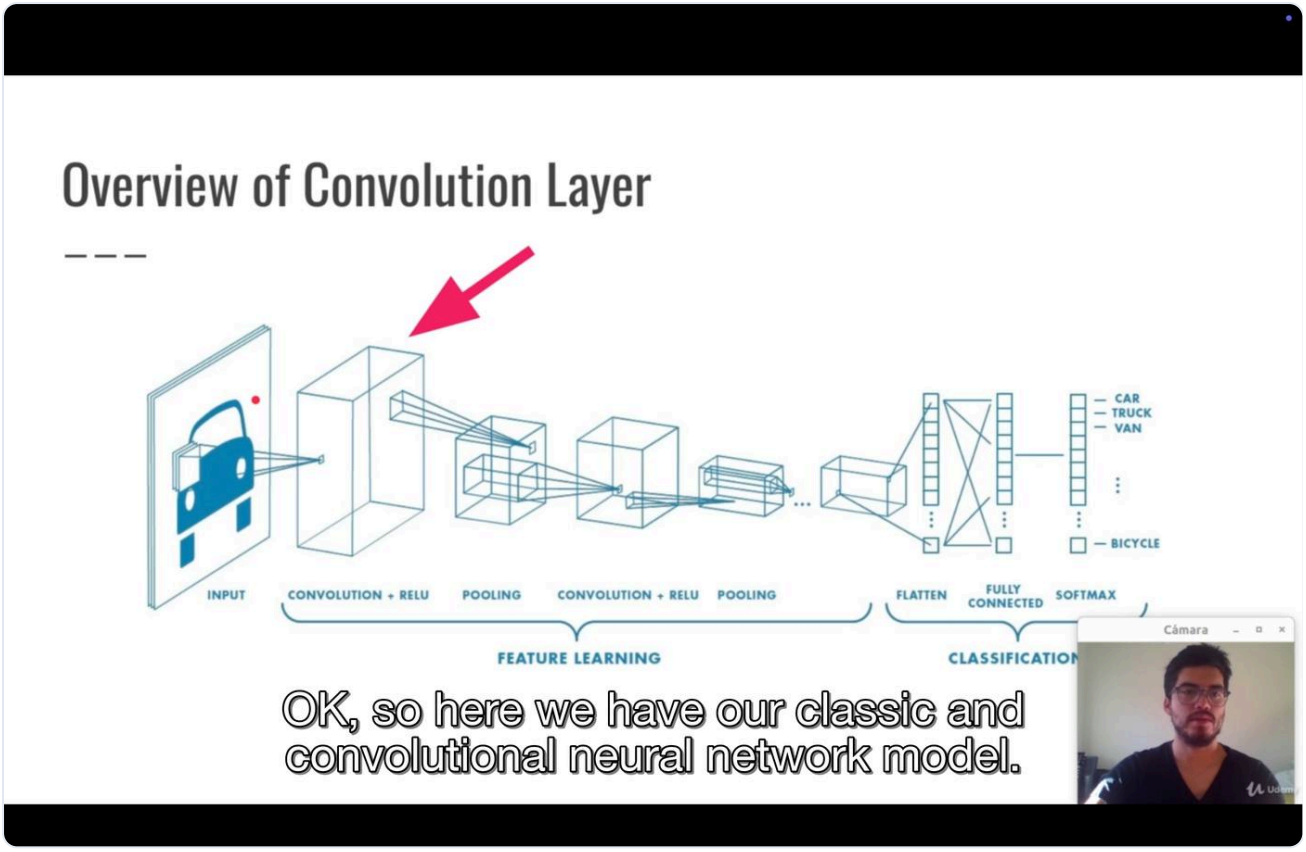



FIGURE 7.2 | Overview of Convolution Layer

OK, so here we have our classic convolutional neural network model. We have the input, the output, the feature learning stage and the classification stage. OK, so in the feature learning stage, we're going to have two important layers. The first one is going to be the convolutional layer, and the second one is going to be the pooling layer, OK?

In this example, we have two convolutions. So here we have the first convolutional layer. Then we have a pooling layer. Then we have another convolutional layer. And then we have another pooling layer. OK, so in order to understand this, we are going to study first the convolutional layer.

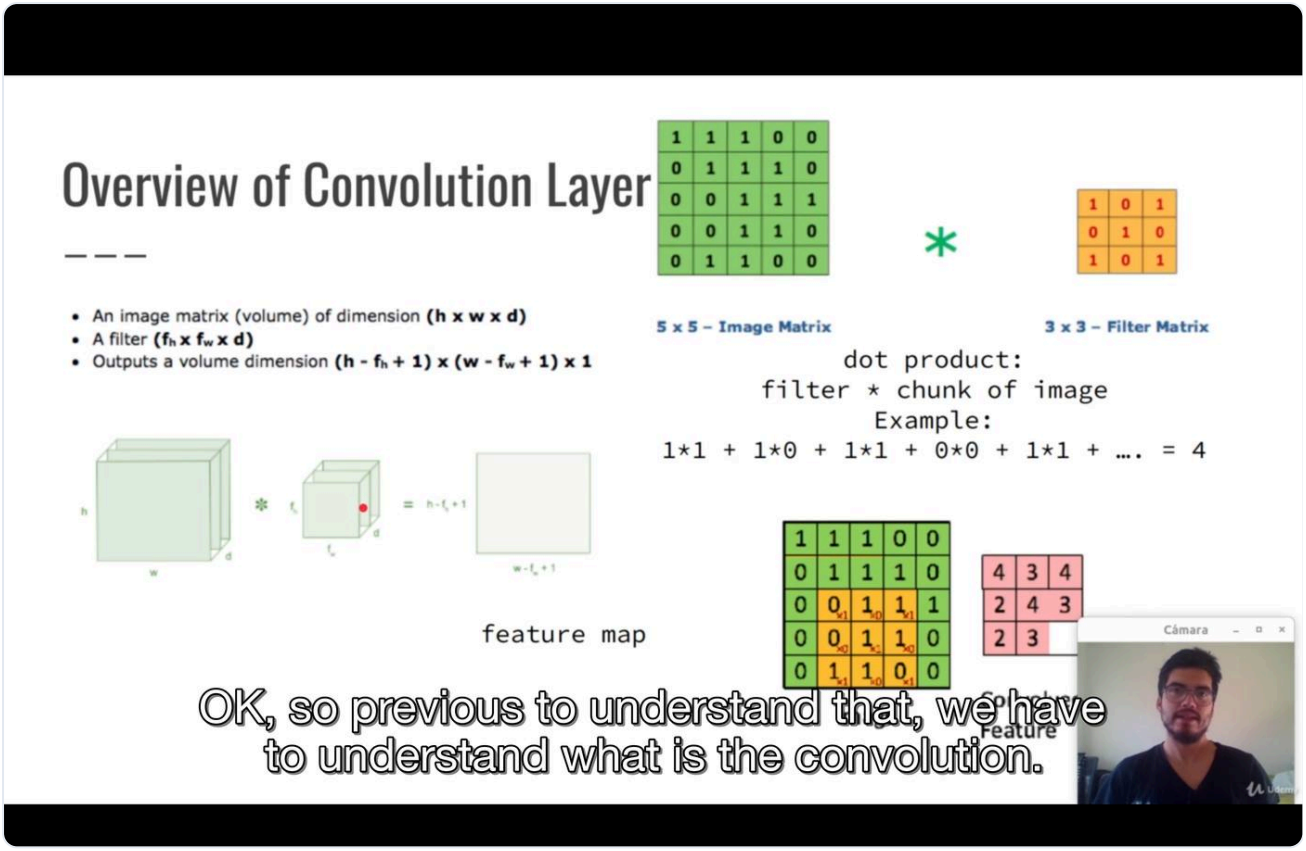



FIGURE 7.3 | Overview of Convolution Layer

OK, so before understanding that, we have to understand what the convolution is. OK, so basically a convolution is a mathematical operation between these two things. Here we're going to have an image, and this image can be any image of height H, width W and depth D.

OK, and then we're going to have a filter with height h, width w and depth d. OK, so the convolution is usually represented with this sign, with this symbol. And so the convolution between the image and the filter is going to output another image, a new image. OK, and this new image is usually called the feature map. OK, so from one convolution between an image and a filter, we're going to have another...

The output is going to be, excuse me, the output is going to be another image.

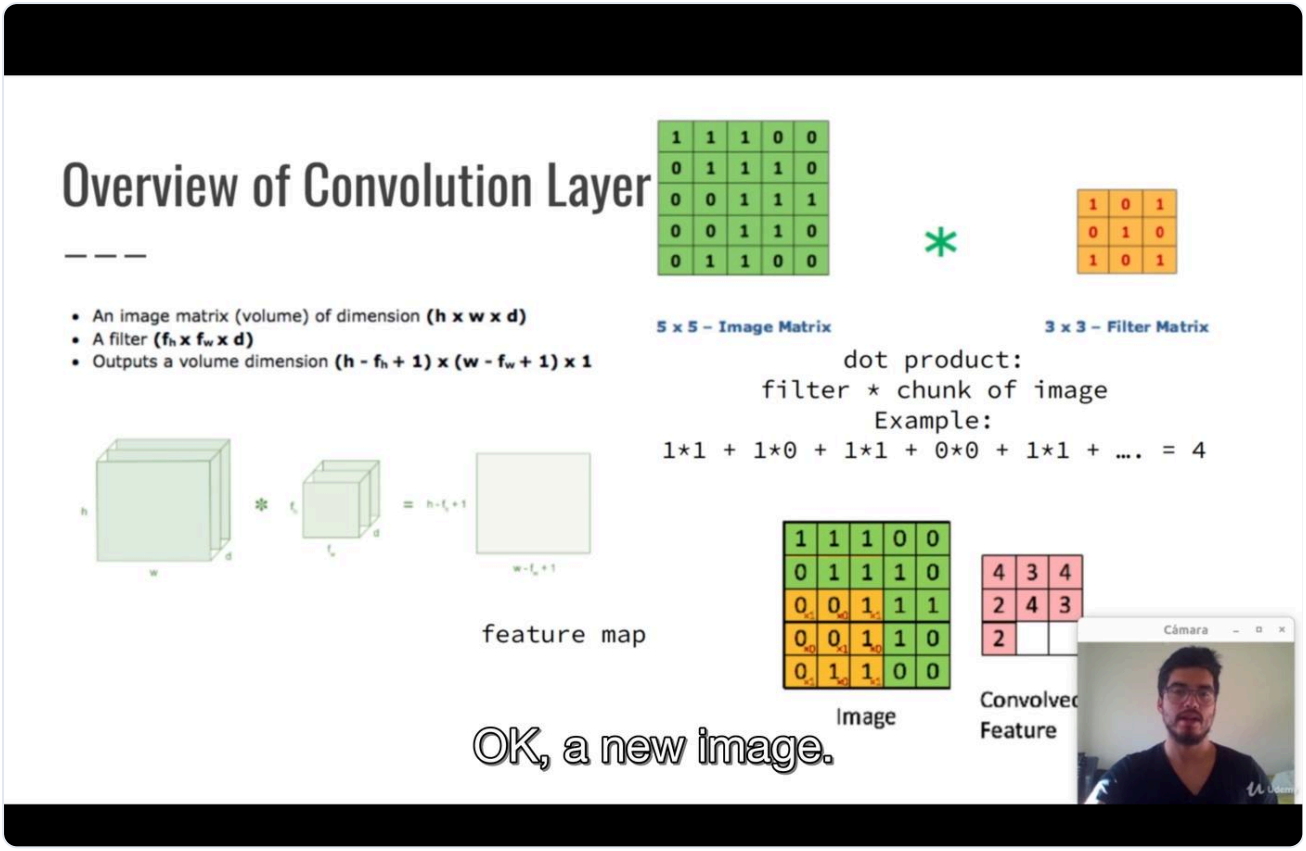



FIGURE 7.4 | Overview of Convolution Layer

OK, a new image. So here we have an example image, and here we have an example filter. OK, so this image is going to be composed only of ones and zeros, and the filter is going to be composed only of ones and zeros.

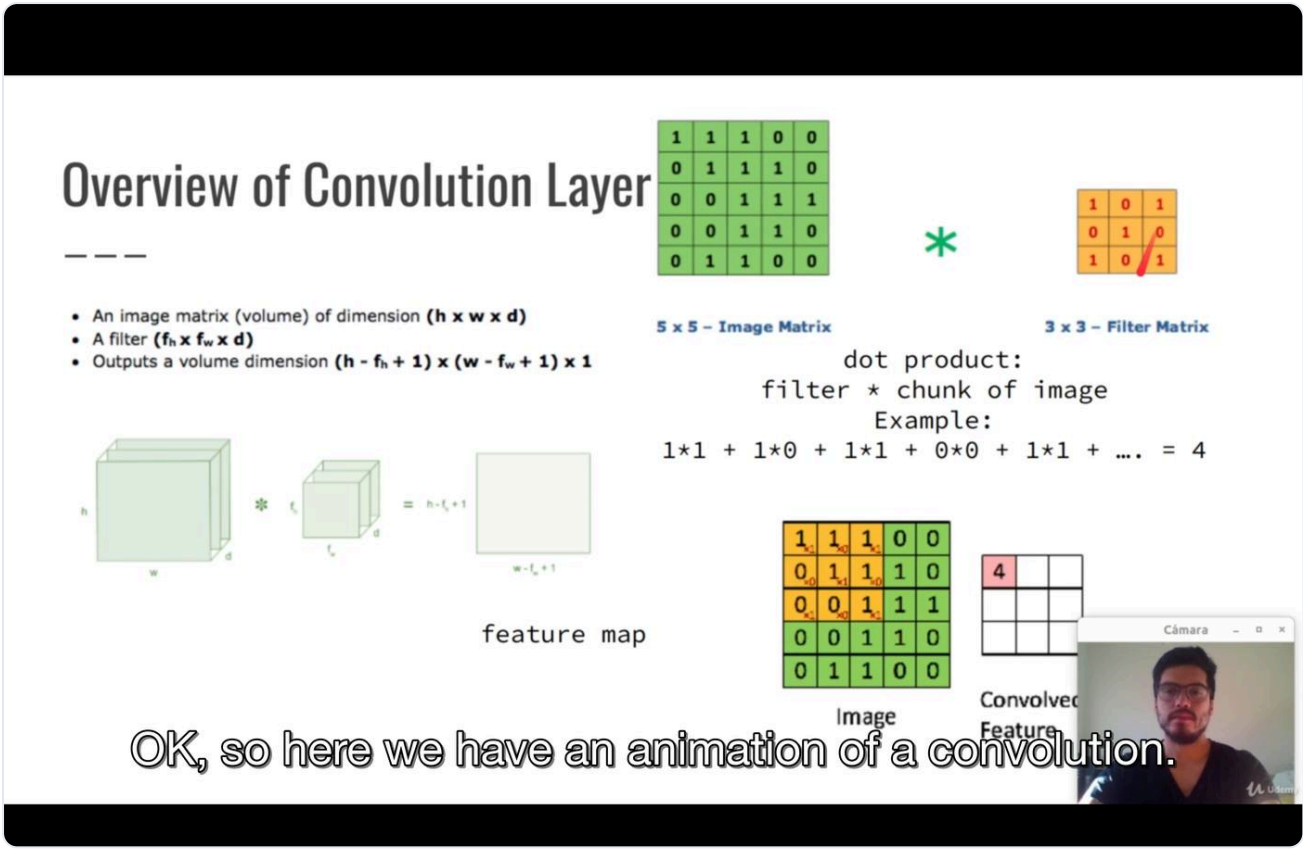



FIGURE 7.5 | Overview of Convolution Layer

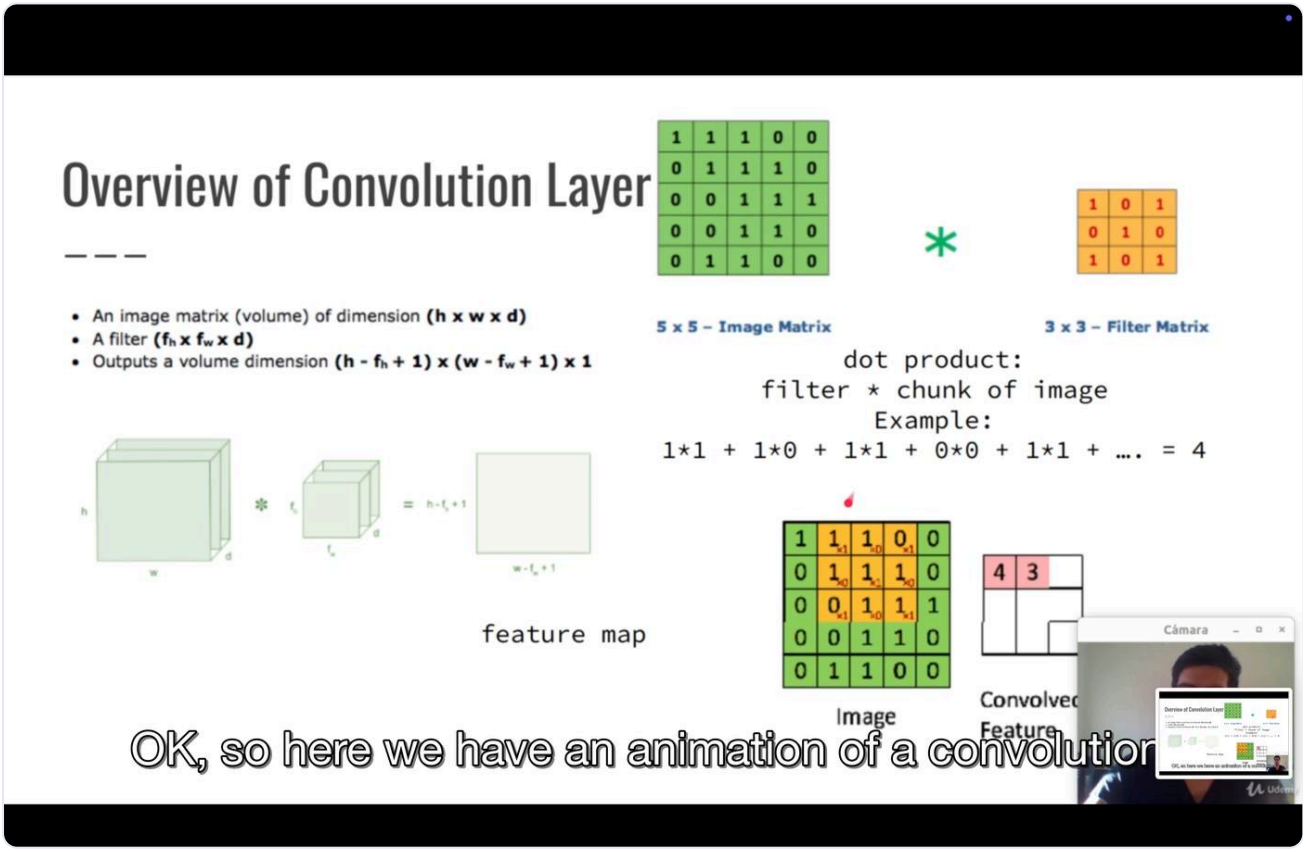



FIGURE 7.6 | Overview of Convolution Layer

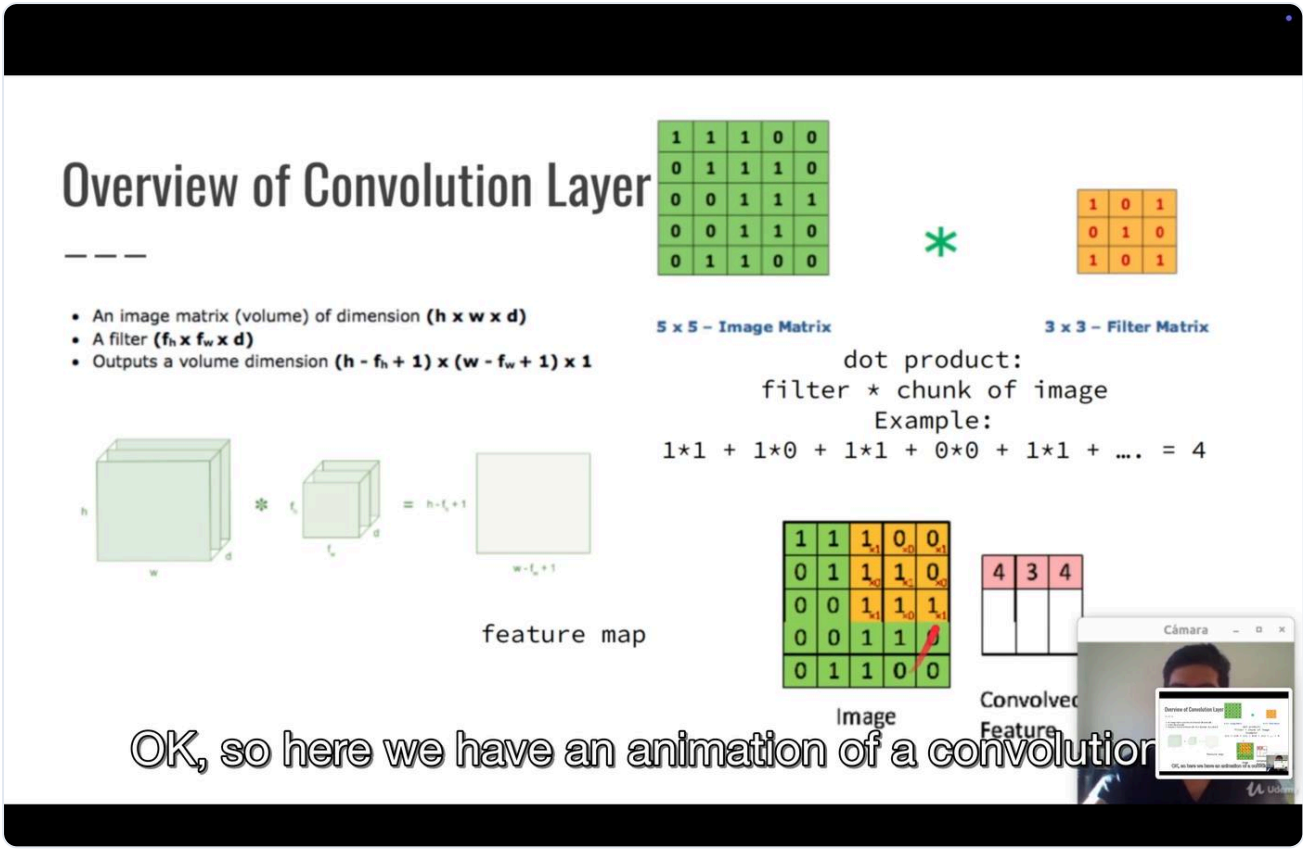



FIGURE 7.7 | Overview of Convolution Layer

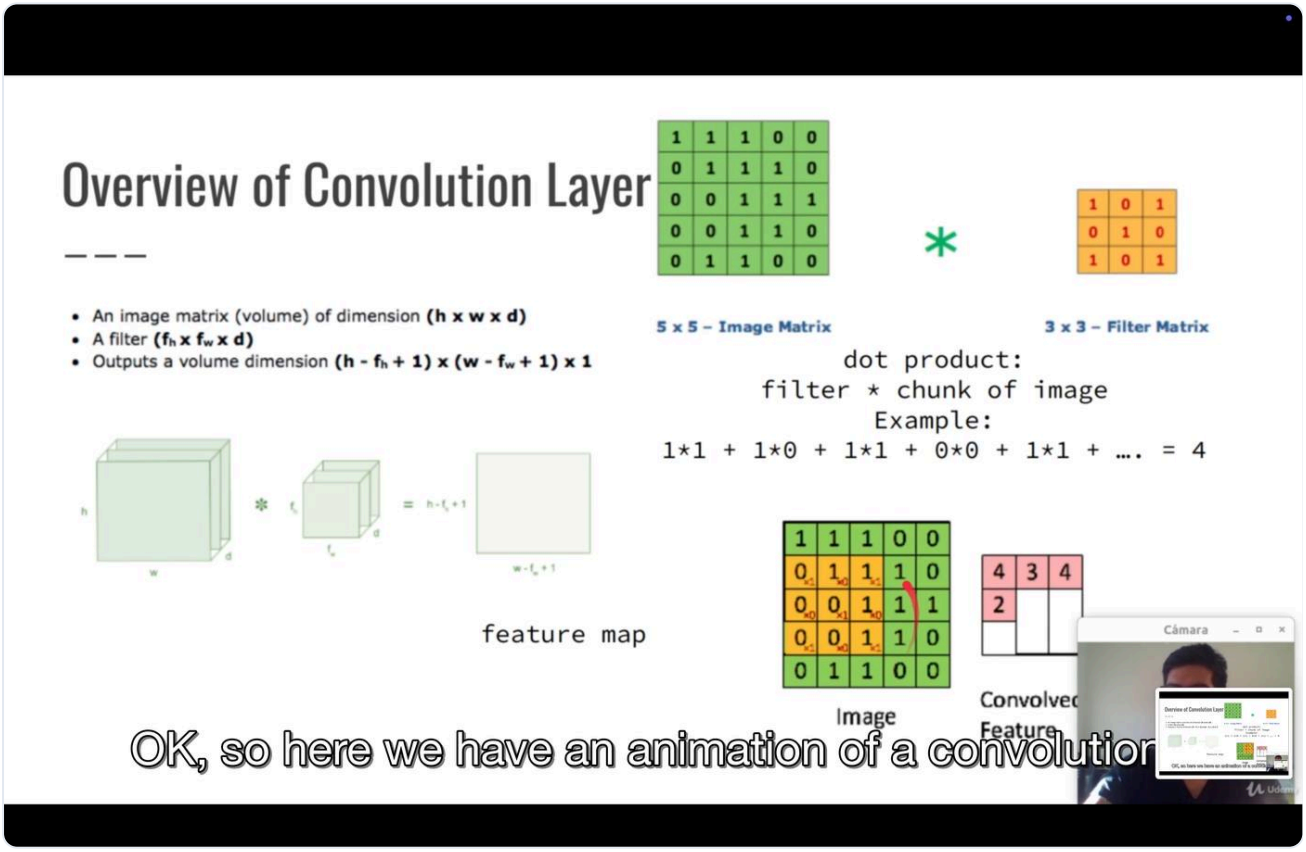



FIGURE 7.8 | Overview of Convolution Layer

OK, so here we have an animation of a convolution.

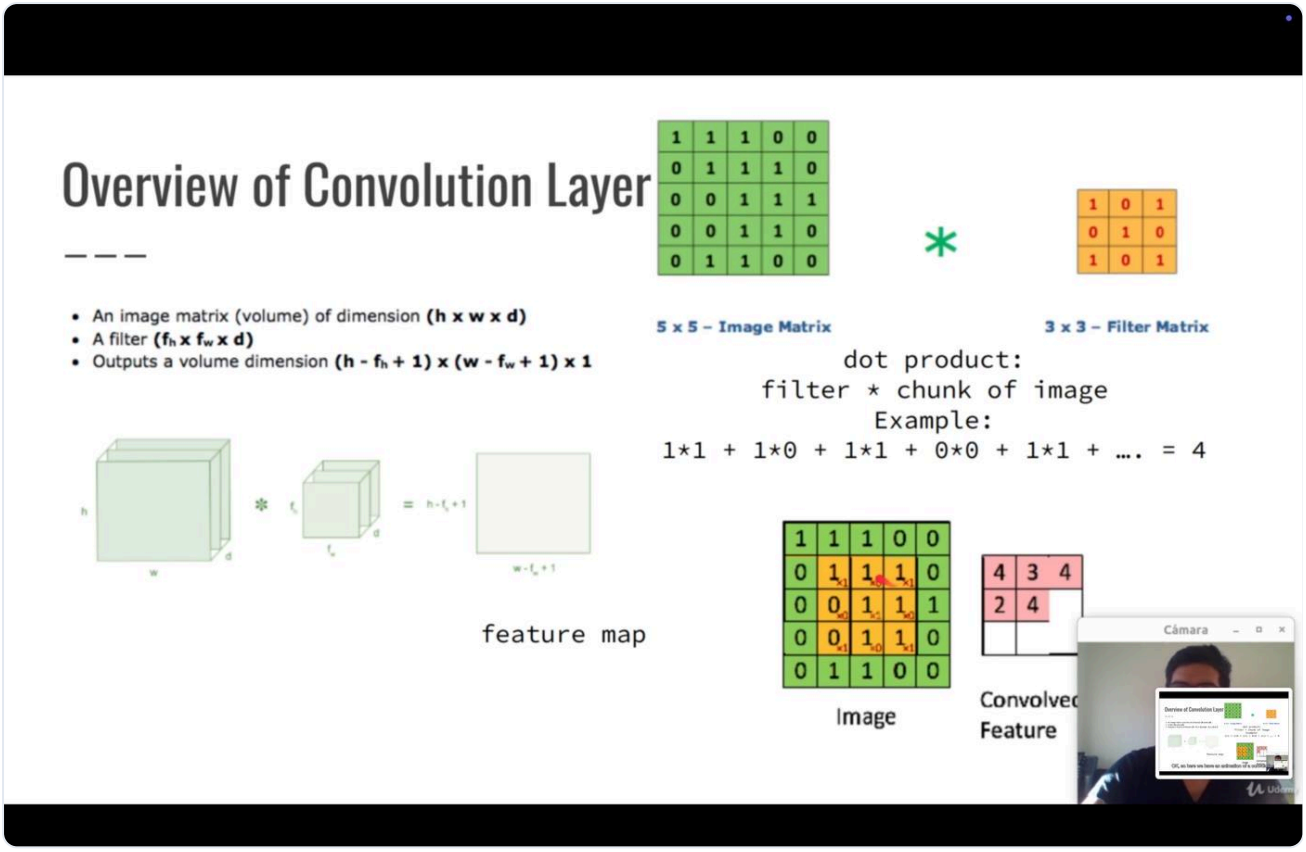



FIGURE 7.9 | Overview of Convolution Layer

OK.

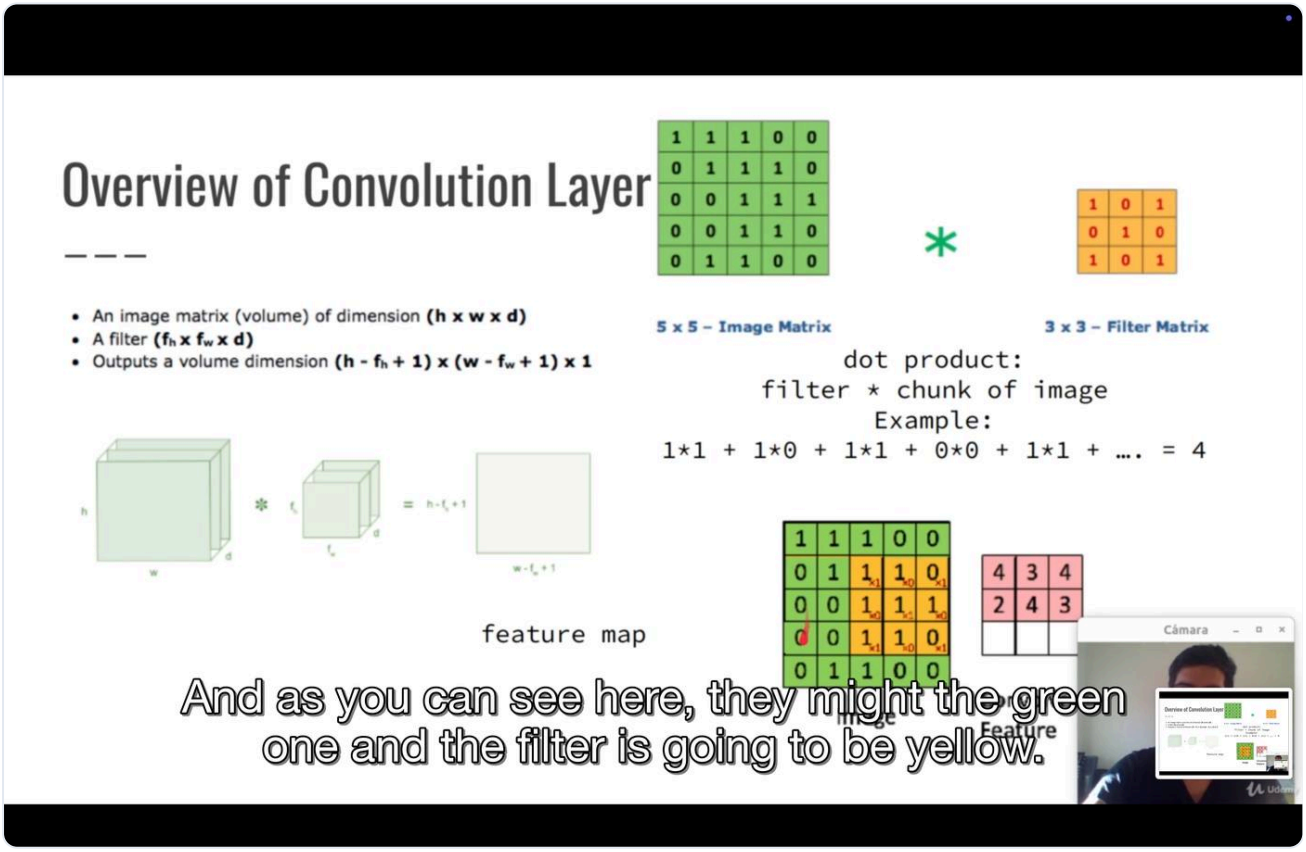



FIGURE 7.10 | Overview of Convolution Layer

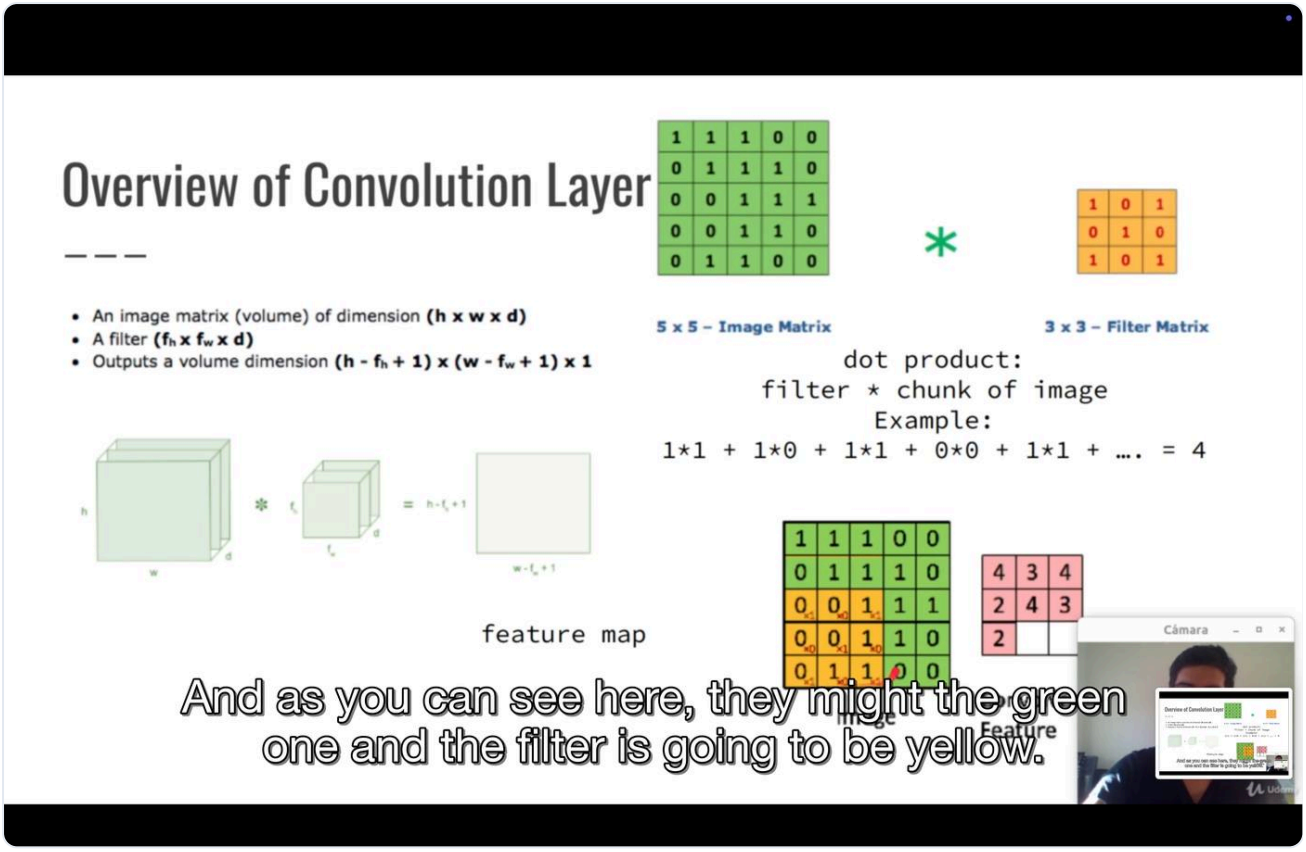



FIGURE 7.11 | Overview of Convolution Layer

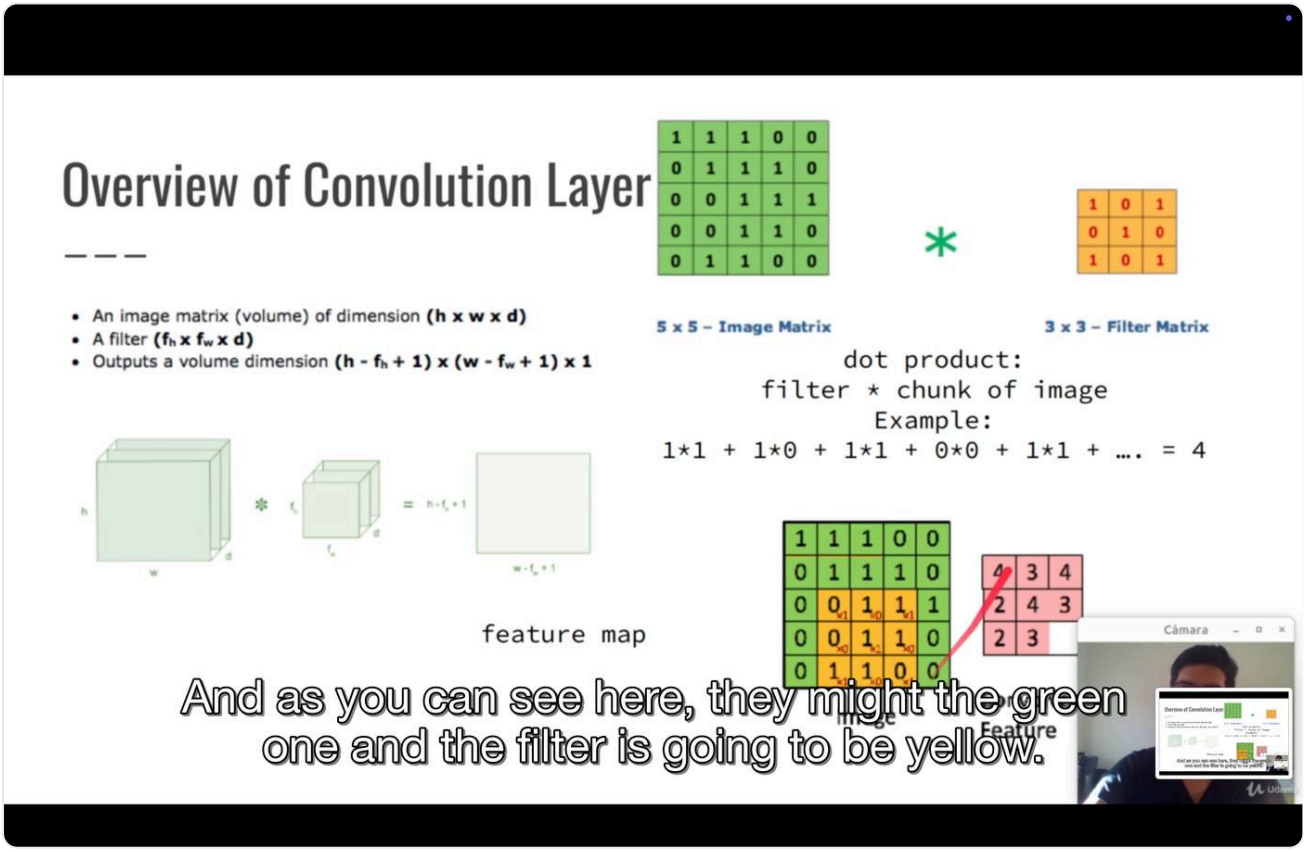



FIGURE 7.12 | Overview of Convolution Layer

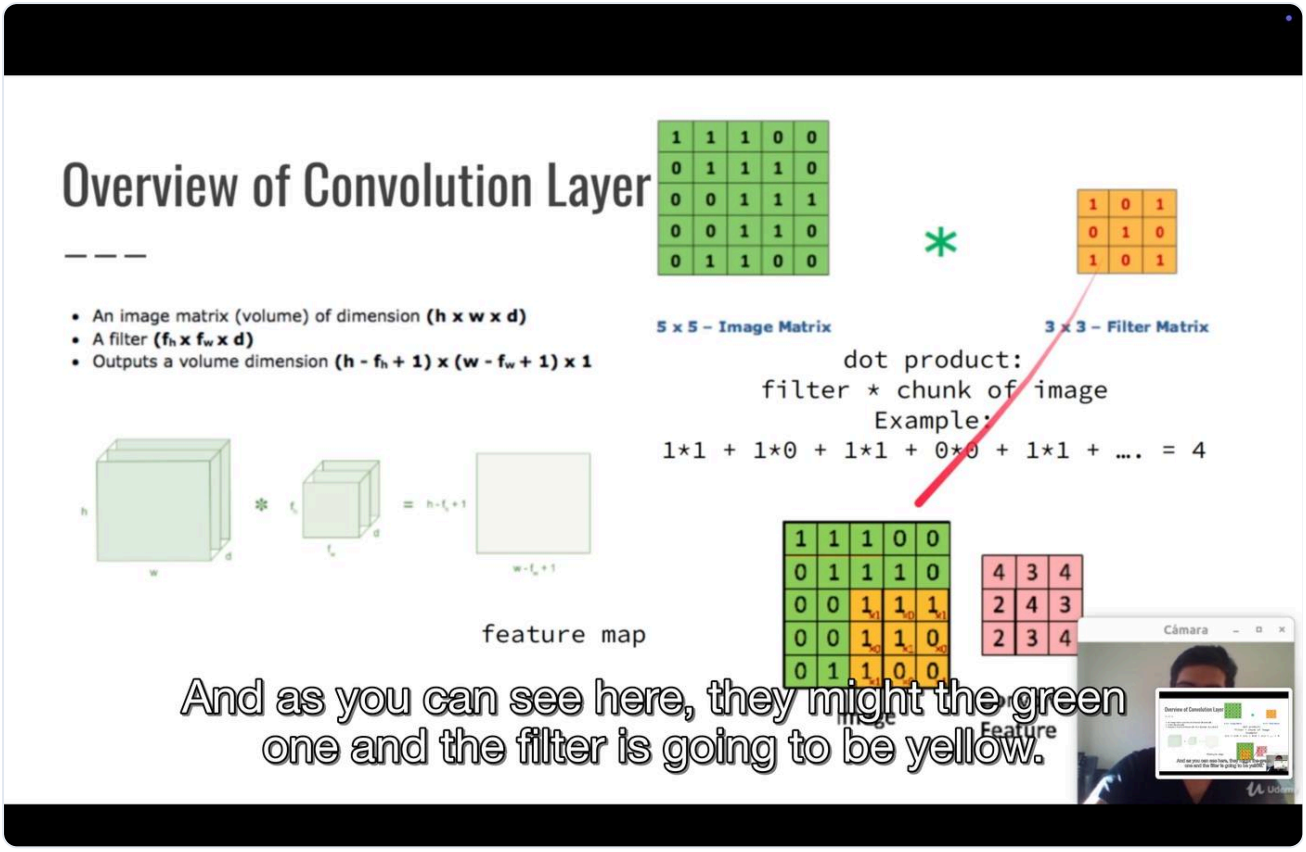



FIGURE 7.13 | Overview of Convolution Layer

And as you can see here, the image is the green one and the filter is going to be yellow. OK, so the convolution operation is going to be a mathematical operation in which we are going to apply the dot product between a specific area, a chunk of the image, and the filter.

OK, so we're going to apply a dot product first. Give me a second.

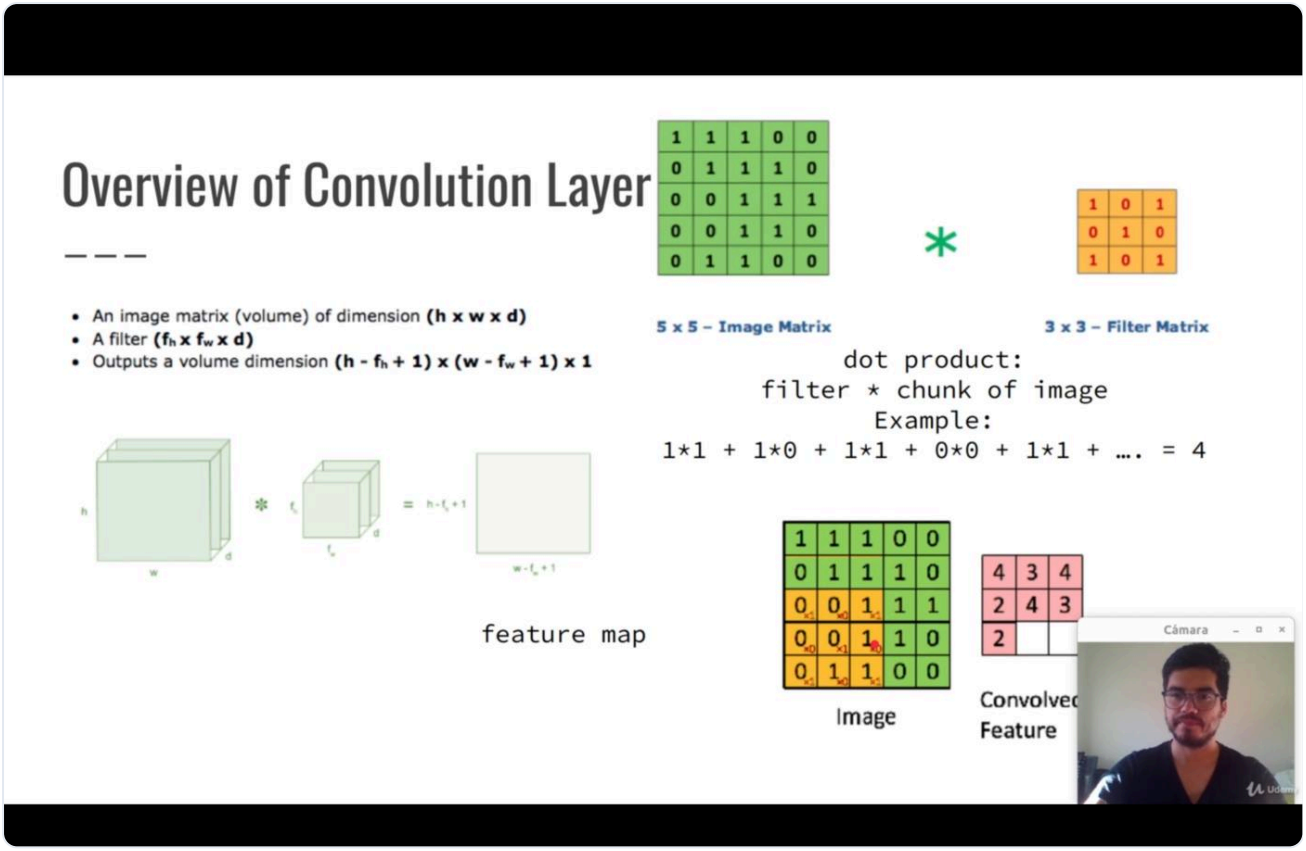



FIGURE 7.14 | Overview of Convolution Layer

And here we're going to have the dot product between the first chunk of this image and the filter, and that is going to output the first element of the convolution. This is going to be, in this example, four, OK. Then we're going to move this filter again to the side, and we're going to apply the dot product between this chunk of the image and the filter, OK, with the same filter. And that dot product is going to output another number.

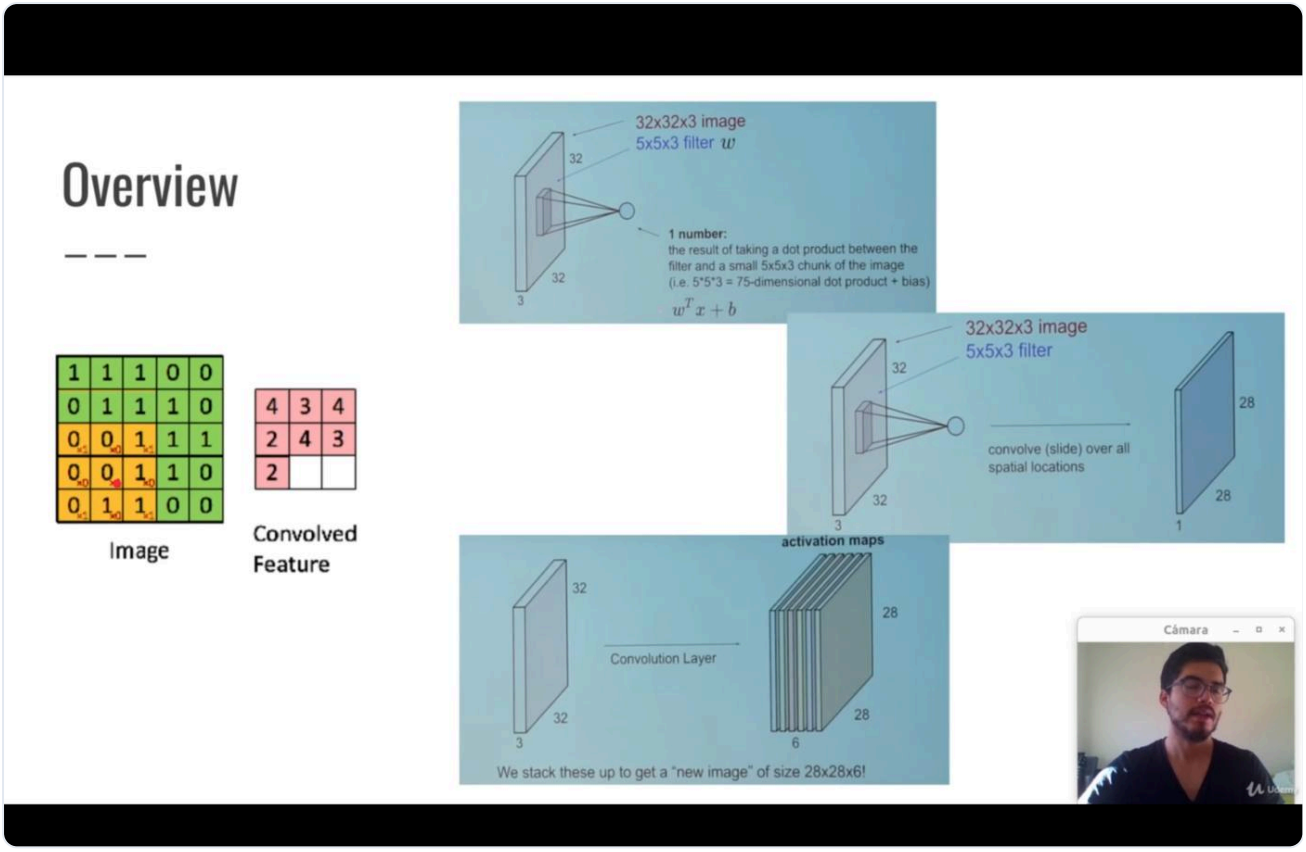



FIGURE 7.15

In this example, it's going to be three. Then we're going to move the filter again to another chunk of the image, and then we're going to do the dot product between this chunk of the image and the filter, and the output is going to be another number, and it is going to be four. Then we're going to move the filter again. We're going to apply the dot product between that chunk of the image and the filter, and we're going to get another number, and then we're going to move on more and more until we reach the end of the image.

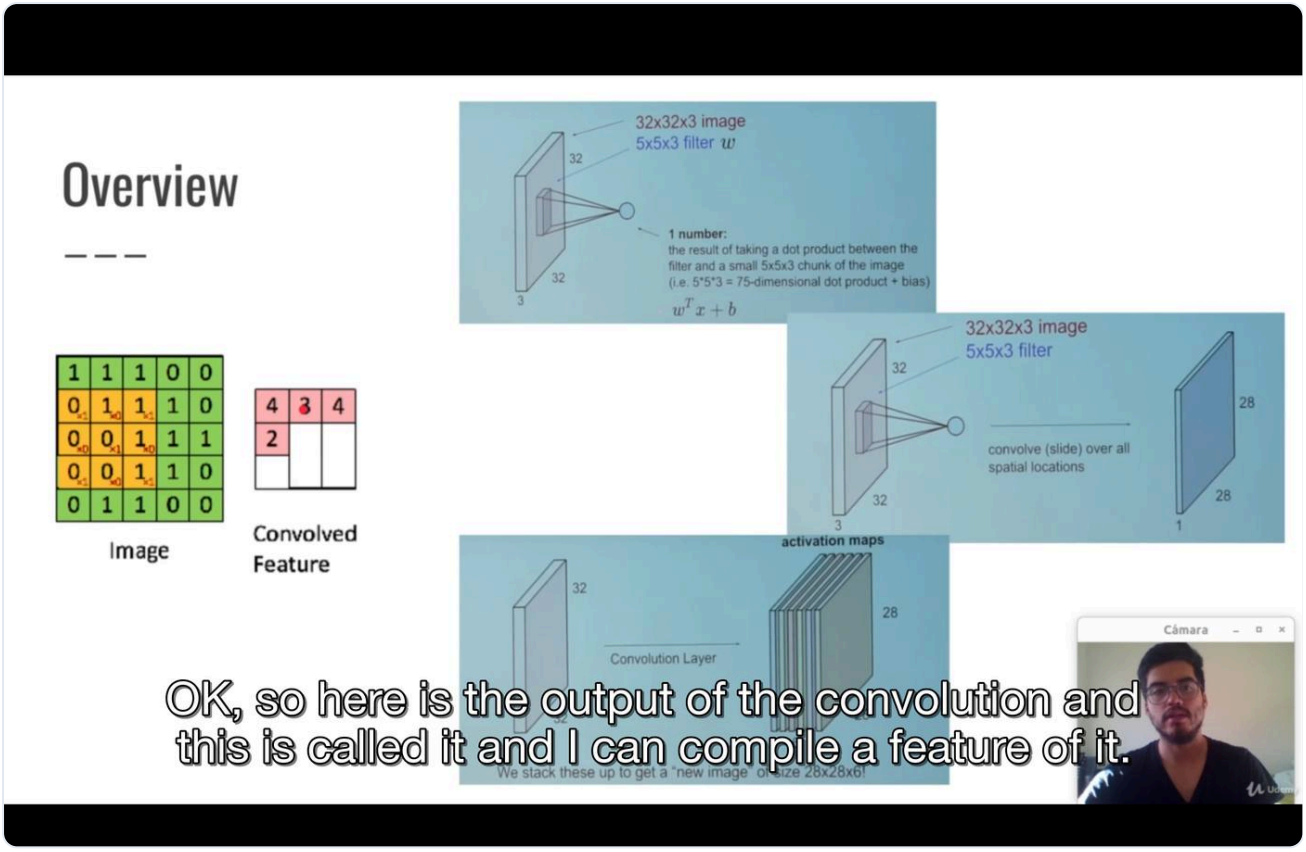



FIGURE 7.16

So here we have the finish, OK, and this final dot product is going to output another number. OK, so basically the convolution operation is an operation between an image and a filter, in which we apply a dot product between a specific chunk of the image and the filter. And with that we get a number, and then we move the filter through the entire image.

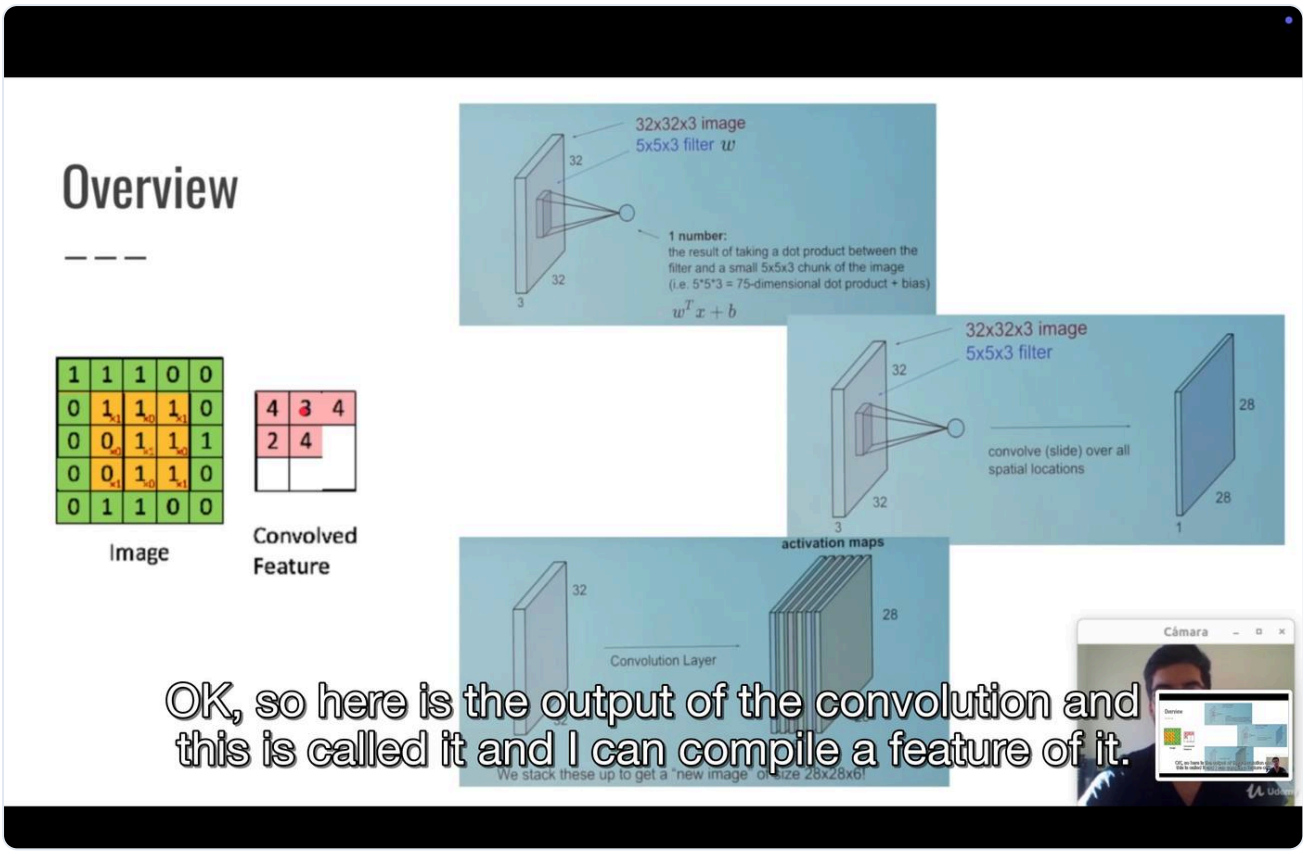



FIGURE 7.17

And in each move we're going to apply the dot product. And with that dot product...

We are going to get a number. OK.

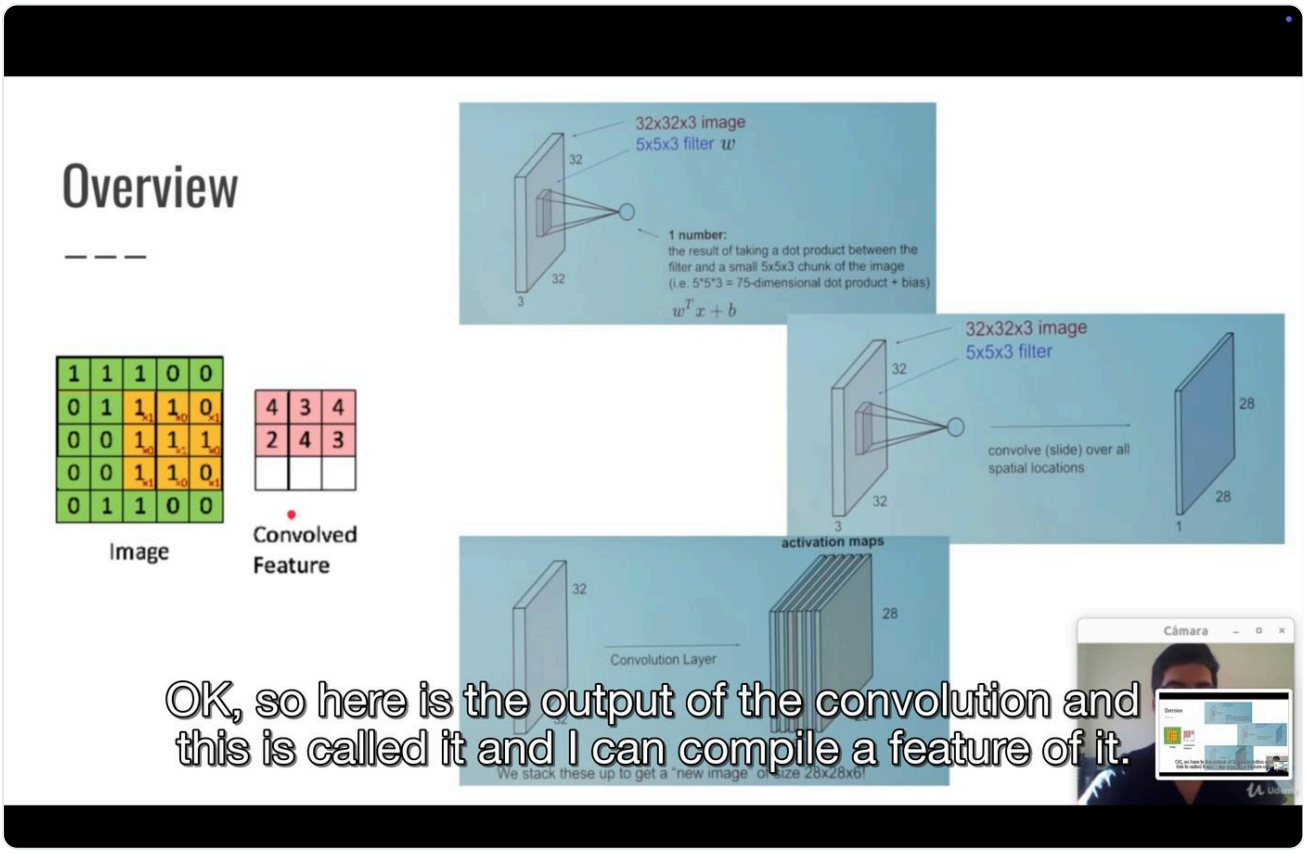



FIGURE 7.18

So this is an example. OK, so for example, if we are applying the convolution, excuse me, if we're applying the filter to this chunk of the image with this square, the output of the first dot product is going to be four in this example, and the dot product is going to be: one by one, plus one by zero, plus one by one, plus zero by zero, plus one by one, plus one by zero, plus zero by one,

plus zero by zero, plus one by one. And then we're going to sum all these different multiplications, and the result is going to be four.

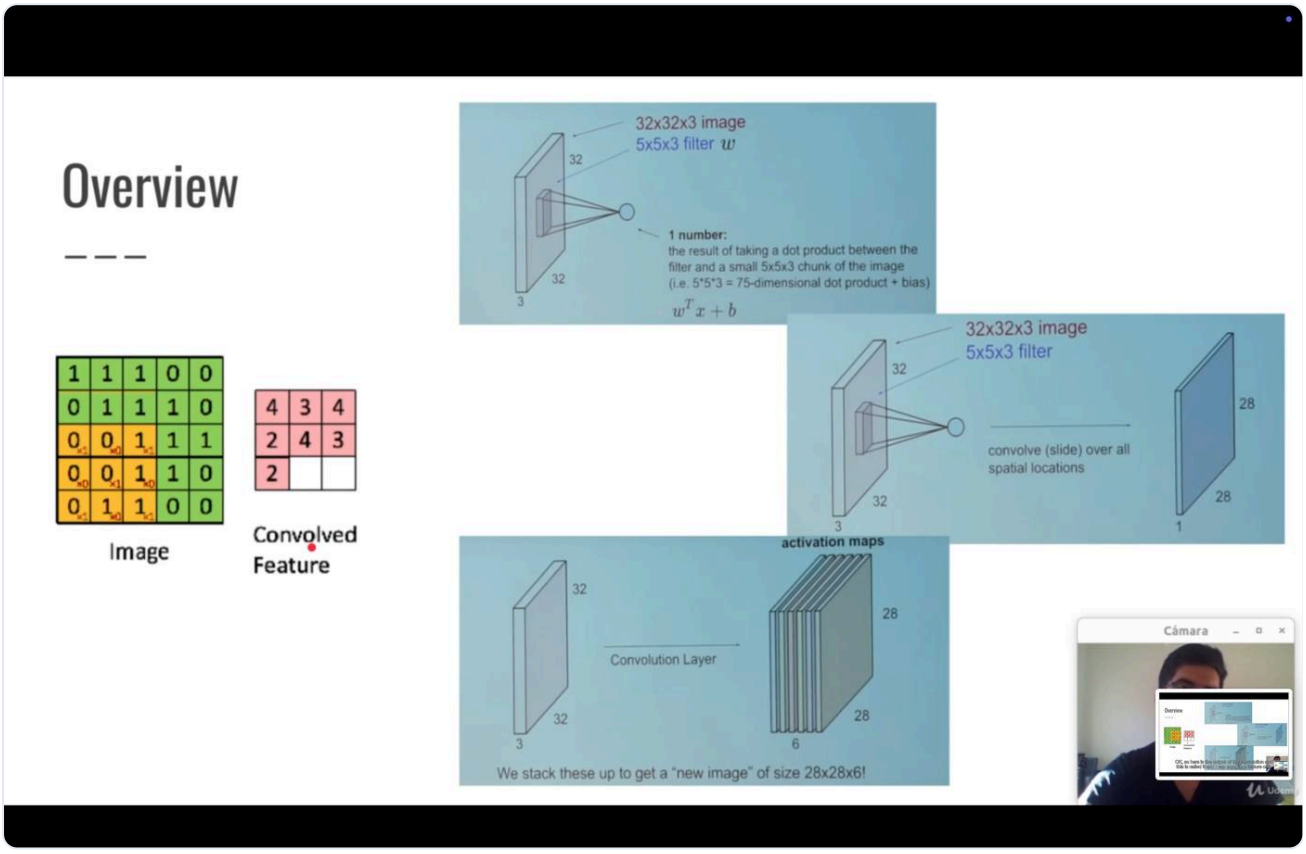



FIGURE 7.19

OK, so that is a convolution. OK, so here again, we have the animation of the convolution, in which we are moving the filter through the entire image. And in each stop of the filter we have the different numbers. OK, so here is the output of the convolution, and this is called the convolved feature, or feature map.

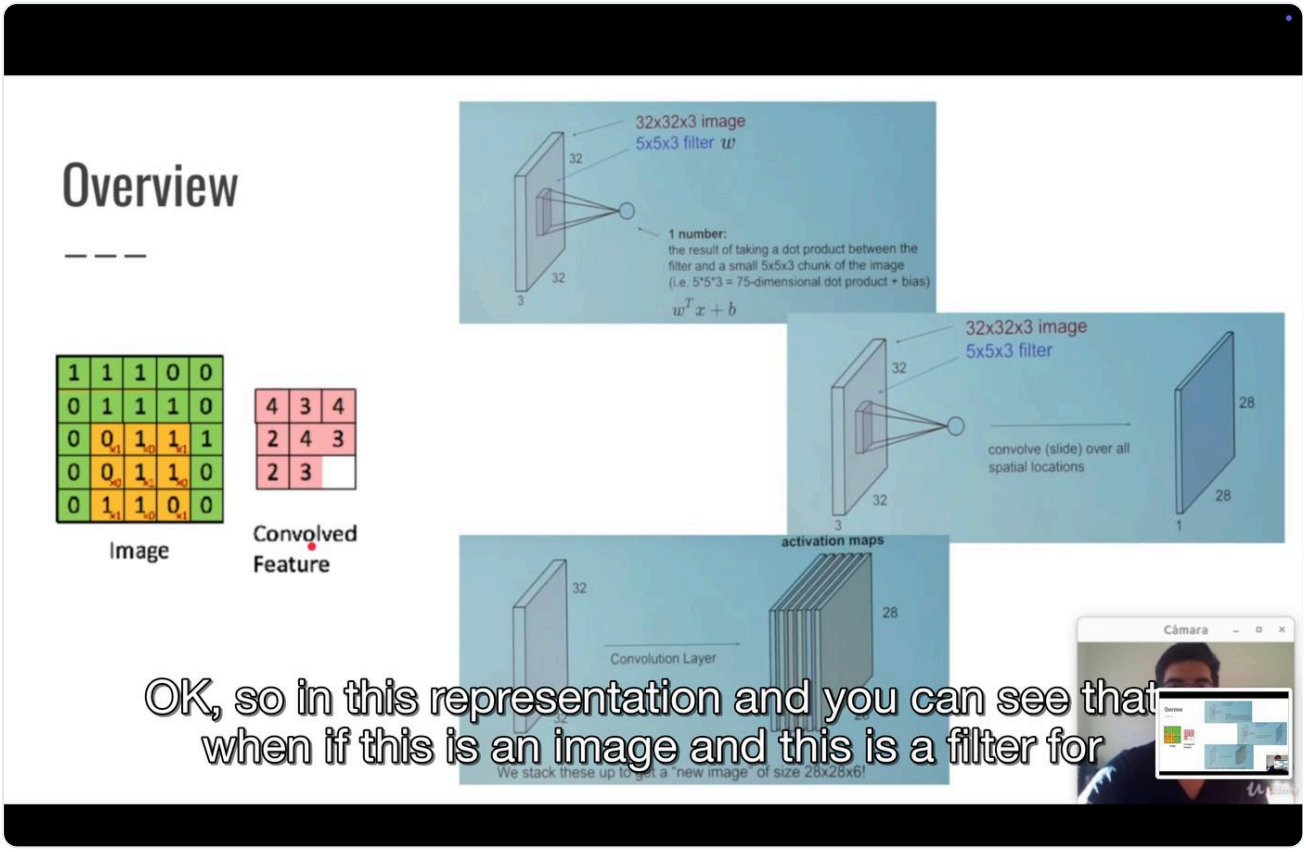



FIGURE 7.20

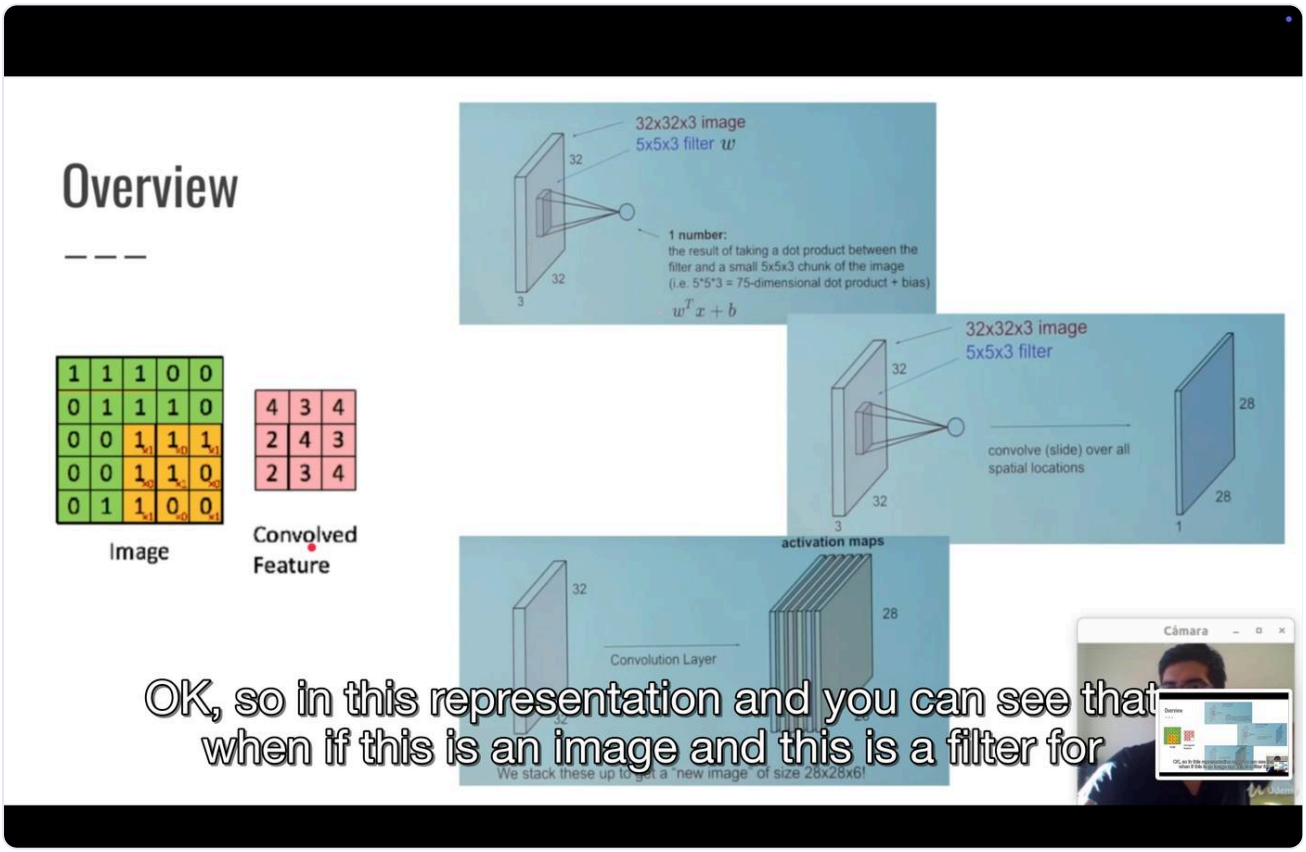



FIGURE 7.21

OK, so in this representation you can see that if this is an image and this is a filter, for each step of the filter on a specific area of the image, we're going to have a number.

Right. So after moving this filter through the entire image, we are going to have this entire image, OK? And each of the pixels, for example, if we are talking about images, each of the pixels, the elements of this matrix, is going to be a point here. And this point here is going to be related to this specific area of the original image. OK, so basically this is the idea of the convolution, in which we are going to move this filter through the entire image.

And for each stop of this filter on the image, we're going to get a number. And as we're moving this filter through different areas of the image, we're going to have different points, we're going to have different numbers. And with these different numbers, we're going to build this output image. OK, so here we're applying one filter.

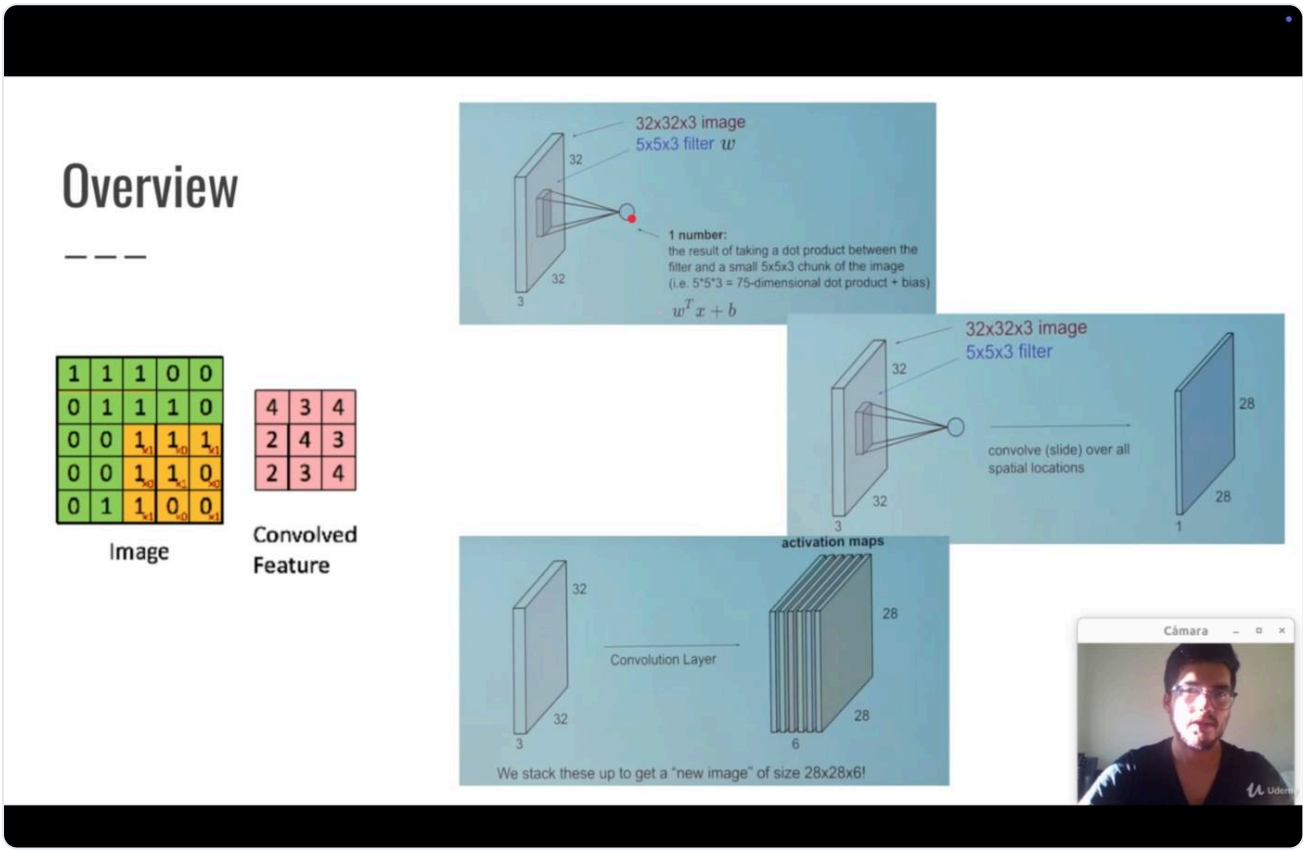



FIGURE 7.22

In this example, we're applying one filter, right, because this is the filter.

OK, so we're moving this unique filter through the entire image. OK, but we can apply different filters. So if you think that for one filter we get one feature map, right, then if we apply different filters, we are going to have different feature maps. So in this example, we have one image and we're applying different filters. And for each filter we are getting a different feature map, or activation map. So in this example, we are applying one, two, three, four, five, six...

I think one, two, three, four, five, six, six different filters, and for each filter we are getting one activation map, or feature map. OK, so let's see what this filter is. OK, so again, we have the convolution between the image and the filter, and this filter in this example has specific numbers. Right.

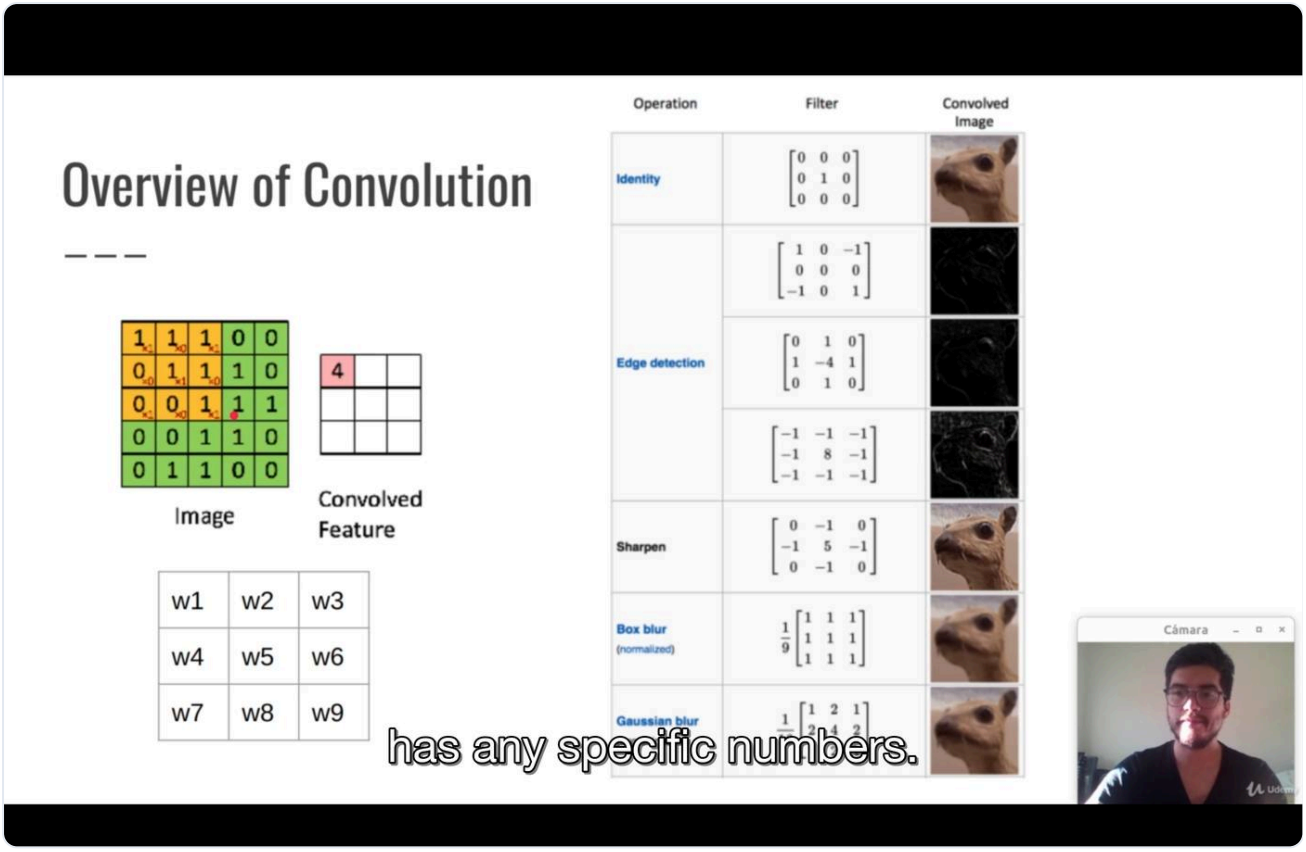



FIGURE 7.23

But this filter can have different numbers. It can have basically any number. So these are going to be general parameters, W1, W2, W3 and so on. So these parameters can have different values.

OK, so these numbers can have different values. Right. So an interesting, an important point here is that depending on the values of these parameters of the filter, we can get a different effect on the image. So, for example, if this is the original image and we apply this filter here, with these numbers, the output of the convolution between this filter and this image is going to be this image here.

And this image here is highlighting the edges of the image. OK, so basically, actually, this filter is for the operation of edge detection.

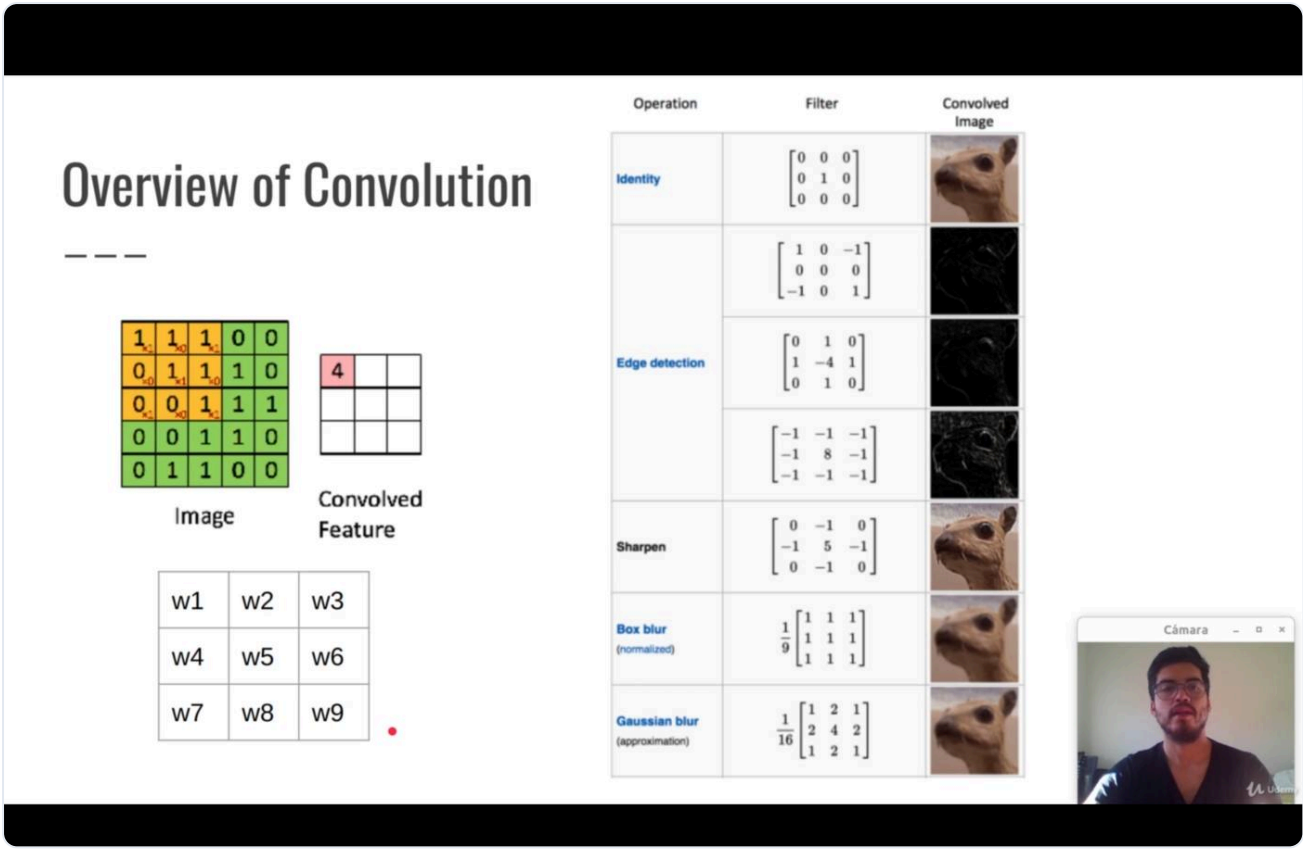



FIGURE 7.24 | Operation Filter Convolved

OK, so again, if we apply this specific filter, we can detect the edges of the image. OK, so that is a very interesting behavior of the filters, because with different filters you can get different effects, and more specifically, with different filters you can get different features of the image.

So for example, with this filter, we're getting the edges of the image. OK, then we can apply, for example, another filter, and this can be this one here. With this filter, if we apply the convolution with this filter, we're going to get a blur effect. And then, for example, if we apply another filter, we're going to have a Gaussian blur. OK, so basically, depending on the values of the parameters of the filter, we're going to have different effects on the original image.

OK, so for example, if we have this car here, we can apply a specific filter in order to detect the different edges of this car. OK, and then with the edges of the car, we can do another operation in order, for example, to identify the wheels, to identify different parts of the car. OK, but basically the filters are the operators that allow you to identify different patterns, or to apply different operations, or basically to detect different important features in the image.

OK, and when we are training the model, the convolutional neural network, what we are learning are these parameters, in the case of the convolution.

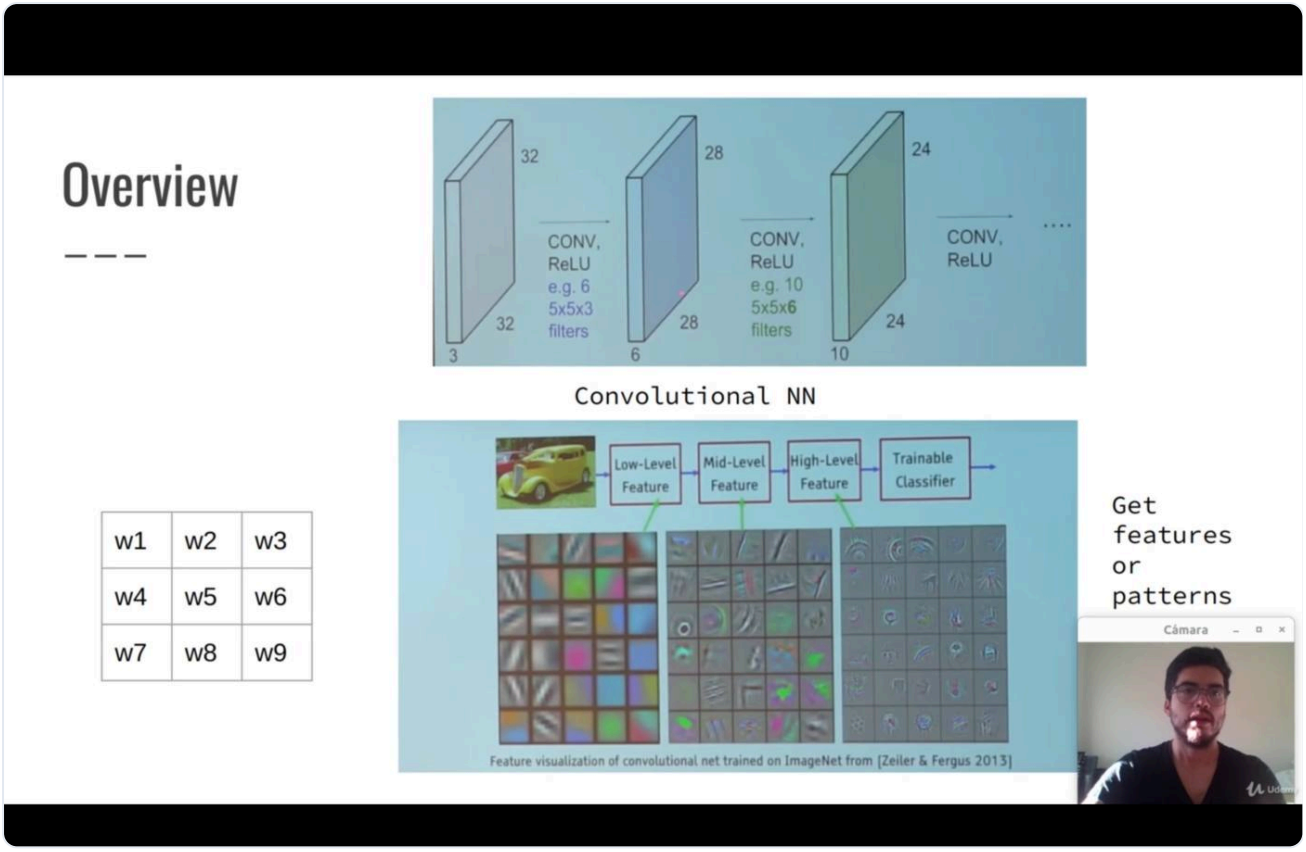



FIGURE 7.25

OK, because these parameters are our parameters, and they can be learned in order to find different features in the original data, OK. So again, we're going to have the parameters here, the filters, excuse me, the parameters of the filters here. And in the convolutional neural network, instead of having one convolution, we usually are going to have different layers of convolutions.

OK, so for example, we are going to have the first convolution here. We're going to apply the first convolution and we're going to have the first feature map here. Then, for example, we can apply another convolution, and then we're going to have this other feature map, with different dimensions, as you can see. OK, so usually we build convolutional networks with different convolutional layers, and that's totally dependent on the problem you're facing, OK, because, for example, you can use one convolutional layer, or you can use two, or one thousand...

One million, I don't know, but basically it is usual, and it's really common, to find different layers of convolutions. OK, and here we have an example in which we have different convolutional layers. OK, so here we are going to have a car image, and the idea is to try to identify if this is a car or not.

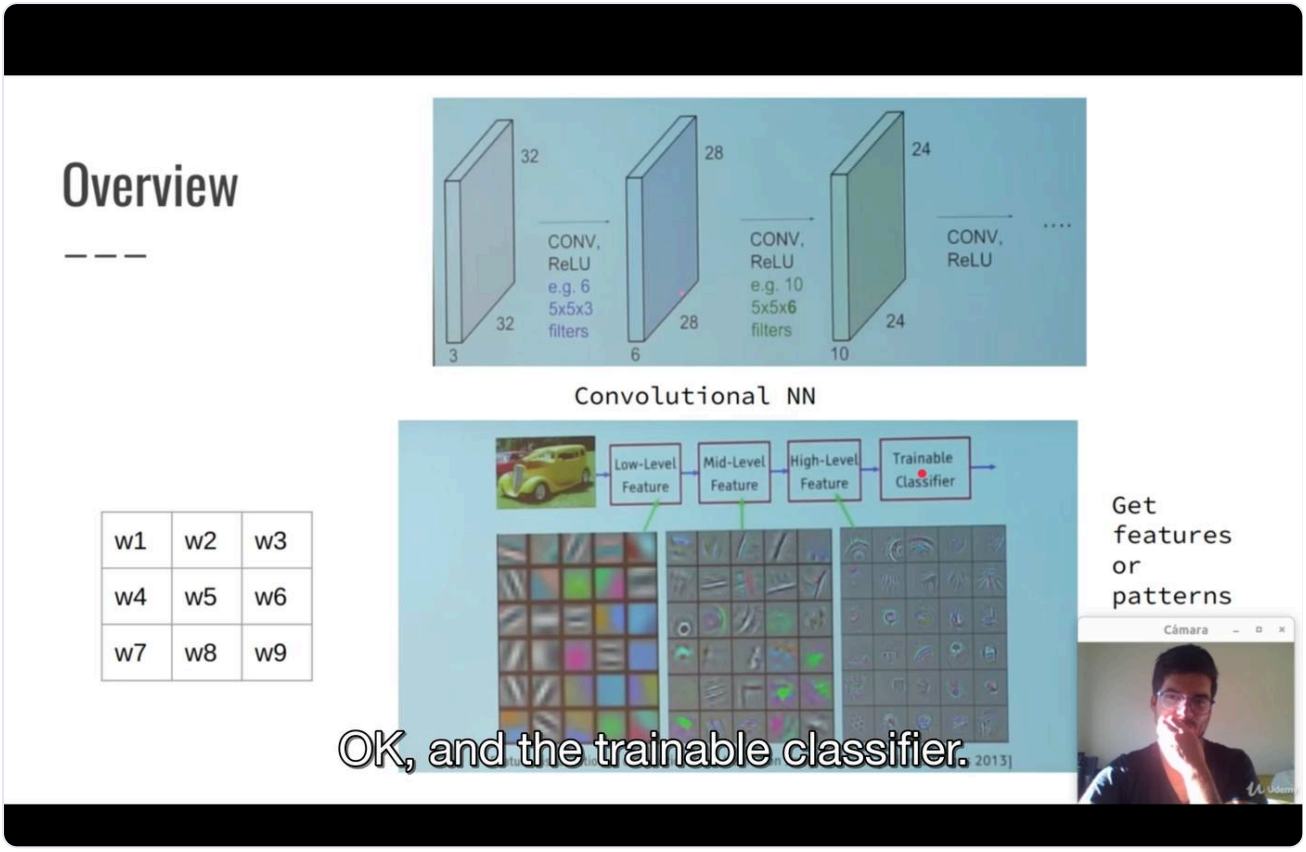



FIGURE 7.26

OK, so here we're going to have the feature learning stage, and then we're going to have the classification stage, OK, the trainable classifier.

OK, so the idea of this is that on each application of one convolution, this convolution is going to find more high-level features. That means that the first convolutions are going to learn very low-level, or kind of general, features. And then when you apply more and more convolutions, these convolutions learn more specific patterns. OK. So these are, for example, high-level features. So the idea here is that in each convolutional layer, we're going to be learning different patterns, or we're going to find different patterns in the input image, or different features that we can use in order to identify what this image is.

OK. So now I have to talk about two very important topics about convolution and convolutional layers, and these are parameter sharing and local connectivity.

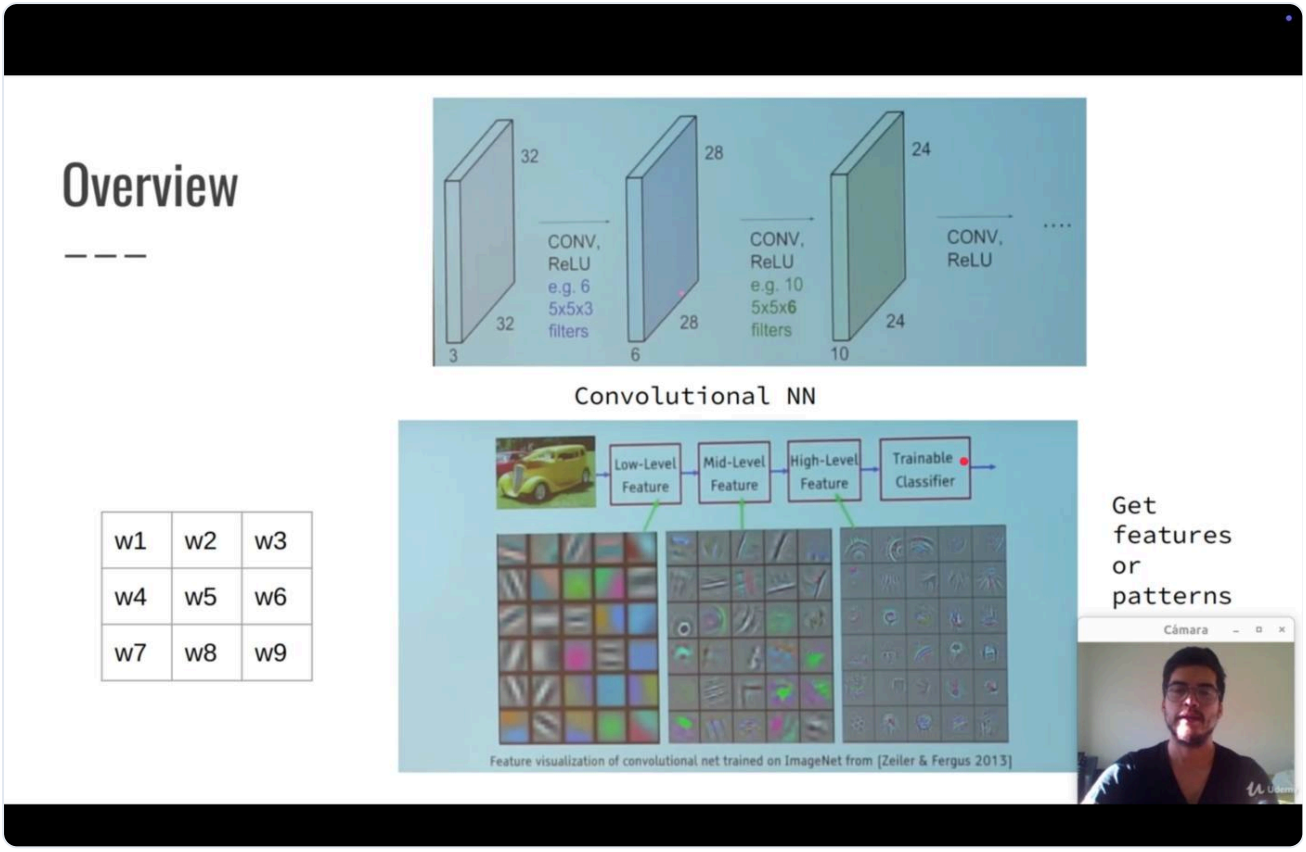



FIGURE 7.27

OK, so when we have this image, and then we have the filter, and then we are applying the convolution, we are getting this number, right. And the filter can be this one here, with these parameters. Right. So when we apply the dot product, we are going to get one number, right. Then we're going to move this filter to another area of this image, and we're going to have another number.

Right. But at the end of the day, we're going to move the same filter through the entire image, and we're going to get the feature map. Right.

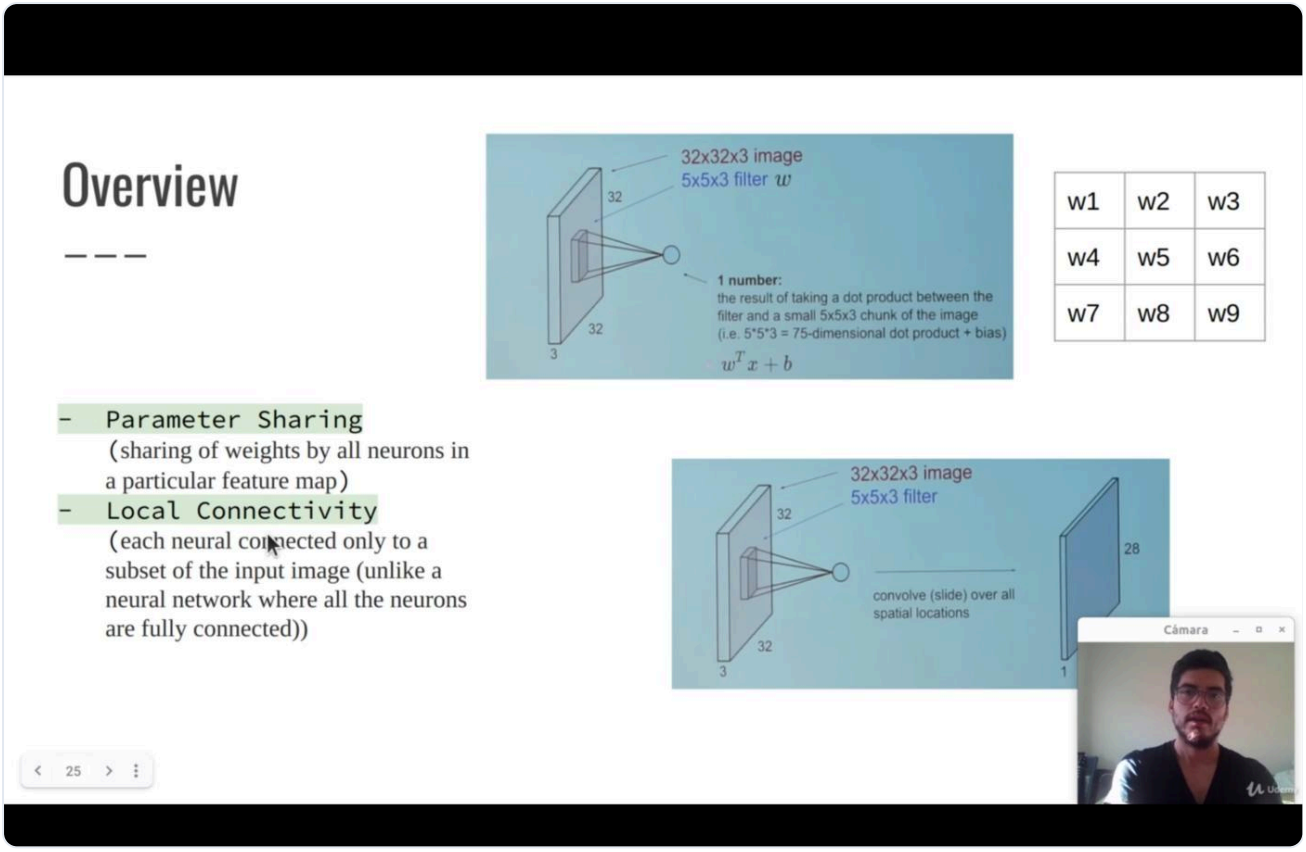



FIGURE 7.28

So basically, what we are doing here is moving the same filter. We're not changing this filter in order to get one feature map. Right. We are using the same filter with the same parameters. OK, so that means that if you think of this point as a neuron, you are going to have the same parameters for the different neurons that build this feature map.

OK, so each point of this feature map is going to have the same parameters for the filter. OK, and that is parameter sharing. OK, then we're going to have the concept of local connectivity. That means that when you are getting this point here of this feature map, this specific point is connected with this specific area.

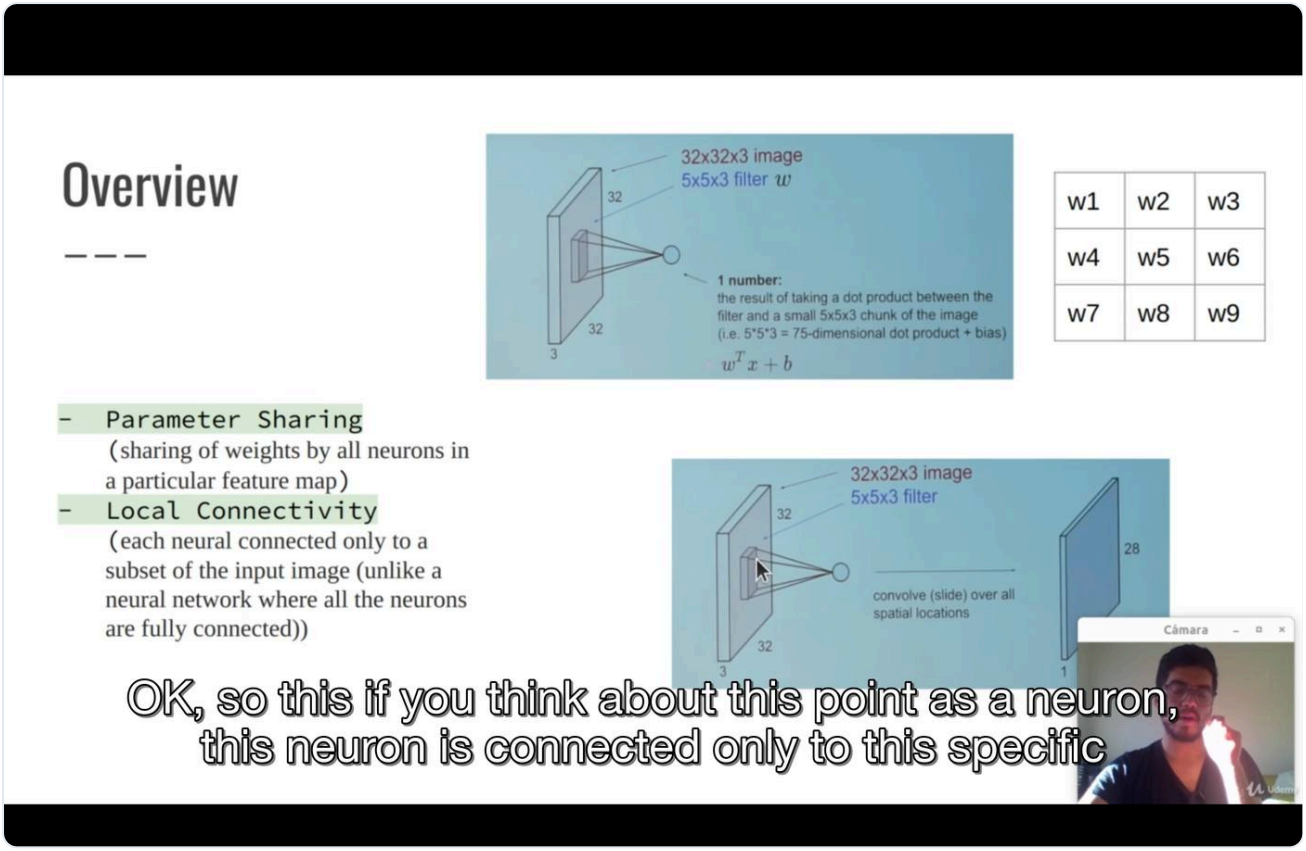



FIGURE 7.29

OK, so if you think about this point as a neuron, this neuron is connected only to this specific area; it is not connected to the entire image.

OK, it is connected only to this area. And this is totally different from the fully connected layer, because if you think that this is a neuron from a fully connected layer, this neuron should be connected to each of the elements, or pixels, of this image. OK, so this neuron would be connected to the entire image here. OK, but now, as we are using convolution, we are going to have this local connectivity, because this neuron is going to be connected only to this specific area, not to the entire image.

OK, so finally we have to see the different concepts that we're going to have in the convolution. OK, so the first concept is that we can have different filter sizes. So, for example, we can have a filter size of three by three. We can have a filter size of five by five, seven by seven, nine by nine and so on. OK, so the filter size is going to be a parameter that you can set when you are training your model. OK, so different filters are going to output different shapes of the feature map.

OK, and usually when we are talking about a filter, we also call this filter a kernel. OK, another very important concept is padding. And basically when you are applying the convolution, we usually add extra pixels to the image in order to fit the kernel to the image. OK, so the most common way to add different pixels here is to add zeros, and this is zero padding. OK, so you can add different... there are different padding methods, but one of them is zero padding.

OK, and basically the padding is how many pixels you add to the border of this image. OK, again, this is a parameter you can set when you're training your model. OK, so this is a tunable parameter. OK, then we're going to have the concept of stride, and the stride basically means how many

pixels you move your filter when you are moving the filter through the image.

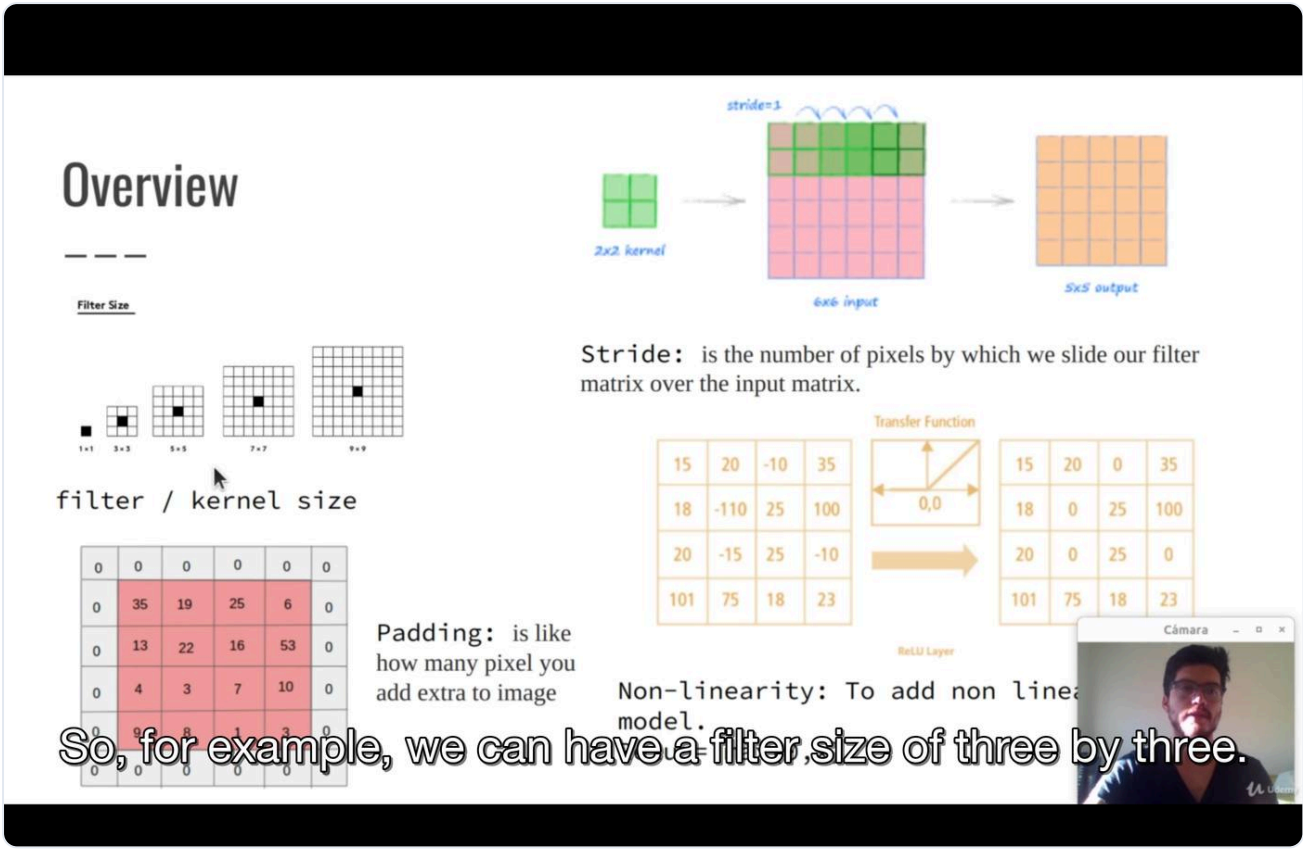



FIGURE 7.30 | Filter Size

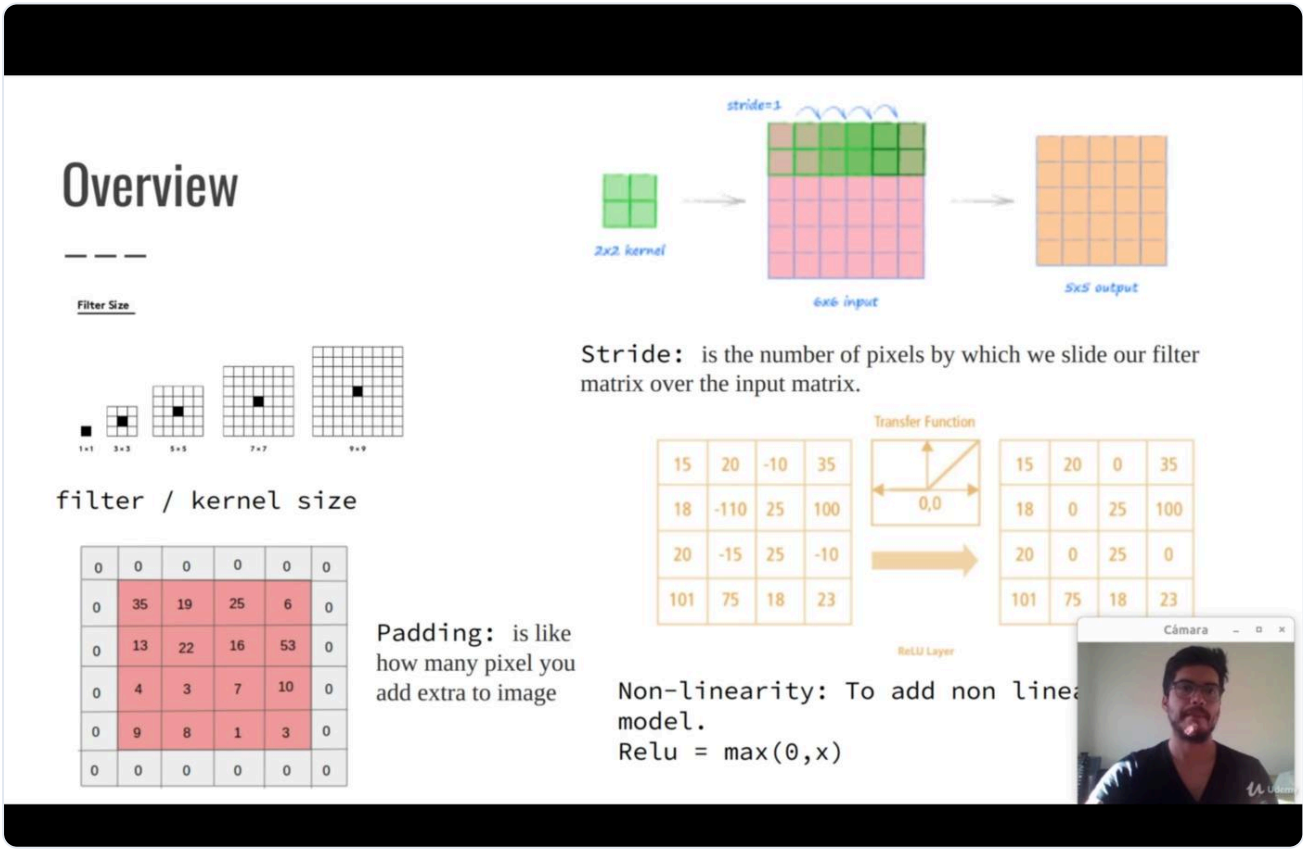



FIGURE 7.31 | Filter Size

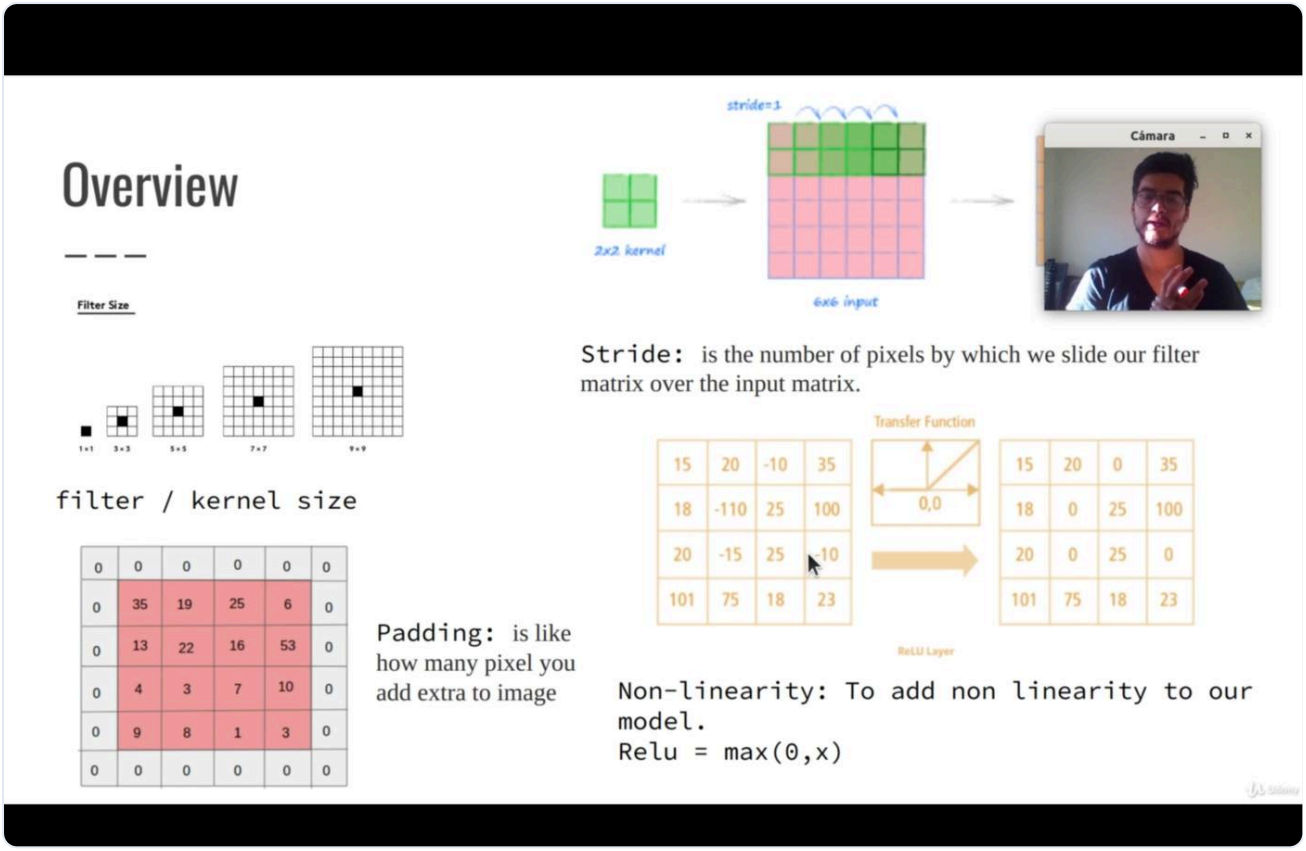



FIGURE 7.32 | Filter Size

OK, so the stride is the number of pixels by which we slide our filter matrix over the input matrix.

OK, so in this example, this is the original kernel, the filter, and this is the first iteration of the convolution. And after the first dot product, we move this filter to here, or here, I'm not sure. But let's suppose that we're moving it like this. OK, so we have a stride of one here. So here is the initial position, and after the first iteration, we're going to move the filter here. So we are moving the filter one pixel per iteration.

OK. So in this example, we have a stride of one, OK. And different strides are going to generate different shapes of the output. OK, and again, this is another parameter you can set when you are training your model. OK, and finally, another very important point here is that we are going to add non-linearity to our model in order to learn complex patterns. OK, so in order to add this non-linearity, we are going to add a kind of activation function. So if this is a feature map, this is the output after the convolution, we're going to have this feature map.

OK, so to this feature map, we're going to apply the nonlinearity function. OK, so for example, this nonlinear function can be ReLU, and this is the max between zero and the number itself. So this nonlinearity function is an element-wise operation. So we're going to apply it element by element; for example, in this example, we are going to apply the ReLU function to each of these numbers. OK, so, for example, if we have this matrix here, we are going to apply the ReLU function to this element.

And this is going to be the maximum between zero and 15. So the output is going to be 15. For this element it is going to be 20. But for this element, the maximum between zero and minus 10 is going to be zero. So we're going to have a zero here, then [unclear], and so on. So, for example, here we have minus 110, and the maximum is going to be zero. OK, so usually we do this to add non-linearity, and we add this after the convolution. So for each feature map we're going to apply the non-linearity function.

OK, and again, this is not a parameter. This is a function that you have to add after the convolution. Instead of adding ReLU, you can use another activation function, but usually when we are talking about convolution, we usually use ReLU.

LECTURE 8

# Overview of Pooling Layer

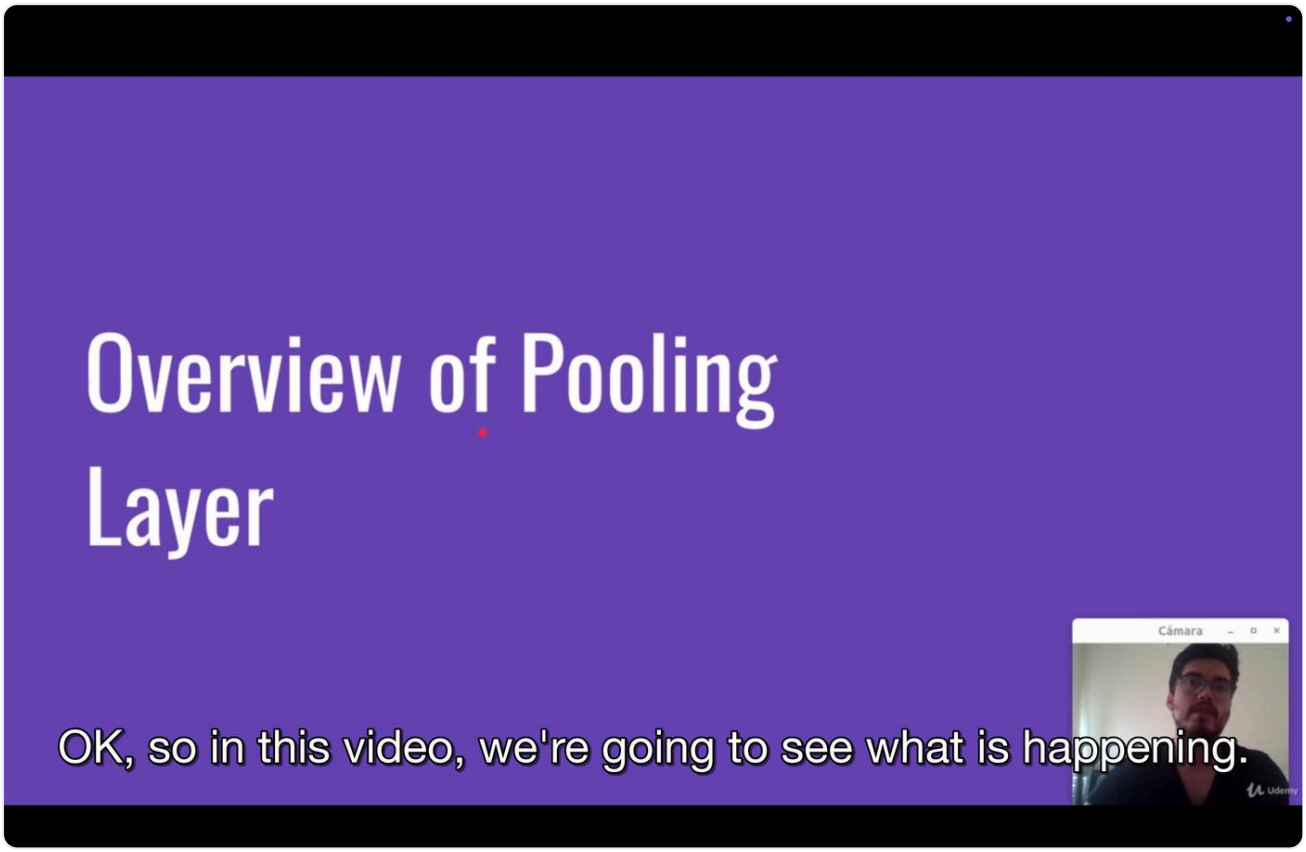



FIGURE 8.1

OK, so in this video, we're going to see what pooling is.

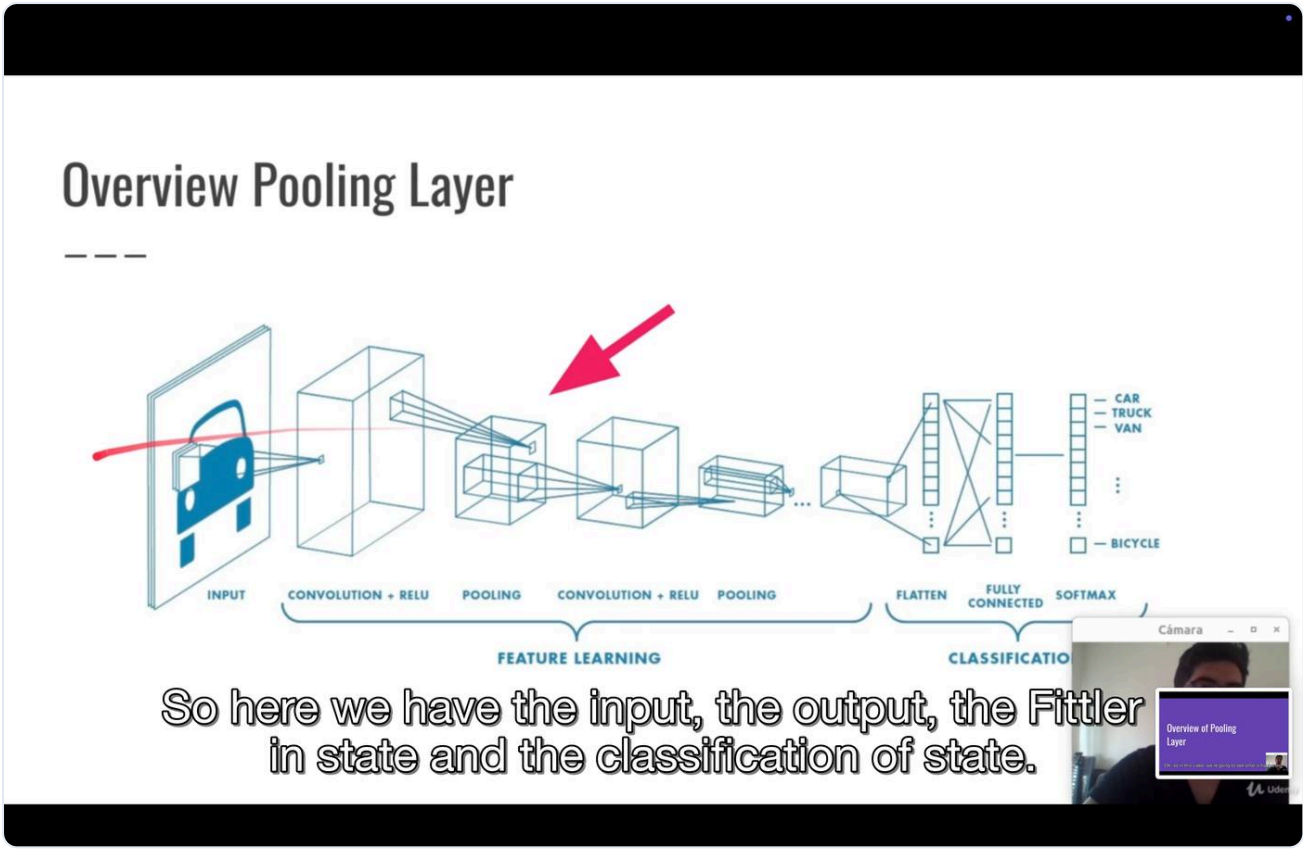



FIGURE 8.2 | Overview Pooling Layer

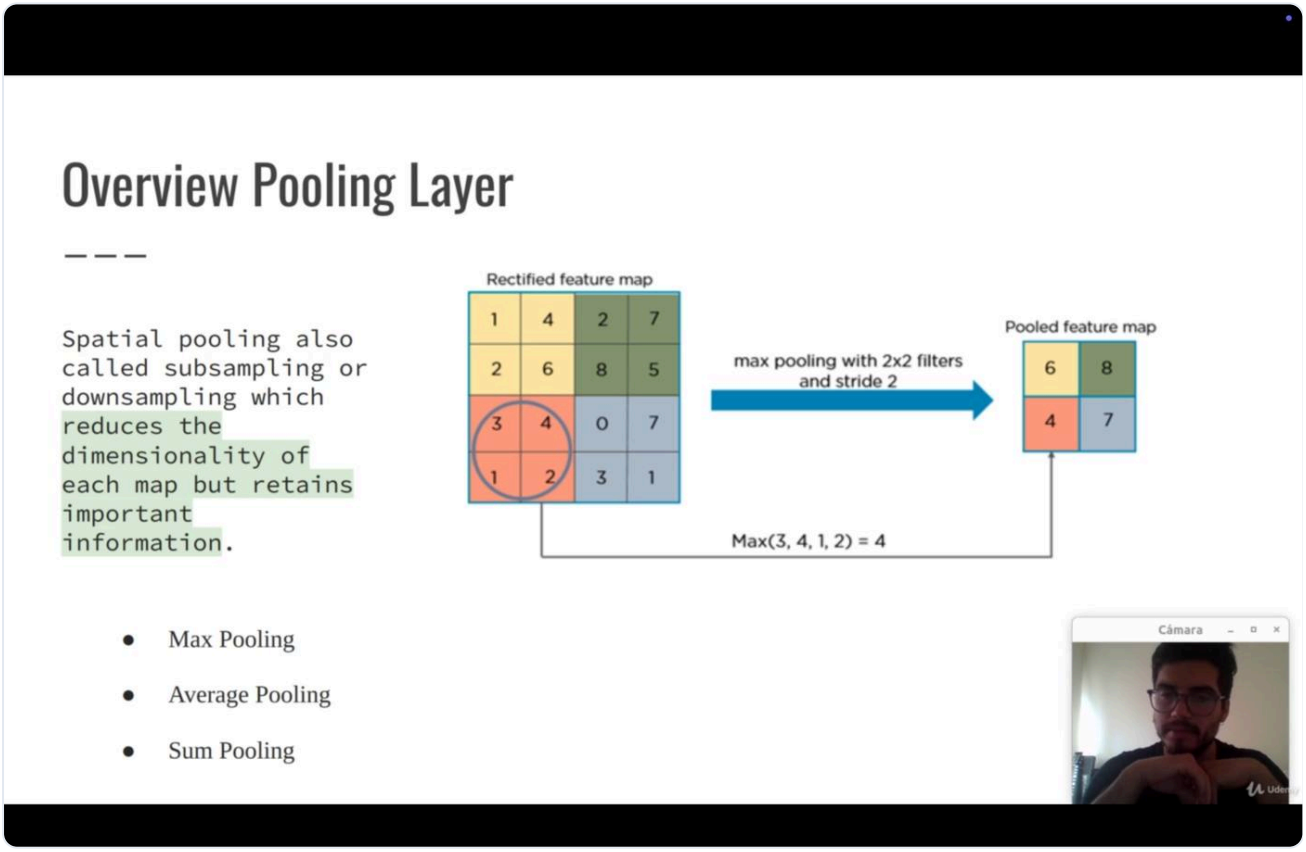



FIGURE 8.3 | Overview Pooling Layer

So here we have the input, the output, the feature learning stage and the classification stage. OK, so the first layer was the convolutional layer. And basically we apply the convolution between the filter and the input. And the outputs of this convolutional layer are going to be the different feature maps. Then to each of these feature maps, we're going to apply the pooling layer. So let's see what this layer is.

Basically, the pooling layer is an operation that we apply in order to reduce the dimensionality of the feature map. OK, so for example, let's say that this is our feature map, after applying the ReLU function or any activation function, and this is the output. OK, so the idea of the pooling layer is that we want to reduce the dimension of the feature map to this, for example. So how can we reduce this dimension? Well, we can apply different operations. The most common approach is max pooling, [unclear], which I'm going to explain. Then we can also apply average pooling or sum pooling.

OK, so in this example, we are applying max pooling. And basically, max pooling is the operation in which first we take a chunk of the feature map. And from this chunk, this area of the feature map, we're going to apply the maximum operation; we're going to get the maximum of this area. OK, so, for example, if we are taking an area of four pixels, one, two, three, four, we're going to take the maximum value of these numbers.

So the first operation is going to be the maximum between one, four, two and six. So the maximum here is going to be six. So this is going to be the number you are going to retain, or you are going to keep. So this is the output of the first operation: six. Then we're going to apply the pooling again between these four different elements, between two, seven, eight and five.

And the maximum is going to be eight. So you are going to keep that eight. Then you are going to apply the pooling again with these four numbers, and this is the notation that we have here. So you are going to apply the maximum between three, four, one and two, and the maximum is going to be four. So you are going to retain that number four. And finally, you are going to apply the maximum between zero, seven, three and one, and the maximum is going to be seven. So you are going to keep that seven here.

OK, so that is the way in which we apply the pooling. And specifically in this example, we're applying max pooling, because for each chunk of this image, we are applying the maximum function. OK, but instead of applying, for example, max pooling, we can apply average pooling. So instead of getting the maximum, we can get the average of these four numbers. Now, for each of these groups of four numbers, the average of these four numbers, or we can apply the sum: one plus four plus two plus six.

And the result is going to be this number here. OK, so in this way, we can reduce the dimensionality of the feature map, but also we're going to keep the important information. So let's come back to our schema. This is the input image. Then we have the convolutional layer. And if we apply the pooling layer, you can see how this big feature map here is reduced to this image here. And this reduction is because we are applying the pooling layer here. So we are reducing the dimension from these to these.

So in that way, we are going to have a smaller amount of data. And finally, that is the goal of pooling: to reduce the dimensionality of the feature maps.

LECTURE 9

# Overview of Fully Connected Layer

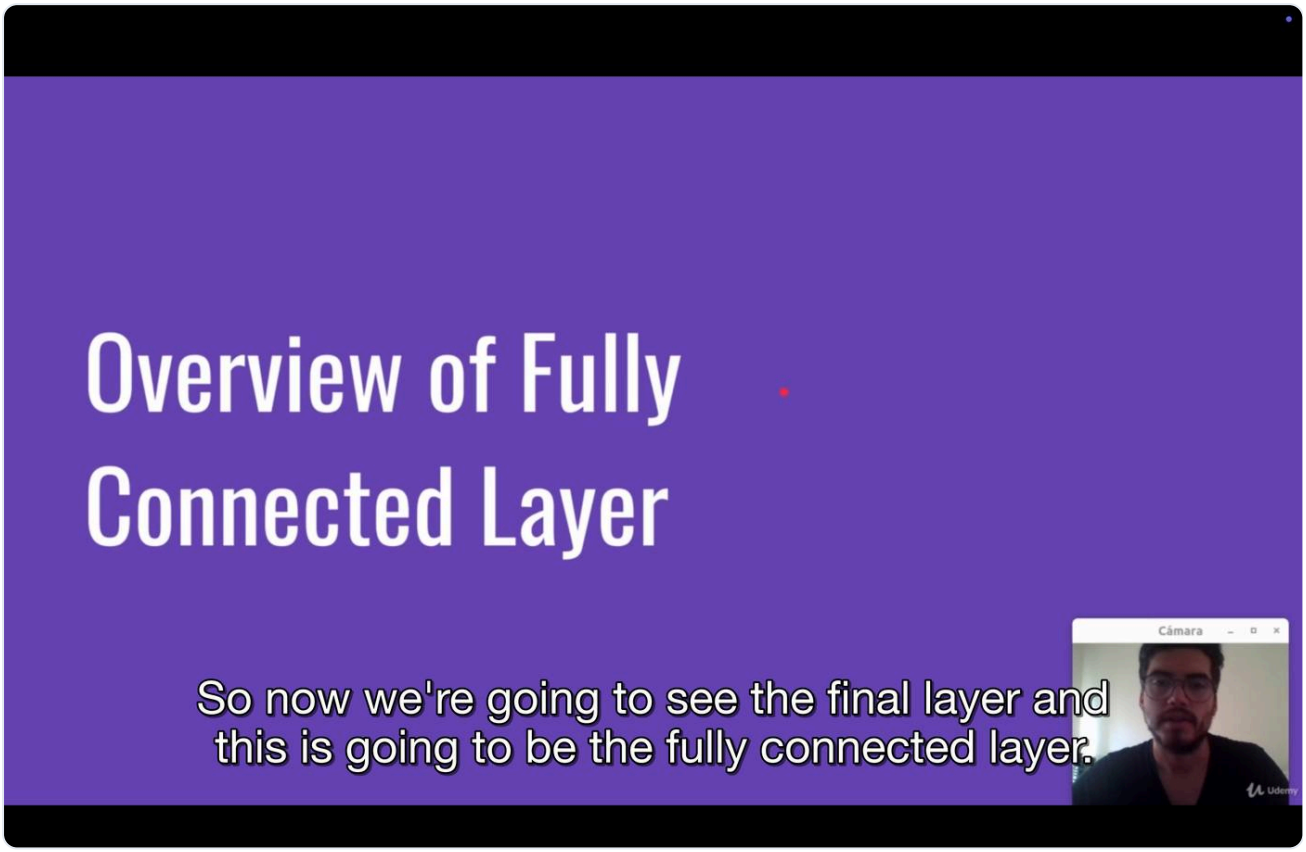



FIGURE 9.1 | Overview of Fully

So now we're going to see the final layer, and this is going to be the fully connected layer.

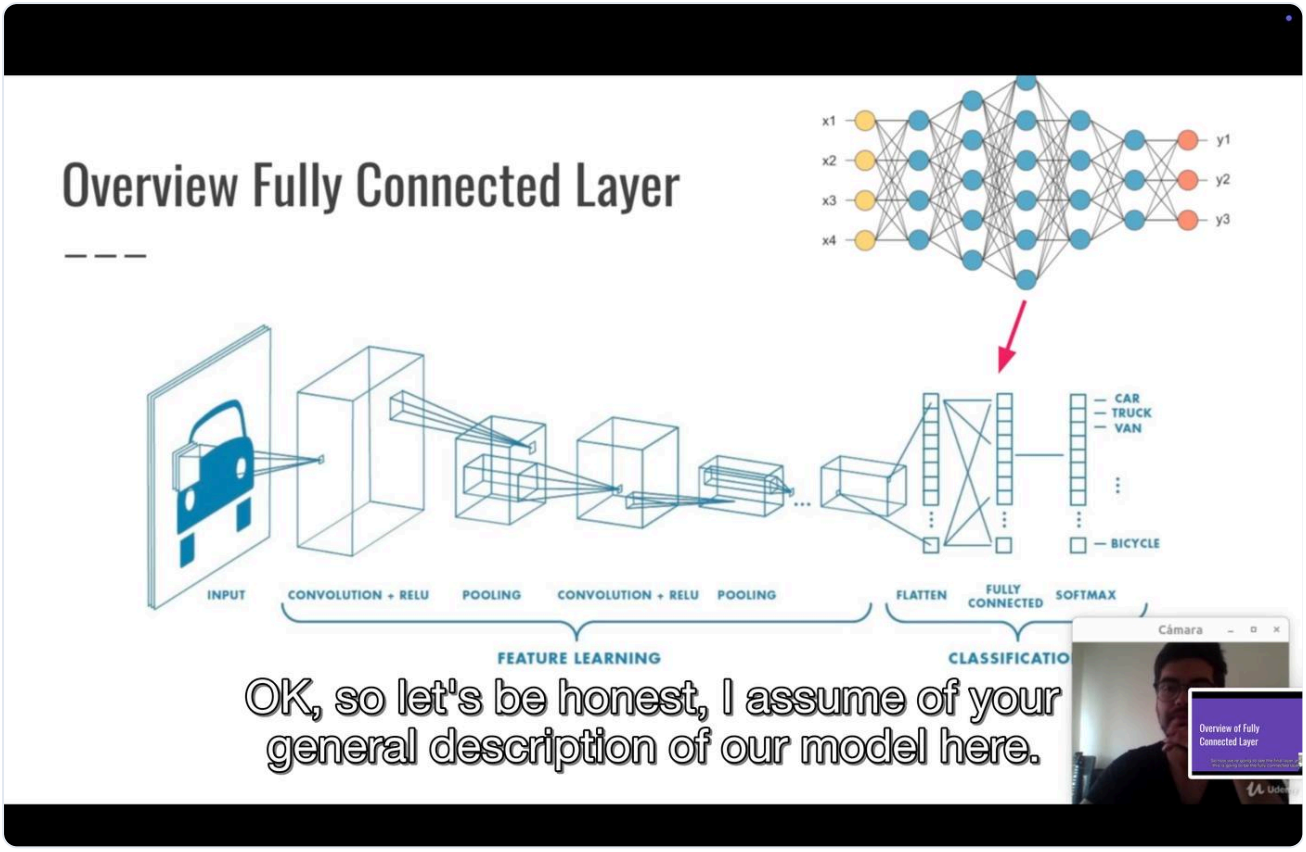



FIGURE 9.2

OK, so let's see again the general description of our model here. We have the input here, and we have the output here. We have the feature learning stage, and then we have the classification stage. OK, so in this example, we have two convolutional layers: convolution one and convolution two. OK, so convolution one is going to apply the convolution between the filters and the input images.

Right. And the outputs of the first convolution are going to be the different feature maps. Right. Then to these feature maps we're going to apply a pooling layer, and we are going to reduce the dimensionality of these feature maps. OK, these reduced feature maps are going to be fed into the second convolutional layer, in which we are going to try to find more features, or more detailed features, in the feature maps.

And then the outputs are going to be new, different feature maps from the second convolutional layer. And to these outputs, these different activation maps, we're going to apply another layer in order to reduce the dimensionality of these feature maps. OK, so the output of the feature learning, in this example, is going to be this. OK, and basically these are the different features that our model automatically learns from this data.

At this point, we have different features that we can use for anything that we want.

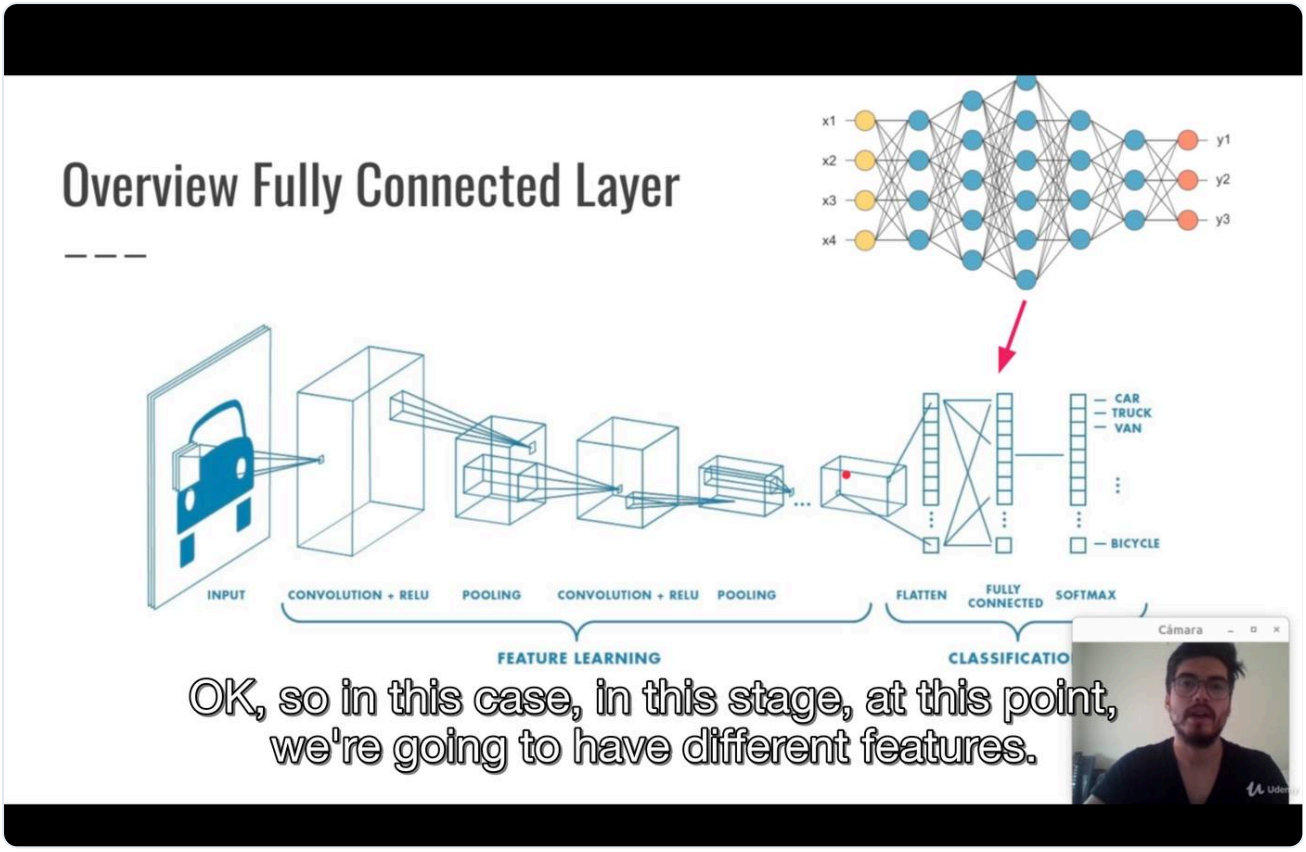



FIGURE 9.3 | Overview Fully Connected Layer

OK, so in this stage, at this point, we're going to have different features. So, for example, we can have different features, like whether it has wheels, lights, or any details that you can find in the images. OK, so in this example, we have features. We only have features. OK, so then in the classification stage we take these different features, and then we try to find different patterns in order to use these features to classify into the different classes.

OK, so the idea of the classification stage is that we are going to use the different features that we learned previously, and we are going to use these features in order to identify how to differentiate the different classes. OK, so for example, if we have a car or a bicycle, we're going to have features that only the car has and features that only the bicycle has. OK, and with these features, we can use these features and their values to identify the different classes.

OK, so again, in this feature learning stage, we are going to have the output, and the outputs are going to be the different features. OK, so for example, this can be the number of wheels, the color, I don't know, but we're going to have different features like this, which are going to be very, very abstract, and usually we're not going to understand them. But the idea is that we're going to have features that describe the original data.

OK. And in the classification stage, we're going to take these different features, and we're going to use them in order to find patterns that allow us to differentiate between the different classes. OK, how can we do this?

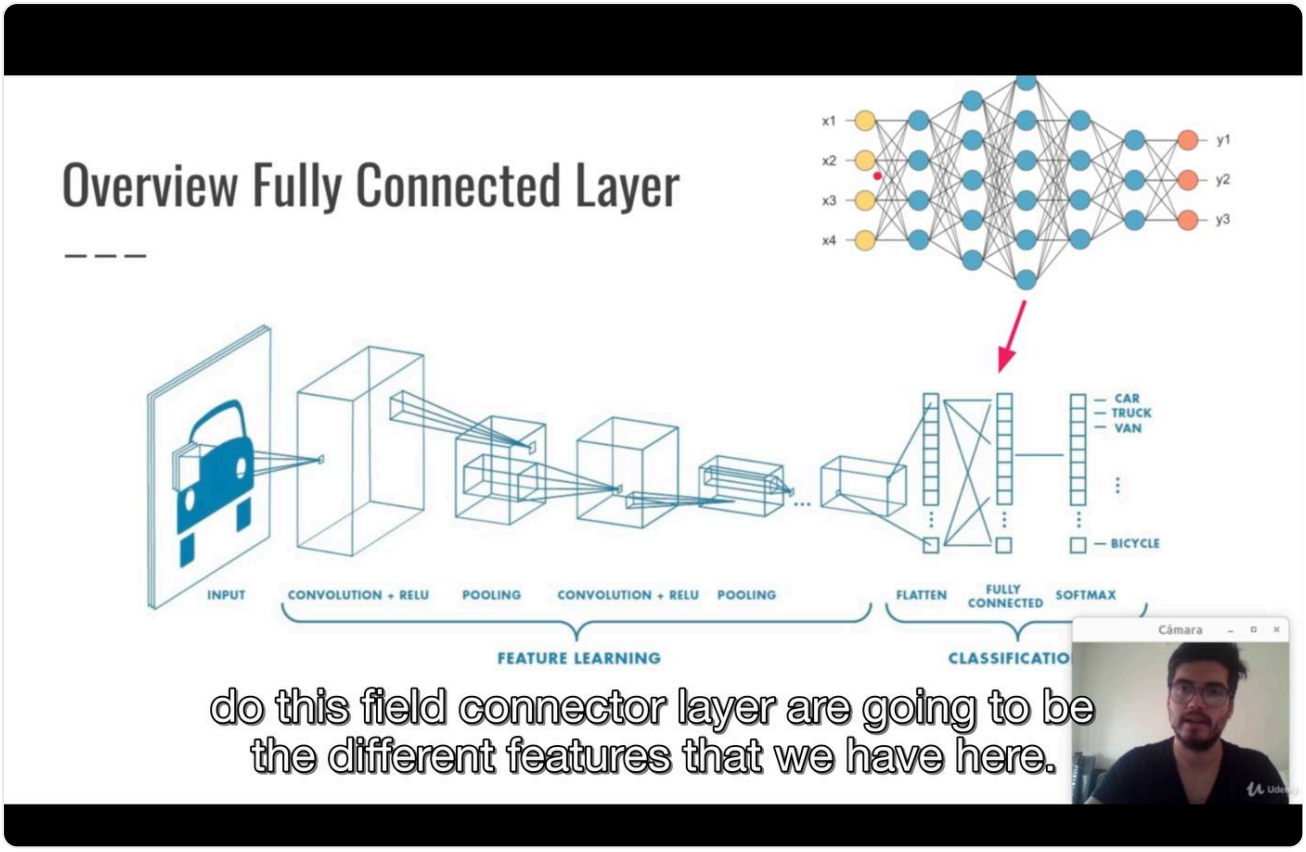



FIGURE 9.4 | Overview Fully Connected Layer

Well, usually we are going to use a fully connected layer, in which, for example, the inputs to this fully connected layer are going to be the different features that we have here. And then we're going to apply different hidden layers here.

And the output is going to be the different classes that we are working with. So, for example, in this example, each of the outputs should be each of these classes: car, truck, van, bicycle and all the other classes. OK, but basically, again, the inputs are going to be the different features that we learned previously, and the outputs are going to be the different classes. OK.

LECTURE 10

# Typical Convolutional Neural Network architecture

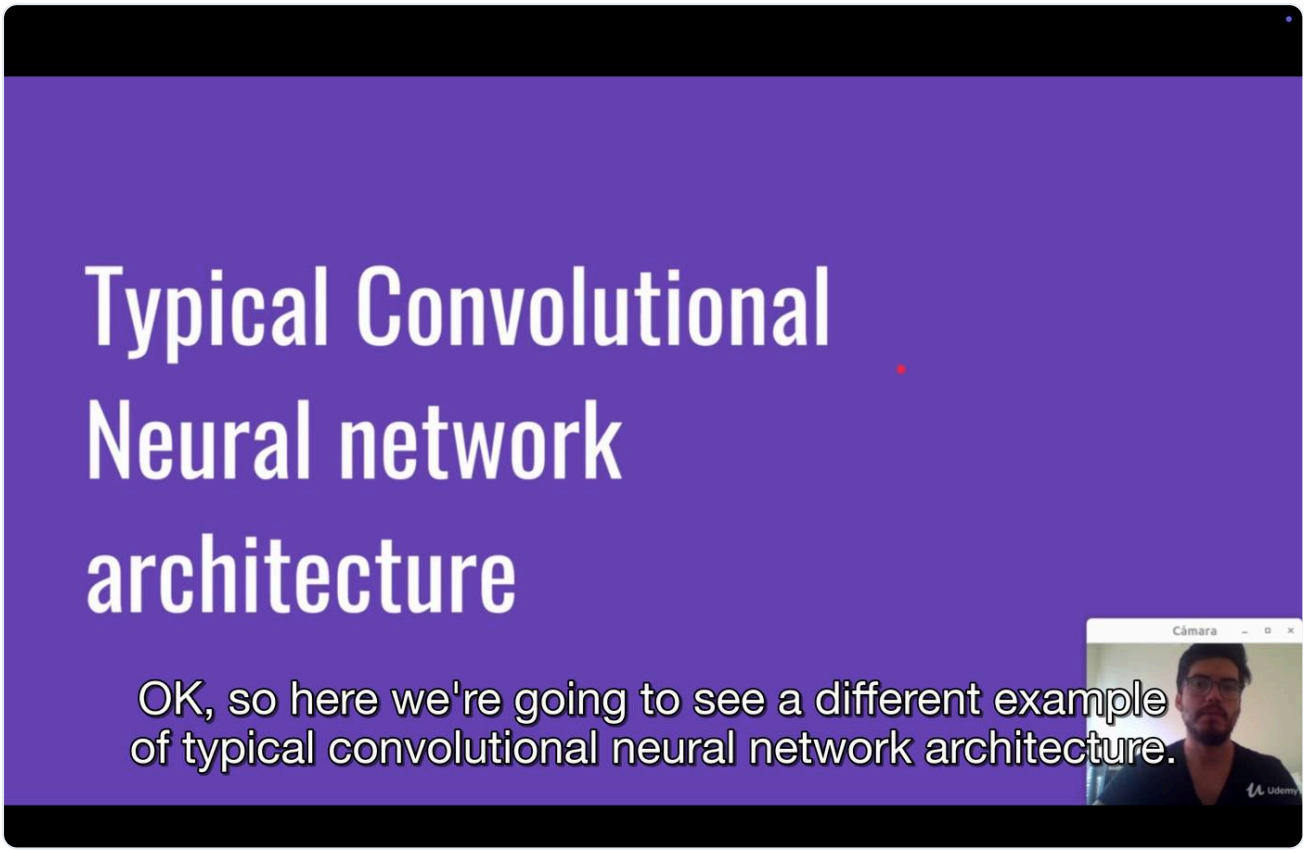



FIGURE 10.1

OK, so here we're going to see different examples of typical convolutional neural network architectures.

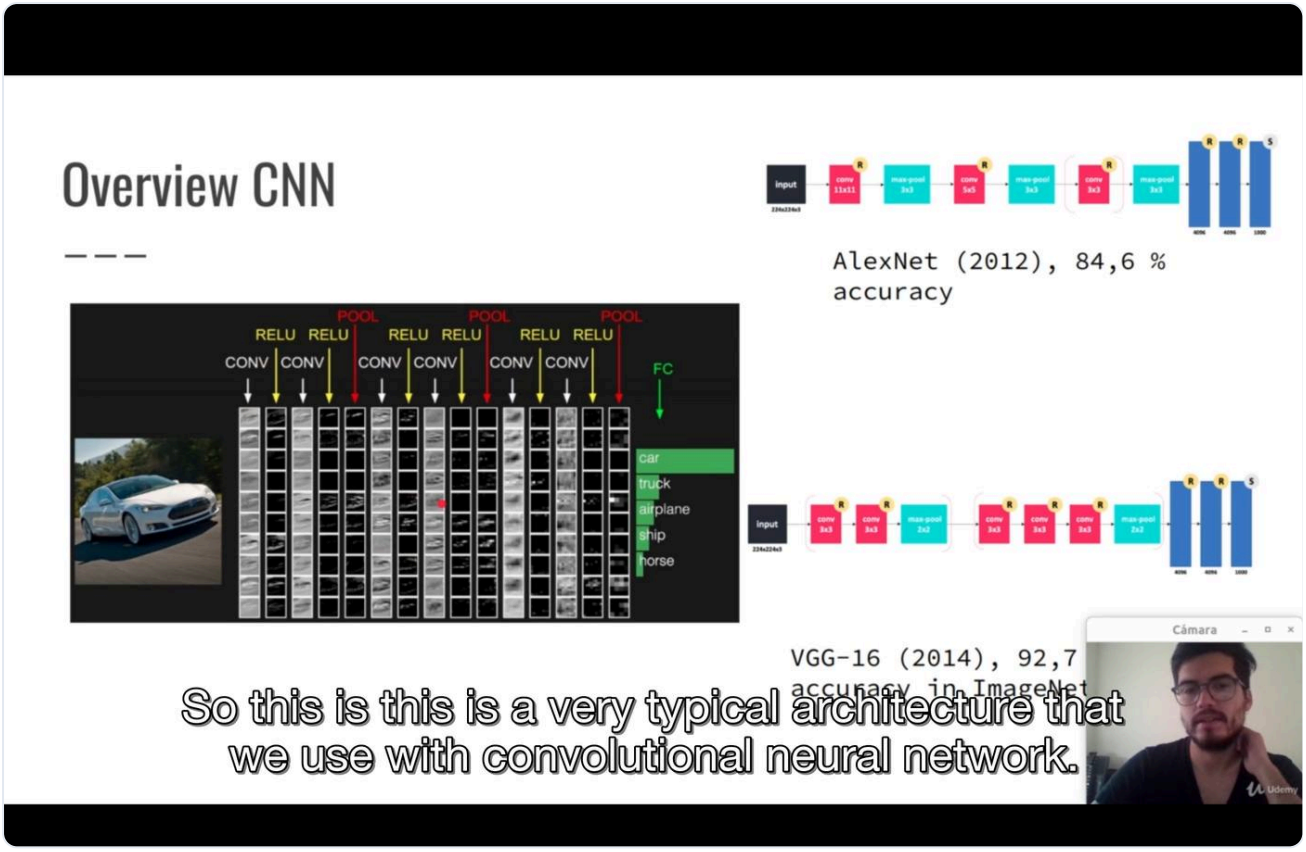



FIGURE 10.2

So this is a very typical architecture that we use with convolutional neural networks. So in this example, we are classifying different classes, in this example car, truck, airplane, ship, horse, and we are feeding different images. OK, so as you can see here, we have a set of different layers: convolutional, convolutional, pooling, convolutional, convolutional, pooling, and so on.

OK, so in this example, we have one, two, three, four, five, six, six different convolutional layers. And each of these layers has the ReLU function, the activation function. And after two convolutions, we have the pooling layer in order to reduce the dimensionality. So as you can see here, we can be very flexible with the architecture. And the idea here is that you have to find the correct, or the appropriate, architecture for your data. OK, so as you can see here, the different convolutions are getting different results.

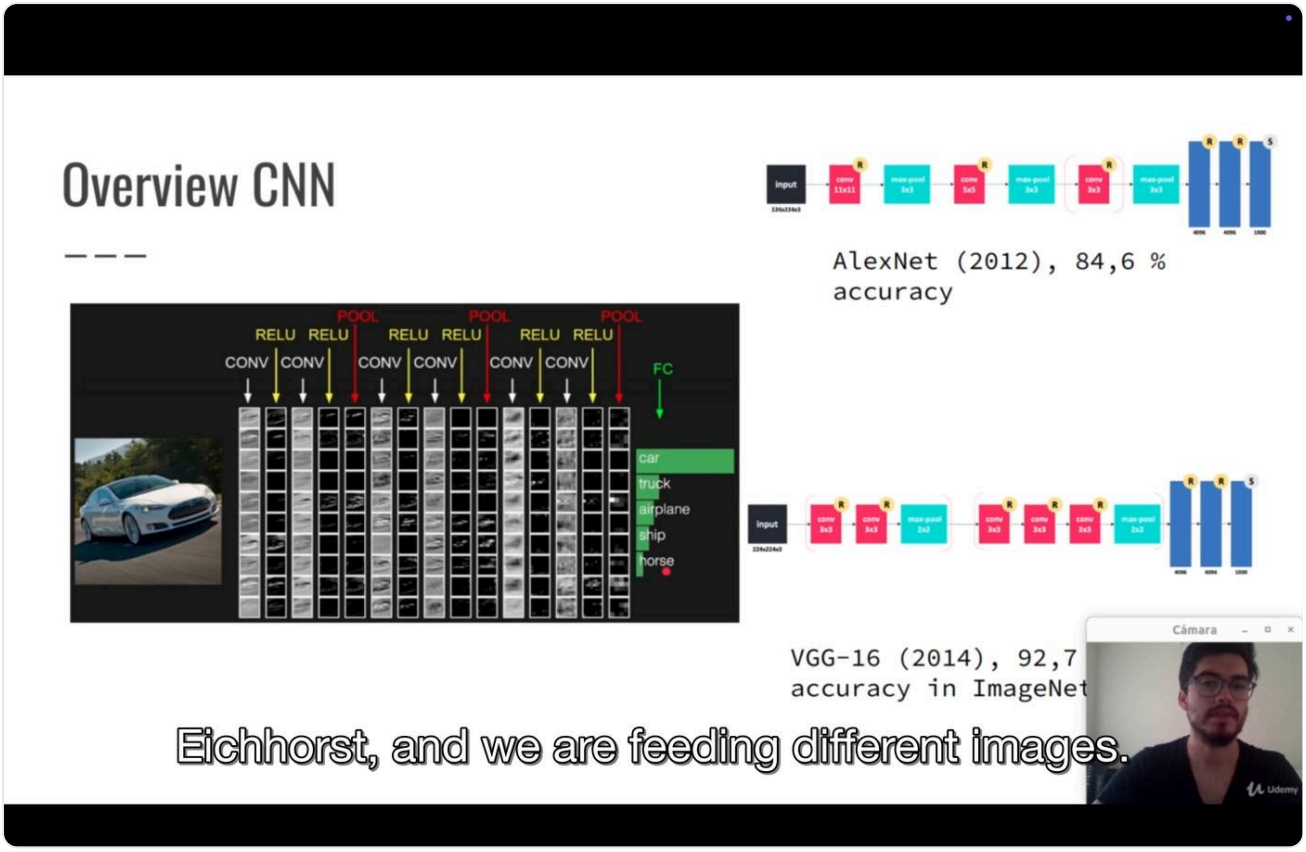



FIGURE 10.3

So, for example, these results, these different feature maps, are totally different from these features, because the functions of the different convolution layers are going to be different. OK, so this is the feature learning stage, and in the final stage here, we're going to have the different features. Right. And then this specific model is going to take these different features that we learned previously.

And we're going to apply a fully connected layer in order to try to find the patterns to identify the different classes. OK, so in this part, we're going to take the different features, and then we're going to use these different features in order to classify the different classes. OK, here we have another example. This is the AlexNet model.

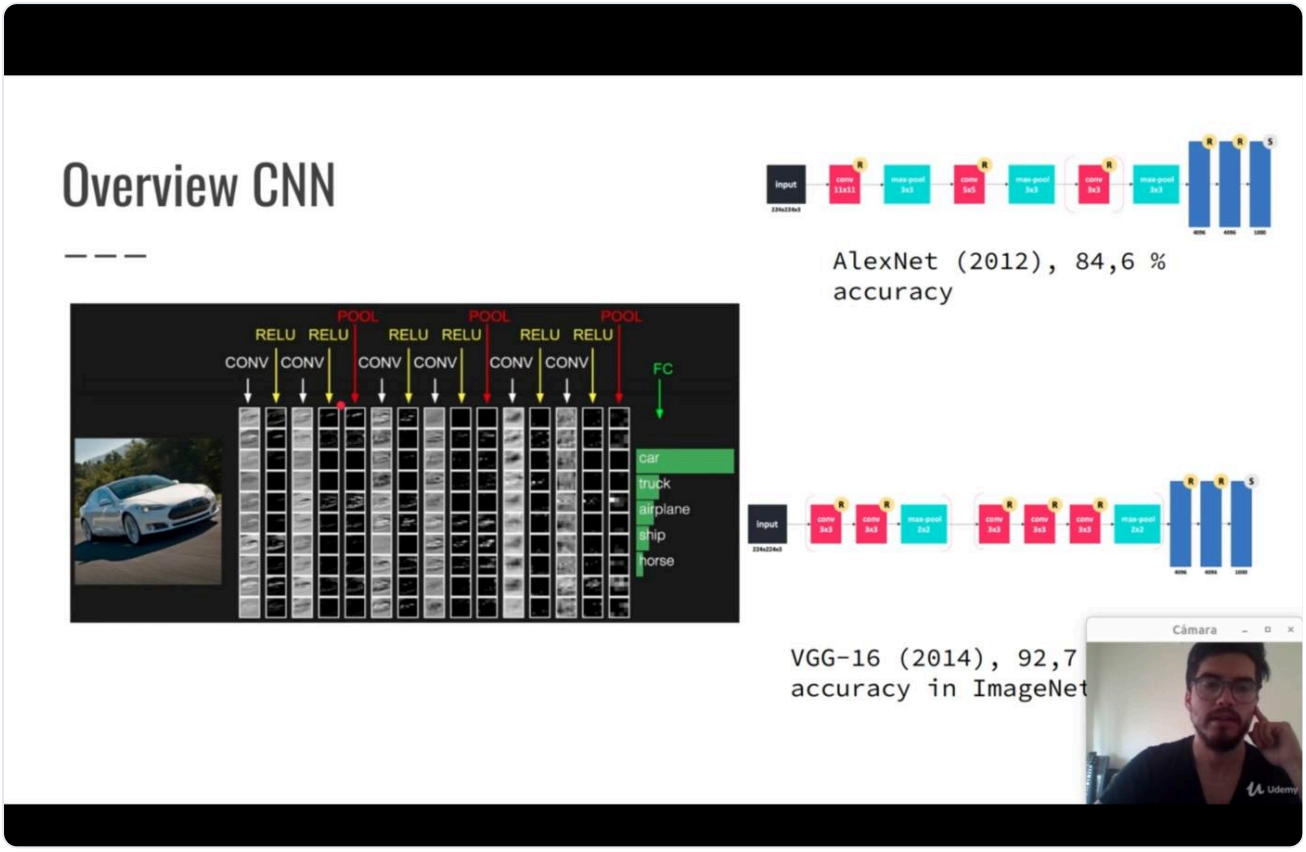



FIGURE 10.4

This was presented in 2012. And this is a very famous model, because it got a very good accuracy level; this is 85 percent.

This was really good for that time, for that year. And basically, the architecture was this. We had different images here of height 224, with the same width, and three different channels, probably for the R, G and B channels. And then we apply a different set of convolutions and pooling layers. In this example, we're applying max pooling layers. And so here we have the output of the first layer. Then we have another layer of convolution and max pooling, and finally we have the fully connected layers.

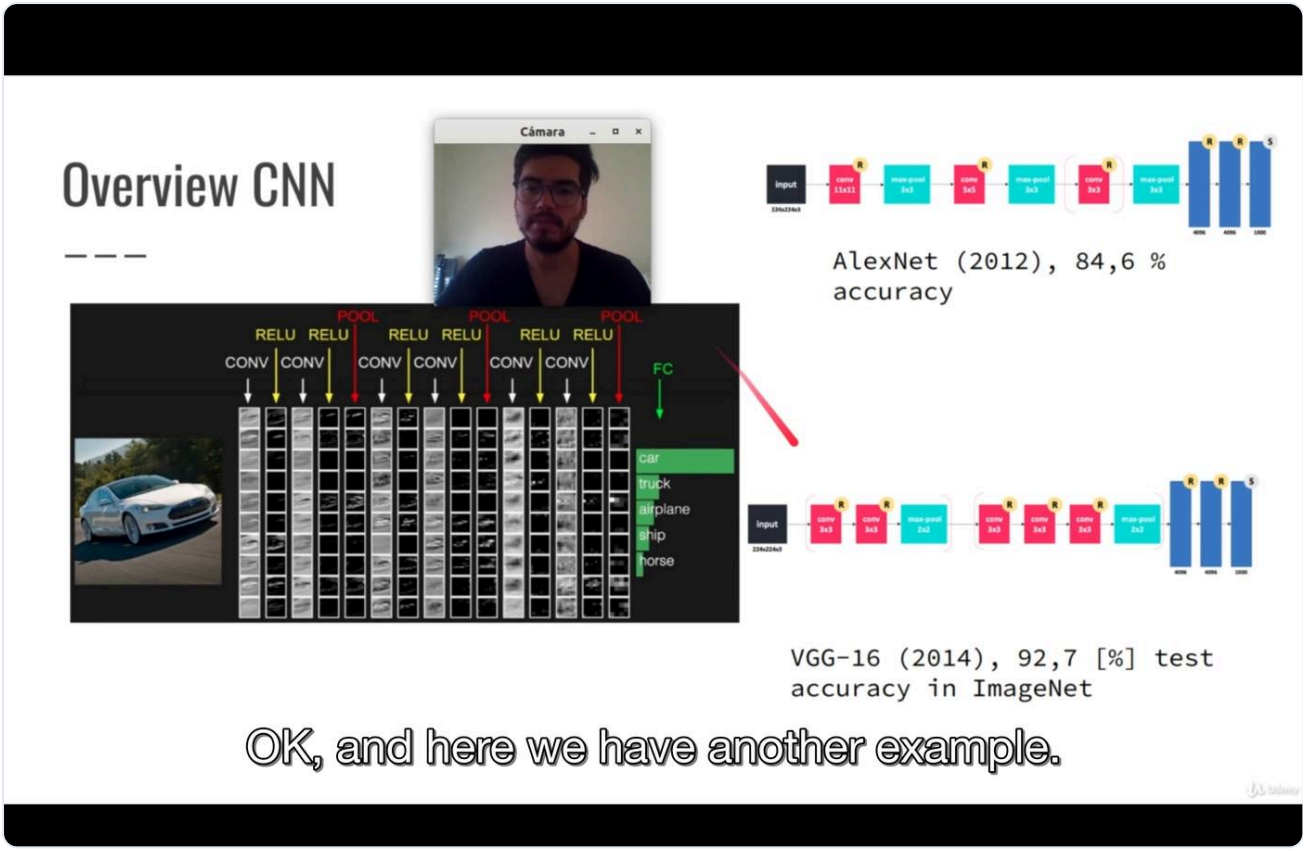



FIGURE 10.5

OK, and here we have another example. This is VGG16, and this is later than AlexNet.

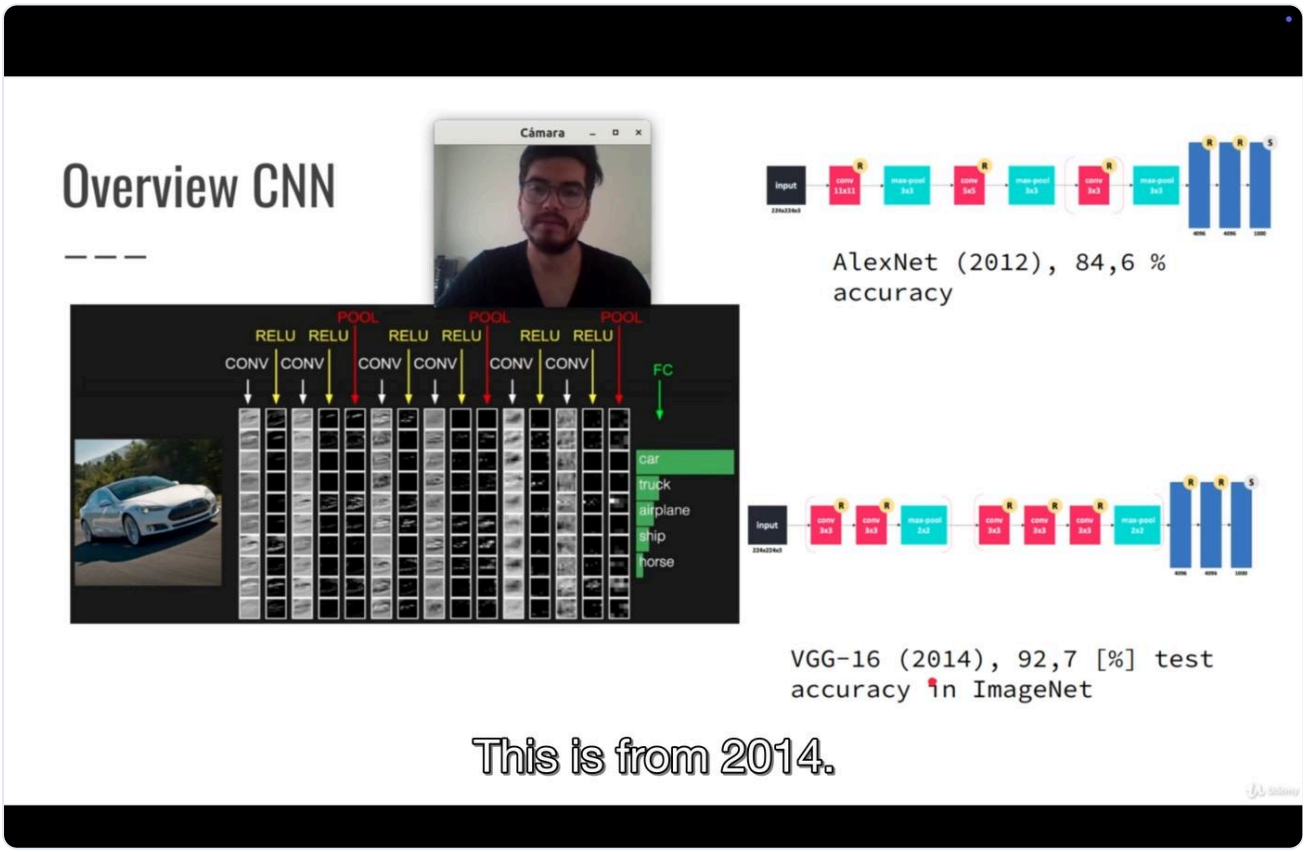



FIGURE 10.6

This is from 2014. And this got 92 percent accuracy on a specific dataset. This was ImageNet, and you can see the different architecture here. Here we have the input, and then we have a set of different convolutions, convolution, and then we have the pooling. Then we have another layer here, and this is convolution, convolution, convolution and then the pooling.

As you can see, this is different from here. And then we have the different fully connected layers. OK, actually these models are really complex, and this is really a high-level abstraction of the model, because it has more and more details. But this is a general overview of this model. And as you can see here, the different models have different options for the convolutions and the pooling, and how you put the convolutions: if you have one or two or three convolutions, and then the max pooling or the average pooling, and so on.

So the architectures of these networks, again, are totally flexible. And this totally depends on the dataset that you are working with. OK, so the big challenge here is to try to find the correct model for your specific data.

LECTURE 11

# Mathematical view of training a Neural Network

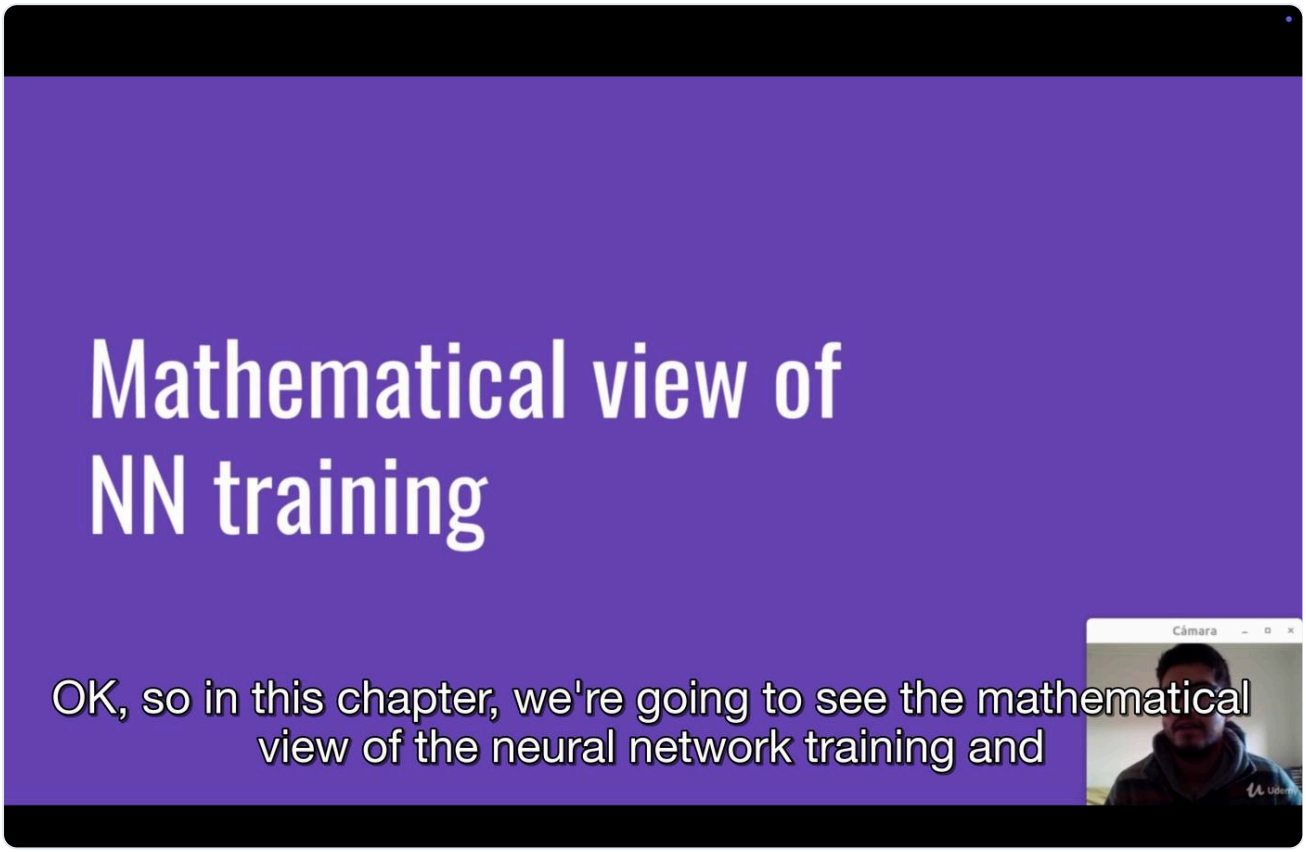



FIGURE 11.1

OK, so in this chapter, we're going to see the mathematical view of neural network training and learning, and this chapter is going to be a little bit difficult because it's going to have a lot of math, OK, but in a very general way. But I think it is necessary if you want to understand neural networks more deeply. OK, so let's begin. We're going to have our model, our fully connected model, and we have the input layer, the hidden layers and the output layer.

Right. And as we were talking about previously, the neural network is finally going to be a function, right? Because we are going to have the activation function in the output layer. And this activation function is going to be the activation function of C8, and C8, for example, of this neuron eight, is going to be a function of neurons seven, six, five and so on. Right. So basically the equation of this neural network is going to be a series of nonlinear and linear functions.

If we take this function and we apply the C formula, for example for this neuron eight, we're going to have that this C is going to be the sum of the different connections multiplied by the parameters, right, the weights, and the bias terms. OK, so finally, the function of this neural network is going to be a function of nonlinear functions plus linear functions. And this function is going to have these parameters. Right. So at the end of the day, we can see that the function of our neural network is going to be a function that depends on the parameters W and B: this is the weight of the connections, and this is the bias term of each neuron.

Right. So basically what this is saying is that if we change the parameters, for example the weights, the function is going to be different. So for different values of weights and biases, we're going to have different values of the function, and basically we're going to have different functions. OK, so if we change the values, the function is going to change, OK? So the next step is to have a measurement of how our model is doing.

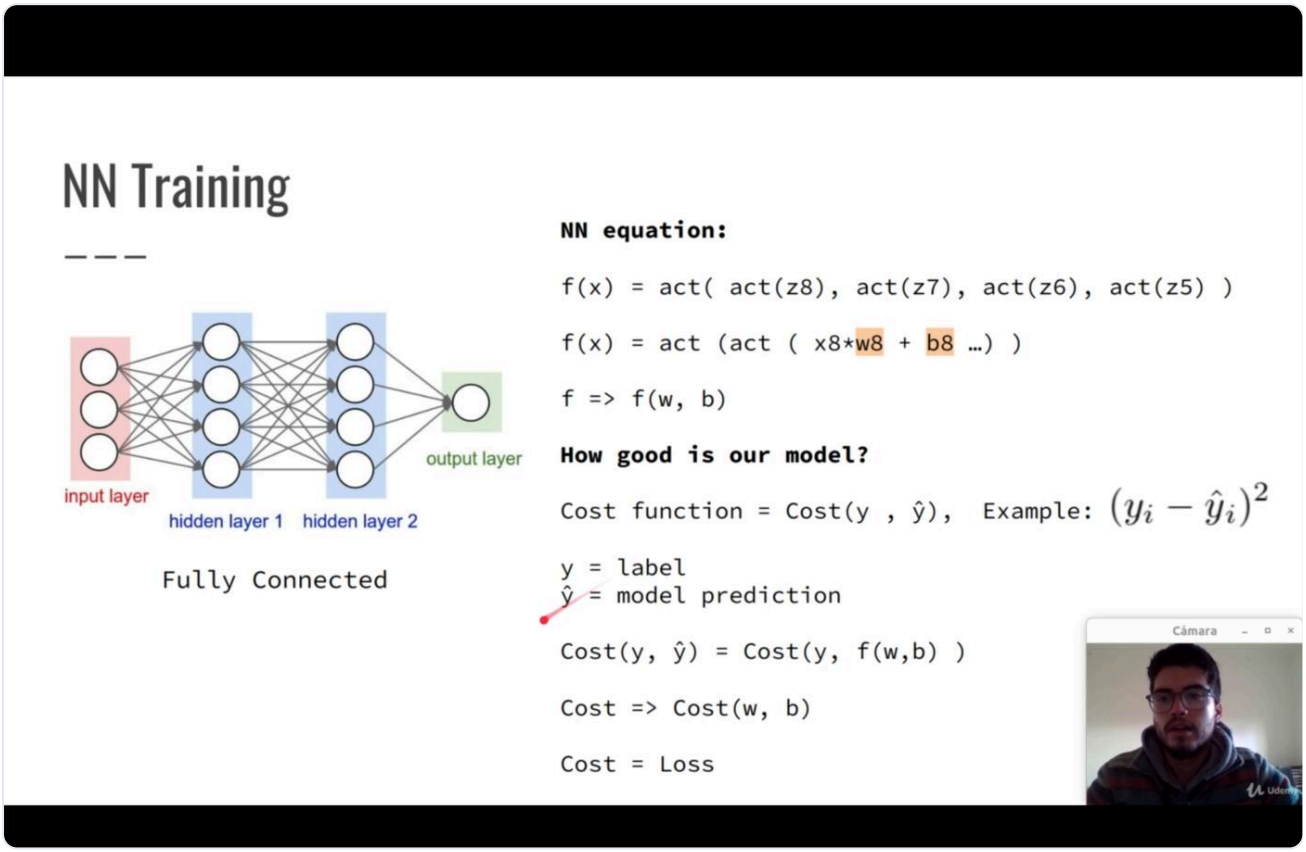



FIGURE 11.2

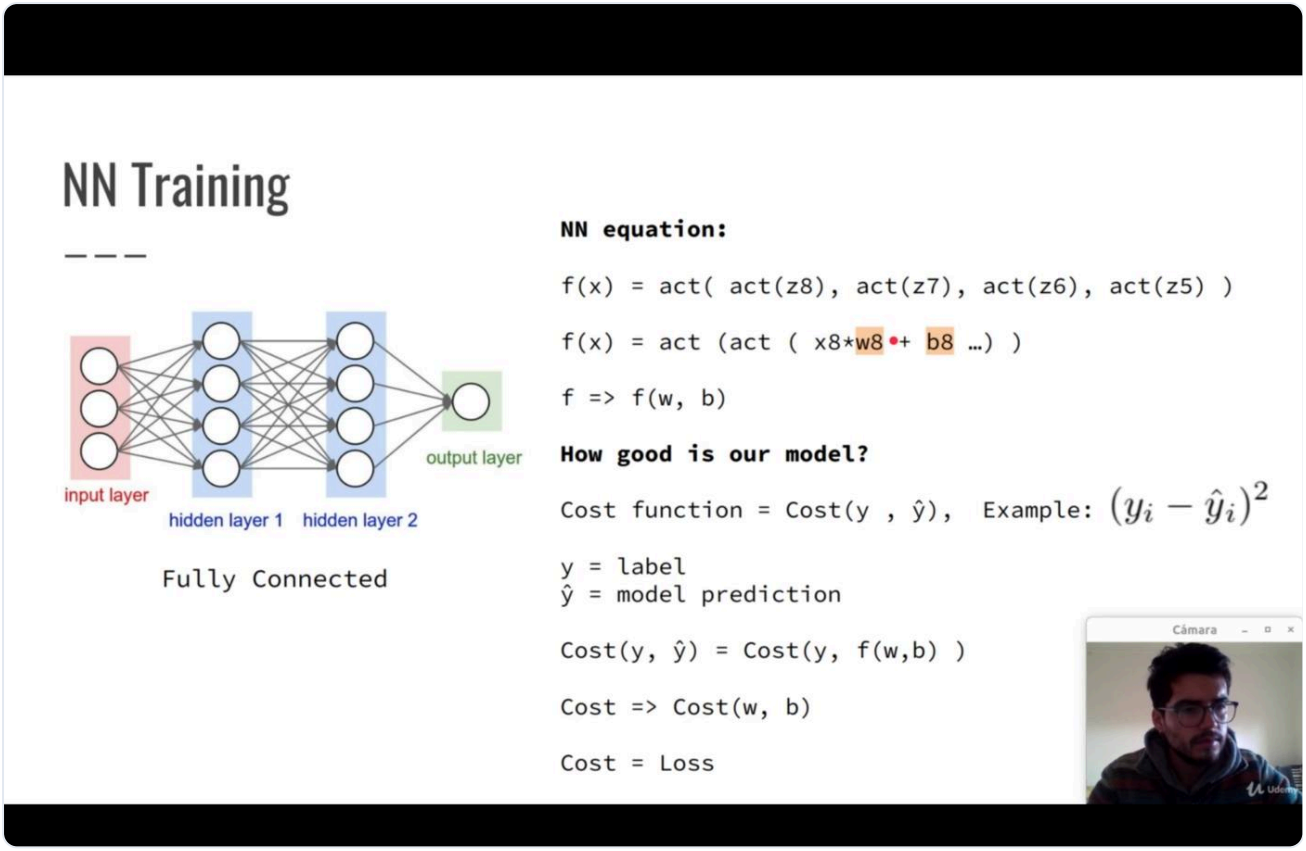



FIGURE 11.3

How good is our model? OK, so in a neural network, we use the cost function, and the cost function is basically a function that tells us

how good our model is. OK, so this cost function, sometimes called the loss function, is a function of Y, and Y is the label, basically the real label. For example, if we have the handwritten number of a nine, the label is going to be nine. OK, so the cost function is going to be a function of the labels, but also of the model predictions, and this is Y hat. Right. So this cost function is going to be a function of the labels and predictions, and this cost function is a function that is going to tell us if the labels are similar to the model predictions.

OK, so this is going to measure if the model is predicting the same labels, OK, the real labels. OK, so if we have a big cost function value, our model predictions are going to have different values from the labels, the real labels. OK, so the more mistakes our model has, the greater the cost function is going to be, and the smaller the number of errors that our model has, the smaller the value of the cost function is going to be, OK.

So basically a cost function is measuring the difference between the predictions and labels. OK, so the bigger, the more errors; the smaller, the fewer errors. OK, so for example, a cost function can be this simple function here, in which we have the label and the prediction for each of the data samples in our dataset. So we take the difference between the real label, excuse me, the real label and the prediction; we take the difference, and then we apply a square.

So then we can add the differences for all the different samples, and that's it. We take the sum of that, and finally we get a number. So, for example, we can have a cost function value of ten. OK, but basically the cost function is measuring the difference between the model predictions and

the real labels. OK, so the cost function is this function here, and in the cost function the label is Y. Right. But the model prediction is our model, our neural network in this example.

Right. And this neural network is going to be a function of the parameters W and B, so we can replace the Y hat, the model prediction, by this function. OK, so at the end of the day, our cost function is also going to be a function of the parameters, the weights and biases. And this means that if we change the parameters of the network, we can change the cost function. So for example, if we set the weights and the bias terms to specific values, we can have a big value of the cost function, or if we change to other values of the weights and bias values, we can have a smaller value of the cost function.

So, for different parameter values, we can have a big cost function or a smaller cost function. So the idea is that the cost function is measuring how good our model is. OK, so the greater the value of the cost function, the more errors our model has, right? So at the end of the day, we want to minimize the cost function. We want to get the minimum value of the cost function, because that means that our model is getting the same labels as the real labels, or, excuse me, the same predictions as the real labels.

OK, so we can express that in a mathematical way. And if you remember from math, if you want to minimize a function, you can take the derivative of that function and set it equal to zero. Right. So this is the thing that we are doing here: we're taking the derivative of the cost function with respect to the parameters, and we are setting this equal to zero. OK, so why can we do this? Well, it is because the cost function is a function of the parameters, in this example the weights and biases.

OK, so this can be easy if we can get an analytical function of this derivative. OK, but it turns out that the cost function is actually very complicated, because, first, the cost function is complicated, and second, the equation for the model, for the neural network, is sometimes really hard, and usually is really hard. So getting the analytical value of the derivative is really hard.

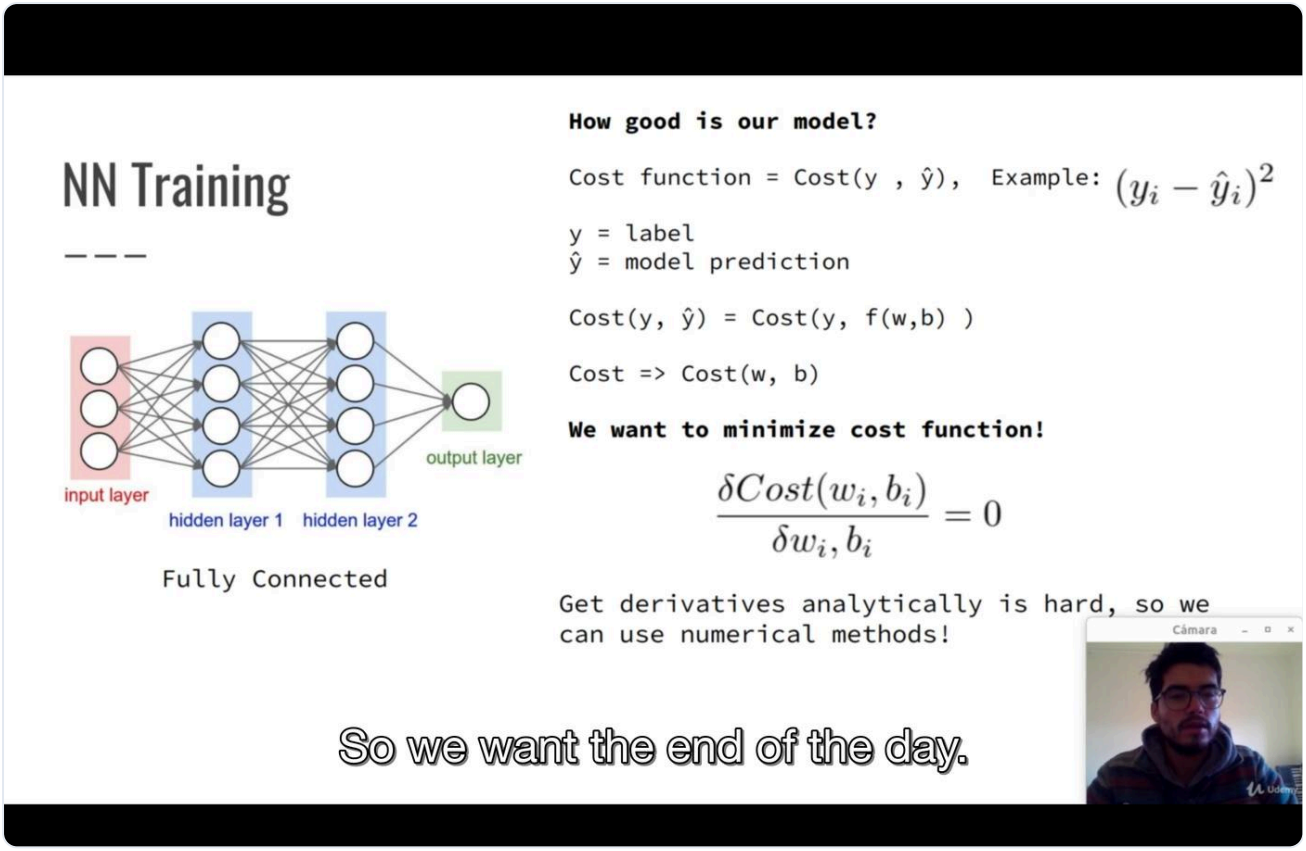



FIGURE 11.4 | How Good Is Our Model?

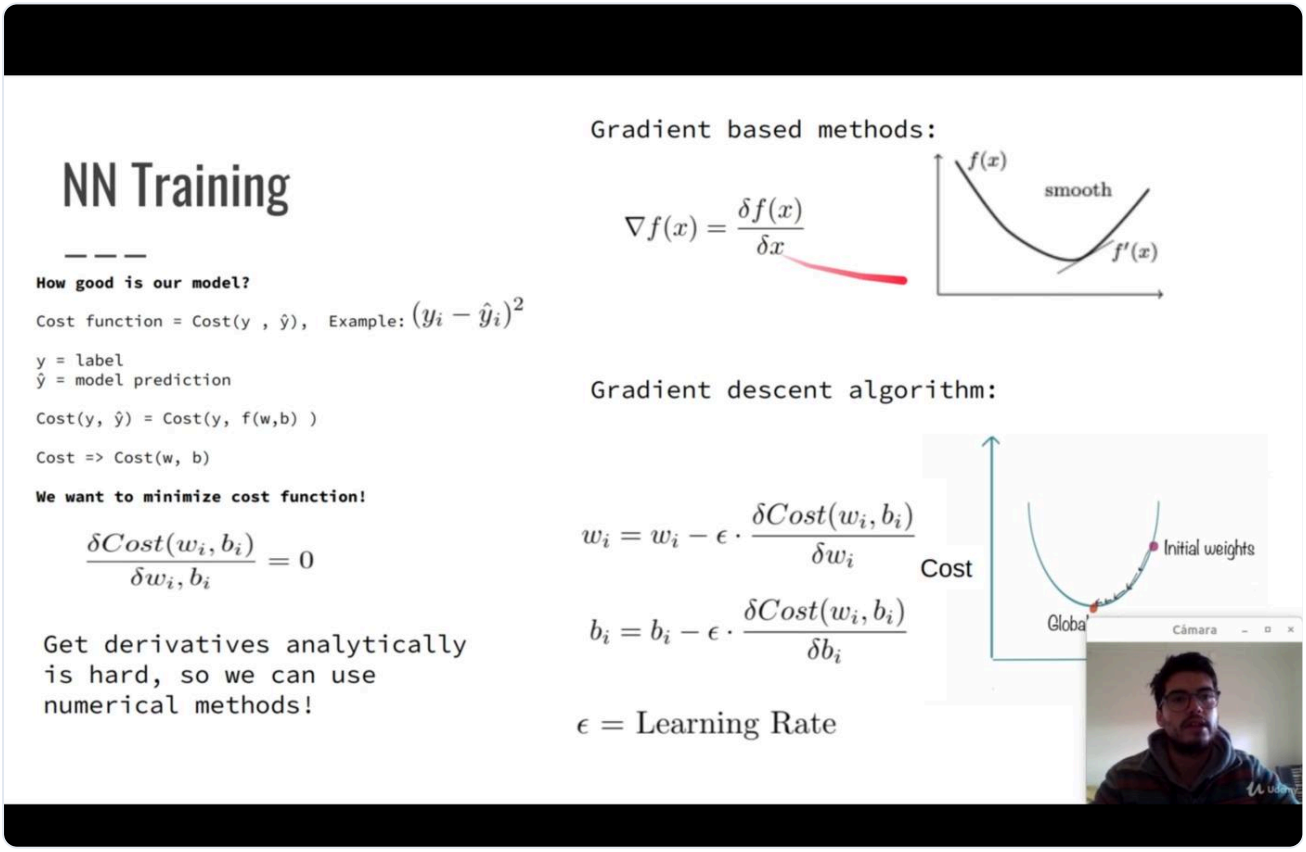



FIGURE 11.5 | How Good Is Our Model?

So instead of using analytical methods, we can use numerical methods. OK, so one very popular type of numerical method is the gradient-based method.

OK, so first of all, we have to remember what the gradient is. OK, so basically the gradient of a function is the derivative of the function with respect to the variable, and geometrically, as we see in this plot, the gradient of a function is the derivative at any point. So for example, if we take the gradient of the function at this point, this is going to be the gradient. Right. And the gradient is positive here. So this is saying that if we take the gradient, the derivative, of this function, at this point it is going to be positive.

And that means that at this point the function is increasing its value. Right. On the other hand, if we take the gradient at this point, this is going to be negative. Right. And that means that our function is decreasing at that point. OK, so basically the gradient is telling us if the function is increasing or decreasing. Right. So, based on this concept, we have different, excuse me, we have different algorithms that allow us to minimize a function using the gradient.

OK, so one of the most famous methods to find the minimum value of a function is the gradient descent algorithm. OK, so based on the idea that we want to minimize the cost function, we can use the gradient descent algorithm, and the output of this algorithm is going to be these two formulas. OK, so basically what this is saying is that we can minimize, or we can find the minimum value of, the function if we use this formula for the different parameters.

OK, so what does this mean? This means that the parameter, in this example W_i, is going to be the value of W_i minus epsilon, and epsilon is the learning rate, and the learning rate is basically a number. This can be 0.1, 0.2, any number. OK, but usually you are going to use small numbers. OK, so you can have the weight value equal to the weight value minus epsilon multiplied by this term. And this term is the gradient, the derivative of the function.

But this gradient is with respect to the specific parameter, W in this example. OK, so this method, using this formula, basically means that we are minimizing this cost function. OK, so if we use this formula, we're ensuring that we're minimizing this cost function, and the same formula is used for the bias. OK, so how does this work graphically? So the idea is that we have this cost function here. This is the cost function, this curve here, and here we have the different values of the weight parameter.

OK, this weight. OK, so the idea is that we kind of start here, at any random number, the initial weight. And the idea is that we want to know in which direction our cost function is decreasing. OK, so in order to know that, we can use the gradient. So that's the reason why we have this equation here, because this gradient is saying that in this direction we are decreasing the value.

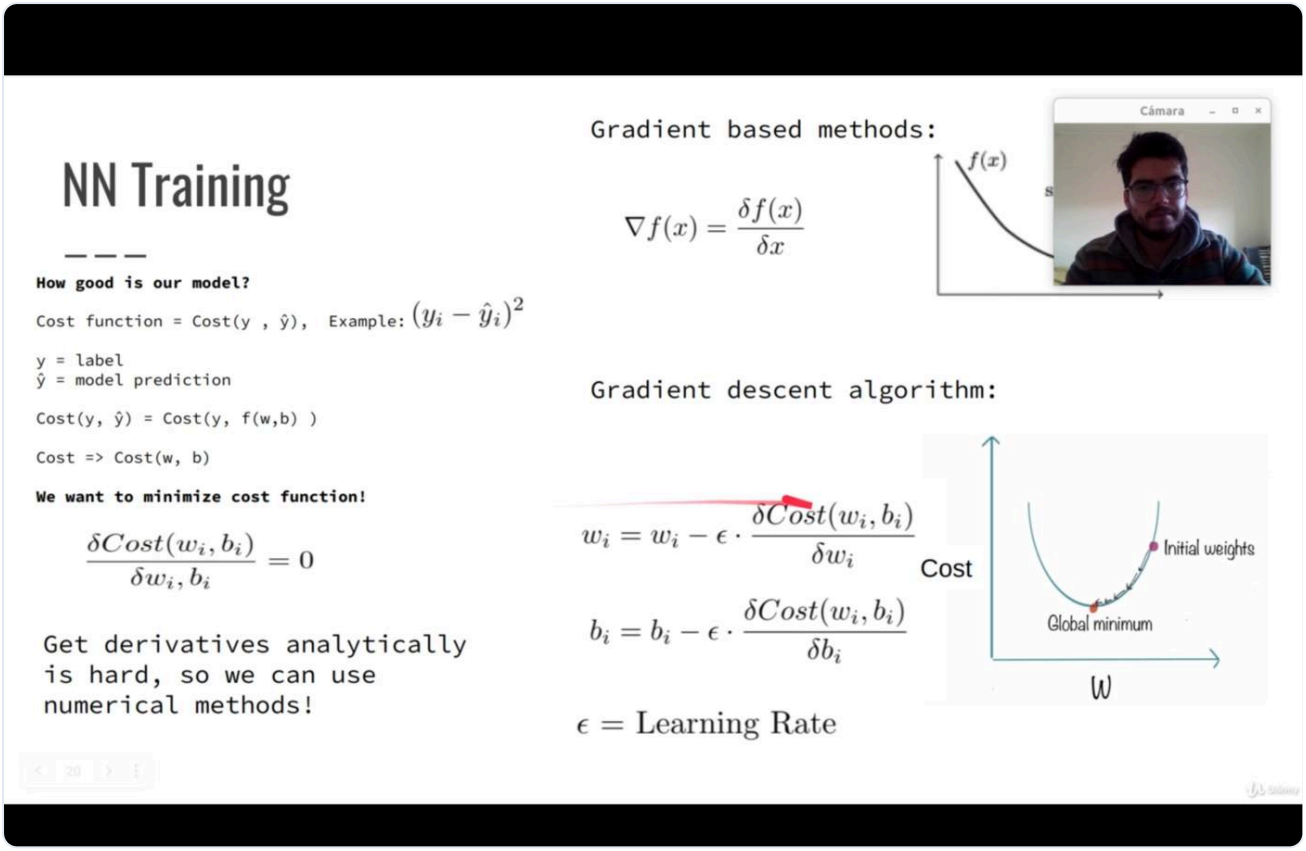



FIGURE 11.6 | Gradient Based Methods:

OK, so we are again using the gradient to find the direction in which the function, the cost function, is decreasing.

OK, so if we want to decrease the cost function value, we have to follow this direction. OK, so then we're going to update the parameters to this point. So these parameters at this point are going to be updated using this formula. So you are going to use the previous value, this is the initial weight, and then you are going to move in the direction in which the cost function decreases, and you are going to move this distance, the epsilon distance, OK. Then you can update the parameters using the same formula, following the decreasing direction.

Then again, again and again, and finally you are going to find the minimum of the function. OK, so basically what this is doing is following the direction in which the cost function is decreasing. So if you take the W values, if you change the W values following the gradient direction, you are going to find the value of the minimum. So basically at this point you are minimizing this. OK, so in this way you can find a minimum of this function.

OK. So this formula here is the key point. OK, so we can find the parameters of the cost function, and of the formula of our neural network model, if we follow this formula. OK. The problem with this is that, analytically, we have this gradient formula here. So we also have to compute the derivative of the cost function with respect to the parameters. So this is a problem, because it's really hard.

So instead of using an analytical derivative, we use the backpropagation algorithm. OK, so the backpropagation algorithm is an algorithm that allows us to compute the derivative of any function using the chain rule. OK, I don't want to go deeper into this, because this can be a little bit

difficult. But at the end of the day, you have to understand that instead of computing this formula directly, we can use other formulas, and they are easier.

Excuse me. So we can compute the derivative of the cost function with respect to C, and then multiply this with the derivative of C with respect to the parameters. OK, so these steps are going to be easier to compute, instead of computing this. OK, so finally the idea is that we are going to use the gradient descent algorithm, using this formula here for our parameters, right, in order to minimize the cost function. This is known, this is known, but this is more complicated.

Right. And instead of computing it in the analytical way, we are going to use the backpropagation algorithm, OK? So in order to find, or to optimize, the parameters in order to find the best neural network model, we're going to use this algorithm. OK, so we're going to repeat these two steps until some optimization condition is satisfied, for example, a minimum. And we're going to repeat these two steps. OK, so basically these two steps are to update the weight parameters and the bias parameters.

OK, so in this formula, the parameters are numbers, right? These are the parameters, numbers.

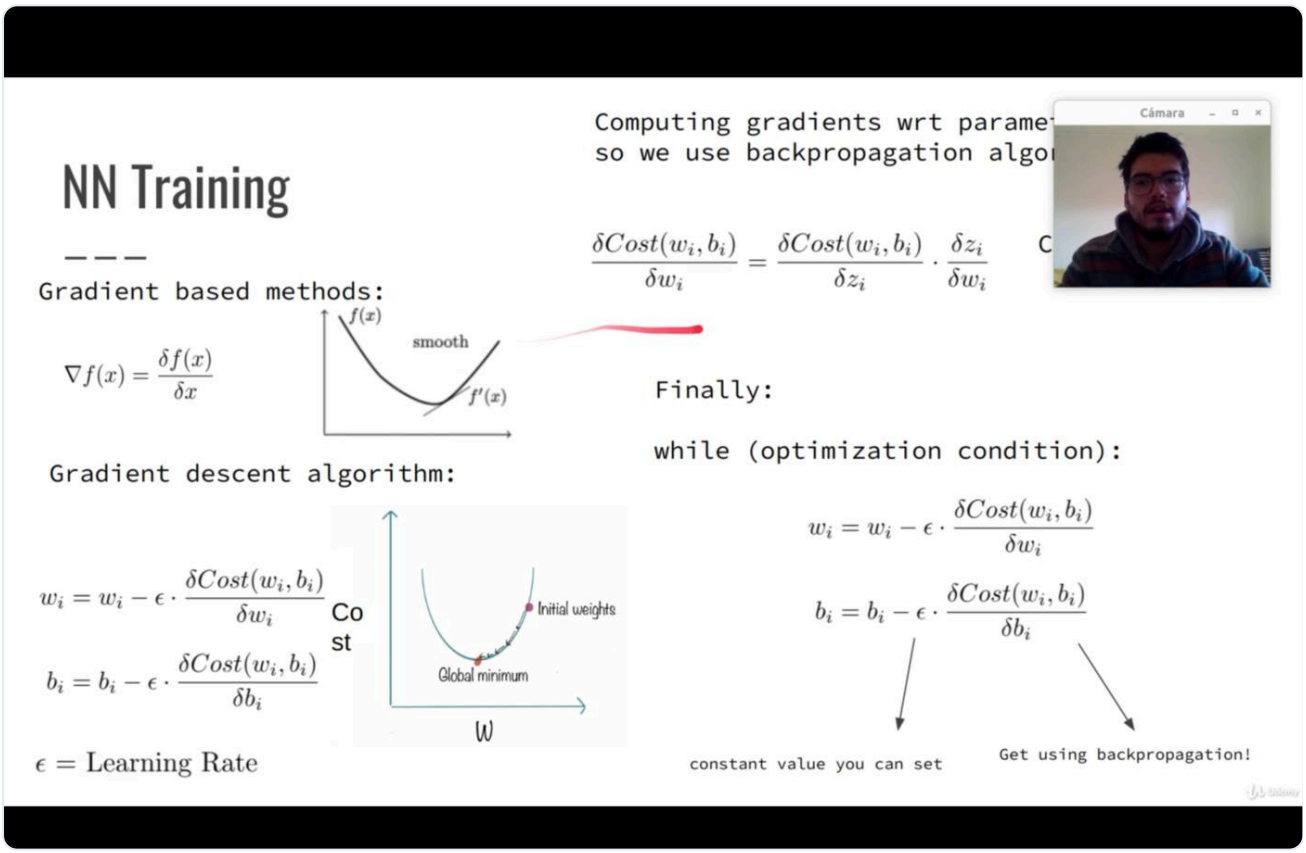



FIGURE 11.7 | Gradient Based Methods:

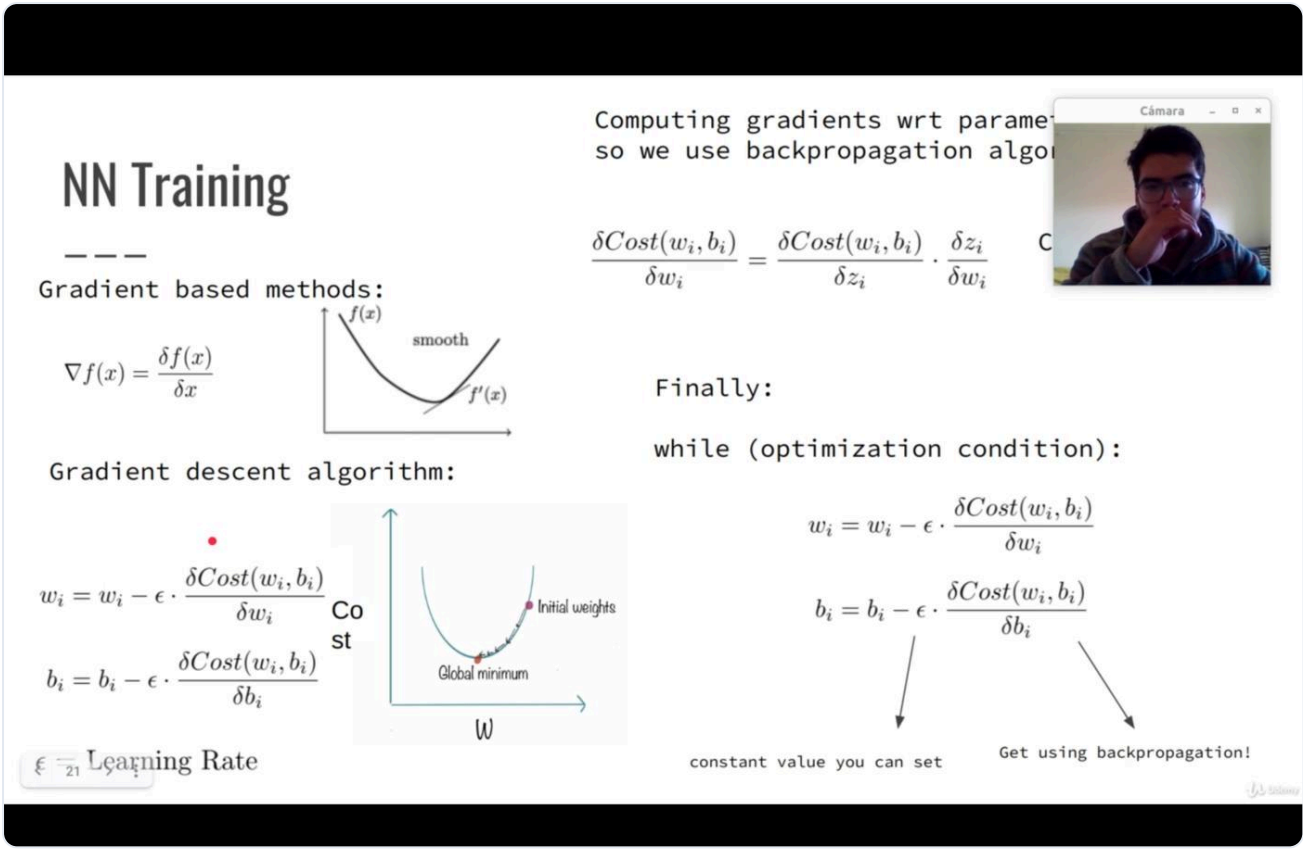



FIGURE 11.8

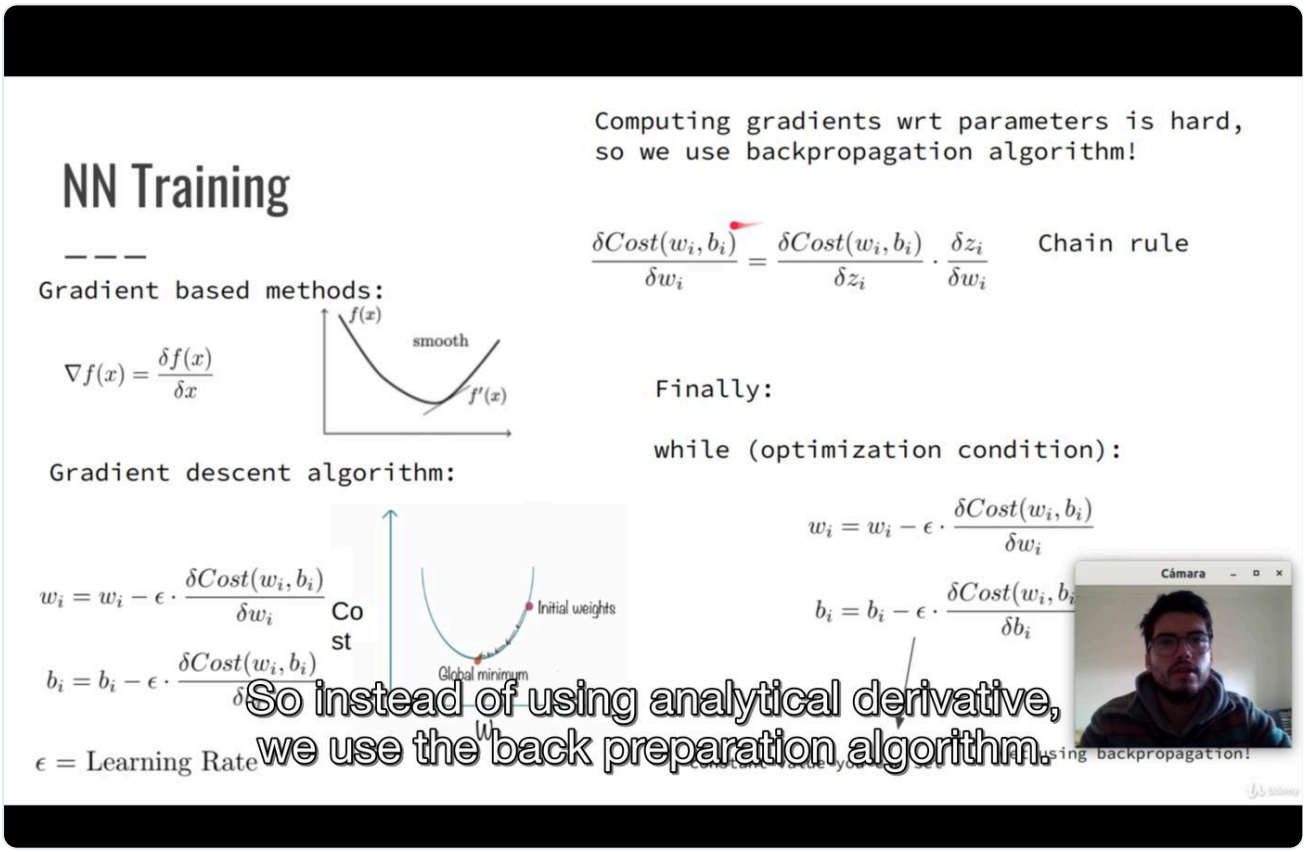



FIGURE 11.9

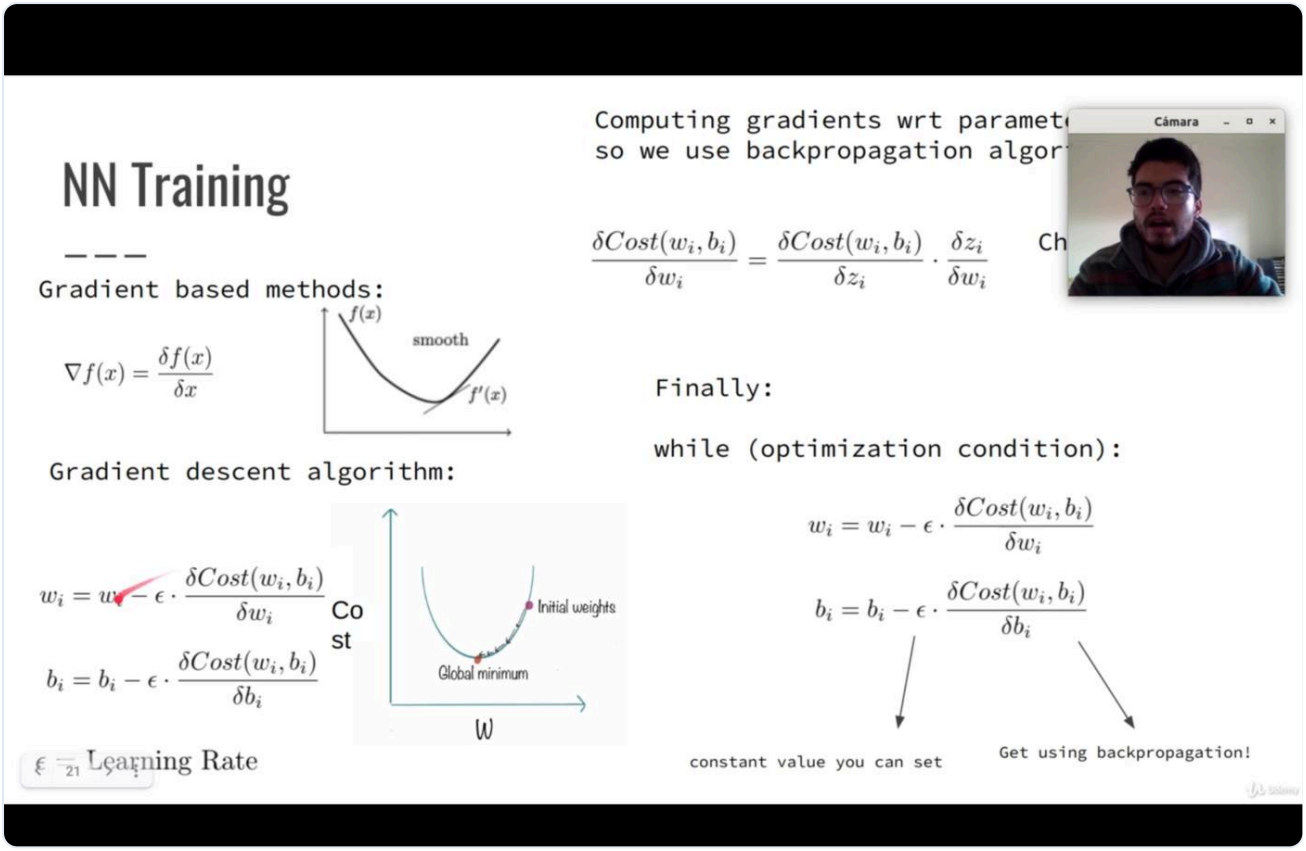



FIGURE 11.10

The learning rate is going to be a constant value that you can set. Usually you can say, for example, I am going to set this to 0.1 or 0.01, OK? And in order to compute this derivative, we are going to use the backpropagation algorithm, OK? So finally, the idea is that we are going to have the function of our model, this one here, and we're going to use the cost function to measure how good our model is.

Then we're going to minimize that cost function with respect to the parameters of our function here. And in order to minimize that, we are going to use the gradient descent algorithm, using this formula here.

LECTURE 12

# Practical view of training a Neural Network

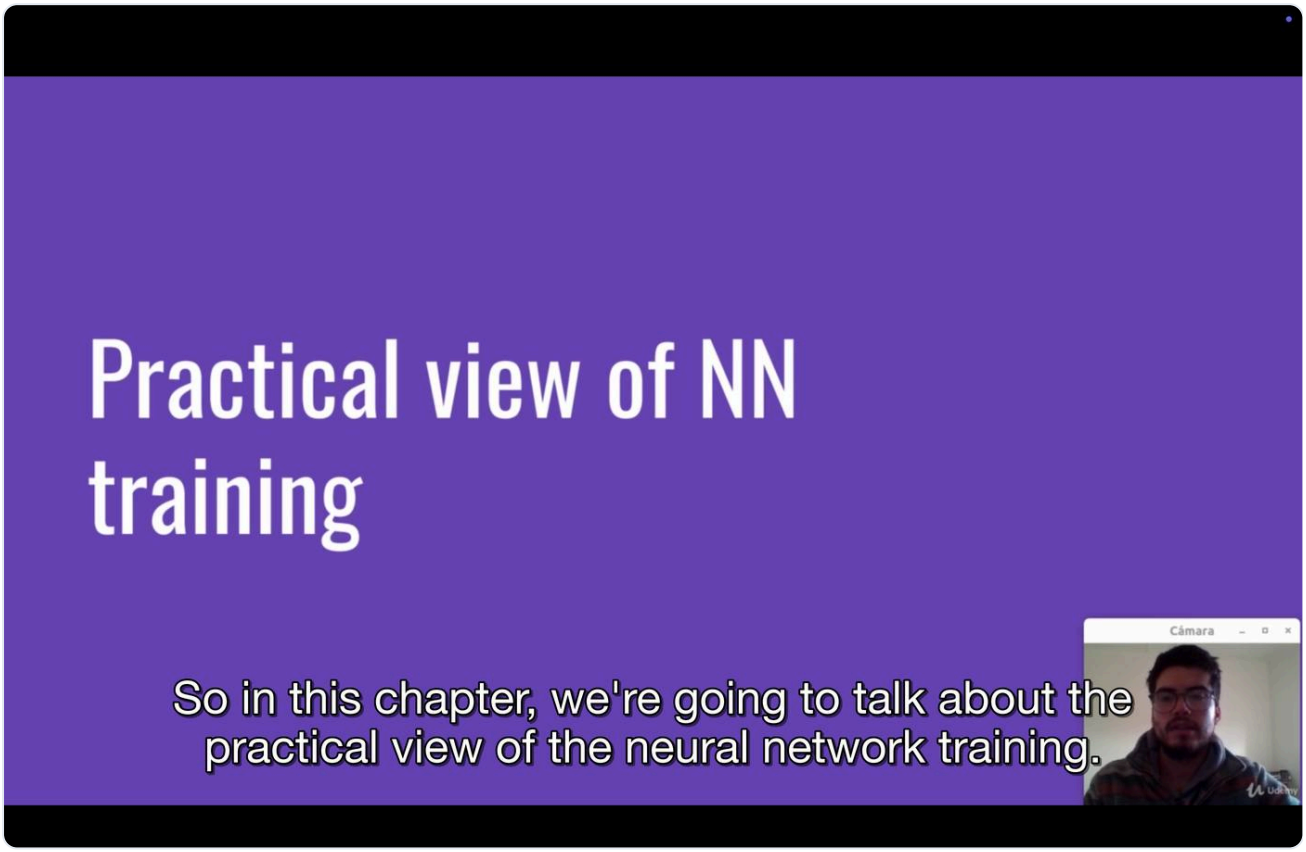



FIGURE 12.1

So in this chapter, we're going to talk about the practical view of neural network training. OK, so in practice we're going to train our model on our dataset.

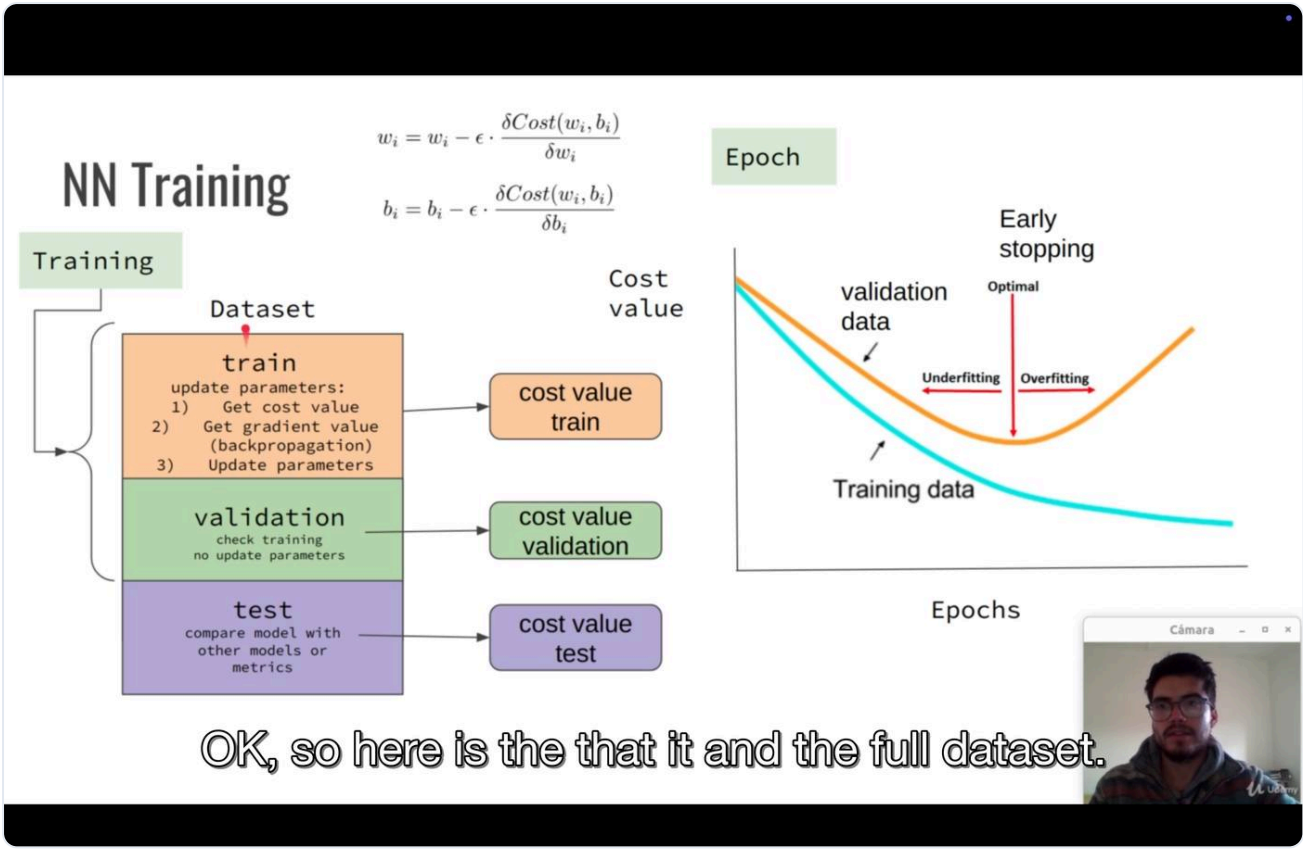



FIGURE 12.2

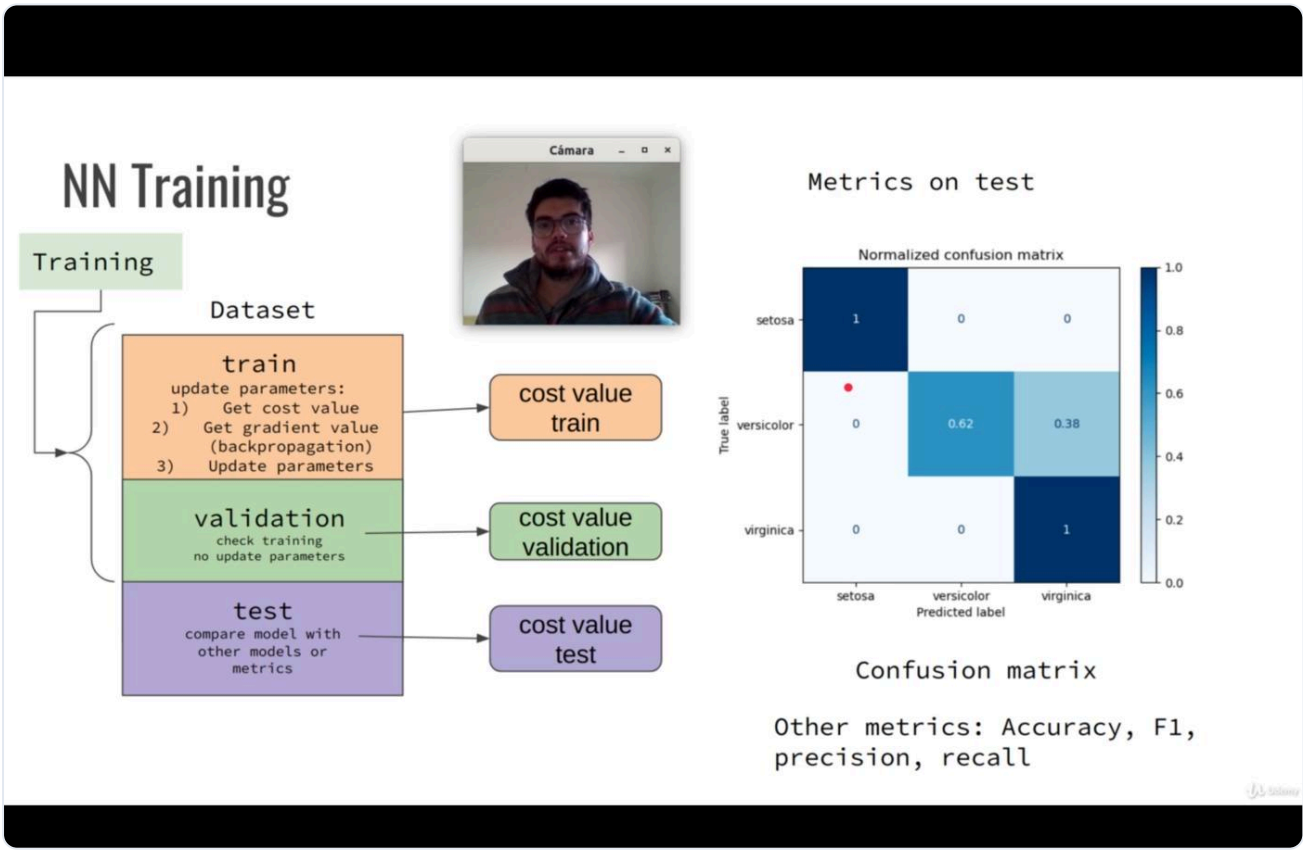



FIGURE 12.3

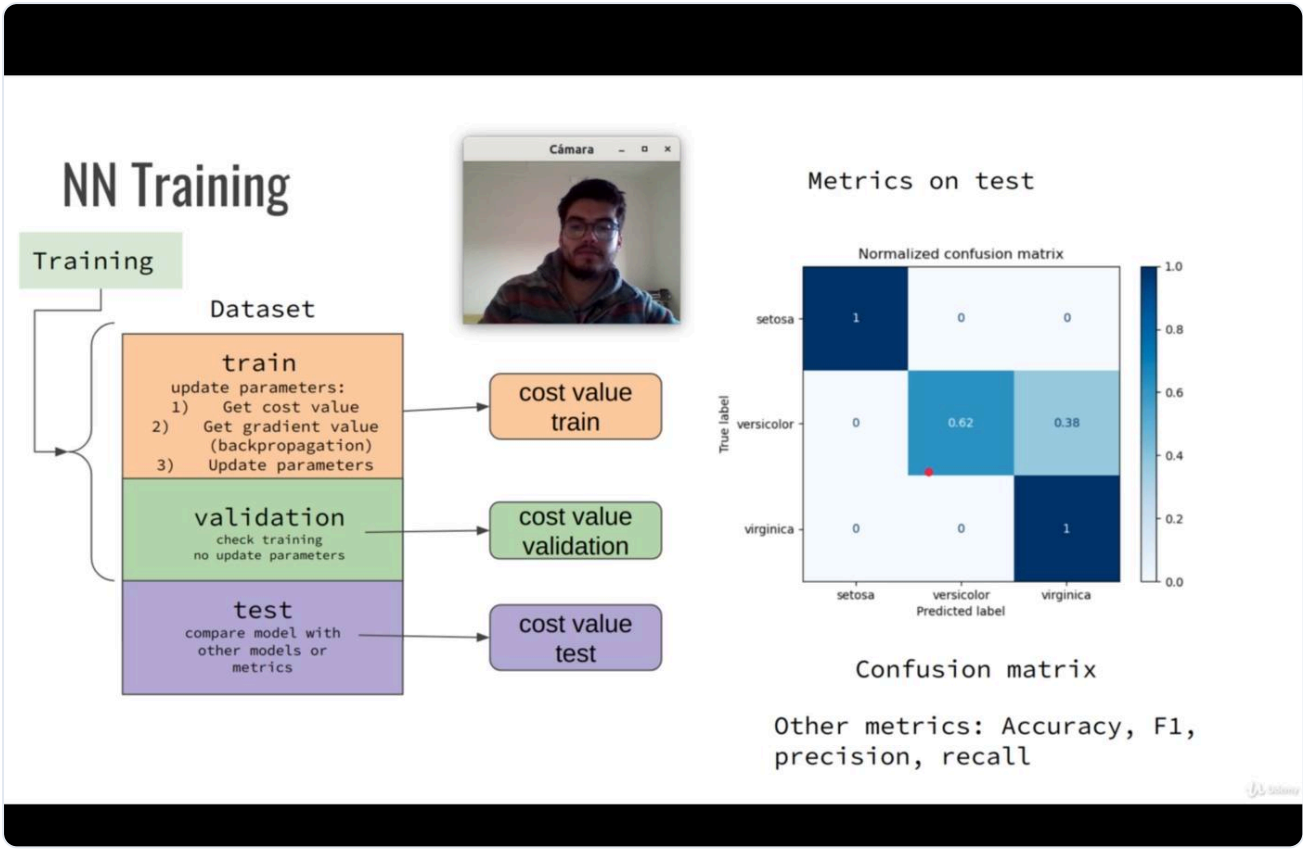



FIGURE 12.4

OK, so here is the data, and the full dataset is this entire figure, OK? In practice, we divide, or split, our dataset into three different groups. OK, the first group is called the train group. The second group is the validation group, and the third one is the test group. OK, the train group we use exclusively to update the parameters.

That means that we apply all the different algorithms that we were talking about previously using this formula. For example, if you are using the gradient descent algorithm, we use this train subset in order to update the parameters. OK, then we're going to have the validation subset. With this subset, we don't update the parameters, OK? We only use this subset to check the training. OK, so with this validation test, excuse me, with this validation group, we can get, for example, a metric in order to analyze how well our training is going.

OK, and finally, we're going to have the test. OK, in the test subset, again, we don't update the parameters. And we usually use this subset to compare our model with other models, or to get metrics on our work. OK, so training means that you are going to find the best parameters for your model. When we are training our model, we are going to use two of these three groups: we're going to use the training and validation.

OK, so with the training subset, we are going to update the parameters, and with validation, we're going to check our training. OK, so, um, when we are in the training phase, we are using the train subset, we are updating the parameters. Right. And in order to update these parameters, we do three different steps. OK, so the first step is to get the cost value using exclusively the train dataset, or the train subset. So depending on the formula that we're using to express the cost function, we're going to get the value.

And this value is going to be a number, for example, ten or one thousand and one, I don't know, whatever. OK, then, in the second step, we are going to get the gradient value of the cost function. OK, so in order to compute this gradient, we need to know previously the cost value. OK, so why are we computing this gradient? Well, because in all the formulas that we use to update our parameters, we're going to have this gradient. And this is the gradient of the cost function with respect to the parameters.

OK, so using this cost function value, we are going to get the value of the gradient. OK, and how do we compute this? Well, we use the backpropagation algorithm. OK, and finally, the third step is to update the parameters. So for example, if we are using the gradient descent algorithm, we are going to use this formula here. OK, so we are going to update our parameters using that gradient. OK, so in the third step, we update the parameters. Before the third step,

we have the previous parameter values; in the third step, we have the new parameters. So basically when we are computing this cost value, we are getting the cost value using the previous parameters, not the updated parameters. OK. So with the training, we update the parameters, OK? And this is important because we update the parameters using the entire training dataset. OK, so we have our entire training dataset, and we update our parameters using the entire dataset.

OK, so from this stage, we're going to get the cost value of the training subset. OK, because this cost value was computed in the first step. OK, so now, in the same training, we take the validation dataset, and from this we are only going to get the cost value. OK, so we're going to apply our cost function. OK. So as you can see here, one time we apply, excuse me, one time we apply the training steps on the training dataset.

And one time we apply the steps on the validation. OK, so basically we use the entire training and validation subsets just one time. OK, one iteration, if you think of it that way. OK, but then we can do another iteration. OK. And we can repeat the same steps that we were doing here. So for example, we are going to do another iteration, and in this iteration we are going to do the same steps. So the first step is going to be to get the cost value.

Right. And this cost value is going to be different from the previous iteration. And this is because the cost value is a function of the parameters, and in the previous iteration, we updated the parameters. Right. So the new cost value in the new iteration is going to be different. OK, because the parameters are changing. Right. So now we're going to get a new cost value, and it's going to be this one in the new iteration. Right. And then with this new cost value, we're going to get the gradient value.

And finally, we're going to update the parameters using the new cost value. So with a new cost value, we're going to get new parameters using this formula. OK, so in this second iteration, we got other, different values of the parameters. OK, this is very important. OK. And in the same iteration, we can go and take the validation dataset and get the cost value. And obviously this is going to be another value, different from the previous iteration. Now we can do another iteration and repeat the same steps.

And you have to think that the cost value and the parameters are going to change, because we are updating the parameters. Right. So in each iteration, we're going to have different cost values and different parameters. OK. Each of these iterations is called an epoch. OK, one epoch is one iteration over the entire dataset used to train, so in this example, train and validation. OK, so for each different epoch we are going to have a different cost value.

OK, so this is a plot in which we have the epochs and the different cost values, and the blue here is going to be the cost value for training, and the orange here is going to be the cost value for validation. OK, so when we are training, we are doing different iterations, different epochs, and in different epochs we are going to have different cost values. OK, it turns out that when we are training our model, if everything is OK, usually we are going to have this kind of plot, in which the training cost value is always going to go down, to decrease its value.

But in the validation, we are going to have a decrease first, then we're going to have a minimum, and then we're going to have an increasing stage. OK, so why is this happening? Well, it turns out that when you are updating the parameters, you are updating the parameters in order to decrease the value in the training, right; you are using the training data to update the parameters. So obviously, if you train your model using this data, your model is going to fit more and more accurately to your data.

So that's the reason why you're going to have this decrease of the cost value in training. But this is an interesting point. It turns out that you are not using your validation data to update your parameters. So everything is going to be fine in the first stage. But then, when your model starts to kind of memorize your training data, it is going to lose generalization. This is a very important property of a model. Generalization is when your model is capable of making good predictions on new data, or unseen data.

OK, so when your model starts to memorize your training data, at the same time it is going to lose the generalization property needed to predict new data. So that's the reason why: your model starts to memorize your training data, but your model is not capable of doing good predictions on unseen data, and this is the validation data. OK, so we're going to have two areas in this plot. We are going to have the underfitting area.

OK, so when we have this underfitting, this means that yes, your model can learn the training data, obviously, but your model is not as good in generalization as in the training. OK, so in this space, your model cannot generalize very well, and your model is not complex enough for learning, or for generalizing to new data. OK, basically you can do more epochs in order to improve your model.

OK, on the other hand, we are going to have the overfitting, and in the overfitting we are going to have this scenario, in which your training data is going to be totally memorized, and your model is going to know every aspect of your training data, but it is going to be totally incapable of generalizing on the validation dataset. OK, so in this area, yes, your model is doing very, very well on the training dataset, but it's going to be very, very bad on your validation data, your unseen data.

OK, so the idea is that you should train your model and you should stop your training at the epoch here in which you have a minimum on the validation data. OK, so the idea is that when you are running different epochs, you should be able to save your model at this epoch, or stop the training at this epoch. OK, so that is the process for training. And you have to remember that this training is only done with the training and validation datasets. OK, now we are going to use the test subset.

OK. And the test subset, we usually use this... give me a second. We usually use this dataset to compare with other models, or to get metrics. OK, so in this example we have a metric, and this is a confusion matrix, and this is a very common metric used in classification problems. So we are going to have this matrix, and in the rows we are going to have the real labels, or the real classes, Setosa, Versicolor and Virginica in this example, and in the columns we are going to have the predicted classes.

And that means that your model predicted that this sample here is of class Setosa, Versicolor or Virginica. OK, so with this matrix, you can basically analyze how your model is doing with the different classes, but also how your model is confusing, for example, the Versicolor class with the Virginica class. OK, on the diagonal you are going to have the correct classes: real Setosa predicted as Setosa, real Versicolor predicted as Versicolor, and real Virginica predicted as Virginica.

But outside the diagonal, for example, in this item, the real label is Versicolor, but your model predicted 0.38, thirty-eight percent, as Virginica. OK, so with this you can check how your model is confusing the different classes. OK, but also we can use other metrics, like, for example, accuracy, precision, recall. These are very famous metrics that we use in classification. Accuracy, for example, is the total amount of correct predictions that your model made. OK, but basically the idea of the test is that we are going to use the test subset to get metrics and analyze our model.

LECTURE 13

# Train with batches

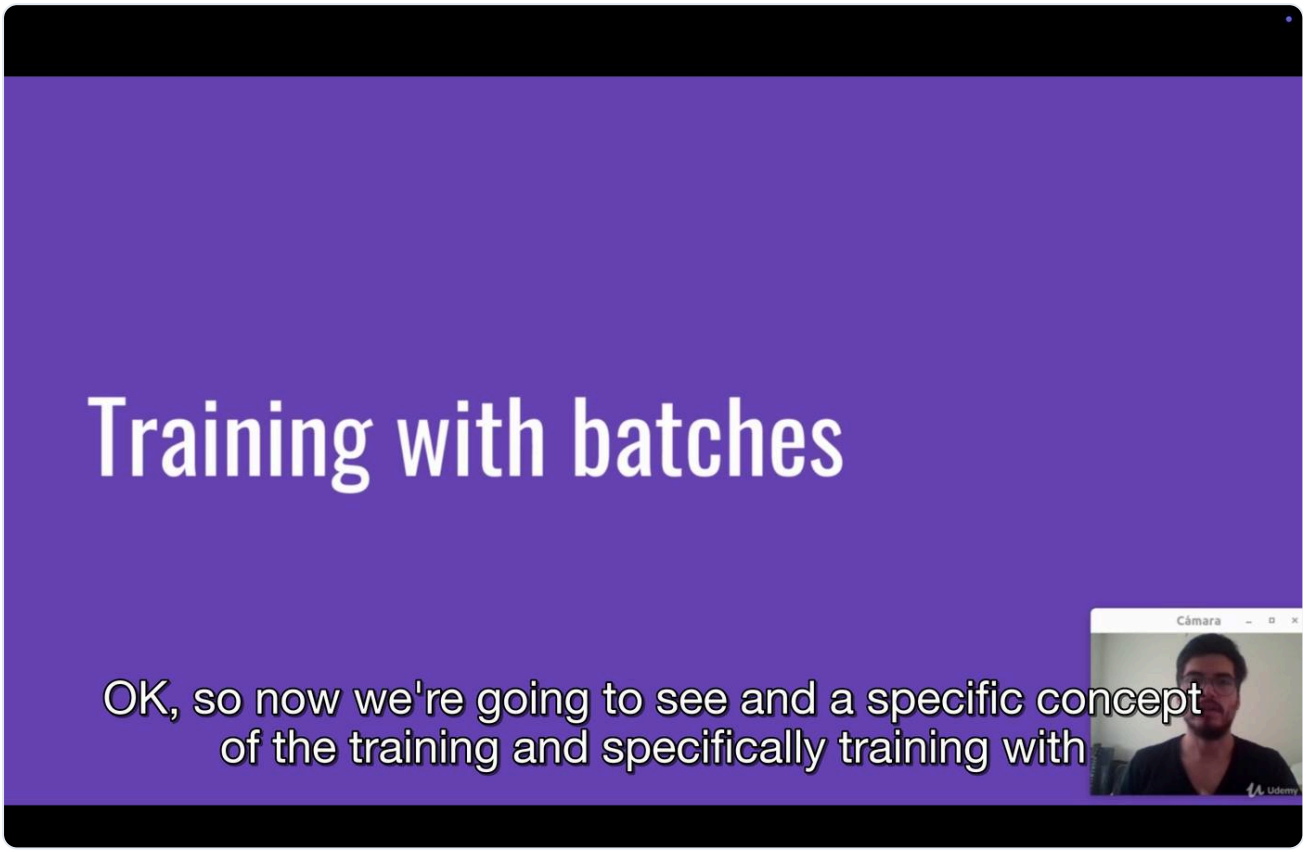



FIGURE 13.1

OK, so now we're going to see a specific concept of training, and specifically training with batches. OK, so when we're training, you have to remember that we are using the training dataset. OK, so when we're using this training dataset, we can organize this dataset, or this training dataset, into different schemas, or different ways. OK, so the first way in which we can use this training data is to use the entire training dataset in order to update the parameters.

That means that we take the entire dataset, and from the entire dataset we get the cost function. Then with the cost function, we get the gradient, and finally we take the gradient values and the cost function and we update the parameters. Right. So basically we update the parameters one time when we're using the full dataset. OK, so that is one way of using the training dataset, OK. But instead of using the entire dataset in order to update the parameters one time, instead of getting the entire dataset, we can get just one sample.

OK, that means, give me a second, that means that we're going to take one sample, one row, from our dataset. We're going to take one sample, and from this one sample we're going to get the cost function. Then we are going to get the gradient, and finally we're going to update the parameters.

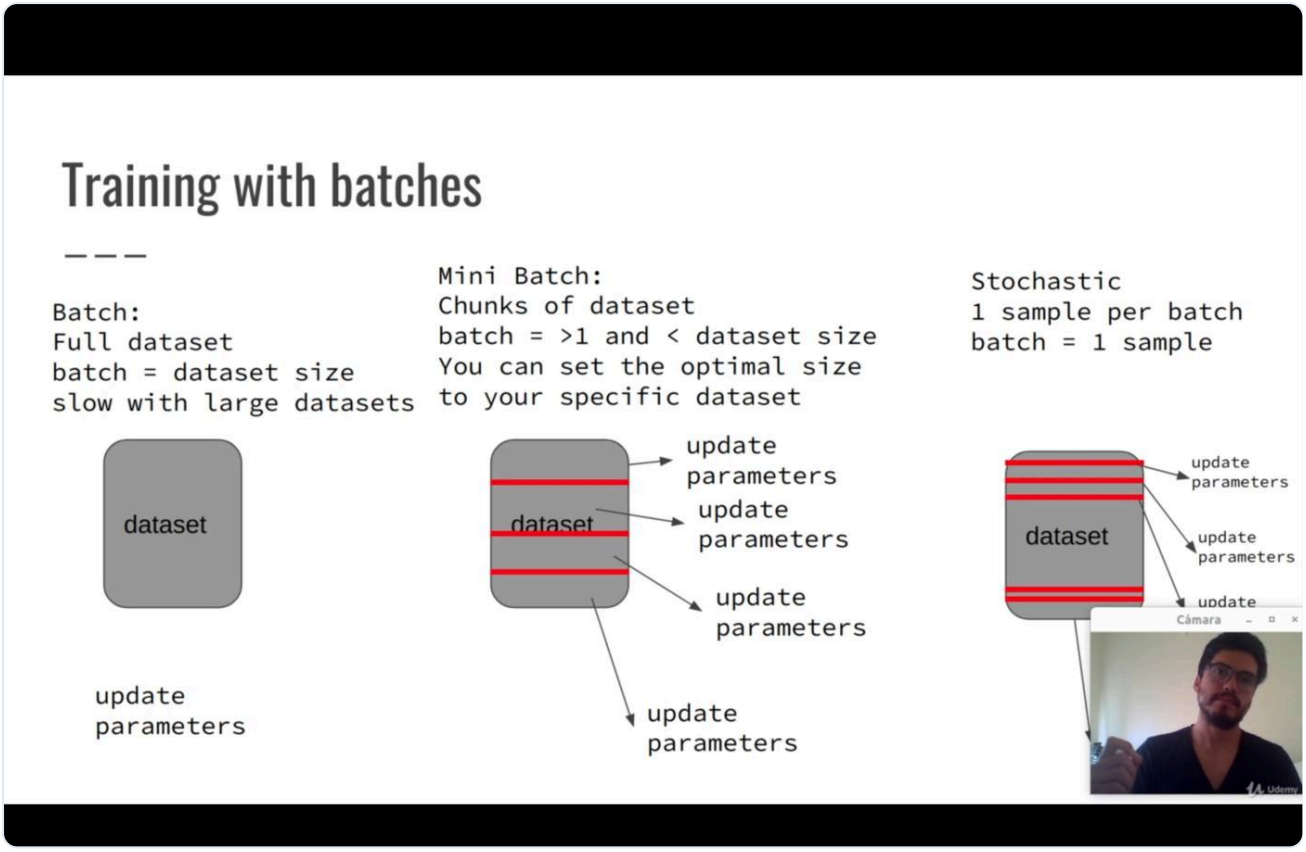



FIGURE 13.2 | Training with Batches

OK, so we're going to update the parameters for one sample. So if we use this approach, we are going to update the parameters for each instance, for each sample of this dataset.

OK, and that means that we're going to update the parameters for each row in our dataset. OK, in our training dataset. OK, so we're going to update them a lot of times, OK? These are the two extremes that we're going to have: we can update the parameters using the entire dataset, or we can update the parameters using one row, or one instance, at a time. OK, but in the middle of these two extremes, we're going to have this approach, and in this approach, instead of taking the full dataset or one instance, we're going to take a chunk of the dataset.

OK, so we're going to take smaller portions of the dataset, and the size of this portion is going to be smaller than the full dataset size and greater than one sample. So, for example, if the entire dataset contains 100 different rows, in this approach we're going to take these 100 different rows in order to update the parameters one time. Right. In this approach, we are going to take one row, and we are going to update the parameters for each row.

So we're going to update the parameters 100 times, because in our example the training set is 100 different rows. But with this approach in the middle, instead of taking 100, or instead of taking one sample, we can take, for example, 10 or 15 or 20 or 50, I don't know.

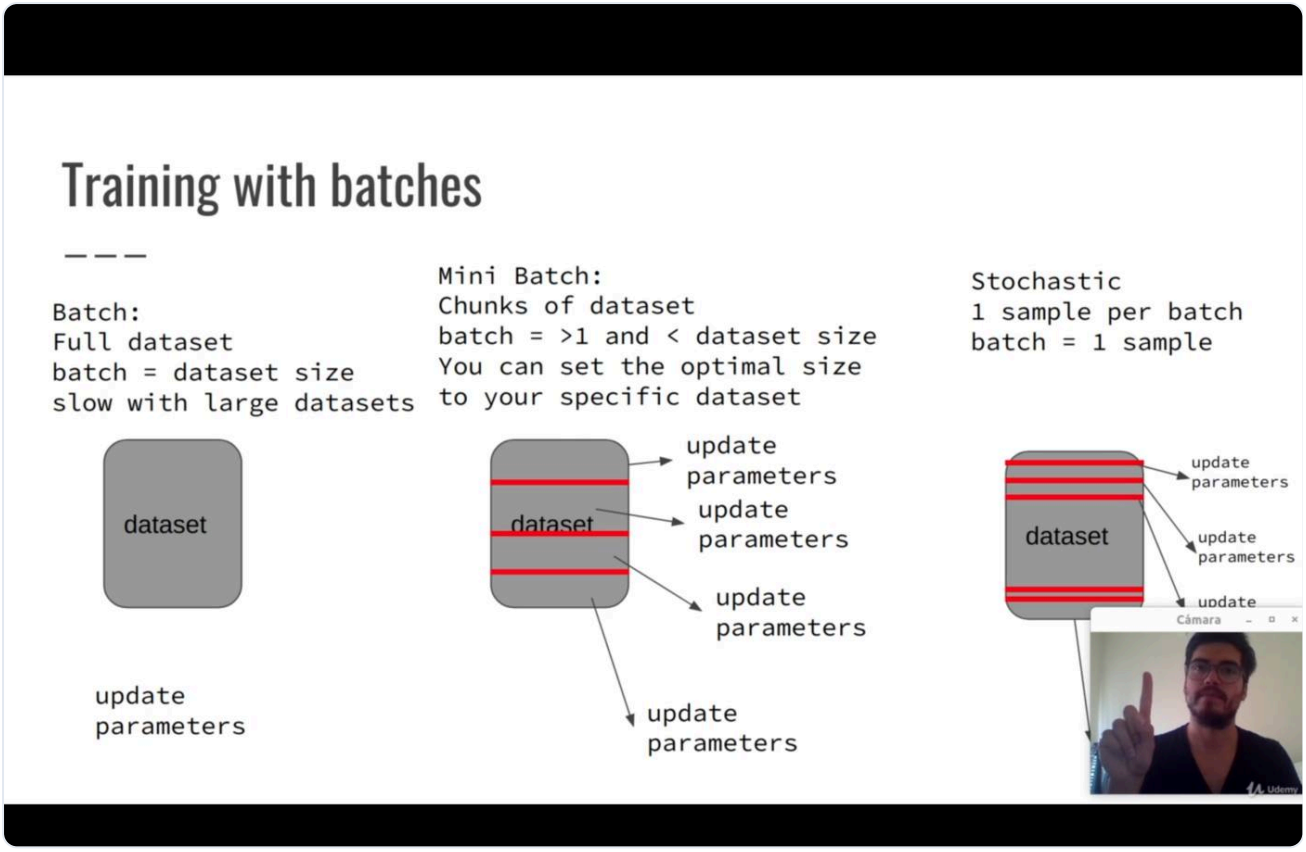



FIGURE 13.3 | Training with Batches

But we're going to take a smaller portion of the dataset, and with this smaller portion, we're going to update the parameters. OK, so that means, for example, that we're going to take, excuse me, we're going to take ten different rows of our dataset, and from these 10 rows

we're going to compute the cost function, then we're going to get the gradient, and finally we are going to update the parameters. OK, then we're going to take another 10 different rows from our training dataset, and we're going to again compute the cost function, compute the gradient, and finally update the parameters. Then we're going to take another ten different rows, and we're going to compute the cost function and the gradient, and finally compute the gradient and update the parameters.

OK, so we're going to apply this update for the different chunks that we have from our training dataset. OK, so, um, these different approaches that you can take have different advantages and disadvantages. OK, so when we are taking the entire dataset in order to update the parameters one time, when you are working with a large dataset, computing the cost function and the gradient and updating the parameters is usually going to be slow, because you have to compute the cost function and gradient from a large dataset.

OK, so this means that this is going to be slow, OK. Sometimes when you have a large dataset, this dataset cannot fit into the memory of the computer you are working on. So you cannot compute the cost function and the gradient with the entire dataset. OK, so you are going to have a problem with the slowness, with the execution time, but also you are going to have problems with the memory.

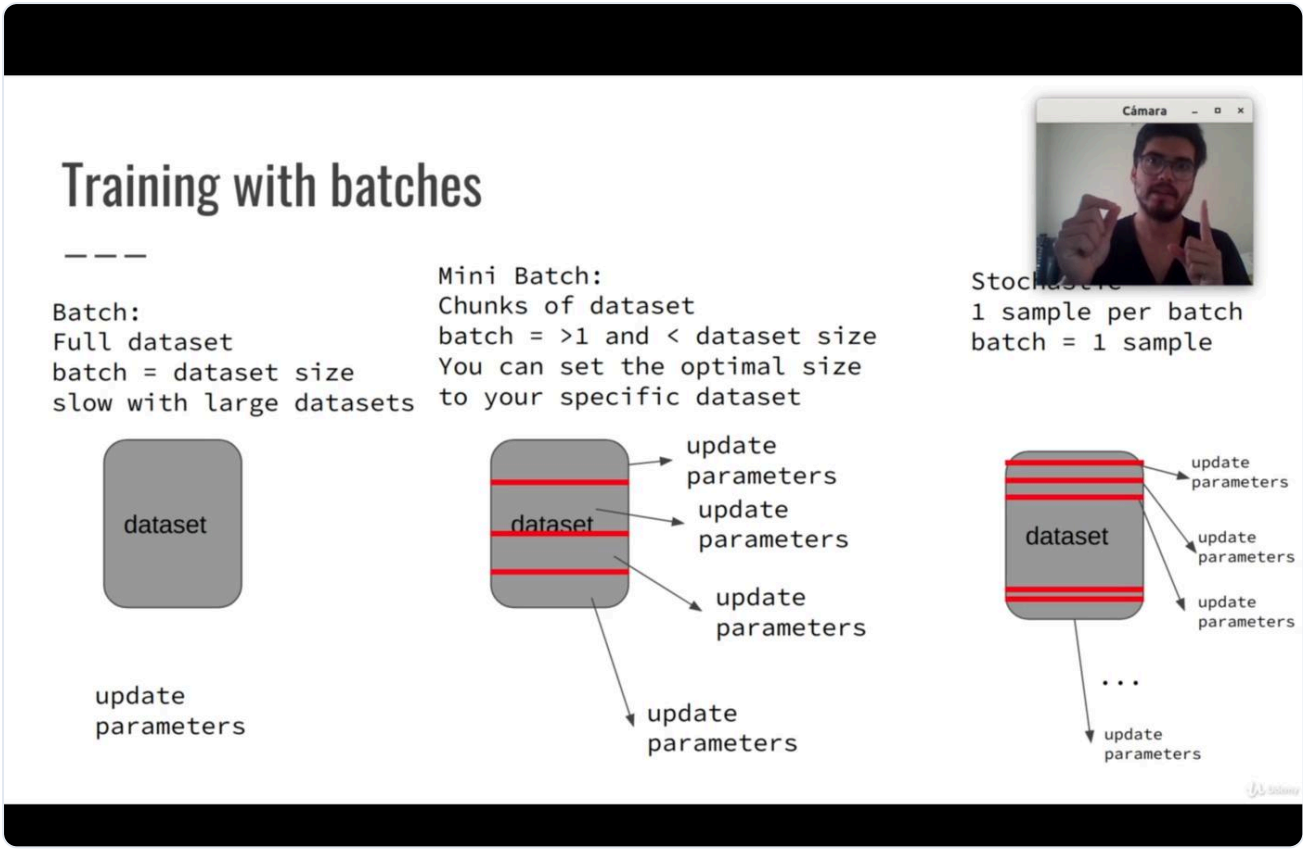



FIGURE 13.4 | Training with Batches

At the other, excuse me, at the other extreme, we are going to have this approach, in which we are using one sample at a time.

Right. This usually is going to be faster, but we are going to have problems with the stability of the training, because we are going to change the parameters all the time. So basically you are going to have a lot of changes in the parameters, and this can affect the stability of the training. And in the middle, we are going to have this approach, in which we can set the size of the batch. So with this option, you again have the option to set an optimal value, to have a trade-off between the slowness and the velocity, or, excuse me, you can have a trade-off between the velocity of the training and the convergence, or the stability, of the training.

OK, so for example, as you can set the size of the batch, you can find, for your specific dataset, for your specific computational resources, the optimal value in order to have a good velocity, but also a good convergence of the training. OK, so when we are using the entire dataset, we are going to talk about batch training, OK? And when we are using this approach, in which we are using one sample at a time, we are going to talk about stochastic training.

And when we're using this training in which you can set the batch size, we're going to call this training mini-batch, OK, because we're using mini portions of the training set. OK, so, um, excuse me. Here we have the different types of training. So we have batch, in which the batch size is equal to the size of the training dataset.

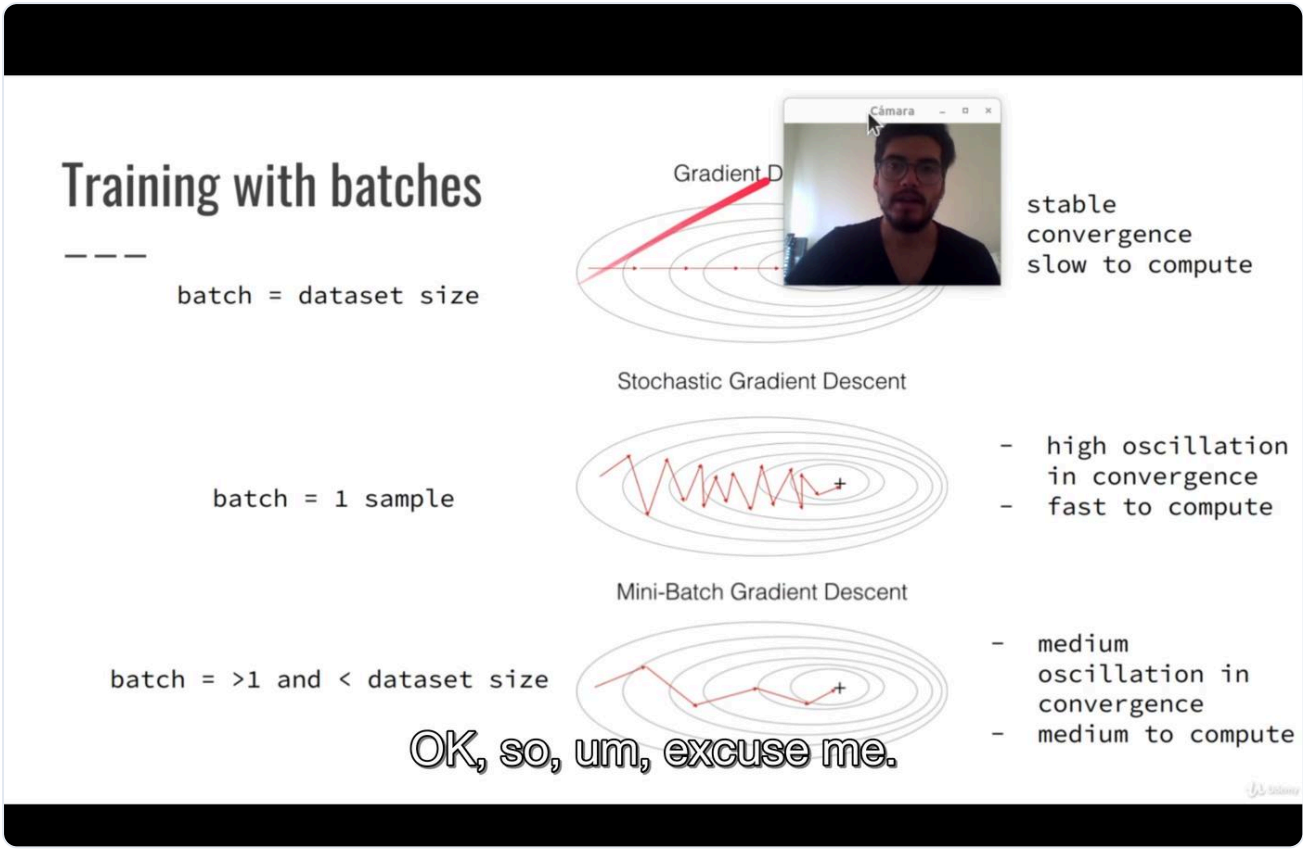



FIGURE 13.5 | Training with Batches Ee 3 Stable

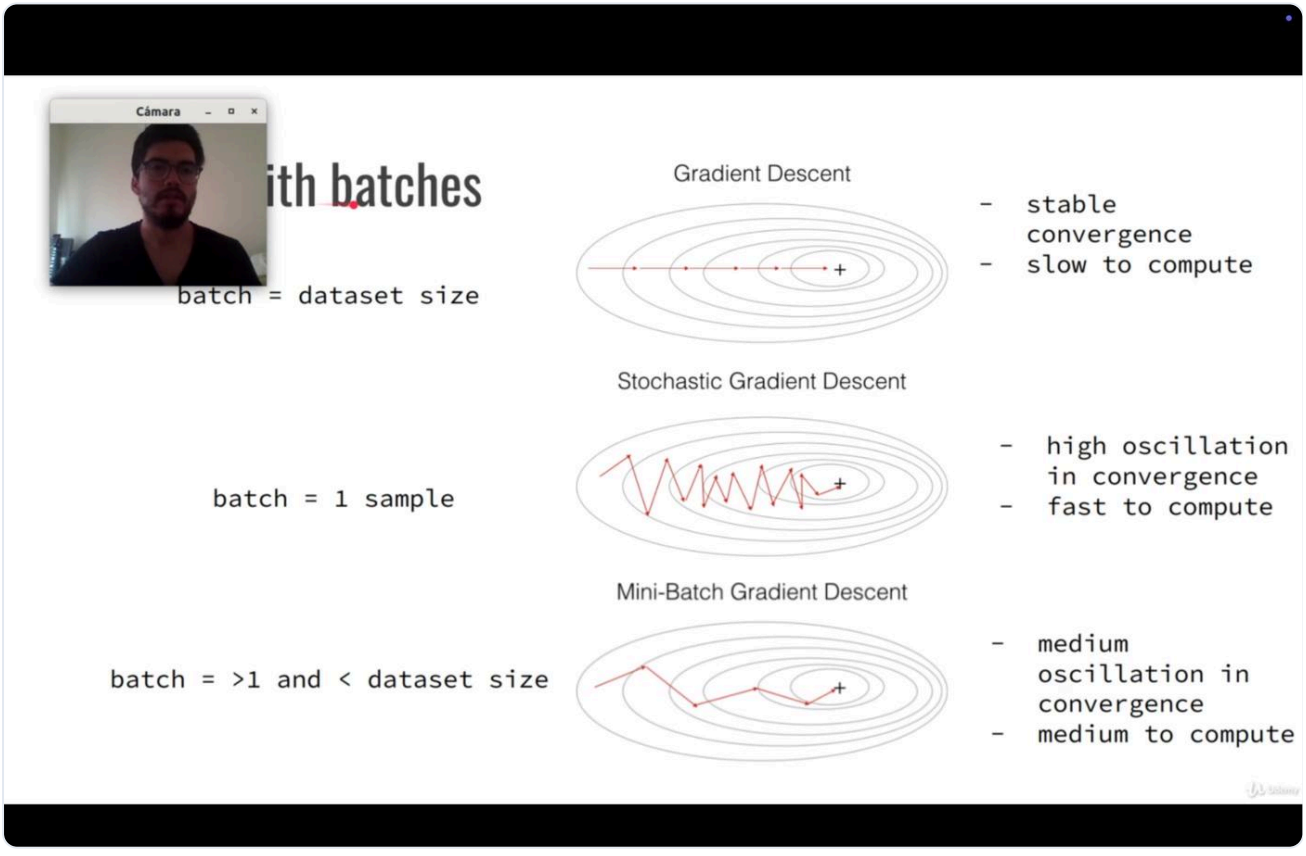



FIGURE 13.6

Then we have stochastic gradient descent, excuse me, stochastic training, and the batch size is going to be one, and then we're going to have mini-batch gradient descent.

OK, excuse me, mini-batch training, and the batch size is going to be greater than one, but smaller than the dataset size. OK, so here we have the different features of these different types of training. So for example, for gradient descent, batch training, as we are using the entire dataset, we are going to take the entire dataset into consideration. We're going to take all the information into consideration. So this is going to be very stable,

because when we are using gradient descent, we are going to use the entire information. So this is going to move kind of straight to the minimum. OK, but the disadvantage of this is that it is going to be slow to compute. OK, then if we are using stochastic, this means that we are going to update the parameters for one sample, for each instance. And the problem with this is going to be the high oscillation in the convergence. OK, because we're going to be updating the parameters a lot of times, for each sample.

We are going to have a different change in the, uh, in the finding of the optimal solution. OK. By the way, the advantage of this is going to be that this is going to be fast to compute. And finally, we're going to have mini-batch. And with the mini-batch approach, we can set an optimal value in order to not have a high oscillation. We can have medium oscillation, but also we can find an optimal value in order to compute something with a decent execution time.

OK, so again, this is going to depend totally on the problem that you are working on, and if you have a large dataset, it is going to be very convenient to use this approach. Or if you have, for example, a small dataset, you can use the gradient descent approach. OK, but again, these different approaches have different advantages and different disadvantages. So that totally depends on the problem that you are working on.

LECTURE 14

# Practical example presentation

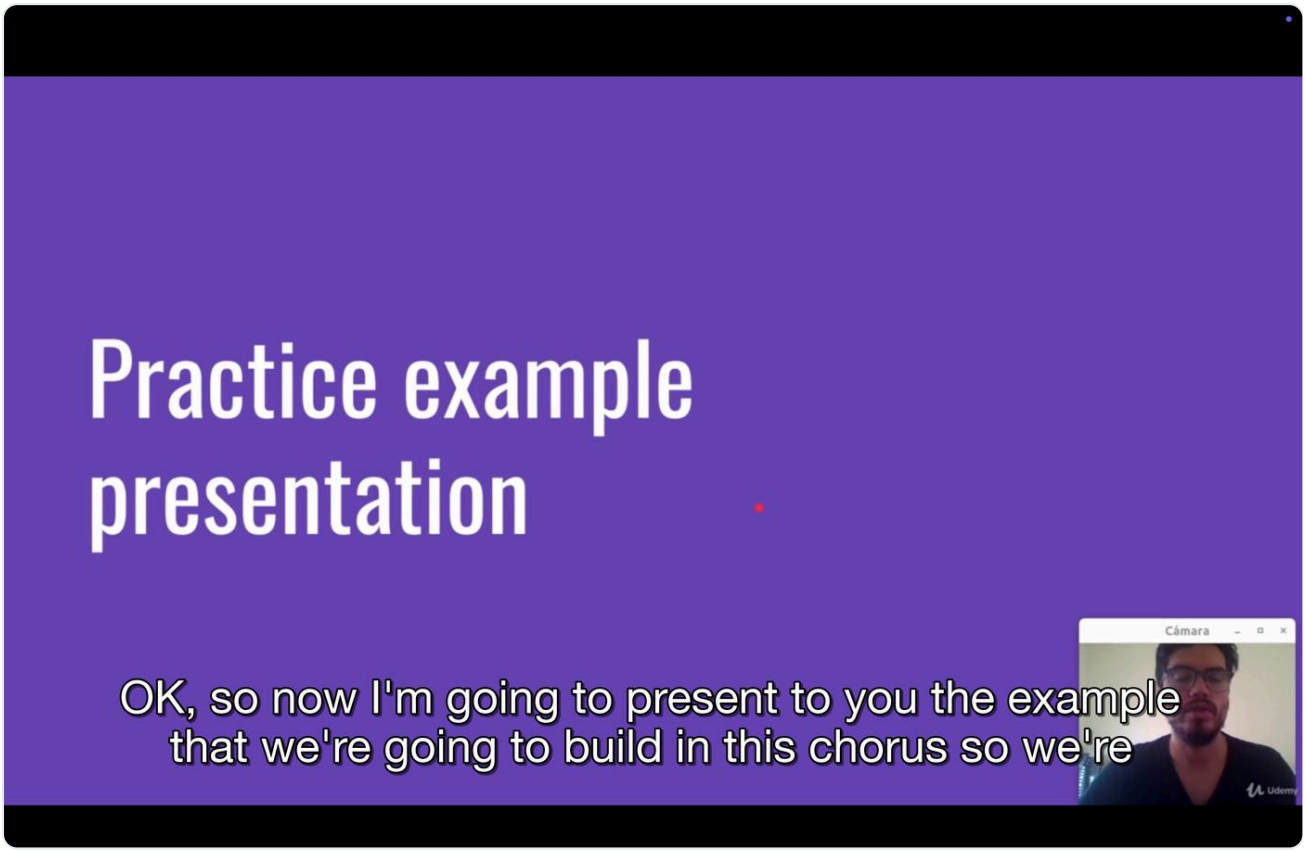



FIGURE 14.1

OK, so now I'm going to present to you the example that we're going to build in this course. So we're going to work with a specific dataset. This is CIFAR-10. This dataset contains 10 different classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck, OK.

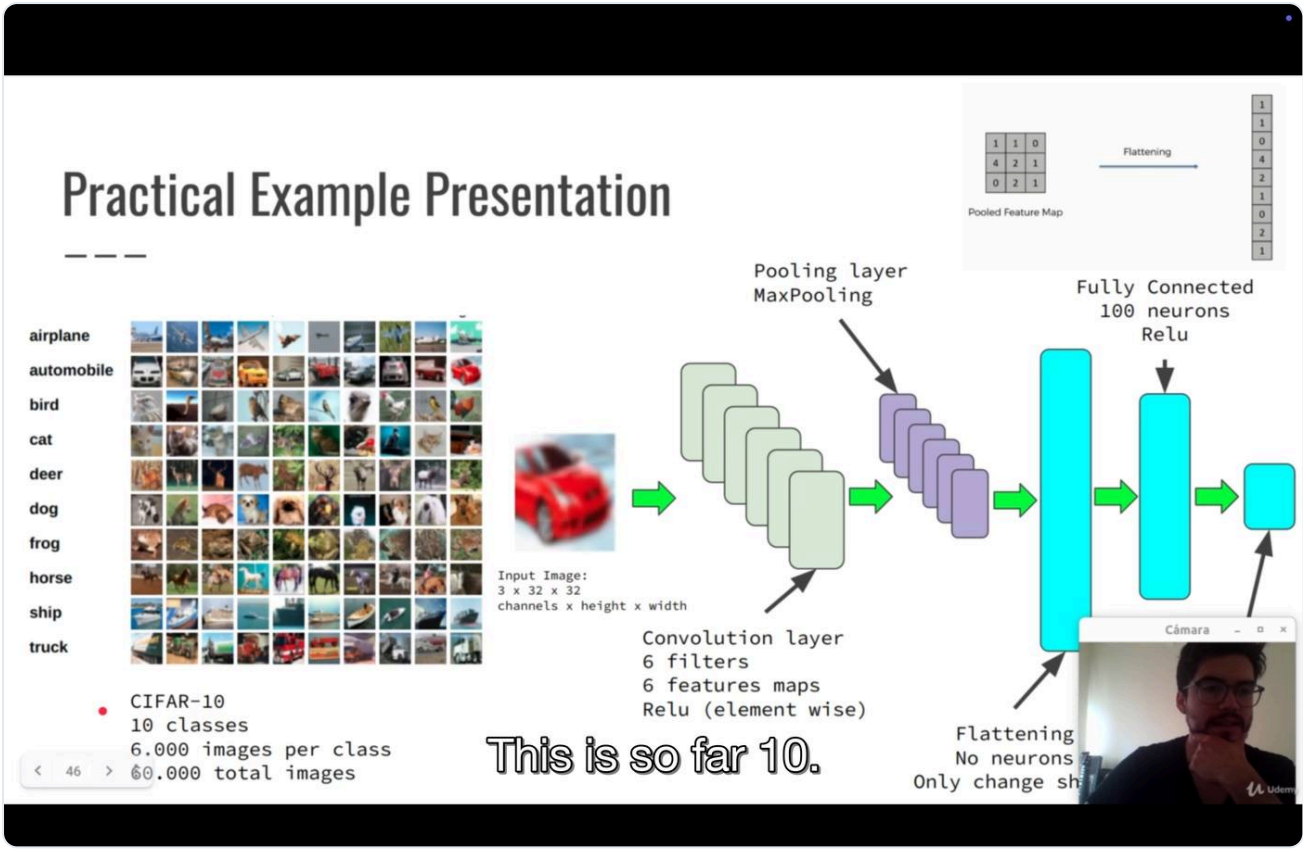



FIGURE 14.2 | Practical Example Presentation

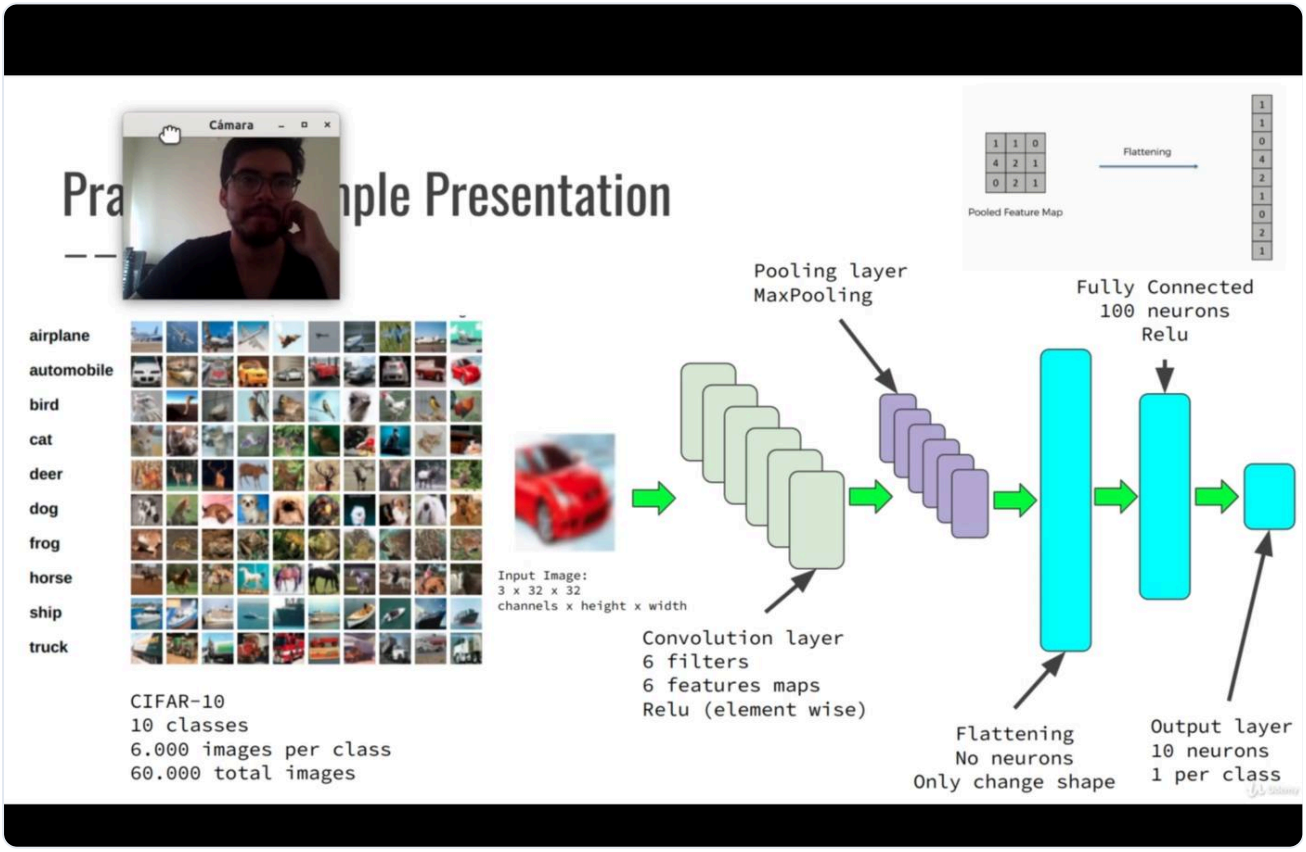



FIGURE 14.3

And for each of these classes, we have 6,000 images per class. So that means that we're going to have 60,000 total images. OK, so the idea of this is to build a convolutional network model in order to classify the different classes.

OK, excuse me, the different images. OK, so we're going to have an image, and we want to classify this image into one of these ten different classes. Right. So this is the one that we're going to build. Give me a second. This is the model that we're going to build, and this is going to be the input, and this is going to be the output. OK, so the input layer is going to be an image. So, for example, this image here, or it can be any of these images here. OK, so the input image is going to be a tensor with dimensions 3, 32 and 32: three for each

channel, 32 is going to be the height, and 32 is going to be the width. So then the shape of this image is going to be three by 32 by 32. OK, um, the output of this model is going to be an output layer that is going to contain one neuron per class. So that means that in the output layer, we're going to have ten different neurons, one neuron per class. OK, so in this model, we are going to have a feature learning stage, and this is going to be composed of a convolutional layer and a pooling layer.

And then we're going to have the classification stage, and this is going to be composed of a flatten layer and then a fully connected layer. OK. In the convolutional layer, we are going to apply the convolution between six different filters and the same input image. OK, so at the end of the day, we are going to generate six different feature maps. OK, and for each of these six different feature maps, we're going to apply ReLU.

And you have to remember the ReLU, or the activation function, is going to be an element-wise operation. That means that for each pixel of these feature maps, we are going to apply the ReLU function. OK, after applying the ReLU function to each of these feature maps, we are going to apply the pooling layer. OK, so we are going to apply, in this example, max pooling. That means that we are going to take the maximum of the different areas in each of these rectified feature maps.

OK, so after applying the pooling layer, we are going to take these six different feature maps, and the output is going to be the same six, but we're going to reduce the dimensionality. OK. So as you can see here, these images here are smaller than these images here, OK? After the pooling layer, we're going to apply a flattening operation. OK, so the idea is to convert these six different images into a flattened layer.

So, for example, here we have an example of flattening. This can be, for example, a pooled feature map. This one here can be this matrix here: one, one, zero, four, two and so on. And then we're going to flatten this image. So instead of having this matrix shape, we're going to have a vector of one-dimensional shape. So the first element of this vector is going to be one. The second is going to be one. The third one is going to be zero, then four, and then two, then one and four and so on, until the final one.

It is going to be one. OK, so basically we are going to transform this matrix, or this tensor, into a one-dimensional vector. OK, so this represents a flattened vector here, and these are going to be the same numbers, or the same values, that we have here, but in a flattened way. OK, so this layer

is not going to be a layer per se, because it is not going to have neurons, because this is only to change the shape of this element. OK, so after this flatten layer, we're going to have these different numbers, right.

So you can think of these numbers as the different features that our model learned from these images. OK, so at this point, we're going to have the different features that we learned. OK, then we're going to have a fully connected layer. In this example, we're going to use 100 different neurons, but you can use the number you want, and for each of these neurons, we're going to apply an activation function. This is going to be ReLU. OK, so the idea of this fully connected layer is to find patterns in these different features in order to identify, or finally to classify, the different classes.

OK, and the outputs of this neural network are going to be connected to the final layer. And as I said previously, in this final layer, we're going to have 10 different neurons, and each neuron is going to represent each class. OK, so again, we are going to have our feature learning stage, and we want to learn different features from these different images. We are not classifying anything here; we are just finding different features in our images.

And at this point, we're going to have the different features. OK, and these are only features, nothing else. We are not classifying anything here. And then with this layer, we want to find the patterns in order to classify these different features. OK, so again, we're going to have our feature learning stage, and then we're going to have our classification stage, using our fully connected layer.

LECTURE 15

# Dataset

See the notebook here.

## Load dataset

OK, so now we're going to start working on our code, OK? So here I am using Jupyter Notebook. It is a tool that allows you to build Python code. So, for example, I'm going to print "hello" here, and you can run the code immediately, OK, but you can use the tool that you want in order to build the code. You can use your own editor. You can use Jupyter Notebook, whatever; you can use, I don't know, Colab, or the code tool that you want to use. OK, but we're going to build everything using Python.

OK, so the first thing that we're going to do here is to load the dataset. Right. So we're going to be using the CIFAR dataset, this one. So we're going to use this dataset, and we are also going to use PyTorch. PyTorch is a framework, or a library, built in Python that allows you to work with deep learning models. And also, we're going to use torchvision, and torchvision is a library that allows you to work with computer vision problems.

And more specifically, this library contains a set of different datasets that you can use easily. OK, so we're going to use this CIFAR dataset using torchvision. OK, so in order to do that, we first have to load the libraries. OK, so I'm going to add a comment here, "Load libraries", and the first library we're going to import is going to be torch. It's going to be PyTorch. I run the code, and it's loading. Everything is OK. The other library we're going to import is going to be torchvision.

So I'm going to run this code, and everything should be OK. So there we have it. Now we're going to load the data. So I'm going to change this to Markdown, and I'm going to add a title, "Load data". OK, so the first thing that we're going to do is... well, this dataset in torchvision comes separated in two parts: the train and the test. OK, so first we're going to load the train part. OK, so the train dataset. So this is going to be trainset. We're going to use the torchvision library, and we're going to use the datasets function, and then we're going to load this CIFAR10 dataset.

OK, and this function has a set of different parameters. The first one is that we are going to say, well, actually, we can download this dataset. So we're going to download this dataset in the same folder, and this is going to be called data. OK, so in this folder, we're going to download the dataset. OK, so we're going to set the download parameter to True in order to download the dataset. And then, as this dataset has two parts, training and test, we're going to select the train part, and we can do this by adding the train parameter and setting it to True.

And that's OK. Now we can run the code, and this should start to download the dataset. OK, as I already had the files, this runs immediately. OK, but if you don't have the dataset, this is going to start to download, and this can take time. OK, so the next thing that we're going to do is to trans-

form this dataset. OK, this dataset here... we are going to use PyTorch, so we have to transform everything to tensors. OK, so in order to do that, we can use another parameter here, and this is transform.

This is a very easy way to add different transformations that we can apply to our dataset. OK, so we're going to create here a variable in which we're going to define the different transformations that we're going to apply to this dataset. OK, so here we're going to create a variable in order to transform, transform. OK, so I'm going to put it over here, and I'm going to use the transforms library. This is a library that we can use from torchvision.

So we're going to import torchvision.transforms, and we're going to call this transforms. OK, so now we can use this name, and we are going to use the Compose function. And this Compose function allows us to add an array of different transformations. OK, so the first transformation that we are going to do is to transform this dataset, this entire dataset, into tensors, OK. So in order to do that, we can use the same transforms and then we can apply the function ToTensor.

And this is a very easy way to transform all this dataset into tensors. OK, so if we run this... excuse me. If I run this, everything is correct, and now I will run this, and everything is correct. So so far we should have this dataset in tensor form, OK, or tensor format. OK. The other thing that we're going to do is to normalize the data. OK, when we're working with this dataset, specifically, this is a set of images. OK, this is a set of images, and each image contains numbers; each pixel contains numbers.

Right. And each pixel can range from zero to one. OK, so when we are working with neural networks, normalizing the data usually helps the training. OK, so we are going to apply, excuse me, we're going to apply a normalization here. OK, and we're going to normalize the data. OK, so we're going to normalize from zero-to-one to minus-one-to-one, OK. So in order to do that, we can use another PyTorch, excuse me, another transform function.

So transforms, and we can use the Normalize function, and the Normalize function is going to receive two parameters, two tuples. So the first one is going to be the means. OK. So you have to add the means first. The parameter is going to be a tuple of means, and then you're going to have another parameter, and this is going to be another tuple, and this is going to be the standard deviations. OK, so this normalization is going to normalize the data using the mean and standard deviation.

OK, um, as we have these images here, we also have that these images have three different channels. OK. And you can think of this as RGB. OK, so each of these channels is for a different color. OK, so we have three different channels. We're going to have three different means. OK. So basically we have to apply the mean and the standard deviation for each channel. OK, so the idea here is that we are going to add the mean for every channel.

So this is 0.5 for the first, 0.5 for the second channel, and 0.5 for the third channel. OK. So these are the means, OK? And this is because this is the range from zero to one, so the mean is 0.5. OK. And then we're going to have the standard deviation for each channel. OK, so this is going to be 0.5, 0.5 and 0.5. OK, so that's all. So again, we are going to take our dataset, and first

we're going to transform every instance to a tensor, a PyTorch tensor, and then to every instance we're going to apply this normalization in order to normalize the data from zero-to-one to minus-one-to-one. OK, so I have run this code, and I can run this code again, and everything is OK. So, um, we have to think here that the training dataset contains 50,000 different instances. OK, so this is a big dataset. So as we're working with a big dataset, we are going to apply mini-batch training.

OK, so in order to do that, we are going to apply this approach. OK, so instead of the entire training dataset, we are going to separate, or split, this entire dataset into mini-batches. OK, so each mini-batch is going to be used to update the parameters. OK, so we can implement that using PyTorch again. And so the first thing that we have to do is, well, the first thing is going to be to create an iterable. And in PyTorch we can use something called DataLoader.

And basically DataLoader is a method that allows you to convert a dataset, the entire dataset, into an iterable with mini-batches. OK, so we're going to create a trainLoader variable, and we are going to use torch.utils.data.DataLoader, and with this function, we're going to create the data loader. So the first argument, or the first parameter, should be the dataset that we're going to use. So this is going to be trainset, and then we have to add the batch size.

So I'm going to create a variable, batch_size, and I'm going to set this variable, I don't know, to one hundred twenty-eight. Um. Then we can add a shuffle parameter. Shuffle means that for each different epoch, you are going to shuffle the dataset in order to get different batches. That means that for every epoch, each batch is going to contain different data; it is not going to be the same data for each batch. OK, so if you want to add that, you have to set the parameter shuffle to True.

OK, and finally, I'm just going to show you that you can work with different workers. So, um, you can add that using the num_workers parameter, and I'm going to set this to two, OK. So now I'm going to run this code, and we have an error here because I forgot a comma here. So I'm going to run it. Um, this is wrong, and this is because this is DataLoader. OK, so everything is OK. So now I'm going to show you what this data loader looks like. So in order to do that, I'm just going to run a test code.

So I'm going to show you what this data loader looks like. So we're going to iterate through this trainLoader. OK. So in order to do it, we can use a for loop. So this is going to be: for data in trainLoader, we are going to print, um, for example, the len of data. OK. So if you see here, we are getting two elements. OK, so let's see what these elements are. I'm going to copy this, uh, and I'm going to remove it from here.

Give me a second, and then I'm going to create a new cell in order to run this. So, um, it is the same. So I'm going to stop this. So, for example, I'm going to take the first element of this. This is going to be a tensor, OK? So when we are using the trainLoader, basically when we are iterating, for each iteration, we are getting two elements. OK, the first element is going to be the data itself. So in this example, it's going to be the different images, and the second element is going to be the labels.

OK, so, um, I'm going to print the labels here. So these are the different labels for the different images. OK, so as we're working with ten different classes, we're going to have classes from zero to nine. OK, so these are the different classes. OK, um, and the important thing here is to try to understand what the batch size is. So in order to get the batch size, I'm going to get the images. OK. So the images are going to be organized in batches. OK. So in order to see that, I'm going to print the shape here.

So here I am taking the images and I am printing the shape. OK, so I'm going to stop this. And as you can see here, we have this shape for each batch. So we have the first batch here. Then we have the second batch here, the third batch here and so on. OK, so for each batch we have a tensor of this shape. OK, so the first dimension of this shape is the batch size. OK. So as you can see here, the batch size is one hundred twenty-eight. So for example, if we change this to 64, just to check, this is going to be...

Now the number is going to change, right, because the dimension of each batch is going to be sixty-four. OK, so this is the batch size. And that means that for each batch we're taking 64 different images from the dataset. OK, then we have this. OK. And the number here is the number of channels of each image, OK? And as I said to you previously, this is an image, and each image contains three different channels, because for each channel we have each color.

OK, so this dimension here is the number of channels. And then we have this matrix here, and these dimensions: the first one is the height, this one here, and the second one is the width of the image. So this is here. OK, so basically, for each image we have three different channels, and each channel contains an image of 32 by 32 different pixels. OK, and each of these pixels contains numbers between minus one and one, because we are normalizing this data.

OK, so again, each of these lines represents the shape of the different batches, or mini-batches. So you have to keep that in mind, because now we're working with batches. OK, so let's continue. I'm going to comment this line of code, and let's continue. OK. So so far we have the data loader, the trainLoader, for the training set. OK, so now we're going to create the same thing, but for the test data, validation and testing. OK, because we are going to split our dataset into training, validation and testing.

OK. So in order to do that... as I said to you previously, so far we have train and test. OK, so if we want to have validation and test, we have to split the test. OK, so first we are going to get the test set of CIFAR, and then we're going to split it. OK, so we are going to load the valid and test dataset, and I'm going to call this testValidSet. And this is going to be torchvision.datasets.CIFAR10. So the first thing is that we're going to load the data into the data folder.

Then we're going to have download True. Then we are going to select the test part of CIFAR. So in order to select that, we're going to set the train variable to False, because we don't want the train part; we want the test part. And finally, we're going to apply the same transformation. So that means that we are going to apply this transformation here, and we're going to transform every instance to a tensor, and then we're going to normalize this data, OK. So we can run it in order to check if everything is correct.

And here we go. So we have this dataset, and this dataset here contains all the instances for the valid and test. Right. So now we have to split this into different subsets. OK, so we're going to split this dataset, and we're going to get the validSet and testSet. Um, in order to do that, we can use the utils functions, then random, excuse me, data, and then random_split. And this function is going to split the dataset. OK, so the first argument here is going to be the dataset that you want to split, and we're going to split this dataset here.

And then you have to add an array saying what size you want for each set. OK, so as we're working with CIFAR, CIFAR contains 10,000 different examples. Here it is; this is the official web page of CIFAR. So CIFAR contains 10,000 examples, or samples, for the test set. So we're going to split this into five thousand and five thousand. OK, so we're going to have five thousand for valid, five thousand for test. OK, um, now we can run this, and everything is correct.

So so far we have the valid and test data. OK, so again, we're going to create, um, data loaders in order to work with the batches. OK, so first we're going to create the validLoader, and this is going to be torch.utils.data.DataLoader. And the first argument is going to be the validSet, because we want to create a data loader using this validSet. Right. Then we're going to add the batch_size variable. And this is going to be the same as we were using for training.

Uh, the same here. So in this example, it should be sixty-four, and then we are going to set shuffle to False, because we don't need to shuffle this dataset. And finally, we're also going to add different workers. We're going to add two different workers. OK, so now we can run this code, and everything is OK. So now we can, for example, test this again. But instead of iterating through the trainLoader, we are going to iterate through the validLoader, just to check.

So here we have the different epochs, excuse me, mini-batches, and for each batch, we have tensors of 64 different instances, basically 64 different images of dimension three by 32 by 32. And these are the different batches. OK. And as you can see here, here we have the last batch, and the last batch, instead of containing sixty-four, only has eight different instances. And this is because we don't have enough data to have sixty-four images again.

Um, so, um, here we have the valid data. OK, so finally we have to create the test data loader. OK, so we're going to call this testLoader. So again, we're going to use torch.utils.data.DataLoader, and then we're going to use the testSet. And then, well, actually, when we are testing, we're not going to train the model. Remember that when we're training, we use mini-batches, right. When we are testing, basically, we are getting different metrics on the dataset; we are not training.

So we don't need the batch size here. Uh, we also don't need the shuffling, because we're not going to shuffle anything. And again, we're going to set num_workers to two, OK. So we can test it. OK, everything is OK. So now we can iterate through the testLoader. OK. So as you can see here, we are iterating over the entire test loader. So we're not using batches. So as we are not using batches, for each iteration we are getting one instance, or one image. OK.

Um, that's all. So for now, we have the different loaders for train, valid and testing. OK, so so far we have created the different structures in order to work with the data. Right. And the last thing that we're going to do, I'm going to remove this cell here, is to add the different classes. OK, and in

order to do that, I'm just going to create an array with the names of the classes. OK, so the first class is going to be plane.

The second one is going to be car. The third one is going to be bird, then we have cat. Then we have deer. Then we have dog. Then we have frog. Then we have horse. Then we have ship. And finally we have truck. OK, so we're going to use these different classes in order to identify the different classes with a string. OK.

LECTURE 16

# Image and Iter Function

See the notebook here.

## Visualize data

OK, so now we're going to display different images, to have the opportunity to visualize them. So I'm going to create a comment here, a comment, "Visualize data". And the first thing that we're going to do is to build a function to display an image. OK. And so I'm going to call this function imgShow. And this is going to receive an image, OK. We're creating a new function because we have to apply different functions before displaying the image.

So as we are taking different steps, I'm going to add all these steps in just one function. OK, so the first thing that we have to do is to unnormalize the data. And this is because we are normalizing the data here. Right. So the data that we're using is normalized. So the first thing is to unnormalize, in order to return to the original data format. To do that, we are going to take the image, divide by two, and add 0.5.

OK, so in this way, we are unnormalizing from minus-one-to-one to zero-to-one. OK, and the second step is that we have a tensor, and in order to plot this thing, we have to use NumPy. So we have to convert this to NumPy, OK. So this is going to be .numpy(), OK, this here. The third step is that, if you remember here, I'm going to iterate, just to display, over this trainLoader and print data[0].shape.

So I'm going to stop this. This is an image, right? So the shape of an image, or the dimensions, are channels, height and width. OK, so when we're plotting, we have to use another shape: as we mentioned, first the height, then the width, and finally the channels. OK, so we have to transpose these things in order to display this image. OK, so, um, image transposed. So we're going to transpose... first, transpose, because the original dimensions are channels...

Well, I'm going to write this to be more clear: from channels, height, width, to... we need first the height, then we need the width, and finally we need the channels. OK, so in order to do that, we can use the NumPy transpose function, in which we have this image here, um, and then we apply the transposition. For the transposition you have to add the new axis order that you want. So for example, if we want first the height, we need to add the index of the height first.

So this would be one; then we need the width, this would be two; and finally we need the channels, so the channels are at index zero. OK. Um, I think we should have a problem here because we need to import NumPy. So we're going to import that library: import numpy as np. So, um, that's OK. OK, so now we can plot the image. OK, so in order to do that, we're going to use the library, and this has a function imshow, in which we can add the image to show.

OK, and finally, we have the plot show function in order to display the image, OK? And again, I'm going to run this. We don't have errors, but we have to import the plot library. OK, so I'm going to import matplotlib, and let me take the name: this is matplotlib.pyplot as plt. OK, so everything is correct now. And now we can use this function in order to display any image. So now we have another job, and this is to get each image. OK, so as we are using loaders, for example, if we want to display images using the trainLoader, basically the structure of the trainLoader is an iterable, right?

Because we can iterate through this object. So in order to get an element from an iterable, we can use some functions to get anything from it without iterating, without using, for example, this. OK, so in order to do that, we need a set of functions in order to get different instances when we're using data. OK, so the first function that we have to use is the iter function, and this iter function

is going to call the iter method on an iterable. OK, and the iter method allows us to get elements from the iterable. OK, so in order to call that method, we need to use the iter function. And so we have to call this iter function on an iterable. So we are passing here the iterable that we want to use. OK, so basically this function is calling the iter method on the iterable object. And this method allows us, again, to get elements from the iterable without iterating through it.

OK, so then, for example, we can get the first element, or the first item, of the iterable. And each item in the iterable is going to be a batch, right. And one batch is going to contain, in this example, sixty-four different images. OK, so we are going to get the first element, OK? And the first element is going to contain different images and different labels. OK, so now we're going to take this dataIter object, and then we're going to call the next method.

OK, so the next method is going to return the first element of the iterable. OK, so that means that, if we are using the trainLoader, we should get this batch. OK, so just to check, because this can be a little bit confusing, I'm going to print this element here, this one here, and I'm going to print the shape. So as you can see here, this element is the first mini-batch. OK, so this mini-batch contains sixty-four different elements. OK, so now I think it is more clear.

OK, so, um, as we are getting sixty-four different images, and we are just using one image at a time in our function, we are going to select different images. So I'm going to create an index, and this is going to be imageIdx. So for example, we're going to take the first, and finally we're going to plot the images. OK, so in order to plot the image, we're going to use our function imgShow, and we are going to pass the images array,

but with this index. OK. So the images we get are going to be this array of different images, and the image at this index selects the first image from here. OK, so now if I run this, now we have the image. OK, so, um, now, for example, we can print the label. So here we have the labels, so I can print the label of this image. So the label here is a number. So instead of printing the number, I'm going to remove this, we can use this array.

OK, so we are going to add classes here, and we should get the name, OK. So as you can see here, we have the plane, and this is the image of the plane. Then we can print again: this is the car, and we have another plane here. This is a dog, I think; this is the kind of thing we get. And then we

have the horse, then we have another dog. Then we have a frog, then another frog, then we have a truck and so on. OK, for example, we also can print... as you can see here, we have close to 30 on these axes.

Right. And this is the same dimension. Right. And this is because the shape of each image is 32 by 32. Right. But we can print images[imageIdx] and we can print its shape. So the shape here for each image is going to be 32 by 32. OK, so, um, well, the channels are first here because we are printing the original array, and in this function we are transposing everything. OK, so as you can see here, we have different images in our dataset with different labels.

LECTURE 17

# Create Convolutional Neural Network model

OK, so now we are going to start building, or defining, our neural network. OK, so in order to define a neural network, we are going to use PyTorch, and basically PyTorch is a library that allows us to build deep learning models, or machine learning models. OK, here is the official web page. You can download the library here using your different configurations, but that is it. So we are going to use PyTorch. OK, so first I'm going to create a comment here, "Define convolutional neural network model, CNN", and then we're going to start.

OK, so the first thing that we have to do is to add the comment "define model". And basically when we are building a neural network model using PyTorch, we have to define two different, two important methods. The first one is going to be the init method. And this init method is to define the layers that we want in our model. And then we have the forward method: how data flows through them. OK, so we have to create these two different methods in order to have our model.

OK, so let's begin. First, you have to create a class, and this class has this parameter, nn.Module, and the first thing we have to create is the init method. OK, "define init method". OK, so in order to do that, we're going to use the def keyword, and we're going to add the method, and the parameter is going to be the self object. So you have to add self. OK, so the first thing that we have to do is to load the initial, I don't know, the initial information from the parent.

Well, I don't want to talk about this stuff, but basically you have to add this method in order to have all the required... yeah, the required methods to work with the model. OK, so the first thing that we are going to create is the different layers inside this init method. OK, so if we look at our model, this is the model that we're going to create. So these are the images, and we already have these images. So the first layer is going to be a convolutional layer.

OK, so this convolutional layer is going to have six different filters, and after the filters, we're going to apply our ReLU function. OK, after the convolution layer, we're going to have the pooling layer, a max pooling; then we're going to have a flattening. Then we're going to have a fully connected layer, a hundred neurons and a ReLU function. And finally, we're going to have the output layer, in which we are going to have 10 different neurons. OK, so the first thing that we're going to do is to define the convolution layer.

OK, so here we're going to create a comment, "conv 1". And in order to create a convolution layer, in general you have to use the self variable, then add the name that you want for your layer. So in this example, we are going to add the name conv1, and then we are going to use nn. But we have to import this first. So I'm going to import it first, before I forget. So I'm going to import torch.nn as nn. OK, um, so now we can use this. So OK, so here we can define the layer that we want to use.

OK, so as we are using images, and these images are two-dimensional, we are going to use a convolution of two dimensions, Conv2d. So here we have a set of parameters, and the first parameter is going to be the input channels, and in this example, with these images, we have three different

input channels. OK, so the first parameter should be three. Um, then the second parameter is the output channels, or, for example, how many feature maps.

How many feature maps do you want? So if we come back to our model here, we're going to have six different feature maps. That means that we're applying six different filters. OK, so the parameter here is going to be six, because we are applying six different filters. So we're having six different feature maps, or six different outputs. OK, and finally, the final parameter... we have more, but we're going to add the final parameter, and this is going to be the kernel size, or the filter size.

OK. So in this example, we're going to use a size of five. OK, so with this line of code, we have created this first layer. OK, so again, you have to remember that this is only the definition. We're not saying how the data is going through this. OK, so the next layer is going to be the pooling layer. So we are going to create that layer. This is going to be the pooling layer. And again, we are going to use the self object, and then we're going to add the name of the layer.

In this example, we're going to use the name pool, and then we're going to use nn, and we are going to create the max pooling layer. And as we're using two dimensions of the input, we're going to use MaxPool2d. And again, this has parameters. The first parameter is going to be the kernel size, and in this example, we're going to use a kernel size of two, but you can use the kernel size that you want, and then we are going to set the stride; remember, how many pixels you move, or slide, your filter over your image.

OK, so here we are going to use a stride of two. That means that we're going to move our filter two pixels at a time. OK, so that's all. Here we have our pooling layer. OK, then we have the flattening. But if you remember, flattening is not a layer itself, because it doesn't contain neurons; it is only a reshaping. OK, then we're going to have another layer, and this is going to be the fully connected layer. OK, so let's create that layer. OK, so this is going to be a fully connected layer.

So for the fully connected layer, we are going to use the self object, and then we're going to add the name fc1, and we're going to use nn, and we're going to add the Linear layer. And this is the layer for a fully connected layer. OK, so the parameters here are the input dimension and the output dimension. OK, but well, we are going to have a problem here, well, not a problem, but, um, when you need to set the input dimension here.

OK, so the input dimension should be the output dimension of the pooling layer. Right. Excuse me. So as you are applying a pooling layer, this is resizing the image, right, and previously you were applying a convolutional layer. Right. So this is also changing the dimensions. OK, so the original dimension was 32 by 32 for each image. Then you apply a convolution with a kernel size of five, and then you are applying a pooling. Right. So you have different operations that are actually going to change the shape of the image.

So in practice, it is kind of hard to know what the input dimension to this layer is, because it is not clear what the dimension of the output of the pooling layer is, right? Actually, you can compute that using the formulas, but here I'm going to show you a trick that you can use in order to check the shape. OK, so first, I'm just going to set a random number. So, for example, I'm going to set the number 10 as the input dimension. OK, and this is part of the trick, OK, because then we're going to figure out what the real number is.

And with this real number, we're going to replace this. OK, so this is going to be the input dimension, the temporary input dimension. But now we're going to add the output dimension. OK, so the output dimension basically means how many neurons your layer has. OK, so in this example, we are going to have 100. OK, so the input dimension is going to be 100. OK, but here we're going to add a parameter, and this is going to be hidden_layer, and we're going to add this parameter here in the middle.

OK, so we're going to set the hidden neurons to one hundred. OK, so that's all. OK, then I'm going to show you how to get this input dimension. OK, but let's say that we are OK. OK, so here we have defined this layer, and finally we need the output layer. Right. And the output layer is going to contain one layer, excuse me, one neuron per class. OK, so we're going to have here ten different neurons. OK, so we're going to define that layer here.

Output layer, and the output layer is going to be self.out; this is the name, remember. And we're going to use the Linear layer, and, remember, the parameters are going to be the input dimension and the output dimension. So the input dimension is going to be the output dimension of the previous layer, of this linear layer. OK, so this is the hidden layer dimension. OK, um, this is going to be one hundred, right?

And then the output dimension is going to be ten, because we have ten different classes. OK, so so far we have defined the convolution layer, the pooling layer, the fully connected layer and the output layer. OK, um, for each convolution layer and fully connected layer, we have the ReLU function. OK, so we have to define that too. This is going to be the activation function. So in order to define that, we're going to call this act, and we're going to use the ReLU function.

So this is going to define the activation function. OK, so so far we are ready with the layers, and now we have to define how to move the information inside. OK, so in order to do that, we have to define the forward method. OK, so we are going to define the forward method, and this method is going to receive the object, self, but also it is going to receive the data. OK, so, um, well, the trick to check the dimension of this fully connected layer is to check the shapes of the data.

OK, so we should check the shape of the data at the output of the first convolution, and then we should check the shape at the output of the pooling layer. And knowing this shape, we can add the correct dimension here. OK, so, for example, I'm going to print the shape of the input data. OK, just to show you how to check the shapes. OK, so I'm going to run this model, and everything is OK.

So now I'm going to define the model in order to check the different shapes, in order to build this model correctly. OK, so we're going to define this model, and we're going to create a model variable, excuse me, and we're going to call this model using the correct name. So everything is correct. And now, actually, we can print this model. And this is the model. We have the different layers here: conv1, pool, fc1, out and the activation function.

OK, so, um, I'm going to test the forward method. So in order to test the forward method, we need to pass the data. OK, so we're going to pass one mini-batch to this method. OK. So in order to get that mini-batch, we have to use the same function that we were using previously. So the first

thing is to call the iter method on the iterable. So I'm going to create the same thing that we created previously. We're going to use the iter method, and we are going to use the trainLoader.

Everything is OK now, and I'm going to get the first item. So this is going to return the first batch, right, with the images and the labels. And I'm going to take the dataIter and use the next method in order to get the first element. OK, so for example, I can print the images' shape, and the images are this mini-batch with sixty-four different images. OK, so the idea now is that we are going to use the forward method. OK, so I'm going to pass the data to the model.

OK, and in order to do that, we are going to take the model, use the forward method, and pass the mini-batch. OK, so if we run this code, this is printing this, and this is because we are printing the shape of the input data. So, for example, I'm going to add "hello" here. I rerun the model and I run this, and this is printing "hello", because we have "hello" in our forward method. OK, so using this strategy, or approach, you can check the different shapes of the data.

OK, so let's continue building this model. So the first thing, I'm going to remove this, the first thing that we are going to apply to our input data is the first convolution, right. If we come back here, we have our image, and to our image we're going to apply this convolution layer. OK, so let's do it. So this is going to be convolution one. And the idea here is that we are going to apply convolution one to our x, to our data.

OK, so if we run this, I run this again, and I run this again. We are not printing anything because I have removed this, but everything is correct. OK, so for example, I can print the shape of the output of the first convolution, and this is going to be different from the input data. OK, so this print is printing the shape of the raw input data, and this print is printing the shape of the data at the output of the first convolution.

OK, so after applying the convolution, excuse me, we're printing the shape after applying the convolution. So we should have the different feature maps, with the shape of each feature map. OK, so I'm going to run this, run and run, and voila. This is the input. This is the raw data. Right. This is the input data, with the different channels, and with the height and width, OK. But here we have something totally different.

And so, here we have the mini-batch dimension, and this is not changing anything, because this is the same, and this is basically the mini-batch, OK. But here the output dimension of this data is six, because the output dimension of the convolution here is six, because we're applying six different filters. OK, so that means that we're going to have six different feature maps. OK, and here is another important point: instead of having 32, as in the original image, we are reducing the dimension to twenty-eight.

And this is because of the convolution. OK, so the idea now is that we are applying the convolution, but we have to remember that for each convolution, for each feature map, after that we're applying the activation function. OK, so we have to apply it here. Excuse me. Let me be more clear. We are here in the model, and here, after applying convolution one, we are going to apply the activation function.

OK, so we use self.act, we enclose this, and this should apply the activation function, and this should not change the shape of the tensor, because we are only applying an element-wise operation, which is not changing the shape. OK, so the first layer is ready, right, because we're applying first the convolution and then we are applying the activation function. OK, so the next step is to apply the pooling, and in order to apply the pooling layer, we have to apply this layer here.

Right. So we're going to define x again, and we are going to apply the pool layer, and we are going to apply the layer to the output of the first layer. Right. And the output is x. OK, so that's all: we are applying the pooling layer to the output of the first convolution layer. OK, so again, we are going to print the shape. OK, so now we should have three different prints, and the last print should display the shape after the pooling. OK, so as you can see here, this is the raw data, this is the output of the first convolution, and this is the output of the pooling layer.

OK, so this is the mini-batch, these are the feature maps, and the pooling layer is not changing anything here, because we are applying the pooling to the six different feature maps. So the number of feature maps is the same, but here is the difference. OK, as you can see here, the output of the convolution was twenty-eight by twenty-eight, but after applying the pooling, we are getting 14 by 14. That means that we're reducing the dimension, you see, using the pooling.

OK, and that is the goal of the pooling. Right. The pooling's goal is to reduce the dimensionality of the images, or the feature maps. OK, so in order to check, for example, things, we can change... instead of having a kernel size of two in the pooling layer, for example, we can apply a kernel size of three in the pooling layer. And let's see how this changes. Remember, the output dimension was 14 by 14, and now, with a different

kernel size, we are reducing to 13. OK, so as you can see here, depending on the parameters you set, you are going to have different dimensions. OK, so I'm going to return back to two. So I'm going to run this. OK, so now we have the original one. OK, so now we have the first convolution, and then we have the pooling layer. Right. So if we return back, we have this layer here, the convolution, but also we have the pooling layer.

OK, so the next step is to apply the flattening. OK, so basically the flattening is flattening the feature maps. OK, so instead of having this tensor of 6 by height by width, we're going to have a flattened vector of one dimension, with the size of the multiplication of these dimensions. OK, so let's do it. And here we're going to apply the flattening, and in order to apply the flattening, um, well, the flattening means that we're going to flatten each element in the mini-batch.

OK, so the output of this flattening should keep the batch size, right, because we're going to have different images, and the idea is to flatten each image. OK, so the idea is that we are going to have a batch size, and in this example, we're going to have sixty-four different images. OK, so this will always be the same. OK, we're not touching anything in the batch. OK, but here is the flattening, and, you know, the shape of each image is this one.

OK, so instead of having six by 14 by 14, we should have one dimension. And this dimension is going to be the multiplication of these different dimensions. So the shape of this should be the multiplication of the dimensions. OK, so how can we get this? Well, we can use x.view, the view

function of torch. OK, so the view function is a function that allows us to reshape, because remember, flattening is just a reshape, so we can reshape tensors, and so we can get this shape.

OK, so in order to do that, we can keep this dimension, the batch size, and this dimension here always has to be the same. OK, so for the batch size, we can set the batch size here, 64, but instead of setting the batch size to 64, for example, we can use the minus one dimension, because usually, or sometimes, when you are exploring your training, you can change this number, you can change the batch size. So instead of changing the batch size every time here, you can set minus one, and minus one means that this is going to be the size that allows you to reshape your original vector into these dimensions.

OK, so you set this dimension, and the rest of the dimension is going to be minus one, OK, or simply you can set sixty-four. OK, but here I'm going to add minus one. OK, and here is the key point, because we are flattening this vector this time. OK, so the dimension that we want here is the multiplication of six, 14 and 14. OK, that means that we are going to flatten. You have to think that we have six different images, and each image is a 14 by 14 matrix.

OK, so instead of having a 14 by 14 matrix, we want a vector of one dimension. OK, so that is the reason why we are adding six by 14 by 14. OK, so in this way we can reshape this vector to have the dimension that we want. OK, so in order to check that, we're going to print the shape of this. OK, so now we can run this cell, and as you can see here, well, again, this is the input. This is the output of the convolution. Excuse me.

This is the output of the convolution. This is the output of the pooling. And finally, this is the flattening, the output of the flattening stage. OK, so here you can see that we are keeping the same mini-batch, sixty-four, right, because we have 64 different images. And for each image we have a tensor, a vector, of this dimension: one thousand one hundred seventy-six. OK, so this is the idea of the flattening. Instead of having this tensor here of 6 by 14 by 14, we have just one vector.

OK, so this is the idea of flattening. OK, so so far we have the first convolution, the pooling layer, and then we have the flattening, the flattened vector. OK, so the next thing that we can do is to apply this fully connected layer. But we have to remember that we set the input of the linear layer to the number 10, and this is a totally random number, because now we know that the output of the flattening layer is this number. So now the input of this fully connected layer is going to be this number here.

OK, so I'm going to copy that number and paste it here. OK, so now we have the correct dimension. OK, so that was the trick that I was telling you about previously, because when you are using this printing method to print the shape of the different tensors at each step, you will know exactly what the shape of the tensor is, so that we know what we need in our fully connected layer. OK, so now we're going to apply fully connected layer one. And so we're going to apply self...

And the name of this layer is fc1. So this is going to be fc1. We're going to apply fc1 to the output of the flattening layer. OK, so if we print... this is the output of the fully connected layer, and this is correct, right, because the input layer is this number and the output layer is going to be 100, because 100 is the hidden parameter we set here, OK? And again, this is only applying the fully connected layer, but we have to apply the nonlinearity.

And this is going to be the activation function in the first fully connected layer. OK, so I'm going to run, and this is OK. And we have this fully connected layer, and finally, we will connect this fully connected layer with the output layer. OK, so this is going to be the output layer, and this is going to be self, and we're going to use this name, out, and we're going to apply the output layer to the output of the fully connected layer.

OK, and then we're going to print the shape. We run, and voila, here is the output layer. OK, so finally, for each image in the mini-batch, we are getting 10 different elements, or 10 different items. And this is because we have ten different classes. One element, or one item, represents one class. OK, um, so in this output layer we are not going to apply the activation function. OK, so we are not going to apply this function here.

OK, so finally we have the different layers, right. We have the convolution, pooling, the flattening, the fully connected layer and the output layer. OK, so the last thing that we have to do is to return the output. OK. So in order to return the output, we only have to return this output here. OK, I'm going to add a comment here, "return output". So this will return an array. Right. And we can, for example, print the shape of this. And actually this is the same that we are printing here, and that means that we're having 10 different elements, or 10 different probabilities, for each of the elements in the mini-batch, or for each of the images.

OK, so the last thing that we're going to do is to comment out the prints, because printing was only to check the shape and check that everything is OK. Actually, printing is a very good way to test a model. And I'm going to remove this last line. I'm going to, for example, create a variable, test, in order to print it. OK, so now I'm going to print, for example, the test variable. I'm just going to print the shape, and that is OK. So this is the final output of this model.

We are printing, for the different elements in that specific mini-batch, the ten different probabilities, or numbers, for each class. OK.

LECTURE 18

# Define cost function and optimizer

OK, so now we are going to define the different methods in order to train our model. OK, so I'm going to create a title here, and the first thing that we're going to define is the cost function, and then we're going to define the optimizer. OK, excuse me. So remember, when we are training our model, we need the cost function, or the loss function. And this function allows us to know how our model is doing; basically this function allows us to measure the error that our model is getting in the training.

OK, so with this error, we can first compute the gradient, and then we can update the parameters in order to minimize this cost function. Right. So we're going to define this cost function here. It is going to be costFunction. And we're going to use the nn package, and we're going to use the cross entropy loss function. And this function is used when we're working with multiclass classification problems. OK, so that is the cost function.

And now we're going to define the optimizer algorithm. And this algorithm, remember, allows us to update the parameters. OK, so I'm going to define the optimizer variable, and I'm going to use torch, excuse me, optim. And we're going to use the Adam algorithm. This is a very popular algorithm that we use when we are training neural networks. OK, and the first parameter is going to be the parameters of our model. OK. And keep in mind that we are updating the parameters of these different layers.

OK, but more specifically, where are the parameters? In the convolution, because remember, the convolution has different filters, and these different filters have different parameters. OK, so these are parameters that we are learning. OK, but the pooling layer doesn't have parameters. OK, because the pooling layer is just taking the maximum, in this example, of the different outputs of the convolution. It has no parameters. OK, then we have the linear layer, and the linear layer has parameters in the connections between the neurons, right, and it also has the bias terms.

OK, so again, in a fully connected layer, we have parameters, and these parameters are parameters that our model is learning. And the same thing holds for the output layer, because here we have the different weights and the different biases. OK, so basically we are updating the parameters of the convolution, of the linear, the fully connected, and the output layer. The pooling layer, again, has no parameters, and obviously the ReLU function has no parameters, because it's an activation function.

OK, so these are the parameters that we're going to be learning. And also we are going to add the learning rate, and this is going to be 0.001. OK.

LECTURE 19

# Train model train dataset

OK, so now we're going to start to train our model. OK, so first I'm going to write a comment here, and this is going to be "Train model". And let's begin. So remember our approach: the idea here is that we are going to run different epochs. Right. And in each epoch, we're going to work on the train dataset and the validation. OK, so the first thing that we're going to do is to implement the different epochs. OK, so in order to do that, we are going to iterate over epochs.

OK, so we're going to create "for epoch in", for example, "for i in range(epochs)", and we're going to print the epoch. So first we are going to define the epochs here, and this is going to be epochs. I don't know, I'm going to set five. OK, so now we print that. OK, so that is the first thing that we need: we have to implement the epochs. OK, so in each epoch here, the idea is to get the cost value on the training and validation datasets.

Right. So first let's get the cost value on the training dataset, OK? So in order to get the cost value, we need the labels first, and then we need the predictions. Right. So we shall implement that. But before that, we have to remember that we are working with this approach. Right. We are not working with the entire dataset; we're working with mini-batches. OK, so the idea is that in one epoch, we are going to have different mini-batches.

Right. And in each mini-batch we're going to get the cost function and update the parameters. OK, so let's implement that. So we already have the different epochs, and now we're going to implement the training, the job to be done with the training data. OK, so we're going to add "TRAINING" here, just to have a label. And let's begin with the training set. OK, so we are going to iterate over the different batches that we have in the training set.

OK, so in order to do this, we're going to create another loop, and we're going to use a temporary variable, tmp, and we're going to iterate over the trainLoader. OK, and, for example, just to check, we are going to iterate here over each mini-batch. OK, so let's remember that we have two elements, right? The first element is going to be the data. OK, the mini-batch, so we would print the mini-batch shape.

OK, so as you can see here, we are printing the different mini-batches. And let's print here, for example, the epoch. We're going to add here f"epoch {i}". So, excuse me, I'm going to stop this. Now we're going to run this. So this is for epoch zero, and in the first epoch, we're going to have the different mini-batches. OK, so we're going to iterate over all the mini-batches, OK, as we have 50,000 data samples. OK, so I'm going to try to find the next epoch.

Here you go. OK, so here is the second. OK, so from here, all these different samples are the different mini-batches that we have in just one epoch. OK, so in this second epoch, let me find it, in this second epoch, we are going to iterate again over the different mini-batches. OK, so that's the idea. So I'm going to run this again. We are having problems, but anyway. OK, so the idea now is that we are going to apply this logic here: for each mini-batch in each epoch, we are going to get the cost value.

OK, so the first thing that we are going to do is to get the predictions, OK, because in order to compute the cost value, we need the predictions and the labels, and the labels we already have. So now we only need the predictions. OK, so the first thing that we're going to get here is the data. And this is going to be tmp[0], right? Then we're going to get the labels, and these are going to be tmp[1]. So now we're going to get the model predictions.

OK, we are going to create the predictions variable, and we're going to take the model, and then we're going to apply the forward method. And here we're going to pass the data as the parameter, OK? So if we run this, this should work. But this is not working, because we have an error here with the tmp. OK, so there it is. So let's, for example, print the predictions' shape, and the predictions' shape should be the shape of the mini-batch, right, and the other dimension should be ten, one for each class.

Right. So we print, and there you have it: here we have 64 different instances for one epoch, excuse me, for one mini-batch, and the ten different elements here. I'm going to change the mini-batch size in order to have more data in one mini-batch, and I'm going to set it to 128. OK, so I'm going to run this code again, and basically we have to run everything again. I'm running the code again, and here we go. OK, so now for each batch we're getting one hundred and twenty-eight different images, or instances.

OK, so so far we have the predictions. OK, so now with the predictions and with the labels, we can get the cost function, right. So in order to do that, we're going to create a loss variable, and we're going to use the cost function that we have here. And first we're going to add, excuse me, we're going to add the predictions first, and then we're going to add the labels. OK, so this cost value should be a number for each epoch,

OK, excuse me, for each mini-batch. But OK, so, um, I think it's value... well, I don't remember. Well, we can remove this. Right. OK, so as you can see here, I'm going to stop here. I'm going to also print the predictions in order to display to you the shape of the different mini-batches. OK, so here is the first mini-batch, one hundred twenty-eight different instances. This is the output, and for this output of 128 different instances we're computing the cost value, and the cost value for each mini-batch is going to be one number.

This is the value of the cost function. OK, so if we come back here, the first thing was to get the cost value, right. Now, in the training set, the next step is to compute the gradient value, right, because with this gradient value we can update the parameters. Right. So let's compute the gradient value. OK, so in order to compute the gradient value, we are going to use the backpropagation algorithm, right. So when we're using PyTorch, we can easily compute the backpropagation; we can compute the gradient value using backpropagation because we have a built-in function, and this is backward.

OK, so with this function, we can basically compute the backpropagation. OK, easy. So there you have it; we don't have errors. OK, so now we have to add something important here. When we're working with PyTorch, by default it saves and stores the values of the gradient each time you call the function that we're going to use. So in order to avoid this storing, or the accumulation of the gradient value, we want to get a unique value for each mini-batch.

Right? I mean, each epoch. OK, so we need an absolute value; we don't need an accumulated value. OK, so in order to do that, we always have to set the gradient to zero each time we compute the gradient. OK, so in order to do that, we use the function "gradient to zero", and this is optimizer.zero_grad. OK, so that's all you have to do in order to set the gradient to zero. OK, so now we can run again. I'm going to remove this print here, and we can run.

OK, so this step is to compute the gradient. So at this point, we already have the gradient value. OK, so the last step is to update the parameters using the algorithm that you want to use. OK, so in this example, we are going to update the parameters using Adam, right, because we have defined this method here. OK, so here we are going to update the parameters. And in order to do that, we only have to use the optimizer object, and we're going to use the method step.

OK, so the step method is going to take the gradient from the loss value, and then it is going to update the parameters using Adam. OK, so, um, so now, in each mini-batch and in each epoch, we should get a new update of the parameters. OK, so we don't have any errors, so we can see that we don't have errors. But the most important thing is that here, so far, we update the parameters. OK, so now we are ready with this. We did all the steps that we need in order to update our parameters, OK.

So so far, our model is improving in every mini-batch and in every epoch, because we are updating the parameters in each mini-batch and each epoch. OK, so now, in theory, we should have a decreasing value of the cost function in each iteration. OK, so in order to check that, we're going to plot the cost value. OK, but first, instead of plotting the loss value for each mini-batch, we're going to compute a loss value for the entire dataset, and then we're going to plot it.

OK, so in order to do that, we are going to create here, I will call it, epoch_train_loss. OK, so this is going to be epoch train loss. So we're going to set the initial value. This is the "loss epoch train". So we are going to... give me a second. We're going to sum this value for each batch. OK, so "sum loss values". OK, so in each batch, we are going to update this, and we're going to get the loss value right here. So let's see if we have some error.

It seems we don't. OK, so if we want to plot these values, we have to create an array in order to store the values. Right. So we're going to create the train loss array, excuse me, train loss array. And this is going to be trainLossArray. And this is going to be np.zeros with the shape equal to the number of epochs. Right. So for each epoch we're going to save the train loss value. OK, so now, when we finish this line, we have already finished iterating through the mini-batches, right; here we have an entire iteration over the entire dataset.

OK, so in this line of code, we're going to add the loss value. OK, so we're going to save the loss value of epoch i, excuse me. And we're going to add the loss value of that iteration. Right. And this is going to be this loss value. Right. But as we are summing this loss in each mini-batch, we're going to divide this. So we are going to divide this, because basically we want to normalize this value.

So we're going to divide this by the number of samples in the dataset. OK, so in this example, we're going to divide the train summed loss by the number of instances in the train dataset. OK, so let's run this code to check if we have some error, and it seems we don't. So finally, if we want

to plot this, we already have the cost value on the train, summing over the train, and we have the different epochs. So now we can plot this.

OK, so in order to do that, we're going to create something here, "PLOT" here, and we are going to create the figure here. So give me a second. So here we're going to create a plot, "create plot". And so we're going to create fig, ax, and we're going to use matplotlib, and we're going to create a subplot. So we're going to build two plots. The first one is going to be the plot to plot the cost values, and the second one is going to be the plot to plot a specific metric.

OK, so we're going to create two plots here, and we are also going to add a figure size, a figsize parameter, in order to control the size. OK, so this is going to be 10 by 3. OK, now we can plot. So we are going to plot, first, "plot train loss". Right. So we are going to use the first plot in order to plot the loss value. We're going to add the plot, and we are going to plot the trainLossArray, and we are going to plot from the first element to the last epoch.

OK, and we're going to add a color here. It's going to be red. So now if we run this, excuse me, I also need another line of code in order to plot, and we have to add the fig.canvas.draw function. And with this function we can draw the plot, like, live. OK, well, this is strange. OK, so we're not getting the plot, and this is because we're using Jupyter Notebook, and if we want to add the plot, we have to add "%matplotlib notebook". OK, I think now this is...

Let me check. Yes. So this is "matplotlib notebook". OK, so you have to rerun everything again. And if everything is OK, we will get the plot here. OK, so we have to wait, because it's taking time, because it is a lot of data. So we should see that. OK, so here we are seeing the cost value for the train. OK. So basically, so far we have this plot, in which we can check how this is going on, and as you can see here, this is a preliminary view.

But we are decreasing the cost value. And this is because we are updating the model parameters each time we iterate through the entire dataset in each epoch. OK, so basically we should see that we decrease the cost value, because we are improving the model in each iteration.

LECTURE 20

# Train model validation dataset

OK, so now we are going to start to work with the validation dataset. OK, so the idea of the validation is that we are going to use this dataset to check the training. That means that we are going to get the cost value on this specific dataset, and we are going to plot this data in every epoch. OK, so with this cost value, we can check how our model is doing. For example, we can identify if we are having overfitting or underfitting, and so on.

OK, so the main idea here, and the most important thing here, is that we are not going to update parameters with this validation dataset. Right. We're just going to use it to check the training, OK, and basically we have to get the cost value. OK, so the first thing that we have to do... as we are working with mini-batches, right, we are working with this approach, in which we have different mini-batches, the same as we have with the training.

We first need to iterate through each mini-batch. So let's implement that, OK? So here is our code for training, and I am going to create a new section here. This is going to be "VALIDATION". OK, so the first thing that we have to do is to iterate through our validLoader. So this is going to be a for loop, and we're going to, for example, print the shape, just to check that everything is correct. So here we immediately have an error.

So now let's see... OK, so this is a typo, a typing error. So this is the training set, and here it is the validation set. OK, so here is the code to iterate through each mini-batch in the validation. OK, so the idea of the validation, again, is to get the cost value, and in order to get the cost value you need the labels, and then you need the predictions. OK, so let's first get the labels. So the labels are labelsVal.

So labels, and this is going to be tmp[1]. Right. And with this line of code, we already have the labels. So now we need the predictions. OK. So in order to get the predictions, we first need the data. So this is going to be dataVal, and this is going to be tmp[0]. Then we're going to get the predictions. OK, predictions. So we're going to call this variable predictionsVal, and we are going to apply the forward method of our model, and we're going to pass the data.

OK, so this gets the predictions on the validation data. So here we're having a problem. And this is because... oh, yes. Excuse me. Here we have an error because of the variable [unclear]. OK, so now if we run this code, we have to wait, and everything is OK. OK, so so far we have the predictions. OK, so now the next step is to get the cost function value. OK, so in order to do that, we have to do the same: lossVal, and we use the cost function, and to this cost function we pass first the predictions, and then we pass the labels, labelsVal.

So this should get the loss, right? So here we have a typing error, because this should be labels. So, uh, this should work. OK, so this is working. OK, so now we already have the loss value, right? But we have the loss value for each iteration. Right. So we have to get the value for the entire valid dataset. OK, so in order to do that, we are going to create "error valid epoch", and we are going to create the variable epoch_valid_loss.

We're going to set it to zero. And then in each mini-batch we're going to update this, and sum the error values in each epoch. So we're going to update this value in each mini-batch. Right. So we are going to get the loss item, and this is going to return the value. OK, so now we are getting the loss value for each mini-batch. OK, but finally, we want to get the loss value for the epoch. Right. So in the epoch, if we want the value and we want to plot that value, we should save it in a similar array, as we did with the train.

Right. So here we are going to save the train loss, excuse me, this is going to be the valid loss array, and we are going to change the name here. This is going to be validLossArray. OK, so, um, at the end of the iteration through the entire dataset, we should save the loss value of the epoch, right. So we're going to save that value at index i, summing the items, and then we are going to save this value.

And as we were normalizing previously with the train, we're going to divide the loss value by the number of samples that we have. OK, and in the valid dataset we are using five thousand. Right. Um, so now we... OK, so so far we already have the cost value in each epoch. OK, so now the final step is to plot the loss: "plot valid loss". Right. So we're going to plot this, and we're going to use the validLossArray.

And this is going to be from zero to the last epoch. And we are going to add a color. So, for example, we're going to add the green color. OK, green. OK, so if everything is OK, we should get the plot here, OK? So we are having a problem here, and this is this comma here; it should not be there. OK, so now it runs correctly. So we have to wait a minute. OK, so as you can see here, we have both: the green is the valid, the red is the train, OK, and as you can see here, we're plotting the values, OK?

OK, so now we have this plot. OK. So with this plot, in a real training, you can check how the training is going. OK, so based on this you can decide if the training is OK or if you have to change something. OK, in this other plot, we want to plot a metric. OK, so in this example, we're going to get the accuracy metric, and we want to plot the metric in this plot. OK, and we're going to plot this metric on the validation set, because this is unseen data.

OK, so with this unseen dataset, we can check correctly if our model is doing well or not. OK, so in order to do that, we are going to first create a metric array. So this is going to be metricArray, and this is going to have the same shape as the previous one. OK, so now we are going to compute this metric in each mini-batch. Right. So in order to do that, we have to create the same logic here. We're going to create "metric valid epoch", and this is going to be metric_valid.

And then we're going to set it to zero, and then we are going to compute this: "compute metric value in mini-batch". OK, so we're going to add this value. OK, so in this example, we're going to use the function accuracy_score. This is a scikit-learn function, in which the first parameter is going to be the true values. So this will be the labels, but remember, the labels of this mini-batch. Right. And then we are going to get the prediction values, and this is going to be this value here.

OK, but if you remember, the output of these predictions, for each instance, is ten different values, ten different elements. Right. So we want to select one, right, because that is going to be the prediction. So in order to do that, we're going to select the maximum value among these ten dif-

ferent classes. OK. And this is going to be done for each instance. OK, so in order to do that, we can use torch.argmax. And this is going to return the index of the maximum value in a vector, or the tensor.

OK, so we're going to get the maximum value of the predictionsVal tensor. And we're going to get the maximum value in the column dimension. Right. Dimension one. OK, and then, so far, we're going to have a tensor, OK, with the maximum value for each instance. OK, so basically the shape of this is going to be the number of epochs and one. Right. So then, this is going to be a tensor.

So we have to convert this tensor to NumPy. So in order to do that, we're going to use the detach function, followed by the numpy function. OK, so if we run this code, we first check that we don't have errors in order to continue. So first, we have an error here, and this is because we have to import this library. So we're going to "from sklearn.metrics import accuracy_score", and that should be right. Uh, no, no. This is from sklearn...

Import that. So that is correct. OK, so we return back here, and now we get the accuracy score for each mini-batch. So if everything is correct, we should go to the next epoch, and everything is correct. OK, so here this metric is for each mini-batch. OK, so we want to store the metric for each epoch. OK, so we're going to "save metric value for epoch", right. So in order to do that, we're going to take this array here, metricArray, and then we're going to save this item here, and we're going to take this metric_valid.

Right. And this is the metric value that we're computing in each mini-batch. That's right. Um, but this is going to be summed. So, for example, if this metric is one all the time, for example, if we have ten different mini-batches, we're going to have one plus one plus one plus one plus one. And finally, the number is going to be ten. And that makes no sense with accuracy; accuracy ranges from zero to one. So that doesn't

make sense. Right. So we have to divide this by the number of epochs. OK, give me a minute. So in order to do that, we have to count how many batches we are having. Right. So in order to do that, we are going to have a batch counter, batch_counter, and it is going to be zero. Then for each... excuse me, we're going to update the counter, and for each mini-batch we're going to update it. You can do it in the way that you want,

but you can use this, because this is going to work. OK, um, well, this is an implementation. OK, so finally, we're going to divide this metric by the number of batches that we have. OK, so in that way, we're going to have a mean of these metrics, OK, I mean, based on the number of batches. OK, so finally, we are going to plot this. We're going to plot the metric. So instead of plotting on the zero axis, we are going to plot on one, and we are going to plot the metricArray from zero to the last.

OK, um, if we plot this, we should get the result. OK. We have to wait. And now we have the different plots: one for cost, and the other one for the metric that we're using. In this example, we are getting the metric, excuse me, the accuracy value. OK. OK, so finally, we are going to run this again, because we are running this code out of time, and instead of having five epochs, I'm going to add ten epochs, just to show you the plots here.

So I'm going to run this code again. We have to rerun the model definition, and finally run this code again. So we're going to run this, and I'm going to stop the video and return when the training is ready. OK, so here we have the loss and the metric. OK, so as you can see here, both the train, the red one, is decreasing, and the valid is decreasing too. But they're decreasing at different rates. Right. So, for example, if you want to run a real training, you have to run this training with more epochs.

Right. This is only an example in order to show you how to train this. But if you want to train your model in a realistic way, again, you have to run this code with more epochs. OK, and the metric here, as you can see, is updating each time. That means in each epoch. And that means that in one epoch we are updating the parameters multiple times, because we have different mini-batches. But that means that we are improving the model in each epoch.

OK, so that is a really good behavior, and it is what we want when we are training, because we want that while our training and validation loss are decreasing, the metric is increasing. OK, so again, this is an example, but the idea is the same. Instead of having 10 different epochs, you can run more epochs, maybe hundreds or thousands of different epochs, in order to have a more realistic behavior of your model.

LECTURE 21

# Analyze model on test dataset and metrics

So in this video, we're going to start to work with the test data. OK, so if you remember, the idea of the test data is not to update the parameters; it is just to get metrics on your model, or to compare your model with other models. OK, um, so in this example, we're going to get metrics in order to get an overview of our model. OK, so the first thing that we're going to do is to add a comment here, "Analyze test dataset". Uh, excuse me: "Analyze on test data and get metrics".

OK. So in order to get the metrics, we usually are going to compare the predictions with the labels. So first we're going to need the predictions. OK, so we're working with batches, so we have to iterate through these batches. So we're going to copy this code. We're going to copy the for loop to iterate through the data loader. Then we're going to get the data, the labels, and get the predictions. OK, so we're going to copy this code here.

And instead of using the trainLoader, we're going to use the testLoader, and we can run this code. And this will run without any problem. It should be OK. So everything is OK. So now, these predictions, if you remember, the predictions' shape is going to be shaped by the batch size. And as we are working without batches, we are returning one instance, but we are returning 10 different elements for one instance.

So we only need one element, because this is going to be the prediction, the class. OK, so I'm going to remove this, and we are going to select the maximum value. OK, so in order to do that, we are going to store the predictions in a new array: "store predictions". And this is going to be the predictions array, and this is going to be predictions... it's going to be np... sorry, excuse me, NumPy: np.zeros with the shape equal to the number of instances in the test dataset.

And if you remember, this is five thousand. OK, so now we're going to save the prediction for the instance. OK, so this is going to be predictions, and we should get an index here. Right. But in order to get this index, we can apply the enumerate function, and this enumerate function is going to return an index, an ID, but also it is going to return the data. OK, so with this index, we are returning the index of the batch, OK, and as we're working with a batch of one, basically we are returning the index of the data for each instance, or each image.

OK, so here we're going to use the argmax function, and we're going to get the maximum for each image. OK, so this is going to be the argmax of predictions, and it's going to be dimension one. And then we're going to transform this to a NumPy array. OK, so if we run this code, we have this problem, because this new element has the same name. So we're going to call this predictionsTest, and that should be OK. So at this point, we're going to have the different predictions.

So if we take this prediction and we check the shape, this is going to be five thousand. So, for example, if we print, we're going to have the different predictions, the different classes. OK, so now we can get different metrics, because we already have the predictions on the test, but also we've

got the labels. OK, so first we are going to, uh, excuse me, we're going to get the confusion matrix, OK? And basically in the confusion matrix, we are going to get a matrix in which, in the rows, we are going to have the real labels, and in the columns we are going to have the predictions.

OK, so in order to do that, we're going to write the comment "get confusion matrix". We're going to use the confusion_matrix from sklearn. So first we have to import this. So we're going to import confusion_matrix here, and that should work. Now we come back here. So in the confusion matrix, the first parameter is going to be the labels, or the real values, and this is going to be labels. Then we have to add the predictions, and the predictions are going to be predictionsTest.

And then we can use the normalize parameter. So in this example, we're not going to add any normalization. OK, OK, so here we have an error. And this is because labels, if you print the labels' shape, is an array of one element. So this is because, when we are iterating through the testLoader, we have to get the labels from the entire dataset.

OK, so we're going to store the labels. This is going to be np.zeros; then the shape is going to be equal to five thousand. OK. And then for each instance, we're going to store the label, and this is going to be labels. OK, so now, instead of having labels here, we're going to have labelsTest. OK, so if you print the shape, the shape of this array is going to be 5,000. OK, so now if we run this code, now we have the confusion matrix.

Correct. So if we print, this is the correct matrix. OK, so instead of showing this matrix as a matrix itself, we're going to plot this matrix in a heatmap. OK, so in order to do this, we are going to import the library seaborn: import seaborn as sb. And here we're going to plot. OK, so in order to create this plot... excuse me, here we are going to "plot the confusion matrix". And first we're going to create the figure.

So we're going to use the plot library. We're going to create this plot, and we're going to set the figure size, and this is going to be, for example, seven by five. And then we're going to plot. OK, so we're going to use the seaborn library. We're going to use the heatmap function. And in the heatmap function, we are going to add the confusion matrix. OK, so now if we print this, this is the confusion matrix. OK, so first thing, we can add annotations.

So if we add annot equal to True, we're going to have the numbers. OK, if you see, these numbers are kind of overlapping. So we're going to add the parameter tight_layout to the figure. Tight layout, excuse me, give me a second. This should be tight_layout. OK, so now this is more clear, and what else can we do? For example, we can change the colors to a more clear colormap; a popular colormap is Blues.

So we're going to use the Blues colors. So these are more clear colors, because, you know, white is associated with lower values, and the darker colors are more related to higher values. OK, so, um, we have numbers, right, and this is very complex to understand, because I don't know what they mean. Well, actually, this is the number of cases, or instances, but it is really hard. Maybe instead of having numbers, we can have percentages.

Right. So in order to do that, we have to normalize our confusion matrix. OK, so we are going to add "true". That means that we are going to normalize: the number of cases divided by the true labels. That means, for example, that for class zero we have, I don't know, maybe five hundred in-

stances, and we're going to divide each of these instances in each of these classes... we're going to divide that by the total number of instances of that class.

That means, for example, in this example it should be 11 divided by 500. OK. Well, another thing... well, let's do this first. So now we have percentages, so now it is clear; it is more clear to me. OK, another thing here is that we have the numbers as names. And instead of having that, we can change the names of the labels on the axes, excuse me. So we can add the yticklabels, and we can create an array. You can just copy this, and we're going to use the classes array.

So now we have the real names, the real values, excuse me, the real names, and the same for the xticklabels. OK, and here we need a comma. OK, so now we can analyze the real confusion matrix. OK. So as you can see here, dark colors are related to higher values, OK, and light colors are related to lower values. OK, so in each row we have the real class. So for example, here we take all of the plane class, OK, and the columns are the different predictions.

So for example, this is the class predicted as plane, this is the class predicted as car, the class predicted as bird, deer and so on. OK, so on the diagonal of this confusion matrix, we have the real instances that were predicted as the correct class. That means, for example, for the first class, the plane class, sixty-two percent, excuse me, seventy-two percent of the class instances were predicted as plane. In this example, sixty-five percent of car were predicted as car, forty-nine percent were predicted as bird, and forty-nine percent of deer were predicted as deer.

Forty-nine percent of cats were predicted as cat, and fifty-two percent of [unclear], and so on. And outside the diagonal we have the different confusions that our model is making. OK, so for example, for the class plane, 62 percent, 72 percent, were predicted as plane, right. But, uh, for example, this one, this one: 0.02 percent of planes were predicted as deer. So in this way, we can see that 0.02 percent of the plane

class was confused with deer. Our model confuses the plane with the deer in this percentage. OK, so, for example, the most confused class was... well, the plane was confused the most with the ship class. OK, and if you think... I don't know if plane is related to ship, but this is the most confusing class. So let's see. For example, let's finally try to find a higher number for the confused classes. For example, this one here: the real class car was confused

21 percent with truck. And if you think about it, car and truck are related, right, they are similar. So it makes sense that the car class was confused with truck, right? It is correct. It makes sense. For example, we can see that here the cat class was confused with dog. So that is really common, you know, because you can confuse a cat with a dog. And for example, here the dog was confused with cat. OK, so that is the way in which you can use the confusion matrix.

You can find how a class is doing by following the diagonal, and what classes are being confused with each class. So, for example, here we can see that we are confusing cats with dogs: 17 percent of the cats were confused with dog, and 19 percent of the dog class was confused with cat. OK, so basically with these numbers you can see how you can improve your model, OK, and for example, here you can see: OK, if we have cats and dogs and they are related, can we do anything in order to improve this?

We can, for example, change our model, or maybe we can analyze the data, or I don't know, we can take different options in order to improve our model, OK, in order to avoid the confusion between the classes, OK. I learned that the only thing we can get here is that the best class is the plane class, because we have the higher number on the diagonal. OK, the worst class, the worst performance, was for the classes cat and dog and deer. OK, so these three classes are the worst ones, and this is, you know, OK.

And finally, with this you can check how your model is doing with the different classes, and how your model is confusing different classes. Right. But we can also see other metrics. OK, so we are going to use the classification report, and the classification report is a tool that you can use in order to get different metrics. OK, so first you're going to add the labels; they are going to be labelsTest. And then you have to add the predictions, and it's going to be predictionsTest, and you can run this.

And classification_report is not defined. So we have to import this function, which you can import from sklearn. So now we can run this. OK, so we can print this in order to have a better format. OK, so now we have the different metrics. OK, so in order to understand this, we have to check the different metrics. OK, so here we have different metrics for different classes. Here we have the ten different classes, and here we have different metrics for each class.

OK, we have the first metric, and this is precision, and then we have recall, and then we have F1 score. OK, support is the number of instances. OK, so as you can see here, all the classes are close to five hundred instances. OK, so when we are analyzing, we are analyzing different metrics, and you can choose different metrics, or one metric, and this totally depends on the work, or the project, that you are working on. OK, so for example, we can take the F1 score, and the F1 score represents a kind of mix between precision and recall, and we can, for example, analyze the F1 score for each class.

OK, so the higher the value of the F1 score, the better the metric. OK, so for example, here we can see that the best value was for class one, and class one is car. So the best class in F1 score is class one, and the worst class, for example, was class three, and class three, zero, one, two, three, is the class cat. And this makes sense, because on our diagonal, in our confusion matrix, cat has the lowest value, and so on.

OK, so you can analyze the different metrics. OK, and then we have other metrics that you can analyze in more detail. But basically the most popular one that you can use, or the easy one, is accuracy, and accuracy is the number of correct predictions that your model made, excuse me. So for example, in this example we have fifty-nine percent of correct predictions from our model, OK. And then we have another set of different metrics that I don't want to talk about in detail, but basically with the classification report you can analyze the metrics of your model.

OK, so this is the idea with the test dataset. The idea here is that you want to get different metrics in order to analyze the performance of your model. OK, so here we are just analyzing the model itself. OK, so for example, we can analyze the different classes, the performance in each class. OK, but also, for example, if you want to compare your model with other models, you can use some more general metrics, like accuracy or any of these other metrics.

OK, but the idea is that you get metrics on unseen data, like the test data, and you can use your metrics to analyze the performance of your model, but also to compare with other models.

LECTURE 22

# Practical advices about training

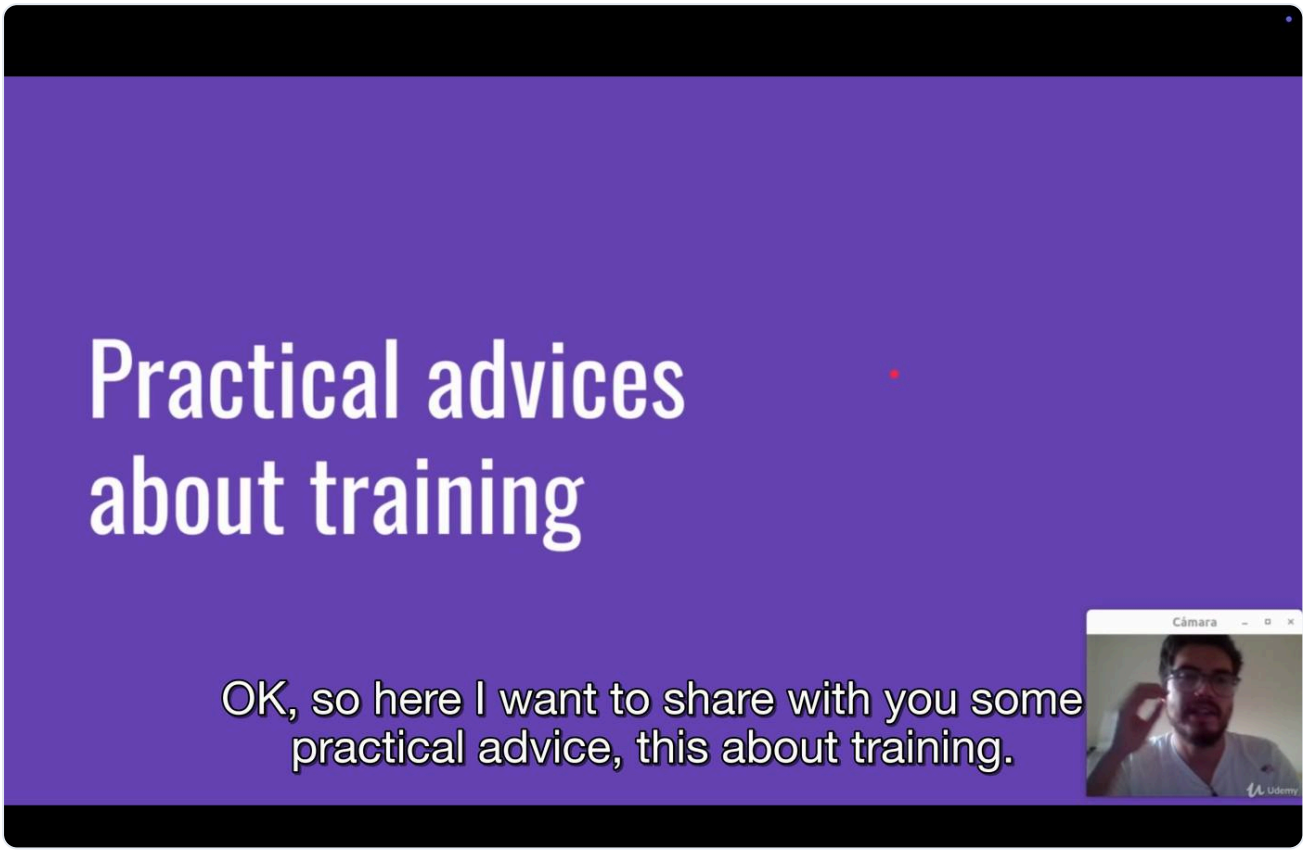



FIGURE 22.1 | Practical Advices

OK, so here I want to share with you some practical advice about training.

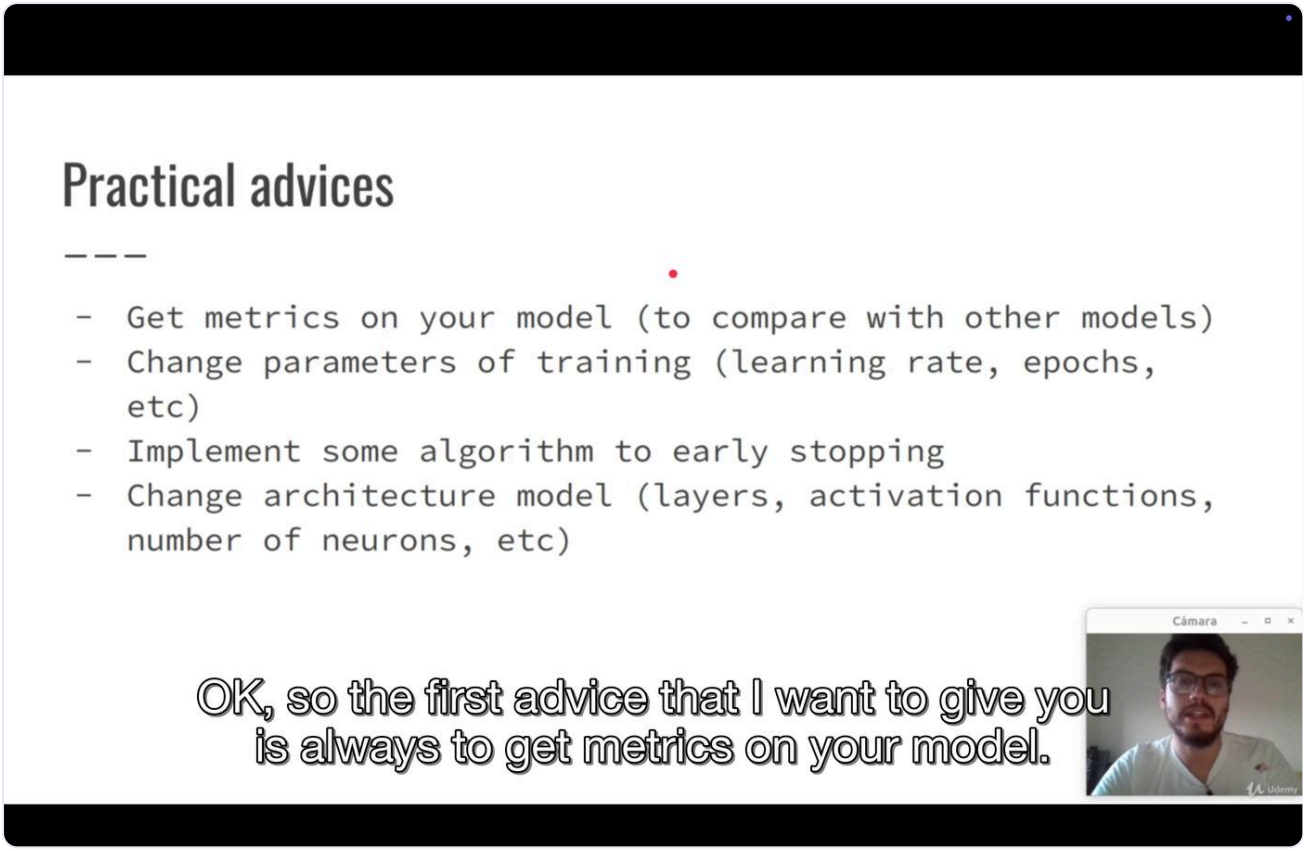



FIGURE 22.2 | Practical Advices

OK, so the first advice that I want to give you is to always get metrics on your model. These kinds of metrics are going to help you measure the performance of your model, but they also allow you to compare your model with other models. OK. Nowadays it's very common to compare different models, so the way in which you compare your models is by using different metrics.

OK, the other practical advice is that you can change parameters in the training. OK, so for example, you can change a lot of parameters: for example, the learning rate, the number of epochs, the hyperparameters of the model, and so on. Or you can try, for example, different training methods: instead of using Adam, you can use, I don't know, stochastic gradient descent, and so on. So the idea here is that you have to explore when you are training a model.

Specifically, with a deep learning model it is very common to explore different options, because everything is trial and error. OK, so basically you have to analyze different options in order to find the optimal set, or configuration, for your model. OK, now, a very typical, very useful tip is that when you are training, you can run your training for millions of different epochs. Right. But what happens when you start to have this overfitting situation?

The idea here is that you have to stop before getting overfitting, OK? The idea is to try to stop the training at the optimal point. OK, so in order to stop your algorithm, you should implement an early stopping algorithm that allows you to save your model when you find an optimum, or minimum, in your cost function. OK, so one of the main ways to implement this algorithm is, for example, by analyzing the loss value on the validation set.

So, for example, you can say: OK, if the validation value is not improving, if it is increasing its value for, I don't know, 10 iterations, then at iteration number 11 you are going to stop the training, OK, because that means that you found the minimum and then you started to increase the cost function value. OK. And the last advice is that the principal, or the main, problem, or not problem, but challenge, with deep learning models is to adapt the model to your specific data.

OK, so for example, when we are working with images, we're going to use convolutional layers, convolutional neural networks. Right. But you can select different numbers of layers, activation functions, neurons, filters, strides, pooling. So you have tons of different options. So basically the idea is: OK, we tested with this architecture, and the metrics are not the best. So let's change some very specific parameter, and maybe with this change we can get a better result, or worse.

OK. So when you are training a deep learning model, it is usually an exploratory analysis in which you have to find the best configuration for your model. OK, you have to test it. OK, so in order to do that, I'm going to show you a general example. So remember, this is our model, and we got an accuracy. I'm going to write this accuracy: accuracy 0.59. OK, so, for example, this is going to be the first model: model one is going to be accuracy 0.59.

OK, but, for example, we can change... I don't know, we can change this in our model. OK, so instead of, for example, having 100 neurons in the fully connected layer... so I'm going to show you here: instead of having 100 neurons here, for example, we can have 1000. So let's see what happens. So now we're going to change this parameter here, and this is going to be one thousand instead of one hundred. So now I'm going to run this code. I'm going to train this model again with ten different epochs.

I'm going to keep the same parameters as previously, but the only thing that I'm going to change is going to be the number of neurons in the hidden layer. OK, so I'm going to run this again, and I'm going to stop the video. I'll return when it is ready. OK, so now the training is ready. We are going to load the metrics again. And here it is, computing the confusion matrix. OK, and now we're going to run the classification report again, and as you can see here, the new accuracy with model two is...

Uh, the accuracy is 0.63. So as you can see here, with one change, we have improved the model by four percentage points. OK, so this is the way in which you can explore different options, because you can have one main idea, one main option, but if you make a little change, you can improve your model. OK, so that is the idea when I tell you that this is a totally exploratory analysis, because you have to test different options, different configurations, different values, different methods, in order to find the best configuration for your own model.

OK, so basically, again, training a deep learning model is usually a trial-and-error process, in which you are iterating over different configurations in order to adapt the model to your specific data.